Learn Being Forward

COURSES · CAMPUS PARTNERSHIPS · PLACEMENTS

COURSE NOTES · BASIC TO ADVANCED

React
Building Modern User Interfaces

Complete notes for the Learn Being Forward React syllabus: every module, topic and subtopic explained step by step, with diagrams, verified examples and a practice checkpoint for each module.

22	3	48	329
modules	levels	minimum days	components and programs run

How to use these notes

These notes follow the **Learn Being Forward React syllabus** exactly: the same 22 modules, in the same order, with every topic and subtopic explained step by step. Each subtopic number (for example **3.7.2**) matches the syllabus, so you can always find where you are.

Level	Modules	Minimum	Outcome
1. Beginner	1 to 8	12 days	Build interactive interfaces with JSX, components, props, state, events, lists, conditional rendering and forms
2. Intermediate	9 to 15	16 days	Handle effects and data fetching, style apps, master hooks and context, add routing, use TanStack Query and React Hook Form, and type everything with TypeScript
3. Advanced	16 to 22	20 days	Manage global state, use React 19 Actions and Suspense, optimise performance, apply advanced patterns, test thoroughly and build full-stack apps with Next.js
Total	1 to 22	48 days	About 8 weeks at 6 study days per week

Read
Read the concept for each subtopic and note the key points in your own words.

Type & run
Type out and run every example. Do not copy and paste code. Compare with the OUTPUT box.

Checkpoint
Finish the practice checkpoint at the end of each module before moving on.

Revise
At the end of each level, revise its modules and build one small project.

Every React example in these notes is real: the component was bundled with **React 19.2** and ReactDOM (development build) using esbuild, run in **Chromium**, and the picture under the code is a screenshot of what the browser displayed, taken after really clicking and typing where the caption says so. Console output and rendered HTML were captured from the same running page, and plain JavaScript examples were executed with **Node.js 22**. Styling uses Tailwind CSS v4 classes compiled with the real Tailwind compiler. TypeScript examples were checked with the real **tsc** compiler, and the end-to-end tests ran with the real **Playwright** test runner. Libraries that could not be installed in the offline build environment (React Router, TanStack Query, React Hook Form, Zod, Redux Toolkit, Zustand, Vitest, Testing Library, MSW) were run against small stand-in modules with the same API, as explained at the start of each module; code for tools that cannot run here (Next.js, Prisma, Auth.js and similar) is labelled **representative**.

HOW TO READ THE BOXES
Dark boxes with a gold top line are **JSX and JavaScript code**. Dashed cream boxes labelled **OUTPUT** show what the program prints. Gold-bordered boxes are notes and tips, red boxes warn about common mistakes, and navy boxes hold the key idea to remember.

Course plan at a glance

Module	Topic	Level	Min days
01	Introduction to React	Beginner	1
02	JavaScript essentials for React	Beginner	2

Module	Topic	Level	Min days
03	Setup and project structure	Beginner	1
04	JSX	Beginner	1
05	Components and props	Beginner	2
06	State and events	Beginner	2
07	Conditional rendering and lists	Beginner	1
08	Forms	Beginner	2
09	Side effects and useEffect	Intermediate	2
10	Styling React applications	Intermediate	1
11	Hooks in depth and custom hooks	Intermediate	3
12	Context API and state management patterns	Intermediate	2
13	Routing with React Router	Intermediate	3
14	Working with APIs and forms at scale	Intermediate	3
15	TypeScript with React	Intermediate	2
16	Global state management	Advanced	3
17	Modern React: Actions, Suspense and concurrent features	Advanced	3
18	Performance and React internals	Advanced	2
19	Advanced patterns and application architecture	Advanced	2
20	Testing React applications	Advanced	2
21	Full-stack React with Next.js	Advanced	4
22	Deployment, projects and interview preparation	Advanced	4

Table of contents

LEVEL 1 · BEGINNER LEVEL · MODULES 1 TO 8

Module 01 · Introduction to React 8

1.1 What is React? 8
1.2 History of React 11
1.3 Why do we need React? 12
1.4 Single-page applications (SPA) 14
1.5 Where is React used? 15
1.6 React vs Angular vs Vue 15

Module 02 · JavaScript essentials for React 17

2.1 Variables and functions 17
2.2 Working with data 19
2.3 Array methods 21
2.4 Conditionals 23
2.5 Modules 24
2.6 Asynchronous JavaScript 24
2.7 Other concepts 26

Module 03 · Setup and project structure 30

3.1 Tools 30
3.2 Creating a project 32
3.3 Project structure 33
3.4 Running and building 36

Module 04 · JSX 38

4.1 What JSX is 38
4.2 JSX rules 39
4.3 JavaScript in JSX 42
4.4 Comments in JSX 44
4.5 Rendering images and assets 45
4.6 JSX security 45

Module 05 · Components and props 49

5.1 Components 49
5.2 Props 53
5.3 Pure components 57
5.4 Thinking in React 59
5.5 Organising components 61

Module 06 · State and events 64

6.1 Handling events 64
6.2 The useState hook 68
6.3 Updating objects and arrays in state 72
6.4 Render and commit 76
6.5 Sharing state between components 77
6.6 Designing state 80

Module 07 · Conditional rendering and lists 85

7.1 Conditional rendering 85
7.2 Rendering lists 89

7.3 Empty, loading and error states 94

Module 08 · Forms 98

8.1 Controlled components 98
8.2 Uncontrolled components 101
8.3 Form submission 103
8.4 Validation 105
8.5 Accessible forms 107

LEVEL 2 · INTERMEDIATE LEVEL · MODULES 9 TO 15

Module 09 · Side effects and useEffect 113

9.1 Effects 113
9.2 Common uses 116
9.3 Data fetching pitfalls 121
9.4 You might not need an effect 122
9.5 useLayoutEffect (overview) 124

Module 10 · Styling React applications 127

10.1 Plain CSS and inline styles 127
10.2 CSS Modules 130
10.3 Tailwind CSS with React 131
10.4 CSS-in-JS (overview) 133
10.5 Component libraries 133
10.6 Icons and animation 134

Module 11 · Hooks in depth and custom hooks 140

11.1 useRef 140
11.2 useReducer 145
11.3 Performance hooks 148
11.4 Other hooks 150
11.5 Custom hooks 152

Module 12 · Context API and state management patterns 159

12.1 Prop drilling 159
12.2 Context 161
12.3 Context with reducers 164
12.4 Common use cases 167
12.5 Context pitfalls 168
12.6 Choosing where state lives 170

Module 13 · Routing with React Router 175

13.1 Routing basics 175
13.2 Navigation 178
13.3 Dynamic routing 181
13.4 Data APIs 186
13.5 Protected routes 190
13.6 Code splitting routes 192
13.7 Framework mode (overview) 193

Module 14 · Working with APIs and forms at scale 198

14.1 API communication 198
14.2 TanStack Query (React Query) 202
14.3 Forms with React Hook Form 210
14.4 File uploads 217

Module 15 · TypeScript with React 223

15.1 Setup 223
15.2 Typing components 224
15.3 Typing hooks 229
15.4 Typing events 234
15.5 Typing API data 236
15.6 Generic components 238

LEVEL 3 · ADVANCED LEVEL · MODULES 16 TO 22

Module 16 · Global state management 245

16.1 When you need a state library 245
16.2 Redux Toolkit 246
16.3 Zustand 251
16.4 Other options (overview) 254
16.5 Choosing a solution 255

Module 17 · Modern React: Actions, Suspense and concurrent features 261

17.1 Concurrent rendering 261
17.2 Suspense 264
17.3 Actions (React 19) 267
17.4 Other React 19 features 273
17.5 Keeping up with React 275

Module 18 · Performance and React internals 279

18.1 How React works 279
18.2 Why components re-render 282
18.3 Optimisation techniques 283
18.4 React Compiler 289
18.5 Measuring performance 290

Module 19 · Advanced patterns and application architecture 295

19.1 Component patterns 295
19.2 Error handling 303
19.3 Portals 305
19.4 Project architecture 307
19.5 Accessibility in React 308
19.6 Internationalisation (overview) 311

Module 20 · Testing React applications 316

20.1 Testing fundamentals 316
20.2 Vitest 317
20.3 React Testing Library 321
20.4 Mocking APIs 327
20.5 Testing hooks 329
20.6 End-to-end testing 331
20.7 Coverage and CI 332

Module 21 · Full-stack React with Next.js 338

21.1 Why a framework 338
21.2 Next.js App Router 340
21.3 React Server Components 343
21.4 Server Functions 347
21.5 Back-end features 348

21.6 Deploying Next.js 350
21.7 Other React frameworks (overview) 351

Module 22 · Deployment, projects and interview preparation 353

22.1 Deployment 353
22.2 Security 355
22.3 Capstone projects 357
22.4 Interview preparation 358
22.5 Next steps 365

LEVEL 1

Beginner level

Outcome: build interactive interfaces with JSX, components, props, state, events, lists, conditional rendering and forms.

8 MODULES · 12 DAYS MINIMUM

Module 01 · Introduction to React	**1 day**
Module 02 · JavaScript essentials for React	**2 days**
Module 03 · Setup and project structure	**1 day**
Module 04 · JSX	**1 day**
Module 05 · Components and props	**2 days**
Module 06 · State and events	**2 days**
Module 07 · Conditional rendering and lists	**1 day**
Module 08 · Forms	**2 days**

MODULE 01 · BEGINNER

Introduction to React

Minimum duration **1 day** | Level **Beginner**

This module explains what React is and why it exists. It defines React as a library for building user interfaces from components, contrasts libraries with frameworks and declarative with imperative UI code, and introduces component-based architecture. It traces React's history from 2013 to React 19 and the React Compiler, shows the problems React solves (manual DOM updates, reuse, efficient updates through reconciliation, one-way data flow), explains single-page applications and client versus server rendering, lists where React is used, and compares React with Angular and Vue. Every React example in this book is a real component run in Chromium.

IN THIS MODULE

1.1 What is React?
1.2 History of React
1.3 Why do we need React?
1.4 Single-page applications (SPA)
1.5 Where is React used?
1.6 React vs Angular vs Vue

1.1 What is React?

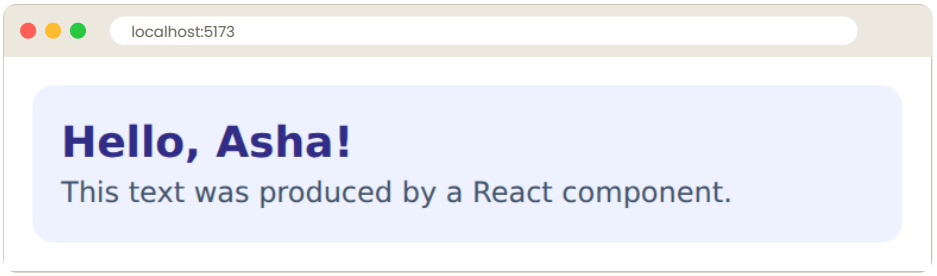
1.1.1 Definition: a JavaScript library for building user interfaces from components

React is an open-source **JavaScript library for building user interfaces**. You describe your interface as a tree of small, reusable pieces called **components**; each component is a JavaScript function that returns what should appear on the screen. When the data changes, React works out what changed and updates only those parts of the page.

The smallest complete React program is a component rendered into an element of the page. Here it is, run in a real browser:

APP.JSX — JavaScript

```
export default function App() {
  const name = "Asha";
  return (
    <div className="rounded-xl bg-indigo-50 p-4">
      <h1 className="text-2xl font-bold text-indigo-900">Hello, {name}!</h1>
      <p className="text-slate-600">This text was produced by a React component.</p>
    </div>
  );
}
```



THE COMPONENT'S OUTPUT, RENDERED BY REACT 19 IN CHROMIUM

Component
A function that returns a description of UI. Name starts with a capital letter: `App`, `ProductCard`.

JSX
The HTML-like syntax inside JavaScript (`<h1>Hello</h1>`). It is compiled to plain function calls (Module 4).

Rendering
React turns the description into real DOM elements, and keeps them in sync with your data.

1.1.2 Library vs framework

A **library** is code you call when you need it; a **framework** calls your code and decides the structure of the whole application ("inversion of control"). React deliberately focuses on one job, the view layer, so routing, data fetching and form handling come from other libraries you choose, or from a React framework such as Next.js that bundles those choices.

	Library (React)	Framework (Angular, Next.js)
Who is in control	your code calls React	the framework calls your code
Scope	one concern: rendering UI	routing, data, forms, build, testing…
Flexibility	pick your own tools	conventions decided for you
Learning curve	small core, but many choices	larger API, fewer decisions
Example	`createRoot(el).render(<App />)`	`ng new`, file-based routes in Next.js

1.1.3 Declarative vs imperative UI

Imperative code says *how* to change the screen, step by step: find this element, change its text, add that class. **Declarative** code says *what* the screen should look like for the current data, and lets React work out the steps. The same counter written both ways:

IMPERATIVE: PLAIN JAVASCRIPT AND THE DOM HTML

```
<button id="btn">Clicked 0 times</button>
<script>
  let count = 0;
  const btn = document.getElementById("btn");
  btn.addEventListener("click", () => {
    count++;                                        // 1. update the data
    btn.textContent = `Clicked ${count} times`;     // 2. update the DOM yourself
    if (count >= 3) btn.style.background = "gold"; // 3. and every other detail
  });
</script>
```



AFTER THREE REAL CLICKS: THE CODE HAD TO CHANGE THE TEXT AND THE COLOUR BY HAND

DECLARATIVE: REACT JavaScript

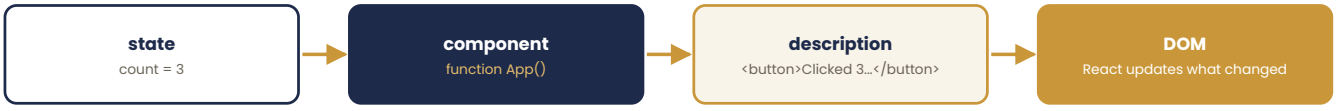
```
import { useState } from "react";

export default function App() {
  const [count, setCount] = useState(0);        // the data
  return (                                       // what the UI looks like for that data
    <button
      onClick={() => setCount(count + 1)}
      style={{ background: count >= 3 ? "gold" : undefined }}>
      Clicked {count} times
    </button>
  );
}
```



AFTER THREE REAL CLICKS: REACT RE-RAN THE COMPONENT AND UPDATED THE DOM TO MATCH

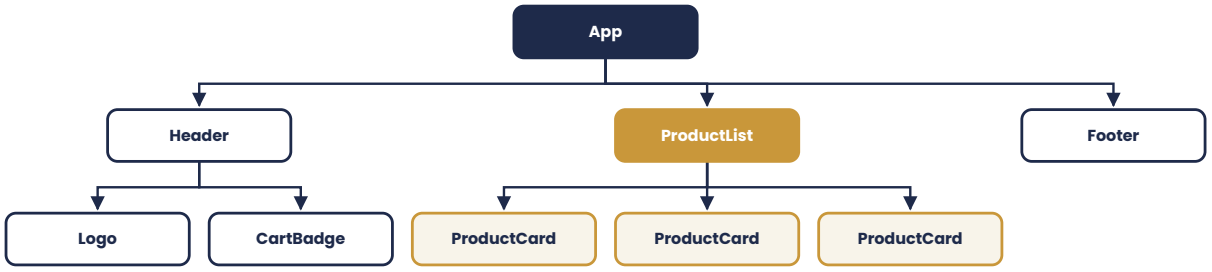
With two or three elements both approaches are fine. In a real application with hundreds of elements that depend on the same data (a cart total shown in the header, the sidebar and the checkout), imperative code must remember to update every one of them, and bugs appear when one is forgotten. Declarative code removes that whole class of bugs: **UI = f(state)**.



UI = F(STATE): THE COMPONENT IS A FUNCTION FROM DATA TO UI

1.1.4 Component-based architecture

A React application is a **tree of components**. Each one owns a piece of the screen, receives data from its parent through **props**, and can contain other components. The same component is reused wherever that piece of UI appears.



A SHOP PAGE AS A COMPONENT TREE: PRODUCTCARD IS WRITTEN ONCE AND USED THREE TIMES

APP.JSX

JavaScript

```
function ProductCard({ name, price }) {
  return (
    <div className="rounded-xl border border-slate-200 p-3">
      <h3 className="font-semibold">{name}</h3>
      <p className="text-slate-500">&#8377;{price}</p>
    </div>
  );
}

export default function App() {
  return (
    <div className="grid grid-cols-3 gap-3">
      <ProductCard name="Idli" price={40} />
      <ProductCard name="Masala dosa" price={80} />
      <ProductCard name="Filter coffee" price={25} />
    </div>
  );
}
```



ONE PRODUCTCARD COMPONENT, THREE USES WITH DIFFERENT PROPS

1.2 History of React

1.2.1 Created by Jordan Walke at Facebook, open-sourced in 2013

React was created by **Jordan Walke**, a software engineer at Facebook, inspired by XHP (an HTML component library for PHP). It was first used in Facebook's News Feed in 2011 and Instagram in 2012, and **open-sourced at JSConf US in May 2013**. The idea of re-rendering the whole UI on every change was controversial at first; the virtual DOM made it fast enough, and the model won.

1.2.2 React Native (2015)

React Native (2015) applies the same component model to native iOS and Android apps: components render to native views instead of DOM elements. "Learn once, write anywhere" became React's motto.

1.2.3 Hooks (React 16.8, 2019)

Before 2019, components that needed state had to be written as **classes**. **Hooks** (`useState`, `useEffect` and others), released in React 16.8 in February 2019, let function components hold state and side effects. Function components with hooks are now the standard; classes are legacy (Module 5).

1.2.4 Concurrent rendering (React 18)

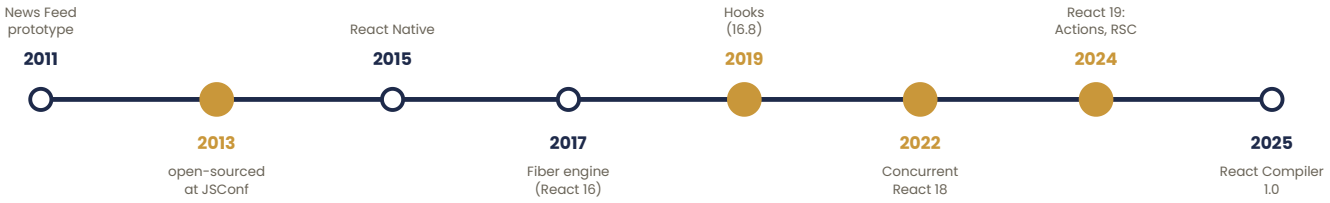
React 18 (March 2022) introduced **concurrent rendering**: React can prepare several versions of the UI at once and interrupt a slow render to handle urgent input, plus automatic batching, `useTransition` and streaming server rendering (Module 17).

1.2.5 React 19: Actions, new hooks and Server Components

React 19 (December 2024) added **Actions** (async functions for forms and mutations), new hooks (`useActionState`, `useFormStatus`, `useOptimistic`, `use`), stable **Server Components** for frameworks, and simpler APIs such as passing `ref` as a normal prop. This book uses React 19.2.

1.2.6 React Compiler

The **React Compiler** (version 1.0 released in October 2025) is a build-time tool that automatically memoises components and values, so most manual `useMemo`, `useCallback` and `memo` calls become unnecessary. It works with React 17 and later through a Babel plugin, and can be switched on in Vite, Next.js and Expo projects (Module 18).



MILESTONES IN THE HISTORY OF REACT

1.3 Why do we need React?

1.3.1 Problems with manual DOM manipulation in large apps

- **Synchronisation bugs**: the same data appears in several places, and every change must be applied to each by hand.
- **Spaghetti event code**: handlers that read from and write to the DOM become hard to follow as features grow.
- **Performance traps**: careless DOM writes cause many layouts and repaints (layout thrashing).
- **No structure**: plain scripts have no standard way to split a page into reusable, testable pieces.

1.3.2 Reusable components

A component bundles markup, logic and (optionally) styles. Written once, it can be used anywhere with different data, shared between projects or published as a library. Teams build design systems from components: `<Button>`, `<Modal>`, `<DataTable>`.

1.3.3 Efficient updates through the virtual DOM and reconciliation

When state changes, React calls the component again to get a new description of the UI (a tree of plain JavaScript objects, sometimes called the **virtual DOM**). It **compares** the new tree with the previous one (**reconciliation**, also called diffing) and applies only the differences to the real DOM, which is the slow part.



Only the changed text node is written; the button element itself is kept.

This is why React can re-render often without rebuilding the page.

RECONCILIATION: COMPARE TWO DESCRIPTIONS, APPLY ONLY THE DIFFERENCE

APP.JSX

JavaScript

```
import { useState } from "react";

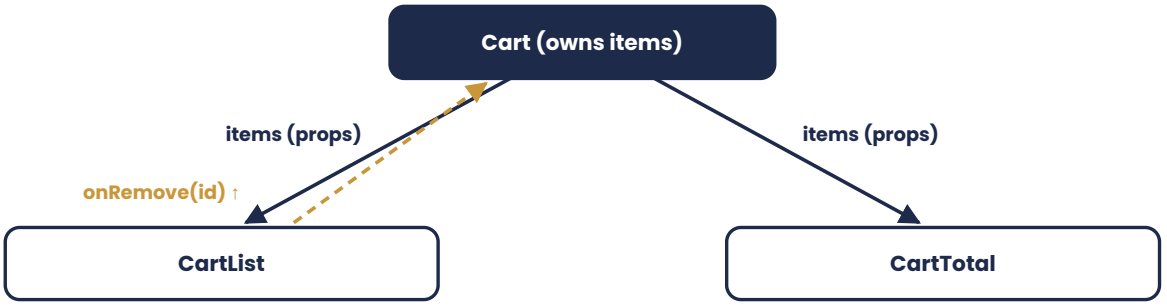
export default function App() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>Clicked {count} times</button>;
}
```

BROWSER CONSOLE (PROOF THAT THE ELEMENT IS REUSED)

```
button element before: saved
same element after 2 clicks? true
text now: Clicked 2 times
```

1.3.4 One-way data flow

Data flows **down** the component tree through props; children never change their parent's data directly. To change it, a child calls a function the parent passed down. Because every piece of data has one owner, you can always trace where a value came from, which makes large applications predictable (Module 6).



DATA DOWN, EVENTS UP

1.3.5 Huge ecosystem and community

React is the most widely used front-end library: in the Stack Overflow Developer Survey it has been the most used web framework or library among professional developers for several years running, and the `react` package is downloaded tens of millions of times per week from npm. Almost every UI need has a well-maintained React library: routing (React Router), data fetching (TanStack Query), forms (React Hook Form), state (Redux Toolkit, Zustand), UI kits (shadcn/ui, MUI), animation (Motion) and testing (Testing Library).

1.3.6 High job demand

Because so many companies build on React, it is one of the most requested front-end skills in job listings, including in India's product companies and service firms. React skills also transfer directly to React Native (mobile) and Next.js (full-stack).

1.4 Single-page applications (SPA)

1.4.1 SPA vs multi-page applications

In a traditional **multi-page application** (MPA), every click on a link asks the server for a new HTML page and the browser reloads everything. In a **single-page application** (SPA), the browser loads one HTML page and a JavaScript bundle once; after that, JavaScript changes the content and the URL without full reloads, fetching only data from the server.



MPA VERSUS SPA NAVIGATION

	Multi-page app	Single-page app
Navigation	full page reload	instant, no reload
First load	fast (HTML ready)	slower (download and run JS first)
Server sends	HTML for each page	HTML once, then JSON
SEO	easy	needs care (or server rendering)
Examples	news sites, blogs	Gmail, Trello, dashboards

1.4.2 Client-side rendering vs server-side rendering (overview)

With **client-side rendering** (CSR), the server sends an almost empty HTML file and React builds the page in the browser. With **server-side rendering** (SSR), React runs on the server first and sends ready HTML, then the browser **hydrates** it (attaches event handlers) so it becomes interactive. SSR gives faster first paint and better SEO; frameworks such as Next.js combine both, along with static generation and Server Components (Module 21).

React itself can render to an HTML string, which is the heart of SSR. The same component rendered both ways:

SSR.JS: RENDERING A COMPONENT TO AN HTML STRING WITH NODE.JS 22 main.js

```
const { createElement: h } = require("react");
const { renderToString } = require("react-dom/server");

function Greeting({ name }) {
  return h("h1", { className: "title" }, "Hello, ", name);
}

console.log(renderToString(h(Greeting, { name: "Asha" })));
```

OUTPUT

```
<h1 class="title">Hello, <!-- -->Asha</h1>
```

NOTE

`<!-- -->` in the output is a marker React inserts between adjacent text pieces, so that hydration in the browser can match each text node exactly.

1.5 Where is React used?

1.5.1 Web apps (Facebook, Instagram, Netflix, Airbnb, WhatsApp Web)

Meta builds Facebook, Instagram and WhatsApp Web with React. Netflix, Airbnb, Uber, Discord, Atlassian, Shopify, and Indian companies such as Swiggy, Zomato and Flipkart use React for large parts of their web front ends. Dashboards and internal tools in most companies are commonly React SPAs.

1.5.2 Full-stack frameworks (Next.js, React Router framework mode)

The React team now recommends starting new apps with a framework. **Next.js** (by Vercel) adds file-based routing, server rendering, Server Components and API routes; **React Router** in framework mode (the successor of Remix) provides routing with data loading and server rendering. Both are covered in Modules 13 and 21.

1.5.3 Mobile apps with React Native

React Native renders components to native mobile views. Components use `<View>` and `<Text>` instead of `<div>` and `<p>`, but state, props and hooks work exactly as in this book. Facebook, Instagram, Discord and Shopify ship React Native apps; Expo is the standard toolkit.

1.5.4 Desktop apps with Electron

Electron packages a web app with Chromium and Node.js into a desktop application. VS Code, Slack, Discord and Figma's desktop app are Electron apps, and many Electron apps use React for their interface.

1.6 React vs Angular vs Vue

1.6.1 Key differences at a glance

	React	Angular	Vue
Type	UI library	full framework	progressive framework

	React	Angular	Vue
Made by	Meta (2013)	Google (2016; AngularJS 2010)	Evan You and community (2014)
Templates	JSX (JavaScript)	HTML templates with Angular syntax	HTML templates (or JSX)
Language	JavaScript or TypeScript	TypeScript	JavaScript or TypeScript
State / reactivity	state + re-render, hooks	signals and RxJS	reactive refs (proxies)
Routing, forms, HTTP	separate libraries	built in	official add-ons
Learning curve	gentle core, many choices	steepest	gentlest
Typical users	startups to big tech	large enterprises	small to large teams, Asia-Pacific

All three are mature and fast; the concepts you learn here (components, props, state, effects, routing) carry over to the others. Learn Being Forward offers separate Angular and Vue courses.

PRACTICE CHECKPOINT

Explain in your own words **declarative vs imperative UI**, and why a component-based library makes large interfaces easier to build and maintain.

Model answer. Imperative UI code lists the steps to change the page (find an element, set its text, add a class). Declarative UI code describes what the page should look like for the current data, and the library works out the steps. In React a component is a function from state to UI, so when the data changes we only change the data; React re-renders and updates the DOM. Components split a large interface into small, independent, reusable pieces with clear inputs (props), so each piece can be understood, tested and reused on its own, and one-way data flow makes it easy to see where each value comes from.

React is a library for building UIs from components: functions that turn data into a description of the screen. It is declarative (UI = f(state)), updates the DOM efficiently through reconciliation, and has the largest ecosystem of any front-end technology.

MODULE 02 · BEGINNER

JavaScript essentials for React

Minimum duration **2 days** | Level **Beginner**

React is "just JavaScript": components are functions, props are objects and lists are arrays. This module reviews exactly the modern JavaScript that React code uses every day: let and const, arrow functions and default parameters; template literals, destructuring, spread and rest, shorthand properties, optional chaining and nullish coalescing; map, filter, reduce, find and some, and how to update arrays and objects without mutating them; the ternary operator and the && pitfall with 0; ES modules; promises, async/await and fetch; and closures and reference equality, which explain many React behaviours. Every program was run with Node.js 22.

IN THIS MODULE

2.1 Variables and functions
2.2 Working with data
2.3 Array methods
2.4 Conditionals
2.5 Modules
2.6 Asynchronous JavaScript
2.7 Other concepts

2.1 Variables and functions

2.1.1 let and const

Use `const` by default and `let` only when the variable must be reassigned. Never use `var`: it is function-scoped and hoisted, which causes surprising bugs. In React almost everything is `const`: components, props, state values and handlers, because each render creates fresh constants rather than changing old ones.

LET_CONST.JS — main.js

```
const course = "React";
let lessons = 22;
lessons = lessons + 1;          // let can be reassigned
console.log(course, lessons);

const student = { name: "Asha" };
student.name = "Ravi";          // allowed: the object is changed, not the binding
console.log(student);

try {
  course = "Vue";               // TypeError: the binding is constant
} catch (e) {
  console.log(e.constructor.name + ":", e.message);
}
```

OUTPUT

```
React 23
{ name: 'Ravi' }
TypeError: Assignment to constant variable.
```

NOTE

`const` makes the **binding** constant, not the value: a `const` object can still be mutated. In React we choose not to mutate state objects, for reasons explained in Section 2.3.2.

2.1.2 Arrow functions and implicit returns

Arrow functions are shorter function expressions. With a single expression after `=>` the value is returned **implicitly**; to return an object literal implicitly, wrap it in parentheses. React code uses arrows everywhere: event handlers, `map` callbacks and often components themselves.

ARROWS.JS — main.js

```
function square(n) { return n * n; }            // function declaration
const square2 = (n) => { return n * n; };         // arrow with a body
const square3 = n => n * n;                       // implicit return
const makeUser = (name) => ({ name, active: true }); // object: wrap in ( )
const greet = () => "Hello";                      // no parameters

console.log(square(4), square2(4), square3(4));
console.log(makeUser("Asha"), greet());
console.log([1, 2, 3].map(n => n * 10));
```

OUTPUT

```
16 16 16
{ name: 'Asha', active: true } Hello
[ 10, 20, 30 ]
```

Form	Example	Notes
Declaration	`function App() { ... }`	hoisted; common for components
Arrow, block body	`const add = (a, b) => { return a + b; }`	needs an explicit `return`
Arrow, expression	`const add = (a, b) => a + b`	returns the expression
Returning JSX	`const Hi = () => (<p>Hi</p>)`	parentheses allow multi-line JSX

2.1.3 Default parameters

A parameter can have a default that is used when the argument is `undefined` (missing). React components use the same syntax inside destructuring to give props defaults (Module 5).

DEFAULTS.JS — main.js

```
function price(amount, discount = 0, currency = "Rs") {
  return `${currency} ${amount - amount * discount}`;
}
console.log(price(500));
console.log(price(500, 0.1));
console.log(price(500, undefined, "INR"));  // undefined -> default used
console.log(price(500, null));               // null is NOT undefined
```

OUTPUT

```
Rs 500
Rs 450
INR 500
Rs 500
```

2.2 Working with data

2.2.1 Template literals

Backtick strings can embed any expression with `${...}` and can span several lines. They are used to build class names, URLs and messages.

TEMPLATE.JS — main.js

```
const name = "Asha", score = 92;
console.log(`${name} scored ${score} (${score >= 75 ? "distinction" : "pass"})`);
const id = 7;
const url = `https://api.example.com/students/${id}?include=marks`;
console.log(url);
const active = true;
const className = `btn ${active ? "btn-active" : ""}`.trim();
console.log(className);
```

OUTPUT

```
Asha scored 92 (distinction)
https://api.example.com/students/7?include=marks
btn btn-active
```

2.2.2 Object and array destructuring

Destructuring unpacks values from objects and arrays into variables. React relies on it constantly: `function Card({ title, price })` destructures props, and `const [count, setCount] = useState(0)` destructures the array returned by a hook.

DESTRUCTURING.JS — main.js

```
const student = { name: "Asha", age: 20, address: { city: "Mysuru" } };
const { name, age } = student;                      // by property name
const { name: fullName, grade = "A" } = student;    // rename + default
const { address: { city } } = student;              // nested
console.log(name, age, fullName, grade, city);

const rgb = [255, 140, 0];
const [r, g, b] = rgb;                              // by position
const [first, , third] = rgb;                       // skip an item
console.log(r, g, b, first, third);

function useCounter() { return [5, () => {}]; }     // like useState
const [count, setCount] = useCounter();
console.log(count, typeof setCount);
```

OUTPUT

```
Asha 20 Asha A Mysuru
255 140 0 255 0
5 function
```

2.2.3 Spread and rest operators

The same three dots do two opposite jobs. **Spread** expands an array or object into its elements, which is how React code copies and updates data without mutation. **Rest** collects the remaining elements into a new array or object, used to pass "all other props" to a child.

SPREAD_REST.JS

main.js

```
// spread: copy and extend
const base = [1, 2, 3];
const more = [...base, 4, 5];
const user = { name: "Asha", city: "Mysuru" };
const updated = { ...user, city: "Bengaluru", age: 21 };   // later keys win
console.log(more, base);
console.log(updated, user);

// rest: collect the remainder
const [head, ...tail] = more;
const { name, ...others } = updated;
console.log(head, tail, name, others);

function sum(...nums) { return nums.reduce((a, b) => a + b, 0); }
console.log(sum(1, 2, 3, 4));
```

OUTPUT

```
[ 1, 2, 3, 4, 5 ] [ 1, 2, 3 ]
{ name: 'Asha', city: 'Bengaluru', age: 21 } { name: 'Asha', city: 'Mysuru' }
1 [ 2, 3, 4, 5 ] Asha { city: 'Bengaluru', age: 21 }
10
```



Spread copies one level deep ("shallow copy"): nested objects are shared, not copied.

To change a nested value, spread each level you change (Module 6).

SPREAD CREATES A NEW OBJECT WITH THE OLD PROPERTIES PLUS THE CHANGES

2.2.4 Shorthand properties and computed keys

SHORTHAND.JS

main.js

```
const name = "Asha", age = 20;
const student = { name, age };                 // same as { name: name, age: age }
console.log(student);

const field = "email";
const value = "asha@example.in";
const form = { ...student, [field]: value };   // computed key: the value of field
console.log(form);

const methods = { greet() { return "hi"; } };  // method shorthand
console.log(methods.greet());
```

OUTPUT

```
{ name: 'Asha', age: 20 }
{ name: 'Asha', age: 20, email: 'asha@example.in' }
hi
```

Computed keys are the basis of the **one change handler for many inputs** pattern in forms: `setForm({ ...form, [e.target.name]: e.target.value })` (Module 8).

2.2.5 Optional chaining and nullish coalescing

`a?.b` returns `undefined` instead of throwing when `a` is `null` or `undefined`, which is invaluable while data from an API is still loading. `a ?? b` returns `b` only when `a` is `null` or `undefined`, unlike `||`, which also replaces `0` and `""`.

OPTIONAL.JS — main.js

```
const user = { name: "Asha", address: null, stats: { followers: 0 } };
console.log(user.address?.city);                    // undefined, no crash
console.log(user.profile?.photo?.url ?? "default.png");
console.log(user.getAge?.());                       // call only if it exists

console.log(user.stats.followers || "none");        // 0 is falsy -> "none" (wrong)
console.log(user.stats.followers ?? "none");        // 0 kept (right)
```

OUTPUT

```
undefined
default.png
undefined
none
0
```

2.3 Array methods

2.3.1 map(), filter(), reduce(), find(), some()

These methods take a callback, never change the original array, and return a new value. They are the core of rendering in React: `map` turns data into a list of elements, `filter` selects what to show, `reduce` computes totals.

ARRAY_METHODS.JS — main.js

```
const products = [
  { id: 1, name: "Idli", price: 40, veg: true },
  { id: 2, name: "Chicken biryani", price: 220, veg: false },
  { id: 3, name: "Masala dosa", price: 80, veg: true },
];
console.log(products.map(p => p.name));                    // transform each
console.log(products.filter(p => p.veg).map(p => p.id));   // keep some
console.log(products.reduce((sum, p) => sum + p.price, 0)); // combine all
console.log(products.find(p => p.price > 100)?.name);      // first match
console.log(products.some(p => !p.veg), products.every(p => p.price > 30));
console.log(products.findIndex(p => p.id === 3));
```

OUTPUT

```
[ 'Idli', 'Chicken biryani', 'Masala dosa' ]
[ 1, 3 ]
340
Chicken biryani
true true
2
```

Method	Returns	Typical use in React
`map`	new array, same length	render a list: `items.map(i => <Item key={i.id} />)`

Method	Returns	Typical use in React
`filter`	new array, fewer items	search, tabs (All / Active / Done), remove an item
`reduce`	one value	cart total, counts per category
`find` / `findIndex`	first match or undefined / index or -1	selected item by id
`some` / `every`	boolean	is anything selected? are all fields valid?
`toSorted`, `toReversed`	new sorted / reversed array (ES2023)	sort without mutating state

WATCH OUT

`sort()`, `reverse()`, `splice()`, `push()` and `pop()` **mutate** the array in place. On React state, use the non-mutating versions: `toSorted()`, `toReversed()`, `toSpliced()`, spread and `filter`.

2.3.2 Immutable updates: adding, removing and updating items without mutation

React decides whether to re-render by checking whether state is a **new** value. If you mutate an existing array, its reference is unchanged and React may not notice. So every update creates a new array or object:

IMMUTABLE.JS — main.js

```
const todos = [
  { id: 1, text: "Learn JSX", done: true },
  { id: 2, text: "Learn state", done: false },
];

const added   = [...todos, { id: 3, text: "Learn effects", done: false }];   // add
const removed = todos.filter(t => t.id !== 1);                               // remove
const toggled = todos.map(t => (t.id === 2 ? { ...t, done: !t.done } : t)); // update one
const renamed = todos.map(t => (t.id === 1 ? { ...t, text: "Master JSX" } : t));
const inserted = [...todos.slice(0, 1), { id: 9, text: "Inserted", done: false }, ...todos.slice(1)];

console.log(added.length, removed.map(t => t.id), toggled[1].done, renamed[0].text);
console.log(inserted.map(t => t.id));
console.log("original untouched:", todos.map(t => `${t.id}:${t.done}`).join(" "));
console.log("same array?", toggled === todos, "| unchanged item reused?", toggled[0] === todos[0]);
```

OUTPUT

```
3 [ 2 ] true Master JSX
[ 1, 9, 2 ]
original untouched: 1:true 2:false
same array? false | unchanged item reused? true
```

Operation	Mutating (avoid on state)	Immutable (use)
add	`arr.push(x)`	`[...arr, x]`
remove	`arr.splice(i, 1)`	`arr.filter(x => x.id !== id)`
update	`arr[i].done = true`	`arr.map(x => x.id === id ? {...x, done: true} : x)`
sort	`arr.sort(fn)`	`arr.toSorted(fn)` or `[...arr].sort(fn)`
object field	`obj.city = "X"`	`{ ...obj, city: "X" }`

2.4 Conditionals

2.4.1 Ternary operator

JSX can only contain **expressions**, not statements such as `if`. The ternary `condition ? a : b` is an expression, so it is the standard way to choose between two pieces of UI inline.

TERNARY.JS — main.js

```
const loggedIn = false, cartCount = 3;
console.log(loggedIn ? "Welcome back" : "Please log in");
console.log(`You have ${cartCount} item${cartCount === 1 ? "" : "s"}`);
const label = cartCount === 0 ? "Empty" : cartCount < 5 ? "Some" : "Many";  // nest sparingly
console.log(label);
```

OUTPUT

```
Please log in
You have 3 items
Some
```

2.4.2 Logical && and the 0 pitfall

`a && b` returns `a` if it is falsy, otherwise `b`. In JSX, `{show && <Badge />}` renders the badge only when `show` is true, because React renders nothing for `false`, `null` and `undefined`. But React **does** render the number `0`, so `{count && ...}` shows a stray "0" when the count is zero:

AND_PITFALL.JS — main.js

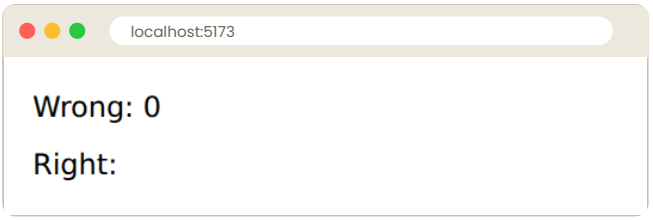
```
const count = 0;
console.log(true && "Badge");    // "Badge"
console.log(false && "Badge");   // false -> React renders nothing
console.log(count && "3 new");   // 0     -> React renders "0"!
console.log(count > 0 && "3 new"); // false -> nothing (correct)
console.log(!!count && "3 new");   // false
```

OUTPUT

```
Badge
false
0
false
false
```

APP.JSX — JavaScript

```
export default function App() {
  const count = 0;
  return (
    <div className="space-y-2">
      <p>Wrong: {count && <b>{count} new messages</b>}</p>
      <p>Right: {count > 0 && <b>{count} new messages</b>}</p>
    </div>
  );
}
```



THE SAME CHECK IN REAL REACT: THE FIRST LINE SHOWS A STRAY 0

2.5 Modules

2.5.1 import and export (named and default)

Each React component usually lives in its own file (a **module**). A module can have many **named** exports, imported with braces and the exact name, and at most one **default** export, imported without braces under any name. React projects typically default-export the main component of each file.

MATH.JS JavaScript

```
// math.js
export const PI = 3.14159;                    // named export
export function area(r) { return PI * r * r; }
export default function circle(r) {           // default export
  return { r, area: area(r) };
}
```

MAIN.JS (RUN WITH NODE.JS 22 AS AN ES MODULE) main.js

```
import circle, { area, PI as pi } from "./math.js";   // default + named (renamed)
import * as maths from "./math.js";                    // everything as an object

console.log(pi, area(2).toFixed(2));
console.log(circle(1));
console.log(Object.keys(maths));
```

OUTPUT

```
3.14159 12.57
{ r: 1, area: 3.14159 }
[ 'PI', 'area', 'default' ]
```

	Named export	Default export
Per file	any number	at most one
Export	`export function Card() {}`	`export default function App() {}`
Import	`import { Card } from "./Card"`	`import App from "./App"`
Name at import	must match (or `as`)	any name you like

2.6 Asynchronous JavaScript

2.6.1 Promises and async/await

Slow operations (network requests, timers, file reads) return a **promise**: an object that will later be **fulfilled** with a value or **rejected** with an error. `async` functions return promises, and inside them `await`

pauses until a promise settles, so asynchronous code reads like normal top-to-bottom code. Errors are caught with `try/catch`.

ASYNC.JS

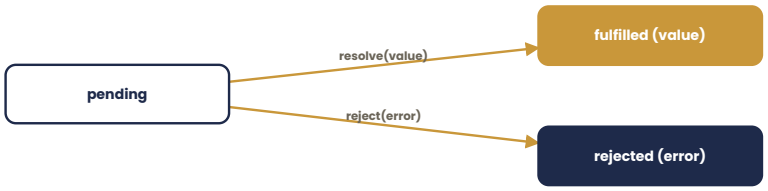
main.js

```
const wait = (ms, value, fail = false) =>
  new Promise((resolve, reject) => setTimeout(() => (fail ? reject(new Error("timeout")) :
resolve(value)), ms));

async function main() {
  console.log("1. start");
  const a = await wait(100, "first result");
  console.log("2.", a);
  const [b, c] = await Promise.all([wait(50, "B"), wait(80, "C")]);   // in parallel
  console.log("3.", b, c);
  try {
    await wait(30, null, true);
  } catch (err) {
    console.log("4. caught:", err.message);
  } finally {
    console.log("5. finally runs either way");
  }
}
main();
console.log("0. this line runs before the awaits finish");
```

OUTPUT

```
1. start
0. this line runs before the awaits finish
2. first result
3. B C
4. caught: timeout
5. finally runs either way
```



THE THREE STATES OF A PROMISE

2.6.2 fetch() and handling JSON

`fetch(url)` returns a promise for a `Response`. Check `response.ok` (fetch only rejects on network failure, not on HTTP 404 or 500), then read the body with `await response.json()`. In the example a small local server stands in for a public API, so the request really travels over HTTP:

FETCH.JS

main.js

```
const http = require("node:http");

// a tiny local API server (stands in for a public API)
const server = http.createServer((req, res) => {
  res.setHeader("Content-Type", "application/json");
  if (req.url === "/users") {
    res.end(JSON.stringify([{ id: 1, name: "Asha" }, { id: 2, name: "Ravi" }]));
  } else {
    res.statusCode = 404;
    res.end(JSON.stringify({ error: "Not found" }));
  }
});

async function getJSON(url) {
  const res = await fetch(url);
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}

server.listen(0, async () => {
  const base = `http://localhost:${server.address().port}`;
  try {
    const users = await getJSON(`${base}/users`);
    console.log("Users:", users.map(u => u.name).join(", "));
    await getJSON(`${base}/missing`);
  } catch (err) {
    console.log("Error:", err.message);
  } finally {
    server.close();
  }
});
```

OUTPUT

```
Users: Asha, Ravi
Error: HTTP 404
```

TIP

In React components, data is fetched in an effect or, better, with a data library such as TanStack Query, which handles loading, errors, caching and retries (Modules 9 and 14).

2.7 Other concepts

2.7.1 Closures

A **closure** is a function that remembers the variables of the scope where it was created, even after that scope has finished. Every event handler in a component is a closure over that render's props and state, which is why a handler "sees" the values from the render that created it (the **stale closure** behaviour in Module 6).

CLOSURES.JS

main.js

```
function makeCounter() {
  let count = 0;                        // private to this closure
  return () => ++count;
}
const next = makeCounter();
console.log(next(), next(), next());   // 1 2 3
const other = makeCounter();
console.log(other());                  // 1: each call has its own count

// a "render" captures the value at that moment
function render(count) {
  return () => console.log("handler sees count =", count);
}
const handlerFromRender1 = render(1);
const handlerFromRender2 = render(2);
handlerFromRender1();
handlerFromRender2();
```

OUTPUT

```
1 2 3
1
handler sees count = 1
handler sees count = 2
```

2.7.2 Reference vs value equality

Primitives (numbers, strings, booleans) are compared by **value**. Objects, arrays and functions are compared by **reference**: two separately created objects are never `===`, even with identical contents, and a mutated object is still `===` to itself. React uses `Object.is` (almost the same as `===`) to decide whether state, props and dependencies changed.

EQUALITY.JS

main.js

```
console.log(5 === 5, "a" === "a");            // value equality
console.log({ a: 1 } === { a: 1 });             // false: two objects
console.log([] === []);                          // false

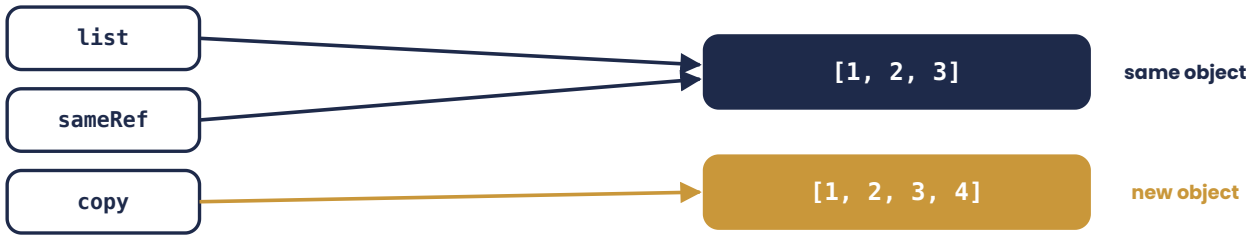
const list = [1, 2];
const sameRef = list;
sameRef.push(3);                                 // mutation
console.log(list === sameRef, list);             // true: React would see "no change"

const copy = [...list, 4];
console.log(copy === list);                      // false: React sees a change

const f1 = () => {}, f2 = () => {};
console.log(f1 === f2);                          // false: functions too
console.log(Object.is(NaN, NaN), NaN === NaN);
```

OUTPUT

```
true true
false
false
true [ 1, 2, 3 ]
false
false
true false
```



TWO NAMES FOR ONE ARRAY VERSUS A REAL COPY

PRACTICE CHECKPOINT

Given an array of products, write **immutable functions** to add, update, remove and filter products, and **fetch data from an API** with async/await.

PRODUCTS.JS: SAMPLE SOLUTION (IMMUTABLE HELPERS) main.js

```
const products = [
  { id: 1, name: "Keyboard", price: 1200, stock: 5 },
  { id: 2, name: "Mouse", price: 600, stock: 0 },
];

const addProduct = (list, product) => [...list, { id: Math.max(0, ...list.map(p => p.id)) + 1,
...product }];
const updateProduct = (list, id, changes) => list.map(p => (p.id === id ? { ...p, ...changes } : p));
const removeProduct = (list, id) => list.filter(p => p.id !== id);
const inStock = list => list.filter(p => p.stock > 0);
const search = (list, text) => list.filter(p => p.name.toLowerCase().includes(text.toLowerCase()));

let v1 = addProduct(products, { name: "Monitor", price: 9500, stock: 2 });
let v2 = updateProduct(v1, 2, { stock: 10, price: 550 });
let v3 = removeProduct(v2, 1);
console.log(v3);
console.log("in stock:", inStock(v1).map(p => p.name));
console.log("search 'mo':", search(v3, "mo").map(p => p.name));
console.log("original unchanged:", products.length, products[1].stock);
```

OUTPUT

```
[
  { id: 2, name: 'Mouse', price: 550, stock: 10 },
  { id: 3, name: 'Monitor', price: 9500, stock: 2 }
]
in stock: [ 'Keyboard', 'Monitor' ]
search 'mo': [ 'Mouse', 'Monitor' ]
original unchanged: 2 0
```

FETCH_TODOS.JS: SAMPLE SOLUTION (ASYNC/AWAIT WITH A LOCAL API)

main.js

```
const http = require("node:http");
const data = [{ id: 1, title: "Learn JSX" }, { id: 2, title: "Learn hooks" }];
const server = http.createServer((req, res) => {
  res.setHeader("Content-Type", "application/json");
  res.end(JSON.stringify(data));
});

async function loadTodos(url) {
  try {
    const res = await fetch(url);
    if (!res.ok) throw new Error(`Request failed: ${res.status}`);
    const todos = await res.json();
    return todos.map(t => t.title);
  } catch (err) {
    console.error("Could not load:", err.message);
    return [];
  }
}

server.listen(0, async () => {
  const titles = await loadTodos(`http://localhost:${server.address().port}/todos`);
  console.log(titles);
  server.close();
});
```

OUTPUT

```
[ 'Learn JSX', 'Learn hooks' ]
```

React code is modern JavaScript: `const`, arrow functions, destructuring, spread, `map` / `filter` / `reduce`, immutable updates, ternaries, modules and async/await. Remember that objects compare by reference, so updates must create new objects, and that handlers are closures over the values of their render.

MODULE 03 · BEGINNER

Setup and project structure

Minimum duration **1 day** | Level **Beginner**

This module sets up a professional React development environment. It covers Node.js and npm, the recommended VS Code extensions and React Developer Tools; creating a project with Vite, why Create React App is no longer recommended, and when to start with a framework instead; the files of a Vite project (index.html, main.jsx, App.jsx, package.json, public and src/assets) and what createRoot() and StrictMode do; and the dev server, production build, preview and hot module replacement. Terminal sessions show the standard output of the official tools; the components themselves were run in Chromium.

IN THIS MODULE

3.1 Tools
3.2 Creating a project
3.3 Project structure
3.4 Running and building

3.1 Tools

3.1.1 Node.js and npm

React code runs in the browser, but the **tools** that build it (the dev server, the bundler, linters, test runners) run on **Node.js**, a JavaScript runtime for your computer. Node comes with **npm**, the package manager that downloads libraries such as React from the npm registry into a `node_modules` folder and records them in `package.json`. Install the current **LTS** version from `nodejs.org` (Vite needs Node 20.19+ or 22.12+).

Checking Node.js and npm

```
$ node --version
v22.22.2
$ npm --version
10.9.4
$ npm install react react-dom        # add packages to the current project
$ npm install -D vite                # -D = development dependency (build tools)
$ npx some-tool                      # run a package's command without installing it globally
```

Command	What it does
`npm install` (`npm i`)	install everything listed in package.json
`npm install pkg`	add a runtime dependency
`npm install -D pkg`	add a development-only dependency
`npm run dev`	run the `dev` script from package.json
`npm uninstall pkg`	remove a package

Command	What it does
`npx cmd`	run a command from a package (downloads it if needed)

> **NOTE**
> pnpm, Yarn and Bun are alternative package managers with the same idea; commands differ slightly (`pnpm add` , `yarn add` , `bun add`). This book uses npm.

3.1.2 VS Code extensions: ES7 React snippets, Prettier, ESLint

Extension	Why
ES7+ React/Redux/React-Native snippets	type `rafce` + Tab to generate a component skeleton; `useState` snippets
Prettier	formats code consistently on save (enable "Format on Save")
ESLint	shows problems as you type, including the Rules of Hooks and missing effect dependencies (Vite's template includes the React ESLint config)
Tailwind CSS IntelliSense	autocomplete for class names if you use Tailwind (Module 10)
Error Lens (optional)	shows errors inline next to the code

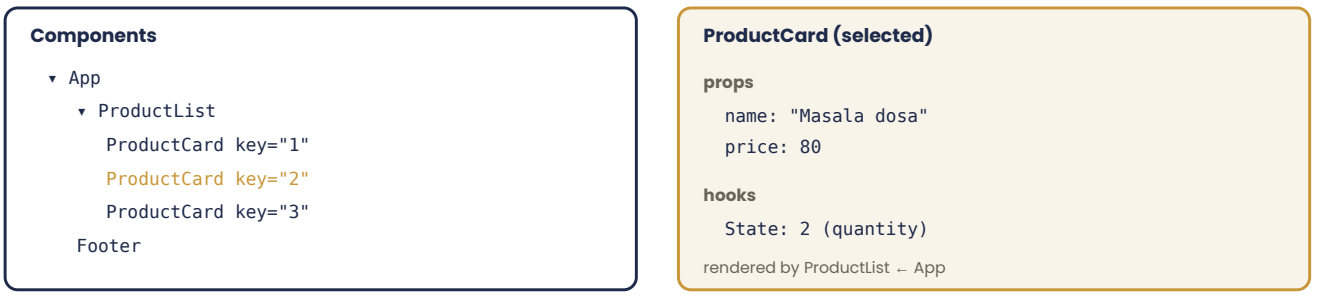
PROFILECARD.JSX (SNIPPET OUTPUT) JavaScript

```
// typing  rafce  + Tab  produces:
const ProfileCard = () => {
  return <div>ProfileCard</div>;
};

export default ProfileCard;
```

3.1.3 React Developer Tools browser extension

React Developer Tools (for Chrome, Edge and Firefox) adds two tabs to the browser DevTools. **Components** shows the component tree of the page, and for the selected component its props, state and hooks, which you can edit live. **Profiler** records renders and shows which components rendered, why and how long they took (Module 18). In development, React prints a reminder to the console if the extension is not installed.



THE COMPONENTS TAB: THE TREE ON THE LEFT, THE SELECTED COMPONENT'S PROPS AND STATE ON THE RIGHT

3.2 Creating a project

3.2.1 Vite: npm create vite@latest

Vite (French for "quick", pronounced "veet") is the standard build tool for React single-page apps. It starts a dev server instantly by serving source files as native ES modules, and builds an optimised bundle for production with Rolldown/Rollup. Create a project with one command and answer the prompts (or pass the template name):

Creating and starting a Vite React project

```
$ npm create vite@latest student-portal -- --template react
Scaffolding project in /home/asha/student-portal...

Done. Now run:

  cd student-portal
  npm install
  npm run dev

$ cd student-portal && npm install
added 152 packages in 9s
$ npm run dev

  VITE v7.1.9  ready in 412 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
```

Template	Gives you
`react`	React with JavaScript (used in Modules 3 to 14)
`react-ts`	React with TypeScript (Module 15)
`react-swc` / `react-swc-ts`	the same, using the faster SWC compiler

3.2.2 Why Create React App is no longer recommended

Create React App (CRA, `npx create-react-app`) was the official starter from 2016. In February 2025 the React team **deprecated** it: it was slow (webpack-based), unmaintained, incompatible with newer tools, and gave no answer for routing, data fetching or server rendering. Old tutorials still use it; for new work use Vite for a single-page app, or a framework.

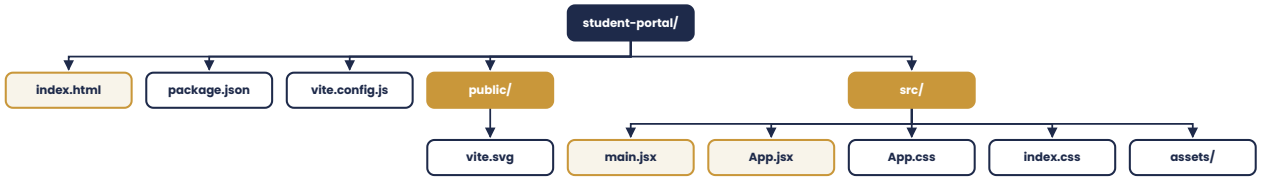
	Create React App	Vite
Status	deprecated (2025)	actively developed
Dev server start	seconds to minutes (bundles first)	well under a second (native ES modules)
Updates after a change	slow rebuilds	instant hot module replacement
Entry file	`src/index.js`	`index.html` + `src/main.jsx`
Env variables	`REACT_APP_*`	`VITE_*` via `import.meta.env`

3.2.3 Framework options: Next.js and React Router (introduced in Modules 13 and 21)

The React documentation now recommends a **framework** for most new full applications, because real apps need routing, data loading, code splitting and often server rendering. **Next.js** (`npx create-next-app@latest`) is the most popular, with Server Components and file-based routing (Module 21). **React Router** in framework mode (`npx create-react-router@latest`) adds data loading and server rendering to the most widely used router (Module 13). A plain Vite app remains the best way to **learn** React and to build client-only apps such as dashboards, so Modules 3 to 20 use Vite.

3.3 Project structure

3.3.1 index.html, main.jsx, App.jsx



THE FILES OF A NEW VITE REACT PROJECT (NODE_MODULES AND CONFIG FILES FOR ESLINT OMITTED)

The three files that matter, in the order the browser meets them:

INDEX.HTML HTML

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>student-portal</title>
  </head>
  <body>
    <div id="root"></div>                          <!-- React renders inside this -->
    <script type="module" src="/src/main.jsx"></script>     <!-- the entry point -->
  </body>
</html>
```

SRC/MAIN.JSX JavaScript

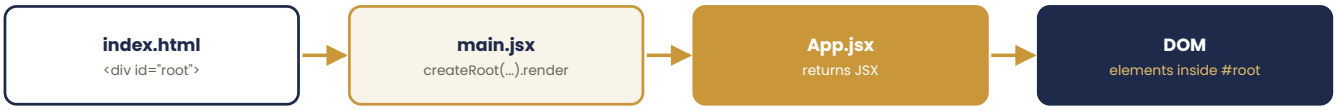
```jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

SRC/APP.JSX (AFTER CLEANING OUT THE STARTER CODE) JavaScript

```
function App() {
  return <h1>Hello React</h1>
}

export default App
```



HOW A VITE REACT APP STARTS

3.3.2 package.json and scripts

PACKAGE.JSON (TYPICAL, TRIMMED) JavaScript

```
{
  "name": "student-portal",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.0",
    "react-dom": "^19.2.0"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^5.0.4",
    "eslint": "^9.36.0",
    "vite": "^7.1.7"
  }
}
```

Field	Meaning
`scripts`	commands run with `npm run <name>`
`dependencies`	needed by the app in the browser (React, router…)
`devDependencies`	needed only to develop and build (Vite, ESLint, test tools)
`^19.2.0`	semantic versioning: any 19.x.y at or above 19.2.0
`package-lock.json`	records the exact installed versions; commit it to Git
`node_modules/`	downloaded packages; never commit it (it is in .gitignore)

3.3.3 public vs src/assets

	public/	src/assets/
How you reference it	absolute path: `<img src="/logo.png">`	import: `import logo from "./assets/logo.png"`
Processed by Vite	no: copied as-is	yes: optimised, hashed file name for caching
Missing file	error only at runtime (broken image)	error at build time
Use for	favicon, robots.txt, files that need a fixed URL	images, fonts and SVGs used by components

HEADER.JSX

JavaScript

```
import logo from "./assets/logo.svg";      // becomes a URL string, e.g. "/assets/logo-B3x9.svg"

export default function Header() {
  return (
    <header>
      <img src={logo} alt="Learn Being Forward" width="40" />
      <img src="/vite.svg" alt="" width="24" />   {/* from public/ */}
    </header>
  );
}
```

3.3.4 createRoot() and StrictMode

`createRoot(domNode)` (from `react-dom/client`) creates a React root that takes over a DOM element; `root.render(<App />)` renders the component tree into it. An app normally has exactly one root. `<StrictMode>` is a development-only helper that renders nothing itself but activates extra checks for its children: components render **twice**, effects run setup, cleanup and setup again, and deprecated APIs produce warnings. The goal is to expose impure code early. In production builds it has no effect.

APP.JSX

JavaScript

```
let renders = 0;               // a module-level counter (impure on purpose)

export default function App() {
  renders++;
  console.log("App rendered, render number", renders);
  return <h1>Hello</h1>;
}
```

BROWSER CONSOLE WITH <StrictMode> (development build)

```
App rendered, render number 1
App rendered, render number 2
```

BROWSER CONSOLE WITHOUT <StrictMode>

```
App rendered, render number 1
```

With StrictMode the component ran twice for one visible render (React 19 dims the second log in the DevTools console). A component that changes outside variables while rendering gives different results each time, and StrictMode makes that visible. Module 5 explains purity.

3.4 Running and building

3.4.1 npm run dev

`npm run dev` starts Vite's development server at `http://localhost:5173`. It transforms JSX on demand, shows compile errors as an overlay in the browser and uses React's development build, which includes helpful warnings (missing keys, invalid hooks) at the cost of size and speed.

3.4.2 npm run build and preview

Building for production (typical output)

```
$ npm run build

> student-portal@0.0.0 build
> vite build

vite v7.1.9 building for production...
✓ 31 modules transformed.
dist/index.html                   0.46 kB │ gzip:  0.30 kB
dist/assets/index-D8b4DHJx.css    1.39 kB │ gzip:  0.71 kB
dist/assets/index-Bx7Kt3Lq.js   195.12 kB │ gzip: 61.34 kB
✓ built in 1.21s

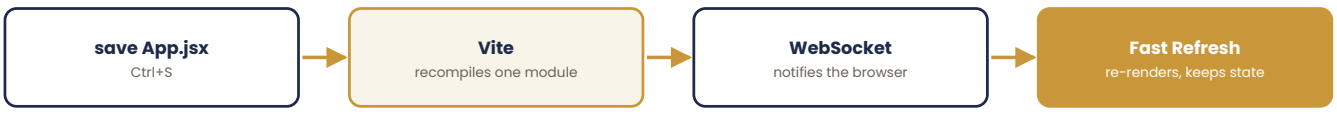
$ npm run preview
  ➜  Local:   http://localhost:4173/
```

`vite build` bundles and minifies everything into the `dist/` folder, uses React's production build and adds content hashes to file names so browsers can cache them forever. `dist/` is a static website you can upload to any host (Module 22). `npm run preview` serves `dist/` locally to check the real build before deploying.

> **NOTE**
> Never deploy the dev server. It is slow, unminified and not secure. Always deploy the output of `npm run build`.

3.4.3 Hot module replacement

Hot module replacement (HMR) swaps changed modules into the running page without a full reload. With React Fast Refresh, editing a component re-renders it **and keeps its state**: change a button colour while a form is half filled and the typed text stays. Editing a non-component file, or a file that exports both components and other values, causes a full reload instead.



HOT MODULE REPLACEMENT IN A FRACTION OF A SECOND

> **PRACTICE CHECKPOINT**
> Create a Vite React project, clean out the starter code, and display your own **"Hello React"** component with React DevTools open.

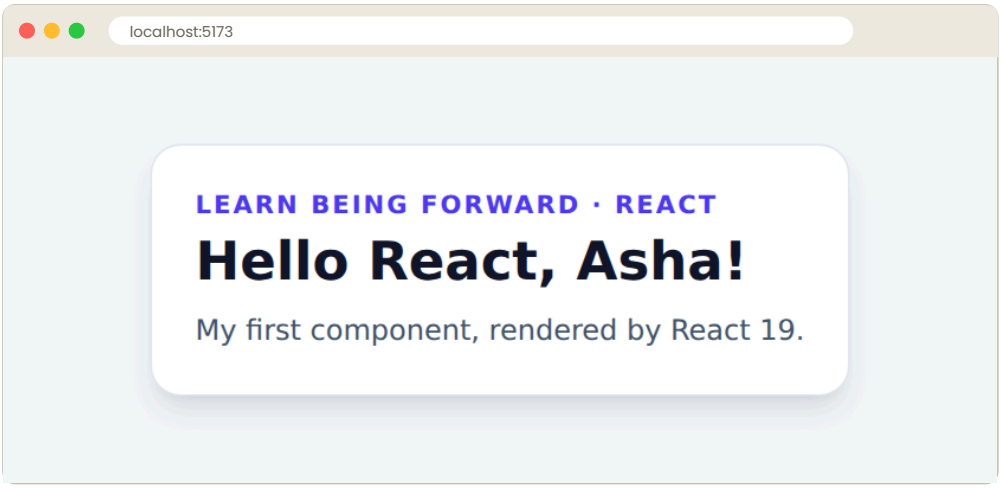
1. `npm create vite@latest hello-react -- --template react`, then `cd hello-react`, `npm install`, `npm run dev`.
2. Delete `src/App.css` and the contents of `src/index.css`, and remove the logo imports from `App.jsx`.
3. Replace `App.jsx` with the component below, save, and watch Fast Refresh update the page.
4. Open DevTools → Components and select `Greeting` to see its props.

SRC/APP.JSX: SAMPLE SOLUTION

JavaScript

```
function Greeting({ name, course }) {
  return (
    <div className="rounded-2xl bg-white p-6 shadow-lg ring-1 ring-slate-200">
      <p className="text-sm font-semibold uppercase tracking-widest text-indigo-600">{course}</p>
      <h1 className="mt-1 text-3xl font-black text-slate-900">Hello React, {name}!</h1>
      <p className="mt-2 text-slate-600">My first component, rendered by React {"19"}.</p>
    </div>
  );
}

export default function App() {
  return (
    <main className="grid min-h-60 place-items-center bg-slate-100">
      <Greeting name="Asha" course="Learn Being Forward · React" />
    </main>
  );
}
```



THE HELLO REACT PAGE

React tooling runs on Node.js and npm. Start learning projects with `npm create vite@latest -- --template react`; the app starts at `index.html` → `main.jsx` (createRoot + StrictMode) → `App.jsx`. Use `npm run dev` while working and deploy the `dist/` folder from `npm run build`.

MODULE 04 · BEGINNER

JSX

Minimum duration **1 day** | Level **Beginner**

JSX is the HTML-like syntax used to describe React UI inside JavaScript. This module shows what JSX compiles to (real compiler output), why React uses it, and its rules: one root element, fragments, closing every tag and camelCase attributes. It covers embedding JavaScript expressions, calling functions and using variables, inline style objects, comments, rendering images and other assets, and the security model: automatic escaping and the deliberately awkward dangerouslySetInnerHTML. Compiled output comes from esbuild and every rendered example ran in Chromium with React 19.

IN THIS MODULE

4.1 What JSX is
4.2 JSX rules
4.3 JavaScript in JSX
4.4 Comments in JSX
4.5 Rendering images and assets
4.6 JSX security

4.1 What JSX is

4.1.1 JSX compiles to JavaScript function calls

Browsers do not understand JSX. A build tool (Vite uses esbuild or SWC, others use Babel) **compiles** each JSX element into a function call that creates a plain object describing the element. With the modern "automatic runtime" the calls are `jsx()` and `jsxs()` from `react/jsx-runtime`; the older "classic" form calls `React.createElement(type, props, ...children)`. Here is the real compiler output for a small element:

JSX (WHAT YOU WRITE) — JavaScript

```
const element = <h1 className="title">Hello, {name}!</h1>;
```

COMPILED JAVASCRIPT (esbuild, automatic runtime)

```
import { jsxs } from "react/jsx-runtime";
const element = /* @__PURE__ */ jsxs("h1", { className: "title", children: [
  "Hello, ",
  name,
  "!"
] });
```

THE SAME JSX — JavaScript

```
const element = <h1 className="title">Hello, {name}!</h1>;
```

COMPILED JAVASCRIPT (esbuild, classic React.createElement)

```
const element = /* @__PURE__ */ React.createElement("h1", { className: "title" }, "Hello, ", name,
"!");
```

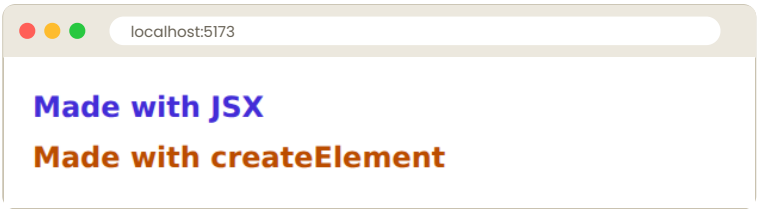
So JSX is only a convenient syntax for creating objects. You could write React without it, and the next example shows that the result is identical:

APP.JSX

JavaScript

```
import { createElement as h } from "react";

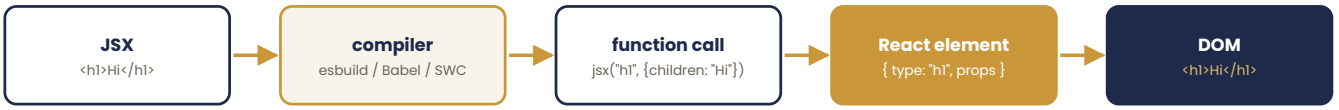
export default function App() {
  const withJsx = <p className="text-indigo-700">Made with JSX</p>;
  const withoutJsx = h("p", { className: "text-amber-700" }, "Made with createElement");
  console.log(withJsx.type, withJsx.props);       // a plain object
  return <div className="space-y-1 font-semibold">{withJsx}{withoutJsx}</div>;
}
```



BOTH ELEMENTS RENDER THE SAME WAY

BROWSER CONSOLE: A JSX ELEMENT IS A PLAIN OBJECT

```
type: a
props: {"href":"/courses","className":"link","children":"Courses"}
key: null
```



FROM JSX TO THE DOM

4.1.2 Why React uses JSX

- **Markup and logic belong together**: a component's structure, data and event handlers change together, so React keeps them in one file instead of separating "HTML" and "JS" by technology.
- **It is just JavaScript**: no special template language to learn for loops and conditions; use `map`, ternaries and variables.
- **Tooling**: the compiler catches unclosed tags and typos; editors autocomplete props; TypeScript type-checks JSX (Module 15).
- **Readability**: nested `createElement` calls are hard to read; JSX looks like the HTML it produces.

4.2 JSX rules

4.2.1 Return a single root element

A component returns **one** value, and two sibling elements are two values. Writing them side by side is a compile error:

CARD.JSX (DOES NOT COMPILE) JavaScript

```
function Card() {
  return (
    <h2>Title</h2>
    <p>Text</p>
  );
}
```

COMPILER ERROR (esbuild)

```
✘ [ERROR] Unterminated regular expression
    <stdin>:4:15:
      4 |     <p>Text</p>
        |               ^
```

esbuild's message is confusing because, once the first element ends, it reads `<p>` as JavaScript. Babel reports the cause directly: *"Adjacent JSX elements must be wrapped in an enclosing tag. Did you want a JSX fragment <>…</>?"* Wrap the siblings in one parent element, such as a `<div>`, or in a fragment when an extra element is not wanted.

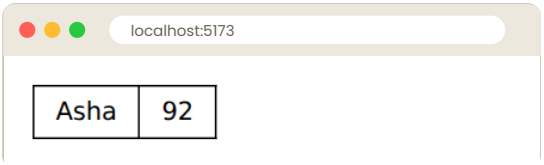
4.2.2 Fragments <>…</>

A **fragment** groups elements without adding a node to the DOM. Use the short syntax `<>...</>`, or `<Fragment key={...}>` when a key is needed in a list. Fragments matter where an extra `div` would break the HTML structure, for example table rows or a flex/grid layout.

APP.JSX JavaScript

```
function Columns() {
  return (
    <>
      <td className="border px-3 py-1">Asha</td>
      <td className="border px-3 py-1">92</td>
    </>
  );
}

export default function App() {
  return (
    <table className="border-collapse text-sm">
      <tbody>
        <tr><Columns /></tr>
      </tbody>
    </table>
  );
}
```



THE FRAGMENT RETURNS TWO <TD> CELLS WITH NO WRAPPER

RENDERED DOM: NO EXTRA ELEMENT FROM THE FRAGMENT

```
<table><tbody><tr><td>Asha</td><td>92</td></tr></tbody></table>
```

4.2.3 Close every tag

JSX follows XML rules: every element must be closed. Void HTML elements are written self-closing (`<img />`, `<br />`, `<input />`), and any element without children may be self-closing, including components: `<Avatar />`.

UNCLOSED TAGS (DOES NOT COMPILE) JavaScript

```
const form = <div><input type="text"><br></div>;
```

COMPILER ERROR (esbuild)

```
✘ [ERROR] Unexpected closing "div" tag does not match opening "br" tag
    <stdin>:1:43:
      1 │ const form = <div><input type="text"><br></div>;
        │                                           ~~~
        ╵                                           br
  The opening "br" tag is here:
    <stdin>:1:38:
      1 │ const form = <div><input type="text"><br></div>;
        ╵                                       ~~
✘ [ERROR] Unexpected end of file before a closing "input" tag
    <stdin>:1:48:
      1 │ const form = <div><input type="text"><br></div>;
        │                                               ^
        ╵                                               </input>
  The opening "input" tag is here:
    <stdin>:1:19:
      1 │ const form = <div><input type="text"><br></div>;
        ╵                    ~~~~~
```

4.2.4 camelCase attributes: className, htmlFor, onClick

JSX attributes become JavaScript object keys and DOM properties, so they use JavaScript names. `class` and `for` are reserved words in JavaScript, hence `className` and `htmlFor`. Multi-word attributes and all event handlers are camelCase. Two exceptions keep their dashes: `aria-*` and `data-*`.

HTML	JSX	HTML	JSX
`class`	`className`	`onclick`	`onClick`
`for`	`htmlFor`	`onchange`	`onChange`
`tabindex`	`tabIndex`	`maxlength`	`maxLength`
`readonly`	`readOnly`	`style="color:red"`	`style={{ color: "red" }}`
`aria-label`	`aria-label` (unchanged)	`data-id`	`data-id` (unchanged)

APP.JSX JavaScript

```
export default function App() {
  return <label class="field" for="email">E-mail</label>;     // HTML names by mistake
}
```

BROWSER CONSOLE: REACT WARNS ABOUT HTML ATTRIBUTE NAMES (DEVELOPMENT BUILD)

```
Error: Invalid DOM property `class`. Did you mean `className`?
Error: Invalid DOM property `for`. Did you mean `htmlFor`?
```

In development React prints these warnings for the first offending attribute it meets (and still renders the element). A quick way to convert existing HTML is an online "HTML to JSX" converter, or the editor's rename refactoring.

4.3 JavaScript in JSX

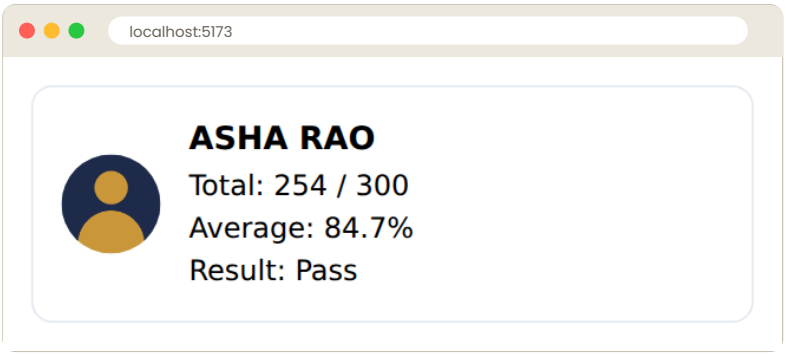
4.3.1 Embedding expressions with {}

Curly braces switch from markup back to JavaScript. Any **expression** (something that produces a value) can go inside: variables, arithmetic, function calls, ternaries, `map`. Statements (`if`, `for`, variable declarations) cannot; compute those before the `return`. Braces work in two places: as text content and as attribute values (without quotes).

APP.JSX

JavaScript

```
export default function App() {
  const student = { name: "Asha Rao", marks: [78, 91, 85] };
  const total = student.marks.reduce((a, b) => a + b, 0);
  const photo = "avatar.png";
  return (
    <div className="flex items-center gap-4 rounded-xl border border-slate-200 p-4">
      <img src={photo} alt={student.name} className="size-14 rounded-full" />
      <div>
        <h2 className="text-lg font-bold">{student.name.toUpperCase()}</h2>
        <p>Total: {total} / {student.marks.length * 100}</p>
        <p>Average: {(total / student.marks.length).toFixed(1)}%</p>
        <p>Result: {total >= 200 ? "Pass" : "Fail"}</p>
      </div>
    </div>
  );
}
```



EXPRESSIONS INSIDE BRACES ARE EVALUATED DURING RENDERING

Inside {}	Rendered as
string, number	the text (`0` included)
`true`, `false`, `null`, `undefined`	nothing
array of elements or strings	each item in order (needs keys, Module 7)
React element	that element
plain object `{ a: 1 }`	**error**: "Objects are not valid as a React child"

4.3.2 Calling functions and using variables

Any part of the UI can be computed: store elements in variables, call helper functions that return JSX or strings, and format values with ordinary JavaScript. Keep complex logic above the `return` so the JSX stays readable.

APP.JSX

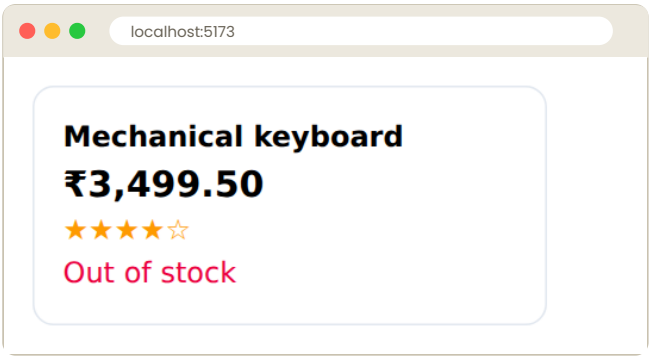
JavaScript

```
function formatPrice(amount) {
  return new Intl.NumberFormat("en-IN", { style: "currency", currency: "INR" }).format(amount);
}

function stars(rating) {
  return "★".repeat(rating) + "☆".repeat(5 - rating);
}

export default function App() {
  const product = { name: "Mechanical keyboard", price: 3499.5, rating: 4, stock: 0 };
  let status;
  if (product.stock === 0) status = <span className="text-rose-600">Out of stock</span>;
  else status = <span className="text-emerald-600">In stock</span>;

  return (
    <div className="w-72 rounded-xl border border-slate-200 p-4">
      <h3 className="font-semibold">{product.name}</h3>
      <p className="text-xl font-bold">{formatPrice(product.price)}</p>
      <p className="text-amber-500">{stars(product.rating)}</p>
      <p>{status}</p>
    </div>
  );
}
```



HELPER FUNCTIONS AND A JSX VALUE CHOSEN WITH IF/ELSE BEFORE THE RETURN

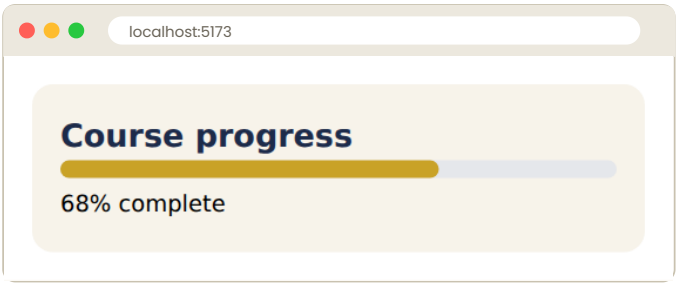
4.3.3 Inline styles with objects

The `style` attribute takes a JavaScript **object**, not a string, hence the double braces: the outer pair enters JavaScript, the inner pair is the object literal. Property names are camelCase (`backgroundColor`, `fontSize`), values are strings, and numbers are treated as pixels for most size properties. Inline styles suit values computed at runtime (a progress width, a user-chosen colour); for everything else use classes (Module 10).

APP.JSX

JavaScript

```
export default function App() {
  const progress = 68;
  const brand = "#1C2B4A";
  const box = { padding: 16, borderRadius: 12, backgroundColor: "#F7F3EA" };   // style object in a
variable
  return (
    <div style={box}>
      <p style={{ color: brand, fontWeight: 700, fontSize: 18 }}>Course progress</p>
      <div style={{ height: 10, background: "#E5E7EB", borderRadius: 5 }}>
        <div style={{ width: `${progress}%`, height: "100%", background: "#C9A227", borderRadius: 5 }}
/>
      </div>
      <p style={{ marginTop: 6, fontSize: 13 }}>{progress}% complete</p>
    </div>
  );
}
```



INLINE STYLE OBJECTS, INCLUDING A WIDTH COMPUTED FROM DATA

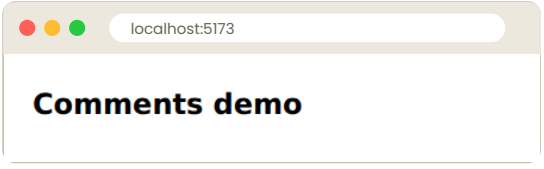
4.4 Comments in JSX

Inside JSX markup, a comment must be a JavaScript comment inside braces: `{/* ... */}`. HTML comments (`<!-- -->`) are not allowed. Outside the markup, and inside a tag between attributes, normal `//` and `/* */` comments work.

APP.JSX

JavaScript

```
export default function App() {
  // a normal comment above the JSX
  return (
    <div>
      {/* a comment inside JSX: not rendered */}
      <h2
        className="font-bold" // a comment between attributes
      >
        Comments demo
      </h2>
      {/* <p>Temporarily disabled paragraph</p> */}
    </div>
  );
}
```



ONLY THE HEADING IS RENDERED

4.5 Rendering images and assets

Images are rendered with `<img>` and an `alt` text. The source can be a URL string (for files in `public/` or remote images), an **imported** asset (Vite returns the final URL), or a value from data. Remember to give images explicit sizes to avoid layout shift, and an empty `alt=""` for purely decorative images.

IMPORTING AN ASSET (VITE) — JavaScript

```
import logo from "./assets/logo.svg";          // Vite: becomes "/assets/logo-a1b2c3.svg"

export default function Header() {
  return <img src={logo} alt="Learn Being Forward" width={120} height={40} />;
}
```

APP.JSX — JavaScript

```
const dishes = [
  { id: 1, name: "Masala dosa", img: "food-dosa.jpg" },
  { id: 2, name: "Idli", img: "food-idli.jpg" },
  { id: 3, name: "Filter coffee", img: "food-coffee.jpg" },
];

export default function App() {
  return (
    <div className="grid grid-cols-3 gap-3">
      {dishes.map(d => (
        <figure key={d.id} className="overflow-hidden rounded-xl border border-slate-200">
          <img src={d.img} alt={d.name} width={160} height={110} className="h-28 w-full object-cover"
/>
          <figcaption className="p-2 text-sm font-medium">{d.name}</figcaption>
        </figure>
      ))}
    </div>
  );
}
```

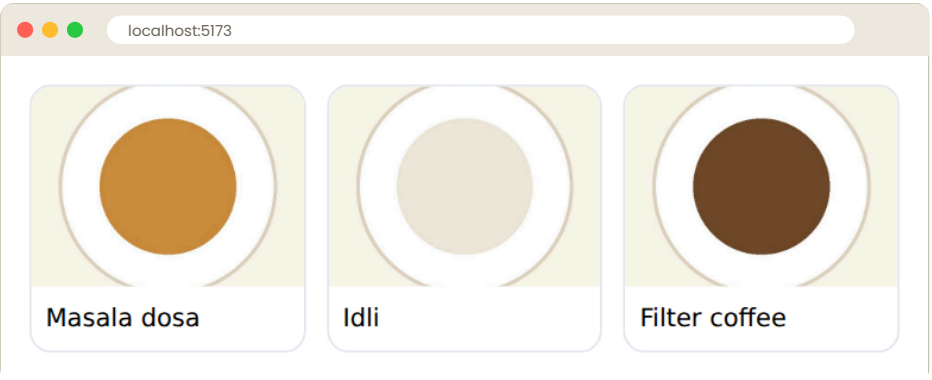


IMAGE SOURCES TAKEN FROM DATA

4.6 JSX security

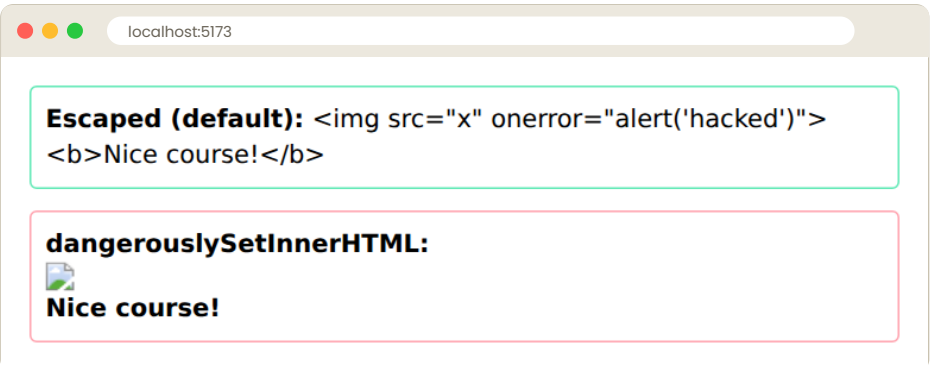
4.6.1 Automatic escaping and dangerouslySetInnerHTML

React **escapes** every value embedded with `{}` before inserting it: `<`, `>` and quotes become text, never markup. So text typed by a user, even if it contains a `<script>` or an `onerror` attribute, is displayed literally and cannot run. This protects against **cross-site scripting** (XSS) by default.

APP.JSX

JavaScript

```
export default function App() {
  const comment = '<img src="x" onerror="alert(\'hacked\')"> <b>Nice course!</b>';
  return (
    <div className="space-y-3 text-sm">
      <div className="rounded border border-emerald-300 p-2">
        <b>Escaped (default):</b> {comment}
      </div>
      <div className="rounded border border-rose-300 p-2">
        <b>dangerouslySetInnerHTML:</b> <span dangerouslySetInnerHTML={{ __html:
comment.replace("onerror", "data-x") }} />
      </div>
    </div>
  );
}
```



ESCAPED TEXT IS SHOWN LITERALLY; RAW HTML IS PARSED (ONERROR REMOVED FOR THE DEMO, A BROKEN IMAGE APPEARS)

`dangerouslySetInnerHTML={{ __html: html }}` bypasses escaping and inserts raw HTML. The name is deliberately alarming. Use it only for HTML you trust or have **sanitised** with a library such as DOMPurify, for example rendered Markdown from your own CMS.

WATCH OUT

Other XSS routes remain your responsibility: never put user input into `href` without checking the protocol (a `javascript:` URL runs code; React 19 blocks it and logs an error), never `eval` user data, and never build HTML strings for `dangerouslySetInnerHTML` from user input without sanitising.

PRACTICE CHECKPOINT

Convert a **static HTML profile card** into JSX that displays data from JavaScript variables.

PROFILE.HTML (THE STARTING HTML)

HTML

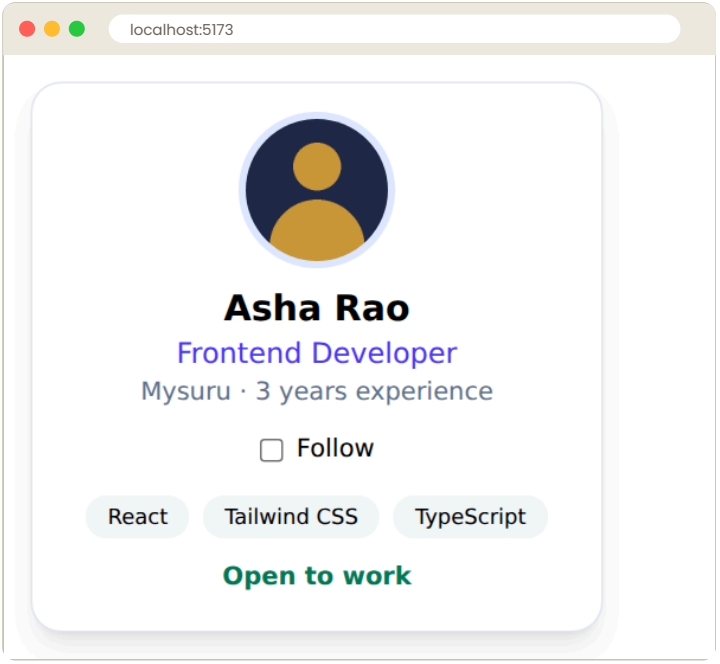
```
<div class="card">
  <img src="avatar.png" alt="Asha Rao" class="avatar">
  <h2>Asha Rao</h2>
  <p class="role">Frontend Developer</p>
  <p>Mysuru · 3 years experience</p>
  <label for="follow">Follow</label> <input id="follow" type="checkbox">
  <!-- skills -->
  <ul><li>React</li><li>Tailwind CSS</li><li>TypeScript</li></ul>
</div>
```

PROFILECARD.JSX: SAMPLE SOLUTION (AS APP.JSX)

JavaScript

```
const profile = {
  name: "Asha Rao",
  role: "Frontend Developer",
  city: "Mysuru",
  years: 3,
  photo: "avatar.png",
  skills: ["React", "Tailwind CSS", "TypeScript"],
  available: true,
};

export default function App() {
  const { name, role, city, years, photo, skills, available } = profile;
  return (
    <div className="w-80 rounded-2xl bg-white p-5 text-center shadow-lg ring-1 ring-slate-200">
      <img src={photo} alt={name} className="mx-auto size-20 rounded-full ring-4 ring-indigo-100" />
      <h2 className="mt-3 text-xl font-bold">{name}</h2>
      <p className="font-medium text-indigo-600">{role}</p>
      <p className="text-sm text-slate-500">
        {city} · {years} {years === 1 ? "year" : "years"} experience
      </p>
      <label htmlFor="follow" className="mt-3 inline-flex items-center gap-2 text-sm">
        <input id="follow" type="checkbox" /> Follow
      </label>
      {/* skills */}
      <ul className="mt-3 flex flex-wrap justify-center gap-2">
        {skills.map(s => (
          <li key={s} className="rounded-full bg-slate-100 px-3 py-1 text-xs">{s}</li>
        ))}
      </ul>
      <p className="mt-3 text-sm font-semibold" style={{ color: available ? "#047857" : "#B91C1C" }}>
        {available ? "Open to work" : "Not available"}
      </p>
    </div>
  );
}
```



THE PROFILE CARD RENDERED FROM JAVASCRIPT DATA

JSX is syntax for function calls that create plain element objects. Return one root (or a fragment), close every tag, use camelCase attributes (`className` , `htmlFor` , `onClick`), put expressions in `{}` and style objects in `style={{ }}` . Values are escaped automatically; `dangerouslySetInnerHTML` is the explicit, risky exception.

MODULE 05 · BEGINNER

Components and props

Minimum duration **2 days** Level **Beginner**

Components are the building blocks of every React app and props are how they talk to each other. This module covers function components, naming, exporting and importing, nesting and the component tree, and legacy class components; passing props, destructuring them, defaults, passing objects, arrays and functions, spreading props, the children prop and why props are read-only; keeping components pure and how StrictMode exposes impure ones; "Thinking in React" (breaking a UI into a hierarchy and building a static version first); and organising components in files and folders. All examples are real multi-file React apps run in Chromium.

IN THIS MODULE

5.1 Components
5.2 Props
5.3 Pure components
5.4 Thinking in React
5.5 Organising components

5.1 Components

5.1.1 Function components

A **component** is a JavaScript function that returns JSX (or `null`). React calls it whenever it needs to know what that part of the screen should show. A component can be declared as a function declaration or as an arrow function assigned to a constant; both are common.

APP.JSX JavaScript

```
function Welcome() {                        // function declaration
  return <h2 className="text-xl font-bold">Welcome to the course</h2>;
}

const Tagline = () => (                     // arrow function
  <p className="text-slate-600">Learn React step by step.</p>
);

export default function App() {
  return (
    <div className="rounded-xl bg-indigo-50 p-4">
      <Welcome />
      <Tagline />
    </div>
  );
}
```



TWO COMPONENTS USED INSIDE APP

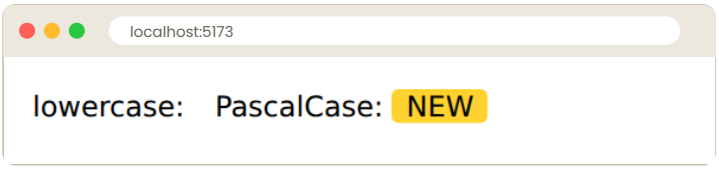
5.1.2 Naming components (PascalCase)

Component names **must start with a capital letter**. JSX uses the case to decide what a tag means: lowercase tags (`<div>`, `<button>`) are HTML elements, capitalised tags (`<Button>`) are components. A lowercase component is silently treated as an unknown HTML element:

APP.JSX JavaScript

```
function badge() {                          // wrong: lowercase name
  return <span className="rounded bg-amber-300 px-2">NEW</span>;
}
function Badge() {                          // right: PascalCase
  return <span className="rounded bg-amber-300 px-2">NEW</span>;
}

export default function App() {
  return (
    <p className="space-x-3">
      <span>lowercase: <badge /></span>
      <span>PascalCase: <Badge /></span>
    </p>
  );
}
```



THE LOWERCASE VERSION RENDERS AN EMPTY <BADGE> HTML ELEMENT INSTEAD OF CALLING THE FUNCTION

WHY: THE COMPILER PASSES A STRING FOR LOWERCASE TAGS JavaScript

```
const a = <badge />, b = <Badge />;
```

COMPILED JAVASCRIPT (esbuild, automatic runtime)

```
import { jsx } from "react/jsx-runtime";
const a = /* @__PURE__ */ jsx("badge", {}), b = /* @__PURE__ */ jsx(Badge, {});
```

5.1.3 Exporting and importing components

Each component normally lives in its own file and is exported so other files can import it. The usual convention is a **default export** named after the file; small helper components that belong together can be **named exports** from one file.

AVATAR.JSX

JavaScript

```
export default function Avatar() {
  return <img src="avatar.png" alt="" className="size-12 rounded-full" />;
}
```

UI.JSX

JavaScript

```
export function Badge() {
  return <span className="ml-1 rounded bg-emerald-100 px-1.5 text-xs text-emerald-700">PRO</span>;
}

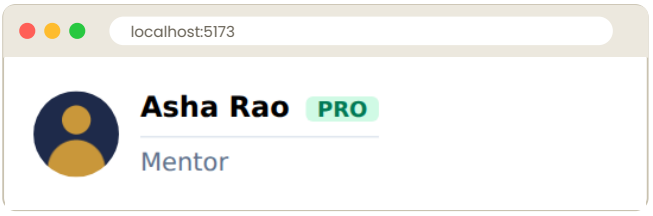
export function Divider() {
  return <hr className="my-1 border-slate-200" />;
}
```

APP.JSX

JavaScript

```
import Avatar from "./Avatar.jsx";                // default import
import { Badge, Divider } from "./ui.jsx";       // named imports

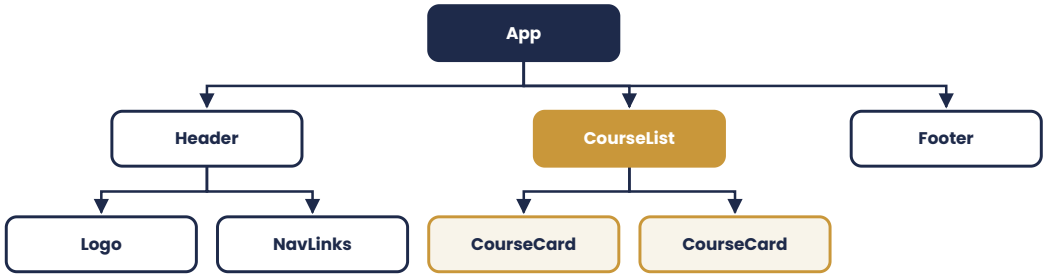
export default function App() {
  return (
    <div className="flex items-center gap-3">
      <Avatar />
      <div>
        <p className="font-semibold">Asha Rao <Badge /></p>
        <Divider />
        <p className="text-sm text-slate-500">Mentor</p>
      </div>
    </div>
  );
}
```



COMPONENTS FROM TWO OTHER FILES

5.1.4 Nesting components and the component tree

Components render other components, forming a **tree** with `App` at the root. The tree is what React DevTools shows and what decides how data flows (always from parent to child). Define components at the **top level** of a file, never inside another component: a component defined inside another is a new function on every render, so React destroys and recreates its DOM and state each time.



A COMPONENT TREE: PARENTS RENDER CHILDREN

DO NOT DEFINE COMPONENTS INSIDE COMPONENTS

JavaScript

```
export default function App() {
  function Row() { return <li>...</li>; }    // BAD: a new Row component on every render of App
  return <ul><Row /></ul>;
}

function Row() { return <li>...</li>; }      // GOOD: defined once, at the top level
export default function App() {
  return <ul><Row /></ul>;
}
```

5.1.5 Class components (legacy, overview)

Before hooks (2019), components that needed state or lifecycle methods were written as ES6 **classes** extending `React.Component`, with `this.props`, `this.state`, `this.setState()` and methods such as `componentDidMount`. They still work in React 19 and appear in older code bases, so you should be able to read them, but new code uses function components and hooks.

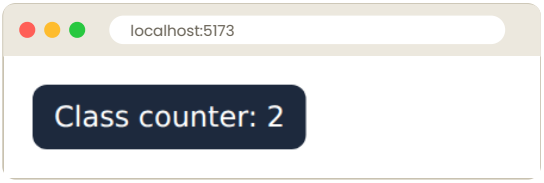
APP.JSX

JavaScript

```
import { Component } from "react";

class Counter extends Component {
  state = { count: 0 };
  render() {
    return (
      <button className="rounded-lg bg-slate-800 px-3 py-1.5 text-white"
              onClick={() => this.setState({ count: this.state.count + 1 })}>
        {this.props.label}: {this.state.count}
      </button>
    );
  }
}

export default function App() {
  return <Counter label="Class counter" />;
}
```



A CLASS COMPONENT AFTER TWO REAL CLICKS

Class component	Function component equivalent
`this.props.label`	`function C({ label })`
`this.state` / `this.setState()`	`useState` (Module 6)
`componentDidMount`, `componentDidUpdate`, `componentWillUnmount`	`useEffect` (Module 9)
`shouldComponentUpdate` / `PureComponent`	`memo` (Module 18)
error boundaries (`componentDidCatch`)	still classes only (or a library), Module 19

5.2 Props

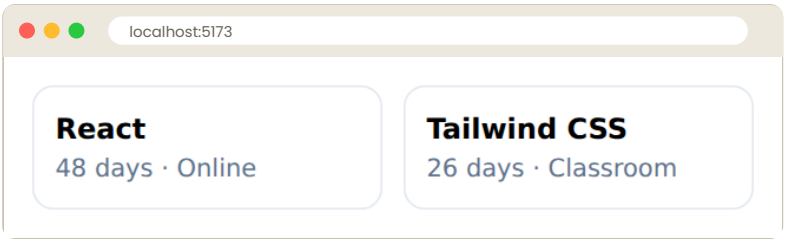
5.2.1 Passing props to components

Props (properties) are the inputs of a component, passed like HTML attributes. Strings can be written in quotes; any other value (numbers, booleans, arrays, objects, functions, JSX) goes in braces. React collects them into one object and passes it as the component's first argument.

APP.JSX — JavaScript

```
function CourseCard(props) {
  console.log(props);
  return (
    <div className="rounded-xl border border-slate-200 p-3">
      <h3 className="font-semibold">{props.title}</h3>
      <p className="text-sm text-slate-500">{props.days} days · {props.online ? "Online" :
"Classroom"}</p>
    </div>
  );
}

export default function App() {
  return (
    <div className="grid grid-cols-2 gap-3">
      <CourseCard title="React" days={48} online={true} />
      <CourseCard title="Tailwind CSS" days={26} online={false} />
    </div>
  );
}
```



THE SAME COMPONENT WITH DIFFERENT PROPS

BROWSER CONSOLE: THE PROPS OBJECT EACH CARD RECEIVED

```
{title: 'React', days: 48, online: true}
{title: 'Tailwind CSS', days: 26, online: false}
```

5.2.2 Reading props with destructuring

Instead of writing `props.title` everywhere, destructure the props object right in the parameter list. It documents at a glance which props the component expects.

COURSECARD.JSX — JavaScript

```
function CourseCard({ title, days, online }) {          // destructured parameters
  return <h3>{title} ({days} days, {online ? "online" : "classroom"})</h3>;
}
```

5.2.3 Default prop values

Give a default with `=` in the destructuring pattern. The default is used when the prop is missing or `undefined` (not when it is `null`, `0` or `""`). The old `Component.defaultProps` property is not supported on

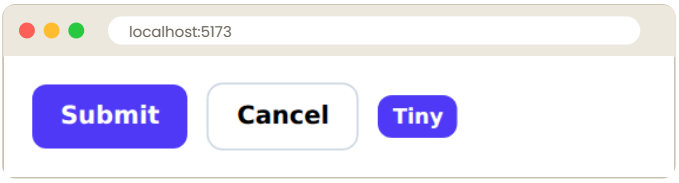
function components in React 19.

APP.JSX

JavaScript

```
function Button({ label = "Submit", variant = "primary", size = "md" }) {
  const colours = { primary: "bg-indigo-600 text-white", secondary: "bg-white ring-1 ring-slate-300"
};
  const sizes = { sm: "px-2 py-1 text-xs", md: "px-4 py-2 text-sm" };
  return <button className={`rounded-lg font-semibold ${colours[variant]} ${sizes[size]}`}>{label}
</button>;
}

export default function App() {
  return (
    <div className="flex items-center gap-3">
      <Button />
      <Button label="Cancel" variant="secondary" />
      <Button label="Tiny" size="sm" />
    </div>
  );
}
```



MISSING PROPS FALL BACK TO THEIR DEFAULTS

5.2.4 Passing objects, arrays and functions as props

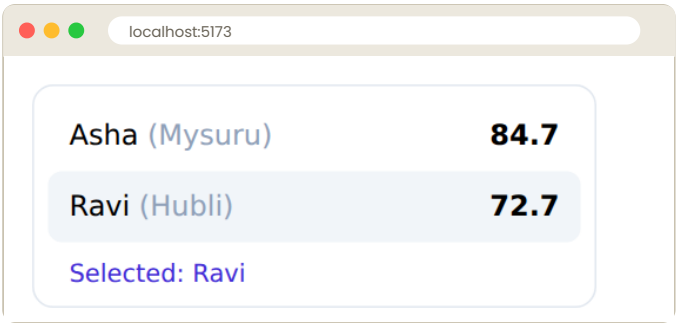
Any JavaScript value can be a prop. Passing an **object** keeps related data together; passing an **array** lets a child render a list; passing a **function** lets a child tell its parent that something happened (the parent decides what to do, Module 6).

APP.JSX

JavaScript

```
function StudentRow({ student, marks, onSelect }) {
  const avg = marks.reduce((a, b) => a + b, 0) / marks.length;
  return (
    <button onClick={() => onSelect(student.name)}
            className="flex w-full justify-between rounded-lg px-3 py-2 text-left hover:bg-slate-100">
      <span>{student.name} <span className="text-slate-400">({student.city})</span></span>
      <b>{avg.toFixed(1)}</b>
    </button>
  );
}

export default function App() {
  const handleSelect = name => (document.getElementById("msg").textContent = "Selected: " + name);
  return (
    <div className="w-80 rounded-xl border border-slate-200 p-2">
      <StudentRow student={{ name: "Asha", city: "Mysuru" }} marks={[78, 91, 85]} onSelect=
{handleSelect} />
      <StudentRow student={{ name: "Ravi", city: "Hubli" }} marks={[66, 72, 80]} onSelect=
{handleSelect} />
      <p id="msg" className="px-3 pt-2 text-sm text-indigo-700">Click a student</p>
    </div>
  );
}
```



AFTER CLICKING RAVI: THE CHILD CALLED THE FUNCTION RECEIVED FROM ITS PARENT

NOTE

This example writes to the DOM directly only to show the callback firing; Module 6 replaces that with state, which is the React way.

5.2.5 Spreading props

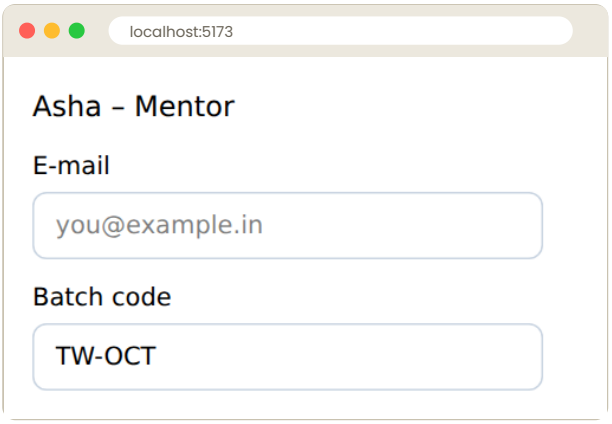
`{...obj}` in JSX passes every property of an object as a separate prop. It is handy when data already has the right shape, and in **wrapper components** that forward "all other props" (collected with rest) to an underlying element, such as `type`, `disabled`, `aria-*` and event handlers.

APP.JSX JavaScript

```
function Input({ label, ...rest }) {                  // rest = every other prop
  return (
    <label className="block text-sm">
      <span className="font-medium">{label}</span>
      <input {...rest} className="mt-1 block w-full rounded-lg border border-slate-300 px-3 py-2" />
    </label>
  );
}

function Profile({ name, role }) {
  return <p>{name} — {role}</p>;
}

export default function App() {
  const user = { name: "Asha", role: "Mentor" };
  return (
    <div className="w-72 space-y-3">
      <Profile {...user} />
      <Input label="E-mail" type="email" placeholder="you@example.in" required />
      <Input label="Batch code" defaultValue="TW-OCT" disabled />
    </div>
  );
}
```



SPREAD PASSES AN OBJECT AS PROPS; REST FORWARDS UNKNOWN PROPS TO THE INPUT

WATCH OUT
Do not spread props blindly everywhere: it hides which props a component accepts and can pass invalid attributes to the DOM. Spread deliberately, in wrapper components.

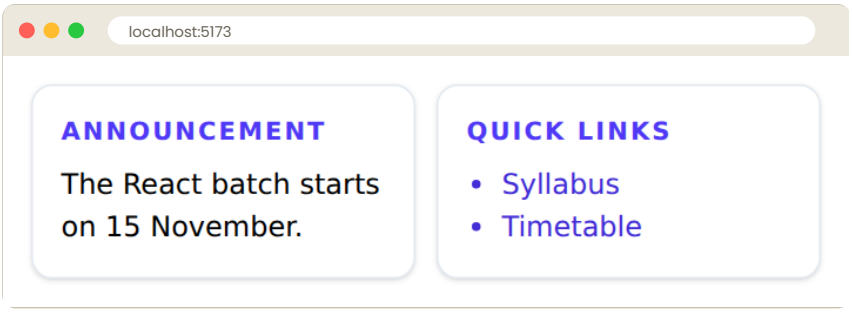
5.2.6 The children prop

Whatever you place **between** a component's opening and closing tags arrives as the special prop `children`. It lets you build generic containers (cards, panels, layouts, modals) that wrap any content, exactly like HTML elements wrap their content.

APP.JSX — JavaScript

```
function Card({ title, children }) {
  return (
    <section className="rounded-xl border border-slate-200 p-4 shadow-sm">
      <h3 className="mb-2 text-sm font-bold uppercase tracking-widest text-indigo-600">{title}</h3>
      {children}
    </section>
  );
}

export default function App() {
  return (
    <div className="grid grid-cols-2 gap-3">
      <Card title="Announcement">
        <p>The React batch starts on 15 November.</p>
      </Card>
      <Card title="Quick links">
        <ul className="list-disc pl-5 text-indigo-700">
          <li>Syllabus</li>
          <li>Timetable</li>
        </ul>
      </Card>
    </div>
  );
}
```



ONE CARD COMPONENT WRAPPING COMPLETELY DIFFERENT CHILDREN

```
<Card title="Announcement">
      <p>The React batch starts...</p>      → props.children
</Card>
```

CONTENT BETWEEN THE TAGS BECOMES PROPS.CHILDREN

5.2.7 Props are read-only

A component must **never change its props**. Props belong to the parent; a child that modified them would make the data flow unpredictable. React even freezes the props object in development, so an assignment throws in strict-mode JavaScript (which modules always are). To change something, the parent changes its own state and passes new props (Module 6).

APP.JSX — JavaScript

```javascript
function Price({ amount }) {
  return <b>{amount}</b>;
}
function Bad(props) {
  try {
    props.amount = 999;          // attempt to modify props
  } catch (e) {
    console.log(e.constructor.name + ": " + e.message);
  }
  return <Price amount={props.amount} />;
}
export default function App() {
  return <Bad amount={100} />;
}
```

BROWSER CONSOLE

```
TypeError: Cannot assign to read only property 'amount' of object '#<Object>'
```

5.3 Pure components

5.3.1 Keeping components pure

React assumes every component is a **pure function** of its props, state and context: given the same inputs it returns the same JSX, and it has **no side effects while rendering**: it does not change variables or objects that existed before the render, does not write to the DOM, and does not fetch data. Side effects belong in event handlers (Module 6) or effects (Module 9). Purity lets React render components in any order, skip them, or render them twice safely.

Pure (fine during render)	Impure (not during render)
compute values from props and state	change a global or module-level variable
create new local variables, arrays and objects	mutate a prop, state or an object passed in
call other pure functions	write to the DOM, start timers, fetch, log analytics
return JSX	`Math.random()` or `Date.now()` to choose what to show (results differ each render)

5.3.2 How StrictMode helps find impure components

In development, `<StrictMode>` renders every component **twice** and keeps the second result. A pure component produces identical output both times; an impure one gives itself away. Here a module-level counter is used to number guests, and StrictMode reveals that the numbers are wrong:

APP.JSX (IMPURE) JavaScript

```
let guest = 0;                          // shared, outside the component

function Cup() {
  guest = guest + 1;                    // side effect during render!
  return <li>Tea cup for guest #{guest}</li>;
}

export default function App() {
  return <ul className="list-disc pl-5"><Cup /><Cup /><Cup /></ul>;
}
```

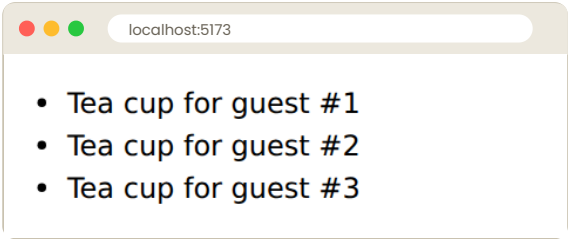


RENDERED WITH <STRICTMODE>: GUESTS ARE NUMBERED 2, 4, 6 BECAUSE EACH CUP RENDERED TWICE

APP.JSX (PURE) JavaScript

```
function Cup({ guest }) {               // pure: the number comes from props
  return <li>Tea cup for guest #{guest}</li>;
}

export default function App() {
  return <ul className="list-disc pl-5"><Cup guest={1} /><Cup guest={2} /><Cup guest={3} /></ul>;
}
```

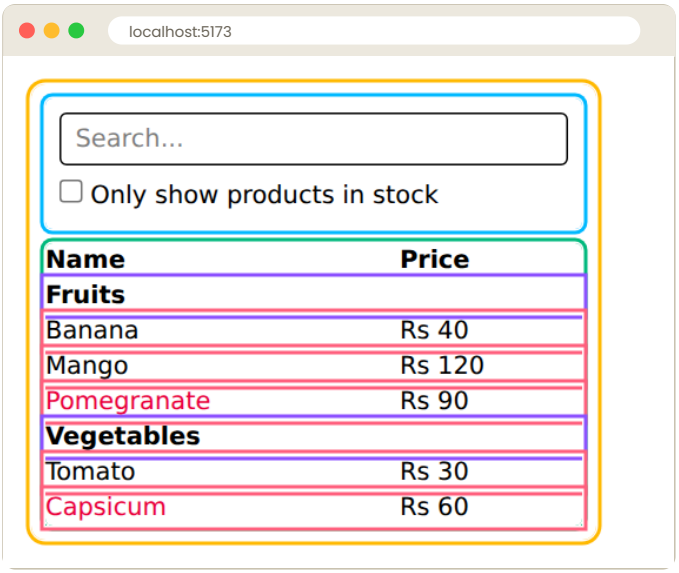


ALSO WITH <STRICTMODE>: CORRECT, BECAUSE RENDERING TWICE CHANGES NOTHING

5.4 Thinking in React

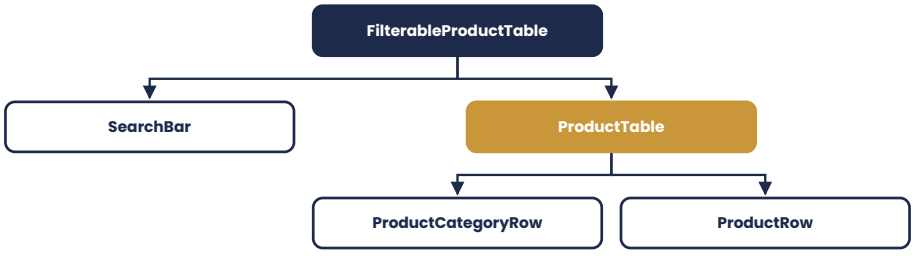
5.4.1 Breaking a UI into a component hierarchy

"Thinking in React" is the method from the official documentation for turning a design into components. Step one: draw boxes around every part of the mock-up and name them. Use the **single responsibility** principle: a component should ideally do one thing; if it grows, split it. Data models often suggest the split: one component per kind of data item.



THE MOCK-UP WITH EACH COMPONENT OUTLINED IN ITS OWN COLOUR

Colour	Component	Responsibility
amber	`FilterableProductTable`	the whole app
sky	`SearchBar`	receives the user input
green	`ProductTable`	displays and filters the list
violet	`ProductCategoryRow`	a heading for each category
rose	`ProductRow`	one row per product



THE RESULTING COMPONENT HIERARCHY

5.4.2 Building a static version first

Step two: build a version that renders the UI from the data **without any interactivity**, passing data down through props only. Do not use state yet. This is mostly typing and requires little thinking; adding interactivity (state) is a separate step done in Module 6, where this same example becomes interactive. Building top-down works for small apps, bottom-up (smallest components first, testing each) for large ones.

APP.JSX: THE STATIC VERSION

JavaScript

```
const PRODUCTS = [
  { category: "Fruits", price: "Rs 40", stocked: true, name: "Banana" },
  { category: "Fruits", price: "Rs 120", stocked: true, name: "Mango" },
  { category: "Fruits", price: "Rs 90", stocked: false, name: "Pomegranate" },
  { category: "Vegetables", price: "Rs 30", stocked: true, name: "Tomato" },
  { category: "Vegetables", price: "Rs 60", stocked: false, name: "Capsicum" },
];

function ProductCategoryRow({ category }) {
  return <tr><th colSpan="2" className="pt-2 text-left text-indigo-700">{category}</th></tr>;
}

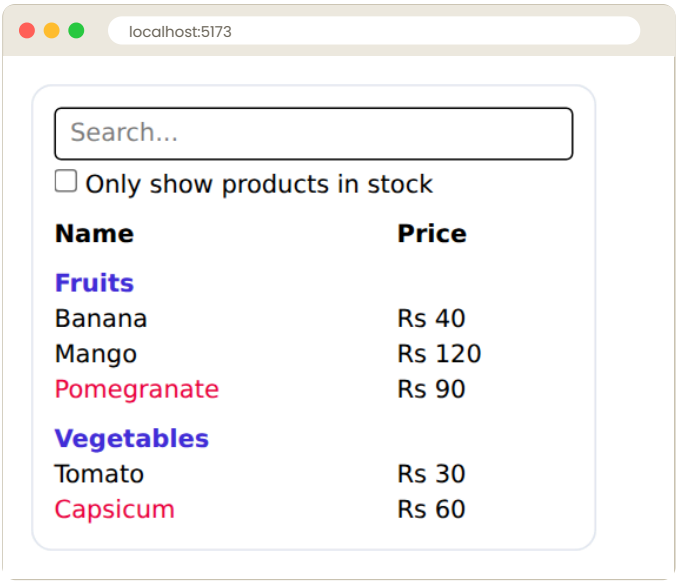
function ProductRow({ product }) {
  const name = product.stocked ? product.name : <span className="text-rose-600">{product.name}</span>;
  return <tr><td>{name}</td><td>{product.price}</td></tr>;
}

function ProductTable({ products }) {
  const rows = [];
  let lastCategory = null;
  products.forEach(product => {
    if (product.category !== lastCategory) {
      rows.push(<ProductCategoryRow category={product.category} key={product.category} />);
    }
    rows.push(<ProductRow product={product} key={product.name} />);
    lastCategory = product.category;
  });
  return (
    <table className="w-full text-sm">
      <thead><tr><th className="text-left">Name</th><th className="text-left">Price</th></tr></thead>
      <tbody>{rows}</tbody>
    </table>
  );
}

function SearchBar() {
  return (
    <form className="space-y-1">
      <input type="text" placeholder="Search..." className="w-full rounded border px-2 py-1 text-sm"
/>
      <label className="block text-sm"><input type="checkbox" /> Only show products in stock</label>
    </form>
  );
}

function FilterableProductTable({ products }) {
  return (
    <div className="w-80 space-y-2 rounded-xl border border-slate-200 p-3">
      <SearchBar />
      <ProductTable products={products} />
    </div>
  );
}

export default function App() {
  return <FilterableProductTable products={PRODUCTS} />;
}
```



THE STATIC VERSION, RENDERED PURELY FROM PROPS

5.5 Organising components

5.5.1 One component per file

Put each significant component in its own file named after it (`ProductCard.jsx` exports `ProductCard`). Small helpers used only by one component can stay in the same file. One component per file makes components easy to find, review, test and reuse, and keeps files short.

5.5.2 Folder structure for components

React has no required structure. Two common layouts, both fine:

By type (small and medium apps)

```
src/
  components/
    Button.jsx
    ProductCard.jsx
    Layout.jsx
  pages/
    Home.jsx
    Products.jsx
  hooks/
  App.jsx
```

By feature (larger apps)

```
src/
  features/
    cart/
      CartPage.jsx
      CartItem.jsx
      useCart.js
    products/
      ProductCard.jsx
  shared/
    Button.jsx
```

ORGANISING BY TYPE OR BY FEATURE

TIP

Start simple (a `components/` folder) and reorganise by feature when the app grows. Co-locate a component with its CSS module and test: `ProductCard.jsx`, `ProductCard.module.css`, `ProductCard.test.jsx`.

PRACTICE CHECKPOINT

Build a **product listing page** from reusable `ProductCard`, `Badge` and `Layout` components, passing data and children through props.

LAYOUT.JSX JavaScript

```
export default function Layout({ title, children }) {
  return (
    <div className="min-h-80 bg-slate-50">
      <header className="flex items-center justify-between bg-slate-900 px-5 py-3 text-white">
        <b>LBF Store</b>
        <span className="text-sm text-slate-300">{title}</span>
      </header>
      <main className="p-5">{children}</main>
    </div>
  );
}
```

BADGE.JSX JavaScript

```
const tones = {
  new: "bg-emerald-100 text-emerald-700",
  sale: "bg-rose-100 text-rose-700",
  out: "bg-slate-200 text-slate-600",
};

export default function Badge({ tone = "new", children }) {
  return <span className={`rounded-full px-2 py-0.5 text-xs font-semibold ${tones[tone]}`}>{children}
</span>;
}
```

PRODUCTCARD.JSX JavaScript

```
import Badge from "./Badge.jsx";

export default function ProductCard({ product }) {
  const { name, price, oldPrice, image, isNew, stock } = product;
  return (
    <article className="overflow-hidden rounded-xl bg-white shadow-sm ring-1 ring-slate-200">
      <img src={image} alt={name} className="h-28 w-full object-cover" />
      <div className="space-y-1 p-3">
        <div className="flex flex-wrap gap-1">
          {isNew && <Badge tone="new">New</Badge>}
          {oldPrice && <Badge tone="sale">Sale</Badge>}
          {stock === 0 && <Badge tone="out">Sold out</Badge>}
        </div>
        <h3 className="font-semibold">{name}</h3>
        <p>
          <b>&amp;#8377;{price}</b>{" "}
          {oldPrice && <s className="text-sm text-slate-400">&amp;#8377;{oldPrice}</s>}
        </p>
      </div>
    </article>
  );
}
```

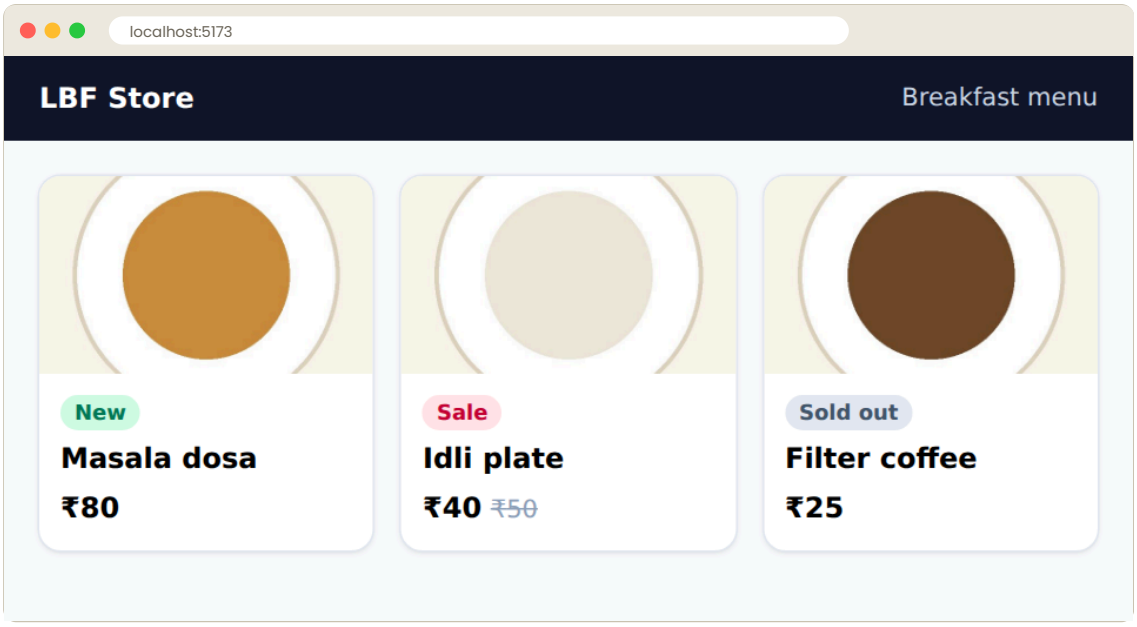
APP.JSX: SAMPLE SOLUTION

JavaScript

```
import Layout from "./Layout.jsx";
import ProductCard from "./ProductCard.jsx";

const products = [
  { id: 1, name: "Masala dosa", price: 80, image: "food-dosa.jpg", isNew: true, stock: 12 },
  { id: 2, name: "Idli plate", price: 40, oldPrice: 50, image: "food-idli.jpg", stock: 30 },
  { id: 3, name: "Filter coffee", price: 25, image: "food-coffee.jpg", stock: 0 },
];

export default function App() {
  return (
    <Layout title="Breakfast menu">
      <div className="grid grid-cols-3 gap-4">
        {products.map(p => <ProductCard key={p.id} product={p} />)}
      </div>
    </Layout>
  );
}
```



THE PRODUCT LISTING PAGE BUILT FROM THREE REUSABLE COMPONENTS

A component is a capitalised function that returns JSX; compose components into a tree and keep each in its own file. Props are the read-only inputs (destructure them, give defaults, pass objects, arrays, functions and JSX), `children` is whatever goes between the tags, and components must stay pure: StrictMode renders twice to catch the ones that are not.

MODULE 06 · BEGINNER

State and events

Minimum duration **2 days** Level **Beginner**

State is what makes components interactive: data that belongs to a component and changes over time, causing React to re-render. This module covers handling events (onClick, onChange, onSubmit, onMouseEnter), passing handlers without calling them, passing arguments, the event object, preventDefault and propagation; the useState hook, the Rules of Hooks, state as a snapshot, updater functions and batching; updating objects and arrays immutably (and Immer); the render and commit phases; sharing state by lifting it up; and designing state: avoiding redundancy, deriving values and resetting state with key. Every screenshot was taken after really clicking and typing in Chromium.

IN THIS MODULE

6.1 Handling events
6.2 The useState hook
6.3 Updating objects and arrays in state
6.4 Render and commit
6.5 Sharing state between components
6.6 Designing state

6.1 Handling events

6.1.1 onClick, onChange, onSubmit, onMouseEnter

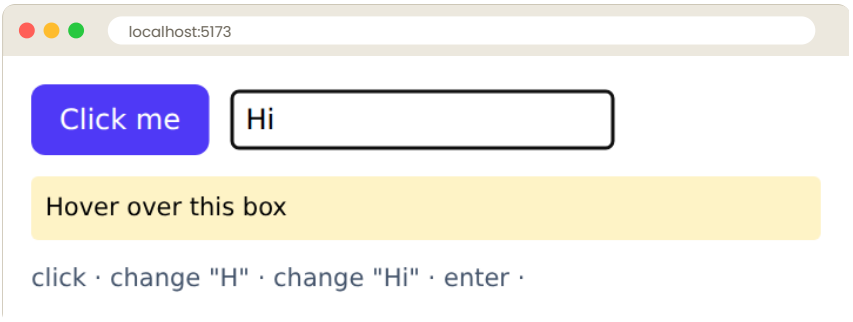
Event handlers are functions passed to JSX elements through props named `on` + event name in camelCase. React attaches a single listener at the root and dispatches **synthetic events** that behave the same in every browser. The most used events:

Prop	Fires when	Typical element
`onClick`	the element is clicked (or activated with Enter/Space on a button)	button, link, card
`onChange`	the value changes on **every keystroke** (unlike the HTML change event)	input, textarea, select
`onSubmit`	a form is submitted (button click or Enter in a field)	form
`onMouseEnter` / `onMouseLeave`	the pointer enters / leaves (does not bubble)	tooltips, previews
`onFocus` / `onBlur`	an element gains / loses focus	input validation
`onKeyDown`	a key is pressed (`e.key` tells which)	shortcuts, Enter to add

APP.JSX

JavaScript

```
export default function App() {
  function handleClick() {
    document.getElementById("log").textContent += "click · ";
  }
  return (
    <div className="space-y-3">
      <button onClick={handleClick} className="rounded-lg bg-indigo-600 px-4 py-2 text-white">Click
me</button>
      <input onChange={e => (document.getElementById("log").textContent += `change "${e.target.value}"
· `)}
             placeholder="Type here" className="ml-3 rounded border px-2 py-1" />
      <div onMouseEnter={() => (document.getElementById("log").textContent += "enter · ")}
           className="rounded bg-amber-100 p-2 text-sm">Hover over this box</div>
      <p id="log" className="text-sm text-slate-600"></p>
    </div>
  );
}
```



AFTER A CLICK, TYPING "HI" AND HOVERING: EVERY HANDLER RAN (LOGGED TO THE PAGE FOR THE DEMO)

NOTE

These first examples write into the DOM only to make the events visible. From Section 6.2 on, interactive data lives in state, which is how React apps are written.

6.1.2 Passing event handlers (not calling them)

Pass the **function itself**: `onClick={handleClick}`. Writing `onClick={handleClick()}` **calls** the function immediately during rendering and passes its return value (usually `undefined`) as the handler. That is a very common beginner bug:

APP.JSX

JavaScript

```
function sayHi() {
  console.log("Hi! (called)");
}
export default function App() {
  return (
    <>
      <button id="right" onClick={sayHi}>Right</button>
      <button id="wrong" onClick={sayHi()}>Wrong</button>
    </>
  );
}
```

```
BROWSER CONSOLE
Hi! (called)
--- page loaded, now clicking both buttons ---
Hi! (called)
```

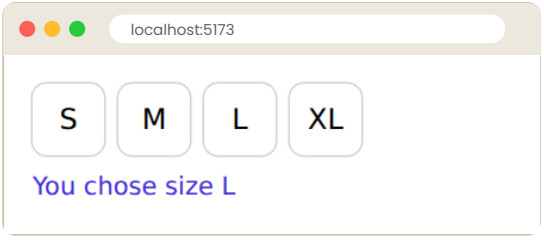
"Hi!" was printed once while rendering, before any click (the wrong button), and once for the click on the right button; clicking the wrong button did nothing. Rule: `onClick={fn}` or `onClick={() => fn(arg)}`, never `onClick={fn()}`.

6.1.3 Passing arguments to handlers

To pass extra data, wrap the call in an arrow function, which creates the handler: `onClick={() => remove(item.id)}`. Handlers are usually named `handleX` inside the component and `onX` when received as a prop.

APP.JSX — JavaScript

```
export default function App() {
  const sizes = ["S", "M", "L", "XL"];
  function choose(size) {
    document.getElementById("out").textContent = "You chose size " + size;
  }
  return (
    <div>
      <div className="flex gap-2">
        {sizes.map(s => (
          <button key={s} onClick={() => choose(s)} className="size-10 rounded-lg ring-1 ring-slate-
300">{s}</button>
        ))}
      </div>
      <p id="out" className="mt-2 text-sm text-indigo-700">Pick a size</p>
    </div>
  );
}
```



AFTER CLICKING L: THE ARROW FUNCTION PASSED "L" TO CHOOSE()

6.1.4 The event object and preventDefault()

Every handler receives an **event object** (`e`) with the same interface as the browser event: `e.target` (the element that fired), `e.currentTarget` (the element the handler is on), `e.key`, `e.clientX` and so on. `e.preventDefault()` stops the browser's default action, most importantly a form submission reloading the page, or a link navigating.

APP.JSX

JavaScript

```
export default function App() {
  function handleSubmit(e) {
    e.preventDefault();                                  // stop the page reload
    const data = new FormData(e.currentTarget);
    console.log("type:", e.type, "| target:", e.target.tagName);
    console.log("submitted name:", data.get("name"));
  }
  function handleKey(e) {
    if (e.key === "Enter") console.log("Enter pressed in", e.target.name);
  }
  return (
    <form onSubmit={handleSubmit}>
      <input name="name" defaultValue="Asha" onKeyDown={handleKey} />
      <button>Send</button>
    </form>
  );
}
```

BROWSER CONSOLE (after pressing Enter in the input)

```
Enter pressed in name
submitted name: Asha
type: submit | target: FORM
```

6.1.5 Event propagation and stopPropagation()

Events **bubble** up the tree: a click on a button inside a clickable card triggers the button's handler and then the card's. Call `e.stopPropagation()` in the inner handler to stop that. (Scroll events and `onMouseEnter` do not bubble.)

APP.JSX

JavaScript

```
export default function App() {
  return (
    <div onClick={() => console.log("card clicked (open details)")} className="card">
      <h3>Masala dosa</h3>
      <button id="fav" onClick={e => { console.log("favourite clicked"); }}>Favourite</button>
      <button id="buy" onClick={e => { e.stopPropagation(); console.log("buy clicked");
}}>Buy</button>
    </div>
  );
}
```

BROWSER CONSOLE

```
--- click Favourite ---
favourite clicked
card clicked (open details)
--- click Buy ---
buy clicked
```



BUBBLING, AND STOPPING IT

6.2 The useState hook

6.2.1 Declaring state const [count, setCount] = useState(0)

A normal local variable cannot drive the UI: it is reset on every render, and changing it does not tell React to render again. **State** solves both problems. `useState(initialValue)` returns a pair: the **current value** and a **setter function**. Calling the setter stores the new value and asks React to re-render the component, which then reads the new value.

APP.JSX

JavaScript

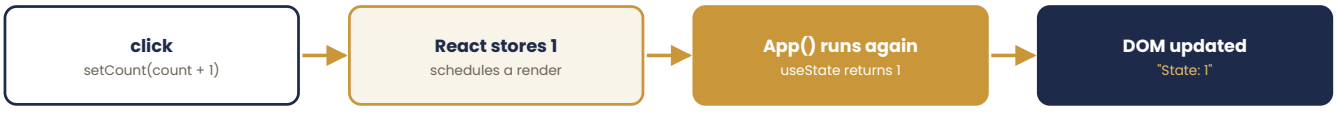
```
import { useState } from "react";

export default function App() {
  let plain = 0;                                  // a normal variable: does not work
  const [count, setCount] = useState(0);          // state: works

  return (
    <div className="space-y-2">
      <button onClick={() => { plain = plain + 1; console.log("plain is", plain); }}
              className="rounded-lg bg-slate-200 px-3 py-1.5">Plain variable: {plain}</button>
      <button onClick={() => setCount(count + 1)}
              className="ml-2 rounded-lg bg-indigo-600 px-3 py-1.5 text-white">State: {count}</button>
    </div>
  );
}
```



AFTER CLICKING EACH BUTTON THREE TIMES: ONLY THE STATE VERSION UPDATES THE SCREEN



WHAT HAPPENS WHEN YOU CALL A STATE SETTER

Part	Meaning
`useState(0)`	declare state with the initial value 0 (used only on the first render)
`count`	the value for this render

Part	Meaning
`setCount`	function to request a new value and a re-render
naming	`[thing, setThing]` by convention: `[isOpen, setIsOpen]`, `[todos, setTodos]`

A component can have as many state variables as it needs, and each instance of a component has its own state:

APP.JSX JavaScript

```
import { useState } from "react";

function Counter({ label }) {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)} className="rounded-lg bg-indigo-50 px-3 py-2 ring-1
ring-indigo-200">
      {label}: <b>{count}</b>
    </button>
  );
}

export default function App() {
  return <div className="flex gap-3"><Counter label="Likes" /><Counter label="Shares" /><Counter
label="Saves" /></div>;
}
```



THREE INSTANCES, THREE INDEPENDENT STATES (CLICKED 2, 0 AND 1 TIMES)

6.2.2 Rules of hooks

Functions whose names start with `use` are **hooks**. React identifies each hook call by its **order** within the component, so the order must be identical on every render. Hence two rules:

1. **Only call hooks at the top level** of a component or custom hook: never inside conditions, loops, nested functions, or after an early `return`.
2. **Only call hooks from React functions**: components and custom hooks (Module 11), not from regular JavaScript functions or event handlers.

BREAKING AND FOLLOWING THE RULES OF HOOKS

JavaScript

```
function Profile({ user }) {
  if (!user) return <p>Loading...</p>;      // early return BEFORE a hook: breaks the order
  const [tab, setTab] = useState("posts");  // BAD: sometimes called, sometimes not

  if (user.admin) {
    const [x, setX] = useState(0);           // BAD: hook inside a condition
  }
}

function Profile({ user }) {
  const [tab, setTab] = useState("posts");  // GOOD: all hooks first, unconditionally
  if (!user) return <p>Loading...</p>;
  ...
}
```

APP.JSX

JavaScript

```
import { useState } from "react";

function Profile({ loggedIn }) {
  const [theme] = useState("light");        // always called
  if (!loggedIn) return <p>Please log in</p>;
  const [tab] = useState("posts");          // called only on some renders
  return <p>Tab: {tab}, theme: {theme}</p>;
}

export default function App() {
  const [loggedIn, setLoggedIn] = useState(false);
  return (
    <>
      <button onClick={() => setLoggedIn(true)}>Log in</button>
      <Profile loggedIn={loggedIn} />
    </>
  );
}
```

BROWSER CONSOLE (after clicking Log in)

```
Uncaught Rendered more hooks than during the previous render.
Error: React has detected a change in the order of Hooks called by Profile. This will lead to bugs and
errors if not fixed. For more information, read the Rules of Hooks: https://react.dev/link/rules-of-
hooks

   Previous render            Next render
   ------------------------------------------------------
1. useState                   useState
2. undefined                  useState
   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

Warning: An error occurred in the <Profile> component.

Consider adding an error boundary to your tree to customize error handling behavior.
Visit https://react.dev/link/error-boundaries to learn more about error boundaries.
```

TIP

The `eslint-plugin-react-hooks` rules (included in the Vite template) flag these mistakes in the editor before you run the code.

6.2.3 State as a snapshot

Setting state does not change the variable you already have; it requests a **new render**. Within one render, `count` is a constant, a **snapshot** of state at the moment of that render. Every handler created in that render sees that snapshot, even if it runs later:

APP.JSX

JavaScript

```
import { useState } from "react";

export default function App() {
  const [count, setCount] = useState(0);
  function handleClick() {
    setCount(count + 1);
    console.log("right after setCount, count is still", count);
    setTimeout(() => console.log("later, the timeout still sees", count), 300);
  }
  console.log("render with count =", count);
  return <button onClick={handleClick}>{count}</button>;
}
```

BROWSER CONSOLE (one click)

```
render with count = 0
right after setCount, count is still 0
render with count = 1
later, the timeout still sees 0
```

The timeout callback was created during the render where `count` was 0, so it logs 0 even after the next render has happened: it is a closure over that render's values (Module 2). This is not a bug; it makes handlers predictable.

6.2.4 Updater functions setCount(c => c + 1)

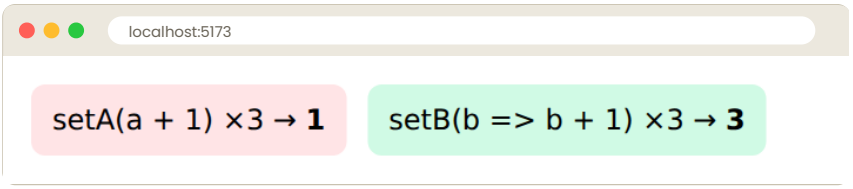
Because of snapshots, calling `setCount(count + 1)` three times in one handler adds only 1: each call computes `0 + 1`. When the new state depends on the previous state, pass an **updater function**. React queues updaters and runs them in order, each receiving the latest pending value:

APP.JSX

JavaScript

```
import { useState } from "react";

export default function App() {
  const [a, setA] = useState(0);
  const [b, setB] = useState(0);
  return (
    <div className="flex gap-3">
      <button className="rounded-lg bg-rose-100 px-3 py-2" onClick={() => { setA(a + 1); setA(a + 1);
setA(a + 1); }}>
        setA(a + 1) ×3 → <b>{a}</b>
      </button>
      <button className="rounded-lg bg-emerald-100 px-3 py-2" onClick={() => { setB(b => b + 1);
setB(b => b + 1); setB(b => b + 1); }}>
        setB(b =&gt; b + 1) ×3 → <b>{b}</b>
      </button>
    </div>
  );
}
```



AFTER ONE CLICK ON EACH: THE VALUE FORM ADDS 1, THE UPDATER FORM ADDS 3

Queued call	n (pending)	returns
`setB(b => b + 1)`	0	1
`setB(b => b + 1)`	1	2
`setB(b => b + 1)`	2	3

6.2.5 Batching of state updates

React waits until all code in an event handler has finished before re-rendering, and applies all the state updates together: **batching**. Ten setter calls in one handler cause **one** render, not ten. Since React 18 batching also applies inside timeouts, promises and native event handlers.

APP.JSX

JavaScript

```
import { useState } from "react";

export default function App() {
  const [name, setName] = useState("");
  const [age, setAge] = useState(0);
  const [city, setCity] = useState("");
  console.log("render:", JSON.stringify({ name, age, city }));
  function fill() {
    setName("Asha");
    setAge(20);
    setCity("Mysuru");
    console.log("handler finished");
  }
  return <button onClick={fill}>Fill</button>;
}
```

BROWSER CONSOLE: THREE UPDATES, ONE RENDER

```
render: {"name":"","age":0,"city":""}
--- click ---
handler finished
render: {"name":"Asha","age":20,"city":"Mysuru"}
```

6.3 Updating objects and arrays in state

6.3.1 Never mutate state

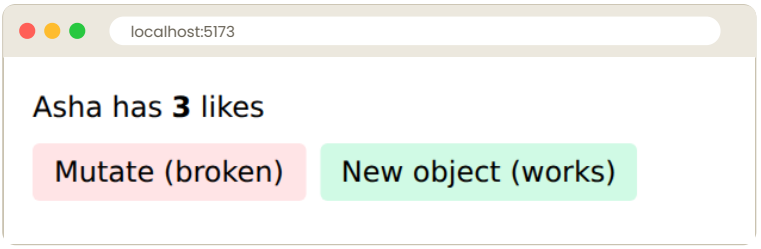
State can hold objects and arrays, but treat them as **read-only**. If you change the existing object and pass the same object to the setter, React compares old and new with `Object.is`, sees the **same reference**, and skips the render, so the screen does not update. Always pass a **new** object or array.

APP.JSX

JavaScript

```
import { useState } from "react";

export default function App() {
  const [user, setUser] = useState({ name: "Asha", likes: 0 });
  return (
    <div className="space-y-2">
      <p>{user.name} has <b>{user.likes}</b> likes</p>
      <button className="rounded bg-rose-100 px-3 py-1" onClick={() => { user.likes++; setUser(user);
}}>
        Mutate (broken)
      </button>
      <button className="ml-2 rounded bg-emerald-100 px-3 py-1" onClick={() => setUser({ ...user,
likes: user.likes + 1 })}>
        New object (works)
      </button>
    </div>
  );
}
```



AFTER CLICKING "MUTATE" TWICE THE SCREEN STILL SHOWED 0; ONE CLICK ON "NEW OBJECT" RE-RENDERED AND REVEALED 3

The mutation did change the object (that is why the final value is 3), but React was never told; the screen only caught up when a correct update happened. Bugs like this appear and disappear unpredictably, so never mutate.

6.3.2 Updating nested objects with spread

Spread copies only one level. To change a nested property, copy **every level on the path** to it:

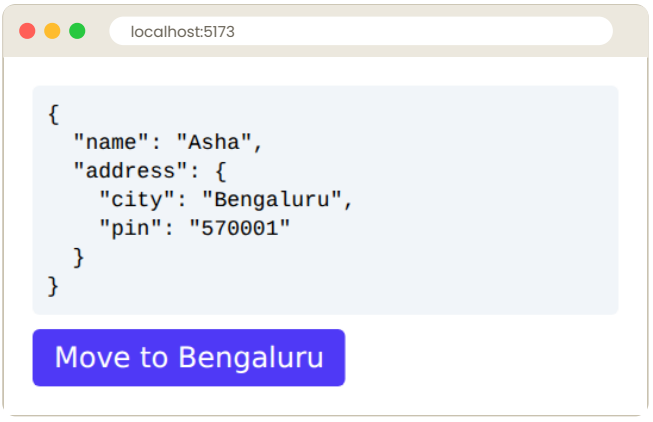
APP.JSX

JavaScript

```
import { useState } from "react";

export default function App() {
  const [student, setStudent] = useState({
    name: "Asha",
    address: { city: "Mysuru", pin: "570001" },
  });

  function moveTo(city) {
    setStudent({
      ...student,                                  // copy the top level
      address: { ...student.address, city },       // copy address and change city
    });
  }
  return (
    <div className="space-y-2">
      <pre className="rounded bg-slate-100 p-2 text-xs">{JSON.stringify(student, null, 2)}</pre>
      <button className="rounded bg-indigo-600 px-3 py-1 text-white" onClick={() =>
moveTo("Bengaluru")}>Move to Bengaluru</button>
    </div>
  );
}
```



AFTER THE CLICK: CITY CHANGED, NAME AND PIN KEPT

6.3.3 Adding, removing and updating array items

The immutable array patterns from Module 2 are exactly what you pass to the setter:

Action	State update
add to end	`setItems([...items, newItem])`
add to start	`setItems([newItem, ...items])`
remove	`setItems(items.filter(i => i.id !== id))`
update one	`setItems(items.map(i => i.id === id ? { ...i, done: !i.done } : i))`
insert at index	`setItems(items.toSpliced(index, 0, newItem))`
sort	`setItems(items.toSorted((a, b) => a.price - b.price))`

APP.JSX

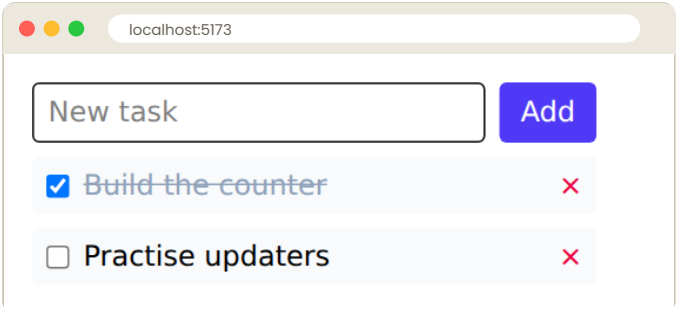
JavaScript

```
import { useState } from "react";

let nextId = 3;
export default function App() {
  const [tasks, setTasks] = useState([
    { id: 1, text: "Read Module 6", done: true },
    { id: 2, text: "Build the counter", done: false },
  ]);
  const [text, setText] = useState("");

  const add = () => { if (text.trim()) { setTasks([...tasks, { id: nextId++, text, done: false }]);
setText(""); } };
  const toggle = id => setTasks(tasks.map(t => (t.id === id ? { ...t, done: !t.done } : t)));
  const remove = id => setTasks(tasks.filter(t => t.id !== id));

  return (
    <div className="w-80 space-y-2">
      <div className="flex gap-2">
        <input value={text} onChange={e => setText(e.target.value)} placeholder="New task"
className="flex-1 rounded border px-2 py-1" />
        <button onClick={add} className="rounded bg-indigo-600 px-3 text-white">Add</button>
      </div>
      {tasks.map(t => (
        <div key={t.id} className="flex items-center gap-2 rounded bg-slate-50 px-2 py-1">
          <input type="checkbox" checked={t.done} onChange={() => toggle(t.id)} />
          <span className={t.done ? "flex-1 text-slate-400 line-through" : "flex-1"}>{t.text}</span>
          <button onClick={() => remove(t.id)} className="text-rose-600">&amp;times;</button>
        </div>
      ))}
    </div>
  );
}
```



AFTER ADDING "PRACTISE UPDATERS", TICKING TASK 2 AND DELETING TASK 1

6.3.4 Immer (overview)

Deeply nested updates with spread become verbose. **Immer** lets you write code that *looks* like mutation on a temporary **draft**, and produces a new immutable object for you. The `use-immer` package provides a `useImmer` hook; Redux Toolkit uses Immer internally (Module 16).

IMMER (REPRESENTATIVE: THE PACKAGE IS NOT INSTALLED IN THIS BUILD)

JavaScript

```
import { useImmer } from "use-immer";

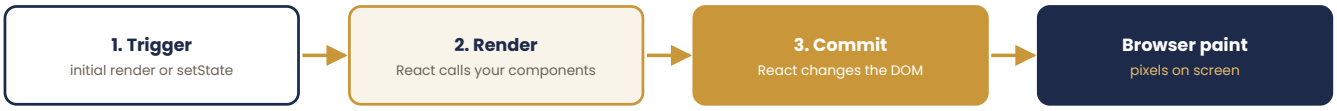
const [student, updateStudent] = useImmer({ name: "Asha", address: { city: "Mysuru" }, marks: [78, 91]
});

function moveAndAddMark() {
  updateStudent(draft => {
    draft.address.city = "Bengaluru";   // looks like mutation, but it is a draft
    draft.marks.push(85);               // Immer builds the new object for you
  });
}
```

6.4 Render and commit

6.4.1 What triggers a render

Displaying UI in React happens in three steps, like a restaurant: **trigger** (the order), **render** (the kitchen prepares it), **commit** (it is served). A render is triggered in exactly two situations: the **initial render** (`root.render()`), and a **state update** of the component or one of its ancestors. When a component re-renders, React also re-renders all the components it returns (its children), unless they are memoised (Module 18).



TRIGGER → RENDER → COMMIT → PAINT

APP.JSX

JavaScript

```
import { useState } from "react";

function Child({ label }) {
  console.log("  render Child", label);
  return <span>{label}</span>;
}

export default function App() {
  const [n, setN] = useState(0);
  console.log("render App, n =", n);
  return (
    <div>
      <button onClick={() => setN(n + 1)}>+1</button>
      <button id="same" onClick={() => setN(n)}>set same value</button>
      <Child label="A" />
      <Child label="B" />
    </div>
  );
}
```

```
BROWSER CONSOLE
render App, n = 0
  render Child A
  render Child B
--- click +1 ---
render App, n = 1
  render Child A
  render Child B
--- click "set same value" ---
render App, n = 1
```

A state update re-rendered App and both children. Setting state to the **same** value (by `Object.is`) skipped the render.

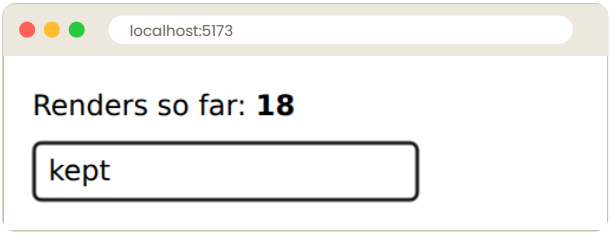
6.4.2 How React updates the DOM

During the render phase React only computes; the DOM is not touched. In the **commit** phase React compares the new result with the previous one and applies the **minimal** DOM operations: if a render produces the same output, nothing in the DOM changes. That is why an input keeps its typed text and focus while the component around it re-renders ten times a second (the timer uses an effect, explained in Module 9):

APP.JSX — JavaScript

```jsx
import { useState, useEffect } from "react";

export default function App() {
  const [time, setTime] = useState(0);
  useEffect(() => {
    const id = setInterval(() => setTime(t => t + 1), 100);
    return () => clearInterval(id);
  }, []);
  return (
    <div className="space-y-2">
      <p>Renders so far: <b>{time}</b></p>
      <input defaultValue="" placeholder="type here" className="rounded border px-2 py-1" />
    </div>
  );
}
```



AFTER TYPING "KEPT" WHILE THE COMPONENT RE-RENDERED TEN TIMES A SECOND: TEXT AND FOCUS SURVIVED

6.5 Sharing state between components

6.5.1 Lifting state up

When two components need the same changing data, move the state to their **closest common parent** and pass it down as props, a technique called **lifting state up**. Here two accordion panels each kept their own `isActive`, so both could be open. Lifting the state to the parent as `activeIndex` lets the parent enforce "only one open":

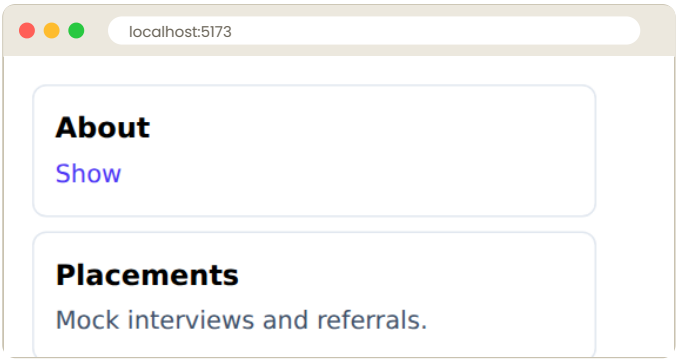
APP.JSX

JavaScript

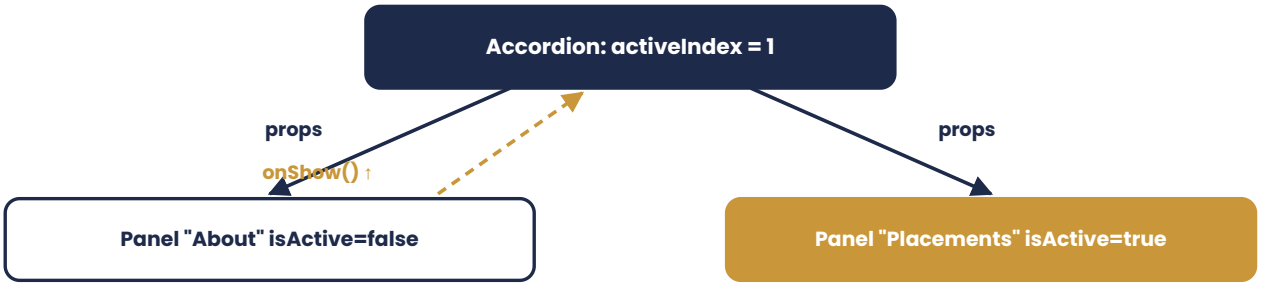
```
import { useState } from "react";

function Panel({ title, children, isActive, onShow }) {
  return (
    <section className="rounded-lg border border-slate-200 p-3">
      <h3 className="font-semibold">{title}</h3>
      {isActive ? <p className="mt-1 text-sm text-slate-600">{children}</p>
                : <button onClick={onShow} className="mt-1 text-sm text-indigo-600">Show</button>}
    </section>
  );
}

export default function Accordion() {
  const [activeIndex, setActiveIndex] = useState(0);       // lifted state
  return (
    <div className="w-80 space-y-2">
      <Panel title="About" isActive={activeIndex === 0} onShow={() => setActiveIndex(0)}>
        A 48-day React course with projects.
      </Panel>
      <Panel title="Placements" isActive={activeIndex === 1} onShow={() => setActiveIndex(1)}>
        Mock interviews and referrals.
      </Panel>
    </div>
  );
}
```



AFTER CLICKING SHOW ON PLACEMENTS: ABOUT CLOSES AUTOMATICALLY



LIFTED STATE: THE PARENT OWNS IT, CHILDREN RECEIVE VALUES AND CALLBACKS

6.5.2 Single source of truth

For each piece of state, decide **which component owns it**. That component is its single source of truth; everyone else receives it as props. Components whose important information comes from props rather than their own state are called **controlled** (by their parent); components with their own state are **uncontrolled**. Duplicating the same data in two places leads to them drifting apart.

6.5.3 Passing callbacks to children

Children change parent state by calling functions passed down as props: `onShow`, `onChange`, `onDelete`. The child decides **when** (a click), the parent decides **what** (which state to update). Completing the "Thinking in React" example from Module 5: the search text and checkbox are state in `FilterableProductTable`, passed down to both `SearchBar` (with setters) and `ProductTable` (to filter).

APP.JSX

JavaScript

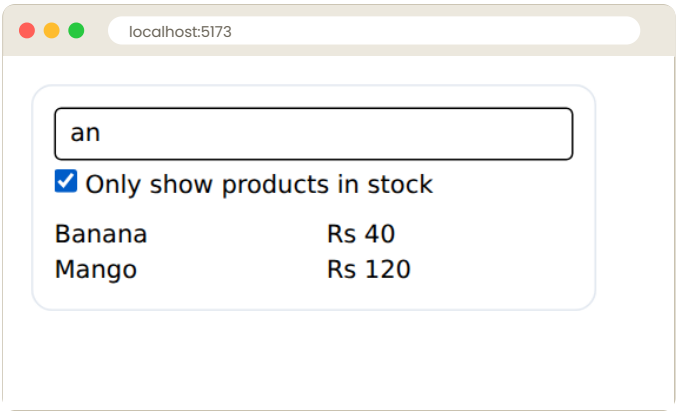
```
import { useState } from "react";

const PRODUCTS = [
  { category: "Fruits", price: "Rs 40", stocked: true, name: "Banana" },
  { category: "Fruits", price: "Rs 120", stocked: true, name: "Mango" },
  { category: "Fruits", price: "Rs 90", stocked: false, name: "Pomegranate" },
  { category: "Vegetables", price: "Rs 30", stocked: true, name: "Tomato" },
  { category: "Vegetables", price: "Rs 60", stocked: false, name: "Capsicum" },
];

function SearchBar({ filterText, inStockOnly, onFilterTextChange, onInStockOnlyChange }) {
  return (
    <form className="space-y-1">
      <input value={filterText} onChange={e => onFilterTextChange(e.target.value)}
             placeholder="Search..." className="w-full rounded border px-2 py-1 text-sm" />
      <label className="block text-sm">
        <input type="checkbox" checked={inStockOnly} onChange={e =>
onInStockOnlyChange(e.target.checked)} /> Only show products in stock
      </label>
    </form>
  );
}

function ProductTable({ products, filterText, inStockOnly }) {
  const visible = products.filter(p =>
    p.name.toLowerCase().includes(filterText.toLowerCase()) && (!inStockOnly || p.stocked));
  return (
    <table className="w-full text-sm">
      <tbody>
        {visible.map(p => (
          <tr key={p.name}><td className={p.stocked ? "" : "text-rose-600"}>{p.name}</td><td>{p.price}
</td></tr>
        ))}
      </tbody>
    </table>
  );
}

export default function FilterableProductTable() {
  const [filterText, setFilterText] = useState("");
  const [inStockOnly, setInStockOnly] = useState(false);
  return (
    <div className="w-80 space-y-2 rounded-xl border border-slate-200 p-3">
      <SearchBar filterText={filterText} inStockOnly={inStockOnly}
                 onFilterTextChange={setFilterText} onInStockOnlyChange={setInStockOnly} />
      <ProductTable products={PRODUCTS} filterText={filterText} inStockOnly={inStockOnly} />
    </div>
  );
}
```



AFTER TYPING "AN" AND TICKING THE CHECKBOX: BANANA AND MANGO (IN STOCK, CONTAINING "AN") REMAIN

6.6 Designing state

6.6.1 Avoid redundant and duplicated state

Good state is **minimal**. Principles from the React documentation:

Principle	Bad	Good
Group related state	`x` and `y` always updated together as two states	`const [pos, setPos] = useState({ x: 0, y: 0 })`
Avoid contradictions	`isSending` and `isSent` both true by mistake	`status: "typing" | "sending" | "sent"`
Avoid redundancy	`fullName` state next to `firstName` and `lastName`	compute `firstName + " " + lastName` while rendering
Avoid duplication	`selectedItem` object copied from `items`	store `selectedId`, find the item
Avoid deep nesting	trees of nested objects	flatten (normalise) by id

6.6.2 Derive values instead of storing them

Anything that can be **calculated** from props or existing state should be calculated during rendering, not stored in another state variable. Derived values are always correct, because they are recomputed from the source of truth on every render; extra state must be kept in sync by hand and eventually is not.

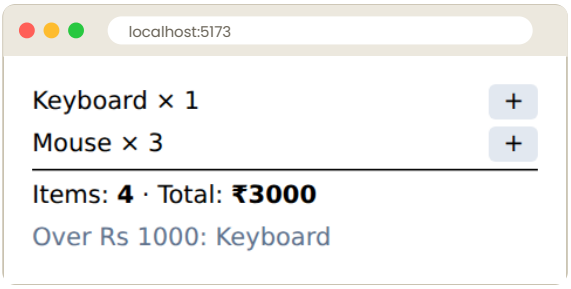
APP.JSX

JavaScript

```
import { useState } from "react";

export default function App() {
  const [items, setItems] = useState([
    { id: 1, name: "Keyboard", price: 1200, qty: 1 },
    { id: 2, name: "Mouse", price: 600, qty: 2 },
  ]);
  // derived: no extra state needed
  const count = items.reduce((n, i) => n + i.qty, 0);
  const total = items.reduce((s, i) => s + i.price * i.qty, 0);
  const expensive = items.filter(i => i.price > 1000).map(i => i.name);

  const inc = id => setItems(items.map(i => (i.id === id ? { ...i, qty: i.qty + 1 } : i)));
  return (
    <div className="w-72 space-y-1 text-sm">
      {items.map(i => (
        <div key={i.id} className="flex justify-between">
          <span>{i.name} × {i.qty}</span>
          <button onClick={() => inc(i.id)} className="rounded bg-slate-200 px-2">+</button>
        </div>
      ))}
      <p className="border-t pt-1">Items: <b>{count}</b> · Total: <b>&#8377;{total}</b></p>
      <p className="text-slate-500">Over Rs 1000: {expensive.join(", ")}</p>
    </div>
  );
}
```



AFTER ADDING ONE MORE MOUSE: COUNT AND TOTAL RECALCULATED AUTOMATICALLY

6.6.3 Preserving and resetting state with key

React keeps a component's state as long as the **same component** is rendered at the **same position** in the tree. Changing props does not reset state. When you want a fresh component (a chat box for a different contact, a form for a different student), give it a `key` that changes: a different key tells React it is a different component, so the old state is thrown away.

APP.JSX JavaScript

```
import { useState } from "react";

function Notes({ student }) {
  const [text, setText] = useState("");
  return (
    <textarea value={text} onChange={e => setText(e.target.value)} rows={2}
              placeholder={`Notes about ${student}`} className="w-full rounded border p-2 text-sm" />
  );
}

export default function App() {
  const [student, setStudent] = useState("Asha");
  return (
    <div className="grid w-[460px] grid-cols-2 gap-3 text-sm">
      <div>
        <p className="mb-1 font-semibold">Without key</p>
        <Notes student={student} />
      </div>
      <div>
        <p className="mb-1 font-semibold">With key={"{student}"}</p>
        <Notes key={student} student={student} />
      </div>
      <button onClick={() => setStudent("Ravi")} className="col-span-2 rounded bg-indigo-600 py-1
text-white">Switch to Ravi</button>
    </div>
  );
}
```



TYPED "GOOD AT CSS" IN BOTH BOXES, THEN SWITCHED TO RAVI: ONLY THE KEYED BOX WAS RESET

PRACTICE CHECKPOINT

A **counter**, a **like button**, and a **shopping cart** where items can be added, quantity changed and removed, with the total derived from state.

APP.JSX: SAMPLE SOLUTION JavaScript

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div className="flex items-center gap-2">
      <button onClick={() => setCount(c => c - 1)} disabled={count === 0} className="size-8 rounded
bg-slate-200 disabled:opacity-40">−</button>
      <b className="w-8 text-center">{count}</b>
      <button onClick={() => setCount(c => c + 1)} className="size-8 rounded bg-slate-200">+</button>
      <button onClick={() => setCount(0)} className="text-sm text-slate-500">reset</button>
    </div>
```

```
  );
}

function LikeButton() {
  const [liked, setLiked] = useState(false);
  const [likes, setLikes] = useState(41);
  function toggle() {
    setLiked(!liked);
    setLikes(n => (liked ? n - 1 : n + 1));
  }
  return (
    <button onClick={toggle} aria-pressed={liked}
            className={`rounded-full px-4 py-1.5 ring-1 ${liked ? "bg-rose-50 text-rose-600 ring-rose-
300" : "ring-slate-300"}`}>
      {liked ? "♥" : "♡"} {likes}
    </button>
  );
}

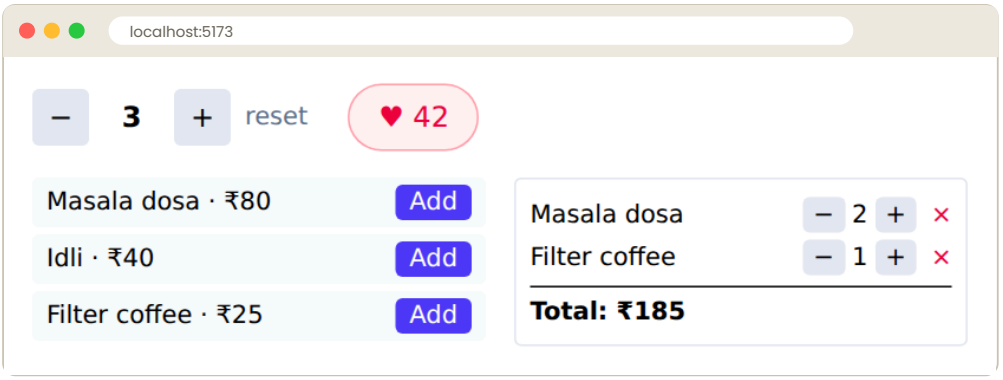
const CATALOGUE = [
  { id: 1, name: "Masala dosa", price: 80 },
  { id: 2, name: "Idli", price: 40 },
  { id: 3, name: "Filter coffee", price: 25 },
];

function Cart() {
  const [cart, setCart] = useState([]);                       // [{ id, name, price, qty }]
  const add = product =>
    setCart(c => c.some(i => i.id === product.id)
      ? c.map(i => (i.id === product.id ? { ...i, qty: i.qty + 1 } : i))
      : [...c, { ...product, qty: 1 }]);
  const changeQty = (id, delta) =>
    setCart(c => c.map(i => (i.id === id ? { ...i, qty: i.qty + delta } : i)).filter(i => i.qty > 0));
  const remove = id => setCart(c => c.filter(i => i.id !== id));
  const total = cart.reduce((s, i) => s + i.price * i.qty, 0);     // derived

  return (
    <div className="grid grid-cols-2 gap-4 text-sm">
      <div className="space-y-1">
        {CATALOGUE.map(p => (
          <div key={p.id} className="flex items-center justify-between rounded bg-slate-50 px-2 py-1">
            <span>{p.name} · &amp;#8377;{p.price}</span>
            <button onClick={() => add(p)} className="rounded bg-indigo-600 px-2 text-
white">Add</button>
          </div>
        ))}
      </div>
      <div className="rounded border border-slate-200 p-2">
        {cart.length === 0 && <p className="text-slate-400">Cart is empty</p>}
        {cart.map(i => (
          <div key={i.id} className="flex items-center justify-between py-0.5">
            <span>{i.name}</span>
            <span className="flex items-center gap-1">
              <button onClick={() => changeQty(i.id, -1)} className="rounded bg-slate-200 px-
1.5">−</button>
              {i.qty}
              <button onClick={() => changeQty(i.id, 1)} className="rounded bg-slate-200 px-1.5">+
</button>
              <button onClick={() => remove(i.id)} aria-label={`Remove ${i.name}`} className="ml-1
text-rose-600">&amp;times;</button>
            </span>
          </div>
        ))}
        <p className="mt-1 border-t pt-1 font-bold">Total: &amp;#8377;{total}</p>
```

```
      </div>
    </div>
  );
}

export default function App() {
  return (
    <div className="space-y-4">
      <div className="flex items-center gap-6"><Counter /><LikeButton /></div>
      <Cart />
    </div>
  );
}
```



AFTER +3 ON THE COUNTER, LIKING, ADDING DOSA TWICE, COFFEE ONCE AND IDLI ONCE, THEN REMOVING THE IDLI

Handle events with `onX={fn}` (pass, do not call). State from `useState` survives renders and its setter triggers a re-render; each render sees a snapshot, so use updater functions when the next value depends on the previous one. Never mutate state: create new objects and arrays. Lift shared state to the closest common parent, derive everything you can, and use `key` to reset a component.

MODULE 07 · BEGINNER

Conditional rendering and lists

Minimum duration **1 day** | Level **Beginner**

Real interfaces show different things in different situations and display collections of data. This module covers conditional rendering with if statements and early returns, the ternary operator, logical &&, returning null, and conditional classes and styles; rendering lists with map() and filter(), keys (why they matter, how to choose them, and the real bugs index keys cause), and nested lists; and the empty, loading and error states every list needs. All examples were run in Chromium, including a demonstration of the index-key bug.

IN THIS MODULE

7.1 Conditional rendering
7.2 Rendering lists
7.3 Empty, loading and error states

7.1 Conditional rendering

7.1.1 if statements and early returns

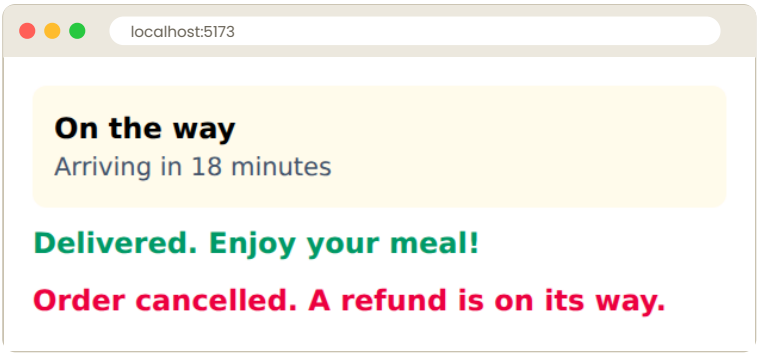
A component is a function, so ordinary `if` statements decide what it returns. For completely different outputs, an **early return** keeps the main JSX simple: handle the special cases first and return, then write the normal case.

APP.JSX

JavaScript

```
function OrderStatus({ status, eta }) {
  if (status === "cancelled") {
    return <p className="font-semibold text-rose-600">Order cancelled. A refund is on its way.</p>;
  }
  if (status === "delivered") {
    return <p className="font-semibold text-emerald-600">Delivered. Enjoy your meal!</p>;
  }
  return (
    <div className="rounded-lg bg-amber-50 p-3">
      <p className="font-semibold">On the way</p>
      <p className="text-sm text-slate-600">Arriving in {eta} minutes</p>
    </div>
  );
}

export default function App() {
  return (
    <div className="space-y-2">
      <OrderStatus status="preparing" eta={18} />
      <OrderStatus status="delivered" />
      <OrderStatus status="cancelled" />
    </div>
  );
}
```



THREE STATUSES, THREE DIFFERENT OUTPUTS FROM ONE COMPONENT

7.1.2 Ternary operator

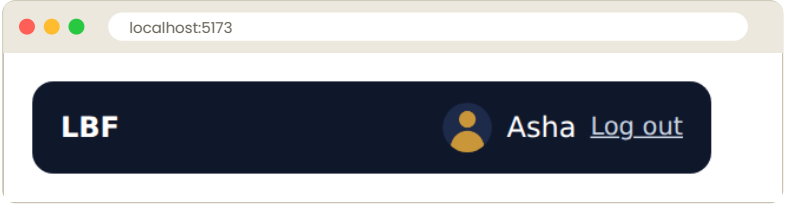
Inside JSX, where statements are not allowed, use `condition ? a : b` to choose between two pieces of UI. Keep ternaries short; if they nest more than one level, move the logic into a variable or a small component.

APP.JSX

JavaScript

```
import { useState } from "react";

export default function App() {
  const [loggedIn, setLoggedIn] = useState(false);
  return (
    <header className="flex w-96 items-center justify-between rounded-xl bg-slate-900 px-4 py-3 text-
white">
      <b>LBF</b>
      {loggedIn ? (
        <span className="flex items-center gap-2"><img src="avatar.png" className="size-7 rounded-
full" alt="" /> Asha
          <button onClick={() => setLoggedIn(false)} className="text-sm text-slate-300 underline">Log
out</button></span>
      ) : (
        <button onClick={() => setLoggedIn(true)} className="rounded bg-indigo-500 px-3 py-1 text-
sm">Log in</button>
      )}
    </header>
  );
}
```



AFTER CLICKING LOG IN: THE TERNARY SWITCHED BRANCHES

7.1.3 Logical &&

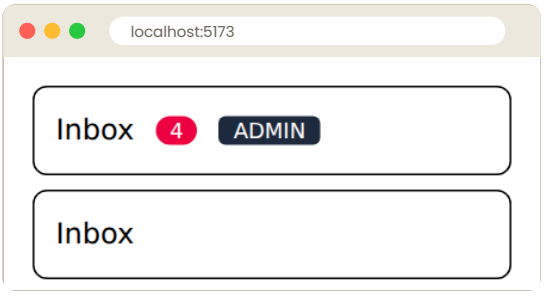
To render something **or nothing**, use `condition && <JSX />`. If the condition is false, React renders nothing. Make sure the left side is a real boolean, because a **0** on the left is rendered (Module 2): write `count > 0 &&`, not `count &&`.

APP.JSX

JavaScript

```
function Inbox({ unread, isAdmin }) {
  return (
    <div className="flex items-center gap-3 rounded-lg border p-3">
      <span>Inbox</span>
      {unread > 0 && <span className="rounded-full bg-rose-600 px-2 text-xs text-white">{unread}
</span>}
      {isAdmin && <span className="rounded bg-slate-800 px-2 text-xs text-white">ADMIN</span>}
    </div>
  );
}

export default function App() {
  return (
    <div className="space-y-2">
      <Inbox unread={4} isAdmin={true} />
      <Inbox unread={0} isAdmin={false} />
    </div>
  );
}
```



BADGES APPEAR ONLY WHEN THEIR CONDITION HOLDS

7.1.4 Returning null

A component can render nothing by returning `null`. The component still exists in the tree (its state is kept); only its output is empty. This is useful for components that decide for themselves whether to show, such as a banner that hides after being dismissed.

APP.JSX

JavaScript

```
import { useState } from "react";

function Banner({ message }) {
  const [closed, setClosed] = useState(false);
  if (closed) return null;                       // render nothing
  return (
    <div className="flex items-center justify-between rounded-lg bg-indigo-600 px-3 py-2 text-white">
      {message}
      <button onClick={() => setClosed(true)} aria-label="Dismiss">&times;</button>
    </div>
  );
}

export default function App() {
  return (
    <div className="w-80 space-y-2">
      <Banner message="New batch starts on Monday" />
      <Banner message="Fees due on the 10th" />
      <p className="text-sm text-slate-500">Page content below the banners</p>
    </div>
  );
}
```



AFTER DISMISSING THE FIRST BANNER, IT RETURNS NULL

7.1.5 Conditional classes and styles

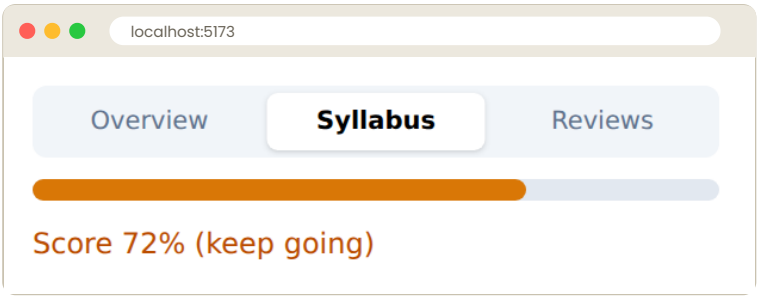
Build the `className` string with a template literal, a ternary, or a small helper such as `clsx` (Module 15). Avoid building class names from fragments like `` `text-${color}-600` `` when using Tailwind; write complete class names (Tailwind Module 15). For values that vary continuously, use an inline style.

APP.JSX

JavaScript

```
import { useState } from "react";

export default function App() {
  const [tab, setTab] = useState("syllabus");
  const tabs = ["overview", "syllabus", "reviews"];
  const score = 72;
  return (
    <div className="w-96 space-y-3">
      <nav className="flex gap-1 rounded-lg bg-slate-100 p-1">
        {tabs.map(t => (
          <button key={t} onClick={() => setTab(t)}
            className={`flex-1 rounded-md py-1.5 text-sm capitalize ${t === tab ? "bg-white font-
semibold shadow" : "text-slate-500"}`}>
            {t}
          </button>
        ))}
      </nav>
      <div className="h-3 rounded-full bg-slate-200">
        <div className="h-3 rounded-full" style={{ width: `${score}%`, background: score >= 75 ?
"#059669" : "#d97706" }} />
      </div>
      <p className={score >= 75 ? "text-emerald-700" : "text-amber-700"}>Score {score}% {score >= 75 ?
"(distinction)" : "(keep going)"}</p>
    </div>
  );
}
```



ACTIVE TAB CLASS, BAR COLOUR AND TEXT COLOUR ALL DEPEND ON DATA

Technique	Use when	Example
`if` / early return	whole outputs differ	`if (!user) return <Login />;`
element variable	a part differs and logic is complex	`let badge = ...; return <div>{badge}</div>`
ternary	one of two pieces inline	`{ok ? <A /> : <B />}`
`&&`	something or nothing	`{isAdmin && <Admin />}`
`return null`	the component hides itself	`if (closed) return null;`

7.2 Rendering lists

7.2.1 map() to render arrays

React renders an **array of elements** in order, so turning data into UI is a `map()`: each item becomes an element. Each element in the array needs a unique `key` prop (Section 7.2.3).

APP.JSX

JavaScript

```
const courses = [
  { id: "html", title: "HTML5", days: 10, level: "Beginner" },
  { id: "css", title: "CSS3", days: 14, level: "Beginner" },
  { id: "react", title: "React", days: 48, level: "Intermediate" },
];

export default function App() {
  return (
    <ul className="w-80 divide-y rounded-xl border border-slate-200">
      {courses.map(course => (
        <li key={course.id} className="flex justify-between px-3 py-2">
          <span className="font-medium">{course.title}</span>
          <span className="text-sm text-slate-500">{course.days} days · {course.level}</span>
        </li>
      ))}
    </ul>
  );
}
```



AN ARRAY OF OBJECTS RENDERED AS LIST ITEMS

7.2.2 filter() before rendering

To show only some items, `filter()` first and then `map()`. The filter criteria usually come from state (a search box, tabs); the filtered list is **derived** during rendering, never stored in separate state.

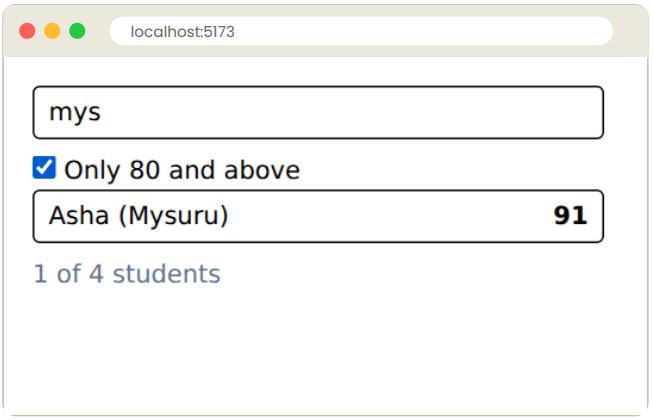
APP.JSX

JavaScript

```
import { useState } from "react";

const students = [
  { id: 1, name: "Asha", city: "Mysuru", marks: 91 },
  { id: 2, name: "Ravi", city: "Hubli", marks: 64 },
  { id: 3, name: "Meera", city: "Mysuru", marks: 78 },
  { id: 4, name: "Kiran", city: "Mangaluru", marks: 85 },
];

export default function App() {
  const [query, setQuery] = useState("");
  const [toppers, setToppers] = useState(false);
  const visible = students
    .filter(s => s.name.toLowerCase().includes(query.toLowerCase()) ||
s.city.toLowerCase().includes(query.toLowerCase()))
    .filter(s => !toppers || s.marks >= 80);
  return (
    <div className="w-80 space-y-2 text-sm">
      <input value={query} onChange={e => setQuery(e.target.value)} placeholder="Search name or city"
className="w-full rounded border px-2 py-1" />
      <label><input type="checkbox" checked={toppers} onChange={e => setToppers(e.target.checked)} />
Only 80 and above</label>
      <ul className="divide-y rounded border">
        {visible.map(s => <li key={s.id} className="flex justify-between px-2 py-1">{s.name}
({s.city}) <b>{s.marks}</b></li>)}
      </ul>
      <p className="text-slate-500">{visible.length} of {students.length} students</p>
    </div>
  );
}
```



AFTER TYPING "MYS" AND TICKING THE CHECKBOX: ONLY MYSURU STUDENTS WITH 80+

7.2.3 Keys: why they matter and choosing good keys

A **key** tells React which array item each element corresponds to between renders. When the list changes (items inserted, deleted, reordered), React matches old and new elements **by key**, so each item keeps its own DOM node and state. Without keys React warns in the console and falls back to matching by position.

APP.JSX

JavaScript

```
const fruits = ["Mango", "Banana", "Guava"];
export default function App() {
  return <ul>{fruits.map(f => <li>{f}</li>)}</ul>;      // no key
}
```

BROWSER CONSOLE (development build)

```
Error: Each child in a list should have a unique "key" prop.

Check the render method of `App`. See https://react.dev/link/warning-keys for more information.
```

Good key source	Example
database id	`key={student.id}`
a unique natural value	`key={product.sku}`, `key={email}`
an id created when the item is created	`crypto.randomUUID()` or a counter, stored with the item
avoid: array index	`key={index}` (only if the list never reorders, filters or inserts)
never: random at render	`key={Math.random()}` (new key every render: everything is recreated)

NOTE

Keys must be unique among **siblings** (not globally), stable across renders, and they are not passed to the component as a prop. Put the key on the outermost element returned by `map`, or on a `<Fragment key={...}>`.

7.2.4 Why array index keys cause bugs

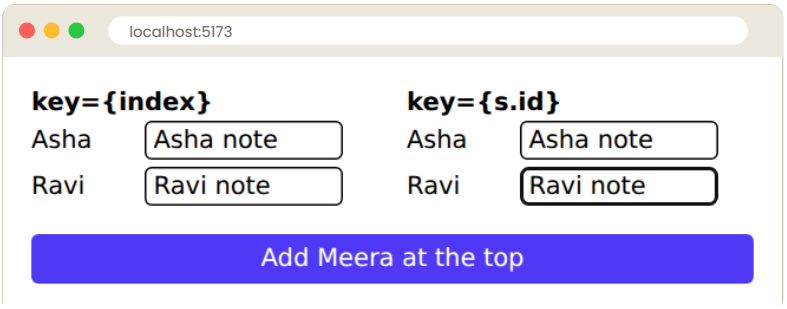
With `key={index}`, keys describe **positions**, not items. When an item is inserted at the top, every item shifts position, so React thinks each existing item changed its data, while each **stateful** child (an input, a checkbox, internal state) stays at its position and ends up attached to the wrong item. The demo below types a note next to each name, then adds a new student at the top:

APP.JSX JavaScript

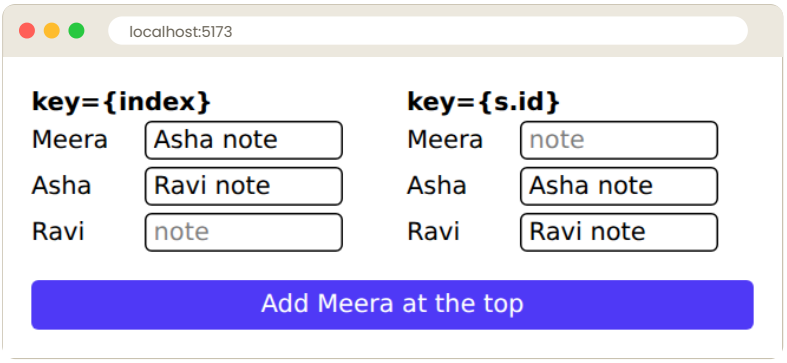
```
import { useState } from "react";

function Row({ name }) {
  return (
    <li className="flex items-center gap-2">
      <span className="w-14 font-medium">{name}</span>
      <input placeholder="note" className="w-28 rounded border px-1 text-sm" />
    </li>
  );
}

export default function App() {
  const [list, setList] = useState([{ id: 1, name: "Asha" }, { id: 2, name: "Ravi" }]);
  const addTop = () => setList([{ id: 3, name: "Meera" }, ...list]);
  return (
    <div className="grid grid-cols-2 gap-4 text-sm">
      <div><b>key={"{index}"}</b><ul className="space-y-1">{list.map((s, i) => <Row key={i} name=
{s.name} />)}</ul></div>
      <div><b>key={"{s.id}"}</b><ul className="space-y-1">{list.map(s => <Row key={s.id} name={s.name}
/>)}</ul></div>
      <button onClick={addTop} className="col-span-2 rounded bg-indigo-600 py-1 text-white">Add Meera
at the top</button>
    </div>
  );
}
```



BEFORE: A NOTE TYPED NEXT TO EACH STUDENT IN BOTH LISTS



AFTER ADDING MEERA AT THE TOP: WITH INDEX KEYS THE NOTES STAYED AT THEIR POSITIONS (MEERA HAS ASHA'S NOTE); WITH ID KEYS THEY MOVED WITH THEIR STUDENTS

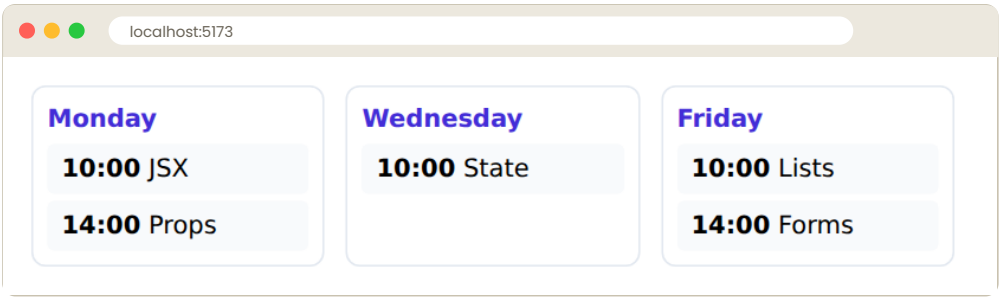
7.2.5 Rendering nested lists

A nested data structure becomes nested `map` calls. Each level needs its own keys (unique among its siblings). If the nested markup grows, extract the inner part into a component.

APP.JSX JavaScript

```
const timetable = [
  { day: "Monday", classes: [{ id: "m1", time: "10:00", topic: "JSX" }, { id: "m2", time: "14:00",
topic: "Props" }] },
  { day: "Wednesday", classes: [{ id: "w1", time: "10:00", topic: "State" }] },
  { day: "Friday", classes: [{ id: "f1", time: "10:00", topic: "Lists" }, { id: "f2", time: "14:00",
topic: "Forms" }] },
];

export default function App() {
  return (
    <div className="grid w-[520px] grid-cols-3 gap-3 text-sm">
      {timetable.map(d => (
        <section key={d.day} className="rounded-lg border border-slate-200 p-2">
          <h3 className="font-semibold text-indigo-700">{d.day}</h3>
          <ul className="mt-1 space-y-1">
            {d.classes.map(c => (
              <li key={c.id} className="rounded bg-slate-50 px-2 py-1"><b>{c.time}</b> {c.topic}</li>
            ))}
          </ul>
        </section>
      ))}
    </div>
  );
}
```



DAYS AND THEIR CLASSES FROM NESTED ARRAYS

7.3 Empty, loading and error states

Every list that comes from user input or a server has more than one state, and the "happy path" is only one of them. Design all four explicitly; a component often uses early returns for them:

State	What to show	Example
loading	a spinner or skeleton placeholders	"Loading students…"
error	a clear message and a way to retry	"Could not load. Retry"
empty	a friendly message and the next action	"No tasks yet. Add your first task"
success	the data	the list

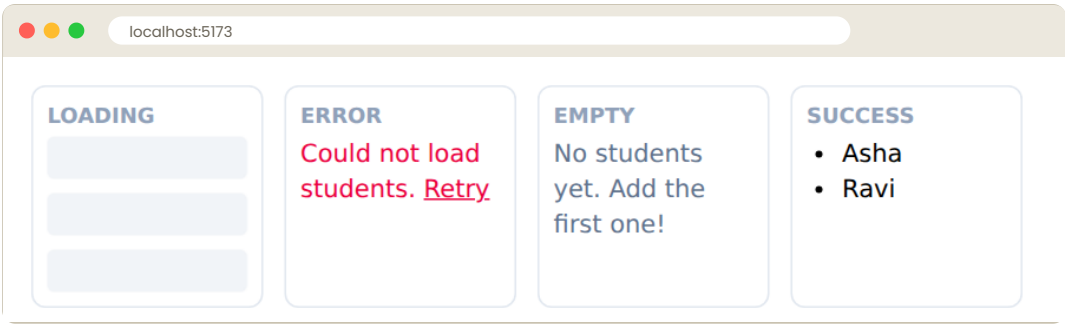
APP.JSX

JavaScript

```
import { useState } from "react";

function StudentList({ status, students, onRetry }) {
  if (status === "loading") {
    return <div className="space-y-2">{[1, 2, 3].map(i => <div key={i} className="h-6 animate-pulse
rounded bg-slate-200" />)}</div>;
  }
  if (status === "error") {
    return <p className="text-rose-600">Could not load students. <button onClick={onRetry}
className="underline">Retry</button></p>;
  }
  if (students.length === 0) {
    return <p className="text-slate-500">No students yet. Add the first one!</p>;
  }
  return <ul className="list-disc pl-5">{students.map(s => <li key={s.id}>{s.name}</li>)}</ul>;
}

export default function App() {
  const data = [{ id: 1, name: "Asha" }, { id: 2, name: "Ravi" }];
  return (
    <div className="grid w-[560px] grid-cols-4 gap-3 text-sm">
      {["loading", "error", "empty", "success"].map(st => (
        <div key={st} className="rounded-lg border border-slate-200 p-2">
          <p className="mb-1 text-xs font-bold uppercase text-slate-400">{st}</p>
          <StudentList status={st} students={st === "empty" ? [] : data} onRetry={() => {}} />
        </div>
      ))}
    </div>
  );
}
```



ONE COMPONENT, FOUR STATES (SHOWN SIDE BY SIDE FOR COMPARISON)

TIP

Module 9 connects these states to real data fetching with `useEffect`, and Module 14 to TanStack Query, which gives you `isPending`, `isError` and `data` for free.

PRACTICE CHECKPOINT

A **to-do app** with add, toggle, delete, filters (All, Active, Completed), an empty-state message and correct keys.

TODOAPP.JSX: SAMPLE SOLUTION (AS APP.JSX)

JavaScript

```
import { useState } from "react";

const FILTERS = {
  all: () => true,
  active: t => !t.done,
  completed: t => t.done,
};

export default function TodoApp() {
  const [todos, setTodos] = useState([
    { id: crypto.randomUUID(), text: "Revise JSX rules", done: true },
    { id: crypto.randomUUID(), text: "Practise map and keys", done: false },
  ]);
  const [text, setText] = useState("");
  const [filter, setFilter] = useState("all");

  function handleAdd(e) {
    e.preventDefault();
    const value = text.trim();
    if (!value) return;
    setTodos([...todos, { id: crypto.randomUUID(), text: value, done: false }]);
    setText("");
  }
  const toggle = id => setTodos(todos.map(t => (t.id === id ? { ...t, done: !t.done } : t)));
  const remove = id => setTodos(todos.filter(t => t.id !== id));
  const clearCompleted = () => setTodos(todos.filter(t => !t.done));

  const visible = todos.filter(FILTERS[filter]);          // derived
  const left = todos.filter(t => !t.done).length;         // derived

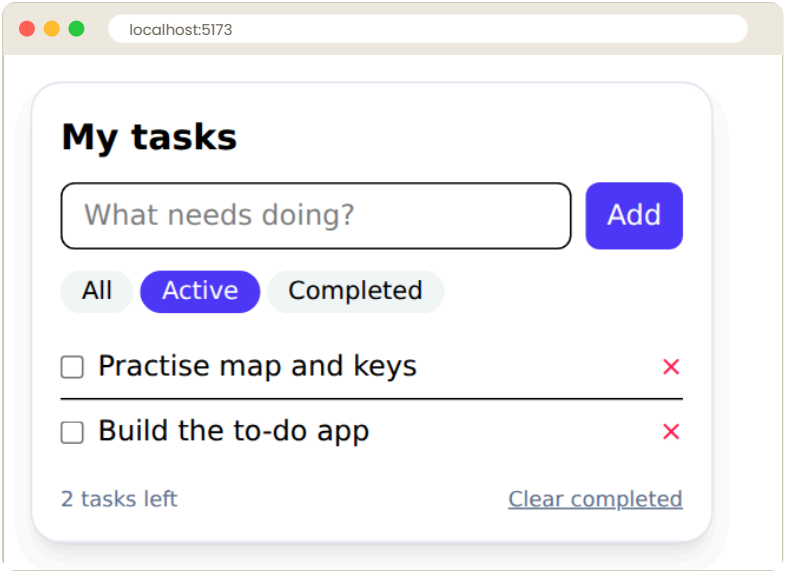
  return (
    <div className="w-96 rounded-2xl bg-white p-4 shadow-lg ring-1 ring-slate-200">
      <h1 className="mb-3 text-xl font-bold">My tasks</h1>
      <form onSubmit={handleAdd} className="flex gap-2">
        <input value={text} onChange={e => setText(e.target.value)} placeholder="What needs doing?"
               className="flex-1 rounded-lg border px-3 py-1.5" aria-label="New task" />
        <button className="rounded-lg bg-indigo-600 px-3 text-white">Add</button>
```

```
      </form>

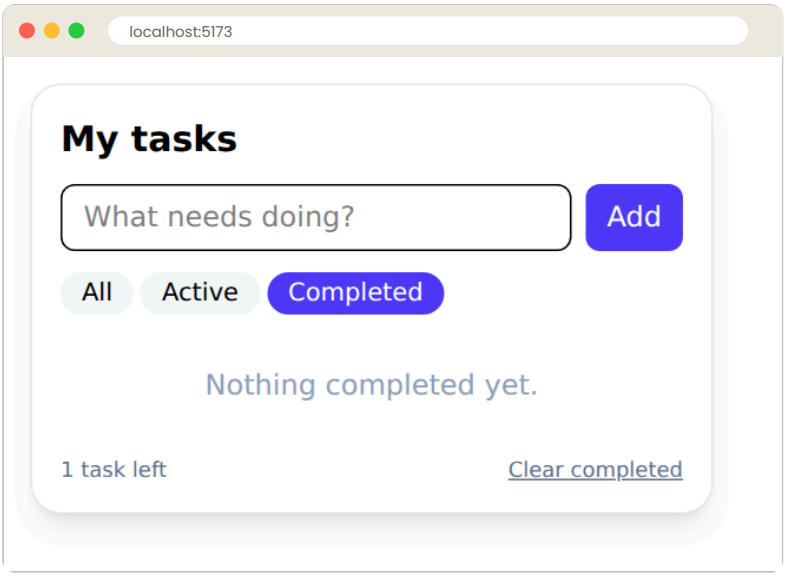
      <nav className="my-3 flex gap-1 text-sm">
        {Object.keys(FILTERS).map(f => (
          <button key={f} onClick={() => setFilter(f)} aria-pressed={f === filter}
                  className={`rounded-full px-3 py-0.5 capitalize ${f === filter ? "bg-indigo-600
text-white" : "bg-slate-100"}`}>
            {f}
          </button>
        ))}
      </nav>

      {visible.length === 0 ? (
        <p className="py-4 text-center text-slate-400">
          {filter === "completed" ? "Nothing completed yet." : filter === "active" ? "All done!" : "No
tasks yet. Add one above."}
        </p>
      ) : (
        <ul className="divide-y">
          {visible.map(t => (
            <li key={t.id} className="flex items-center gap-2 py-1.5">
              <input type="checkbox" checked={t.done} onChange={() => toggle(t.id)} aria-label=
{`Toggle ${t.text}`} />
              <span className={`flex-1 ${t.done ? "text-slate-400 line-through" : ""}`}>{t.text}
</span>
              <button onClick={() => remove(t.id)} aria-label={`Delete ${t.text}`} className="text-
rose-500">&times;</button>
            </li>
          ))}
        </ul>
      )}

      <footer className="mt-3 flex justify-between text-xs text-slate-500">
        <span>{left} {left === 1 ? "task" : "tasks"} left</span>
        <button onClick={clearCompleted} className="underline">Clear completed</button>
      </footer>
    </div>
  );
}
```



AFTER ADDING "BUILD THE TO-DO APP" AND CHOOSING THE ACTIVE FILTER



THE COMPLETED FILTER AFTER CLEARING COMPLETED TASKS: THE EMPTY-STATE MESSAGE

Choose what to render with `if` and early returns, ternaries, `&&` (with a real boolean) or `return null`. Render arrays with `filter` then `map`, give each item a stable, unique `key` from the data (not the index), and always design the loading, error and empty states.

MODULE 08 · BEGINNER

Forms

Minimum duration **2 days** | Level **Beginner**

Forms are how users give data to an application. This module covers controlled components (text inputs, textareas, selects, checkboxes and radio buttons bound to state, and one change handler for many inputs); uncontrolled components read with FormData and their default values; submitting with onSubmit and preventDefault, resetting forms, and an introduction to the React 19 action prop; field-level and form-level validation, showing error messages and disabling the submit button; and accessible forms with labels, useId() and announced errors. Every screenshot was taken after really typing and clicking in Chromium.

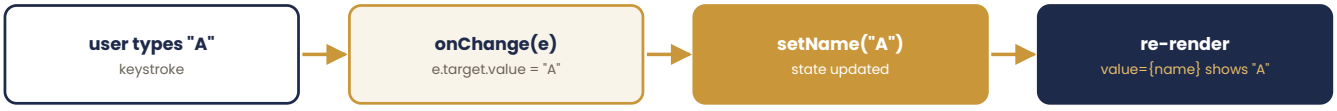
IN THIS MODULE

8.1 Controlled components
8.2 Uncontrolled components
8.3 Form submission
8.4 Validation
8.5 Accessible forms

8.1 Controlled components

8.1.1 Text inputs, textareas and selects bound to state

In a **controlled** input, React state is the single source of truth: the input shows `value={state}`, and every keystroke calls `onChange`, which updates the state, which re-renders the input with the new value. Because the value lives in state, you can validate, transform or display it anywhere as the user types.



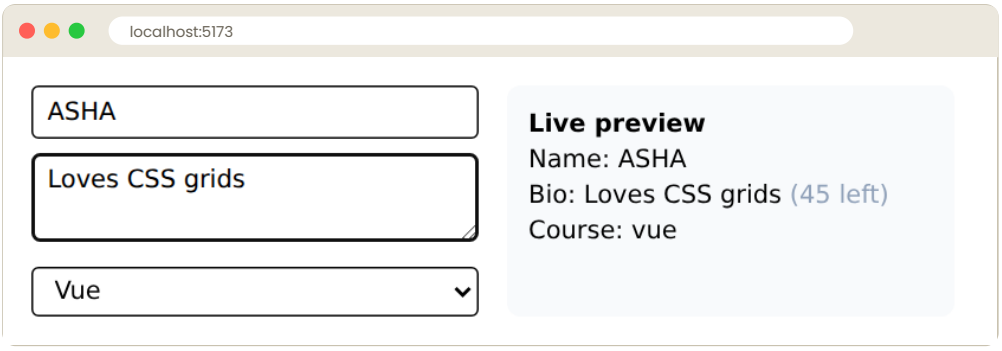
THE CONTROLLED INPUT LOOP

APP.JSX

JavaScript

```
import { useState } from "react";

export default function App() {
  const [name, setName] = useState("");
  const [bio, setBio] = useState("");
  const [course, setCourse] = useState("react");
  return (
    <div className="grid w-[520px] grid-cols-2 gap-4 text-sm">
      <div className="space-y-2">
        <input value={name} onChange={e => setName(e.target.value.toUpperCase())}
               placeholder="Name" className="w-full rounded border px-2 py-1" />
        <textarea value={bio} onChange={e => setBio(e.target.value)} maxLength={60}
                  placeholder="Short bio" rows={2} className="w-full rounded border px-2 py-1" />
        <select value={course} onChange={e => setCourse(e.target.value)} className="w-full rounded
border px-2 py-1">
          <option value="html">HTML5</option>
          <option value="react">React</option>
          <option value="vue">Vue</option>
        </select>
      </div>
      <div className="rounded-lg bg-slate-50 p-3">
        <p><b>Live preview</b></p>
        <p>Name: {name || "—"}</p>
        <p>Bio: {bio || "—"} <span className="text-slate-400">({60 - bio.length} left)</span></p>
        <p>Course: {course}</p>
      </div>
    </div>
  );
}
```



AFTER TYPING "ASHA", A BIO AND CHOOSING VUE: STATE DRIVES THE PREVIEW (AND UPPERCASES THE NAME AS IT IS TYPED)

In React, `<textarea>` and `<select>` also take a `value` prop, unlike HTML where a textarea's value is its content and a select uses `selected` on an option. A `<select multiple>` takes an array.

WATCH OUT

A `value` without an `onChange` makes the input read-only, and React warns: *"You provided a `value` prop to a form field without an `onChange` handler."* Use `defaultValue` for an uncontrolled field or `readOnly` if it should not change. Also never switch between `value={undefined}` and a string: start with `useState("")`, not `useState()`.

8.1.2 Checkboxes and radio buttons

Checkboxes and radios are controlled through `checked` (a boolean), and their handler reads `e.target.checked`. For radio buttons, store the selected value; each radio is `checked={value === "x"}`. A group of checkboxes is usually stored as an array of selected values.

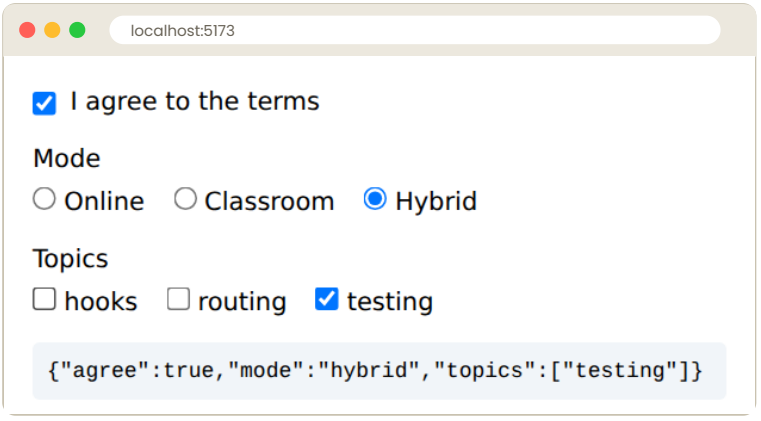
APP.JSX

JavaScript

```
import { useState } from "react";

export default function App() {
  const [agree, setAgree] = useState(false);
  const [mode, setMode] = useState("online");
  const [topics, setTopics] = useState(["hooks"]);
  const toggleTopic = t => setTopics(topics.includes(t) ? topics.filter(x => x !== t) : [...topics,
t]);

  return (
    <div className="space-y-3 text-sm">
      <label className="flex items-center gap-2">
        <input type="checkbox" checked={agree} onChange={e => setAgree(e.target.checked)} /> I agree
to the terms
      </label>
      <fieldset className="flex gap-4">
        <legend className="mb-1 font-medium">Mode</legend>
        {["online", "classroom", "hybrid"].map(m => (
          <label key={m} className="capitalize">
            <input type="radio" name="mode" value={m} checked={mode === m} onChange={e =>
setMode(e.target.value)} /> {m}
          </label>
        ))}
      </fieldset>
      <fieldset className="flex gap-4">
        <legend className="mb-1 font-medium">Topics</legend>
        {["hooks", "routing", "testing"].map(t => (
          <label key={t}><input type="checkbox" checked={topics.includes(t)} onChange={() =>
toggleTopic(t)} /> {t}</label>
        ))}
      </fieldset>
      <p className="rounded bg-slate-100 p-2 font-mono text-xs">{JSON.stringify({ agree, mode, topics
})}</p>
    </div>
  );
}
```



AFTER TICKING "I AGREE", CHOOSING HYBRID, ADDING TESTING AND REMOVING HOOKS

8.1.3 Handling many inputs with one change handler

A form with many fields does not need one state variable and one handler per field. Keep the form in **one object**, give every input a `name` that matches a key, and write **one handler** that uses a computed key: `[e.target.name]`. Handle checkboxes by reading `checked` instead of `value`.

APP.JSX

JavaScript

```
import { useState } from "react";

export default function App() {
  const [form, setForm] = useState({ firstName: "", lastName: "", email: "", city: "Mysuru",
newsletter: false });

  function handleChange(e) {
    const { name, value, type, checked } = e.target;
    setForm(prev => ({ ...prev, [name]: type === "checkbox" ? checked : value }));
  }

  return (
    <div className="grid w-[540px] grid-cols-2 gap-4 text-sm">
      <div className="space-y-2">
        <input name="firstName" value={form.firstName} onChange={handleChange} placeholder="First
name" className="w-full rounded border px-2 py-1" />
        <input name="lastName" value={form.lastName} onChange={handleChange} placeholder="Last name"
className="w-full rounded border px-2 py-1" />
        <input name="email" type="email" value={form.email} onChange={handleChange} placeholder="E-
mail" className="w-full rounded border px-2 py-1" />
        <select name="city" value={form.city} onChange={handleChange} className="w-full rounded border
px-2 py-1">
          <option>Mysuru</option><option>Bengaluru</option><option>Hubli</option>
        </select>
        <label><input type="checkbox" name="newsletter" checked={form.newsletter} onChange=
{handleChange} /> Send me updates</label>
      </div>
      <pre className="rounded-lg bg-slate-900 p-3 text-xs text-emerald-300">{JSON.stringify(form,
null, 2)}</pre>
    </div>
  );
}
```



FIVE FIELDS, ONE HANDLER, ONE STATE OBJECT

8.2 Uncontrolled components

8.2.1 Reading values with FormData

In an **uncontrolled** form, the DOM keeps the values and React does not track every keystroke. You read everything once, on submit, with the browser's `FormData` API: `new FormData(e.currentTarget)` collects every field that has a `name`. This is less code and fewer re-renders, and is ideal when you only need the values at the end (simple contact forms, filters submitted with a button).

APP.JSX

JavaScript

```
export default function App() {
  function handleSubmit(e) {
    e.preventDefault();
    const data = new FormData(e.currentTarget);
    console.log("name:", data.get("name"));
    console.log("course:", data.get("course"));
    console.log("topics:", data.getAll("topics"));        // several checkboxes with one name
    console.log("as object:", JSON.stringify(Object.fromEntries(data)));
  }
  return (
    <form onSubmit={handleSubmit}>
      <input name="name" defaultValue="Asha" />
      <select name="course" defaultValue="react"><option value="vue">Vue</option><option
value="react">React</option></select>
      <label><input type="checkbox" name="topics" value="hooks" defaultChecked /> Hooks</label>
      <label><input type="checkbox" name="topics" value="testing" /> Testing</label>
      <label><input type="checkbox" name="topics" value="routing" defaultChecked /> Routing</label>
      <button>Submit</button>
    </form>
  );
}
```

BROWSER CONSOLE (after clicking Submit)

```
name: Asha
course: react
as object: {"name":"Asha","course":"react","topics":"routing"}
topics: ['hooks', 'routing']
```

NOTE

`Object.fromEntries(formData)` keeps only the **last** value of a repeated name, so use `getAll()` for checkbox groups.

8.2.2 Default values

Uncontrolled fields get their starting value from `defaultValue` (text, textarea, select) or `defaultChecked` (checkbox, radio). Unlike `value`, these set the value only on the first render; later changes to the prop do not update the field (reset it with a `key`, Module 6). To read a single uncontrolled field at any time, use a **ref** (Module 11).

	Controlled	Uncontrolled
Value lives in	React state	the DOM
Set initial value	`useState("x")` + `value`	`defaultValue="x"` / `defaultChecked`
Read the value	any time, from state	on submit (FormData) or with a ref
Instant validation, formatting, dependent fields	easy	harder
Re-renders per keystroke	yes	no
Good for	interactive forms, live preview, wizards	simple forms, file inputs (`<input type="file">` is always uncontrolled)

8.3 Form submission

8.3.1 onSubmit and preventDefault()

Handle submission on the **form**, not on the button's click: `onSubmit` also fires when the user presses Enter in a field, and the browser's built-in validation (`required`, `type="email"`, `minLength`) runs first. Call `e.preventDefault()` to stop the full-page reload that a plain HTML form performs, then send the data with `fetch` or keep it in state.

APP.JSX

JavaScript

```
import { useState } from "react";

export default function App() {
  const [email, setEmail] = useState("");
  const [submitted, setSubmitted] = useState(null);
  function handleSubmit(e) {
    e.preventDefault();                  // no page reload
    setSubmitted(email);                 // here you would call fetch(...)
  }
  return (
    <form onSubmit={handleSubmit} className="w-80 space-y-2 text-sm">
      <input type="email" required value={email} onChange={e => setEmail(e.target.value)}
             placeholder="you@example.in" className="w-full rounded border px-2 py-1" />
      <button className="rounded bg-indigo-600 px-3 py-1 text-white">Subscribe</button>
      {submitted && <p className="text-emerald-700">Subscribed {submitted}. Thank you!</p>}
    </form>
  );
}
```



AFTER TYPING AN ADDRESS AND PRESSING ENTER (NO BUTTON CLICK NEEDED)

8.3.2 Resetting forms

For a controlled form, reset by setting state back to the initial values (keep them in a constant). For an uncontrolled form, call `e.currentTarget.reset()` or use a `<button type="reset">`. A third option for any form is to change its `key`, which remounts it with fresh state.

RESETTING CONTROLLED AND UNCONTROLLED FORMS JavaScript

```
const INITIAL = { name: "", email: "", course: "react" };

function Signup() {
  const [form, setForm] = useState(INITIAL);
  const reset = () => setForm(INITIAL);                    // controlled reset
  function handleSubmit(e) {
    e.preventDefault();
    save(form);
    reset();                                               // clear after a successful submit
  }
  ...
}

function QuickNote() {                                     // uncontrolled reset
  function handleSubmit(e) {
    e.preventDefault();
    save(Object.fromEntries(new FormData(e.currentTarget)));
    e.currentTarget.reset();
  }
  return <form onSubmit={handleSubmit}>...<button type="reset">Clear</button></form>;
}
```

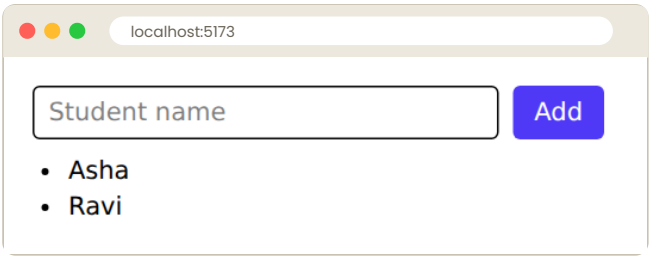
8.3.3 The action prop on forms (React 19, detailed in Module 17)

React 19 lets you pass a **function** to a form's `action` prop. React calls it with the `FormData` when the form is submitted, handles `preventDefault` for you, **resets uncontrolled fields automatically** after the action finishes, and works with new hooks such as `useFormStatus` (pending state) and `useActionState` (result and errors), covered in Module 17.

APP.JSX JavaScript

```
import { useState } from "react";

export default function App() {
  const [names, setNames] = useState([]);
  async function addStudent(formData) {                // receives FormData
    const name = formData.get("name");
    await new Promise(r => setTimeout(r, 200));         // pretend to save
    setNames(n => [...n, name]);
  }
  return (
    <form action={addStudent} className="w-80 space-y-2 text-sm">
      <div className="flex gap-2">
        <input name="name" required placeholder="Student name" className="flex-1 rounded border px-2
py-1" />
        <button className="rounded bg-indigo-600 px-3 text-white">Add</button>
      </div>
      <ul className="list-disc pl-5">{names.map(n => <li key={n}>{n}</li>)}</ul>
    </form>
  );
}
```



AFTER ADDING ASHA AND RAVI WITH THE ACTION PROP: THE INPUT WAS CLEARED AUTOMATICALLY AFTER EACH ACTION

8.4 Validation

8.4.1 Field-level and form-level validation

Field-level validation checks one field on its own (is the e-mail well formed? is the password long enough?); **form-level** validation checks rules involving several fields (do the two passwords match? is the end date after the start date?). Write validation as a **pure function** from values to an errors object, and derive the errors during rendering. Use the browser's built-in attributes (`required`, `minLength`, `pattern`, `type="email"`) as a first line of defence, and always validate again on the server.

VALIDATE.JS — JavaScript

```
function validate(v) {
  const errors = {};
  if (!v.name.trim()) errors.name = "Name is required";
  if (!/^\S+@\S+\.\S+$/.test(v.email)) errors.email = "Enter a valid e-mail";
  if (v.password.length < 8) errors.password = "At least 8 characters";
  if (v.confirm !== v.password) errors.confirm = "Passwords do not match";   // form-level
  return errors;                                // {} means valid
}
```

8.4.2 Showing error messages

Showing every error before the user has typed anything is unfriendly. Track which fields the user has **touched** (left with `onBlur`) and whether they tried to submit, and show a field's error only then. Put the message right below the field, in text, not only as a red border.

APP.JSX — JavaScript

```
import { useState } from "react";

function validate(v) {
  const e = {};
  if (!v.name.trim()) e.name = "Name is required";
  if (!/^\S+@\S+\.\S+$/.test(v.email)) e.email = "Enter a valid e-mail";
  if (v.password.length < 8) e.password = "At least 8 characters";
  if (v.confirm !== v.password) e.confirm = "Passwords do not match";
  return e;
}

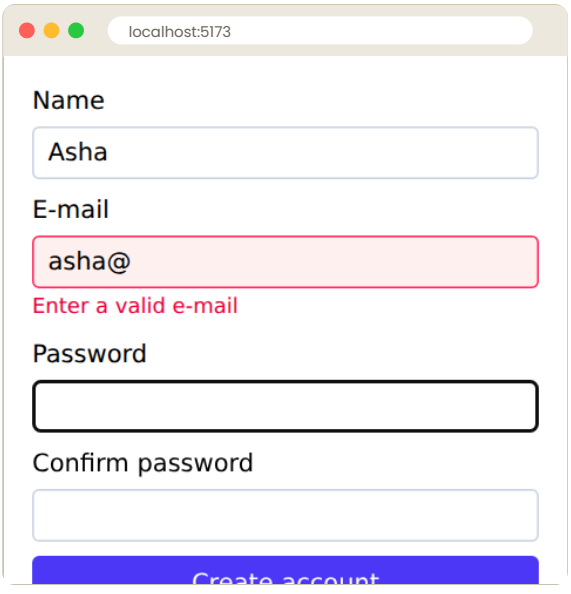
export default function App() {
  const [values, setValues] = useState({ name: "", email: "", password: "", confirm: "" });
  const [touched, setTouched] = useState({});
  const [submitted, setSubmitted] = useState(false);
  const errors = validate(values);                          // derived on every render
  const isValid = Object.keys(errors).length === 0;
  const show = f => (touched[f] || submitted) && errors[f];

  const change = e => setValues({ ...values, [e.target.name]: e.target.value });
```

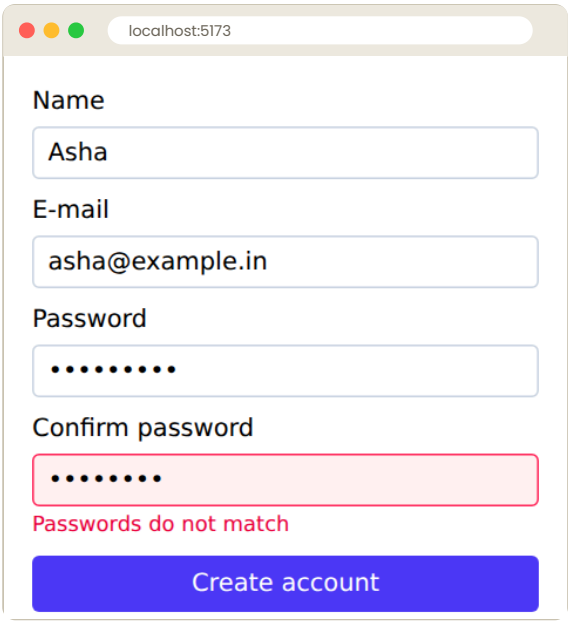
```
  const blur = e => setTouched({ ...touched, [e.target.name]: true });

  function field(name, label, type = "text") {
    return (
      <label className="block">
        <span className="font-medium">{label}</span>
        <input name={name} type={type} value={values[name]} onChange={change} onBlur={blur}
               className={`mt-1 block w-full rounded border px-2 py-1 ${show(name) ? "border-rose-500
bg-rose-50" : "border-slate-300"}`} />
        {show(name) && <span className="text-xs text-rose-600">{errors[name]}</span>}
      </label>
    );
  }

  return (
    <form noValidate onSubmit={e => { e.preventDefault(); setSubmitted(true); }} className="w-72
space-y-2 text-sm">
      {field("name", "Name")}
      {field("email", "E-mail", "email")}
      {field("password", "Password", "password")}
      {field("confirm", "Confirm password", "password")}
      <button disabled={submitted && !isValid} className="w-full rounded bg-indigo-600 py-1.5 text-
white disabled:opacity-40">Create account</button>
    </form>
  );
}
```



AFTER FILLING ONLY THE NAME AND E-MAIL ("ASHA@") AND LEAVING EACH FIELD: ONLY TOUCHED FIELDS SHOW ERRORS



AFTER CLICKING CREATE ACCOUNT WITH MISMATCHED PASSWORDS: EVERY REMAINING ERROR SHOWS AND THE BUTTON IS DISABLED

8.4.3 Disabling the submit button

Disabling the submit button while the form is invalid or while it is being sent prevents double submissions. Two cautions: a disabled button gives no feedback about *why*, so also show the errors (as above, after the first submit attempt), and disabled buttons are skipped by keyboard navigation. Many teams therefore keep the button enabled until the first attempt, then disable it while invalid, and always disable it while a request is pending.

8.5 Accessible forms

8.5.1 Labels, useId() and error announcements

Every input needs a **label** that screen readers announce. Either wrap the input in `<label>` or connect them with `htmlFor` and `id`. Ids must be unique on the page, which is a problem for reusable components rendered several times; `useId()` generates a unique, stable id per component instance (and matches between server and client rendering). Connect error text with `aria-describedby`, mark invalid fields with `aria-invalid`, and use `role="alert"` (or `aria-live`) so new errors are announced.

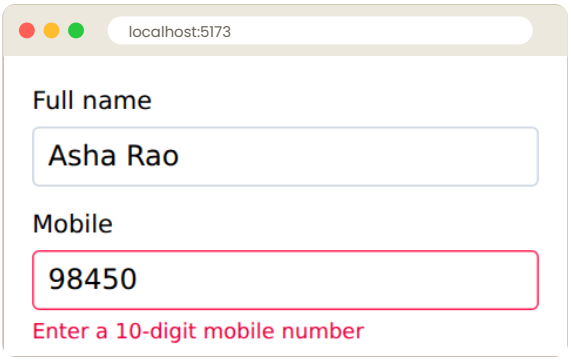
APP.JSX

JavaScript

```
import { useId, useState } from "react";

function TextField({ label, error, ...props }) {
  const id = useId();                                  // unique per instance
  const errorId = `${id}-error`;
  return (
    <div className="space-y-1">
      <label htmlFor={id} className="block text-sm font-medium">{label}</label>
      <input id={id} aria-invalid={error ? true : undefined} aria-describedby={error ? errorId :
undefined}
             className={`block w-full rounded border px-2 py-1 ${error ? "border-rose-500" : "border-
slate-300"}`} {...props} />
      {error && <p id={errorId} role="alert" className="text-xs text-rose-600">{error}</p>}
    </div>
  );
}

export default function App() {
  const [phone, setPhone] = useState("98450");
  const error = /^\d{10}$/.test(phone) ? null : "Enter a 10-digit mobile number";
  return (
    <div className="w-72 space-y-3">
      <TextField label="Full name" defaultValue="Asha Rao" />
      <TextField label="Mobile" value={phone} onChange={e => setPhone(e.target.value)} error={error}
inputMode="numeric" />
    </div>
  );
}
```



TWO INSTANCES OF ONE FIELD COMPONENT, EACH WITH ITS OWN GENERATED ID

RENDERED DOM: LABELS AND ERRORS CONNECTED BY GENERATED IDS

```
<div><label for="_r_0_">Full name</label><input id="_r_0_"></div><div><label
for="_r_1_">Mobile</label><input id="_r_1_" aria-invalid="true" aria-describedby="_r_1_-error"><p
id="_r_1_-error" role="alert">Enter a 10-digit mobile number</p></div>
```

Need	How
A name for every field	`<label htmlFor={id}>` or a wrapping `<label>`; never only a placeholder
Unique ids in reusable components	`useId()` (do not use it for list keys)
Connect help and error text	`aria-describedby={errorId}`
Mark invalid fields	`aria-invalid="true"` (plus a visible message, not only colour)
Announce errors	`role="alert"` or `aria-live="polite"`

Need	How
Group related controls	`<fieldset>` + `<legend>` for radio and checkbox groups
Move focus to the first error on submit	`form.querySelector("[aria-invalid]")?.focus()`

PRACTICE CHECKPOINT

A **student registration form** with controlled inputs, validation messages, a summary preview and a reset button.

REGISTRATION.JSX: SAMPLE SOLUTION (AS APP.JSX)

JavaScript

```
import { useId, useState } from "react";

const INITIAL = { name: "", email: "", phone: "", course: "", mode: "online", terms: false };
const COURSES = ["React", "Angular", "Vue", "Tailwind CSS"];

function validate(v) {
  const e = {};
  if (v.name.trim().length < 3) e.name = "Enter your full name (3+ letters)";
  if (!/^\S+@\S+\.\S+$/.test(v.email)) e.email = "Enter a valid e-mail";
  if (!/^[6-9]\d{9}$/.test(v.phone)) e.phone = "Enter a 10-digit Indian mobile number";
  if (!v.course) e.course = "Choose a course";
  if (!v.terms) e.terms = "Please accept the terms";
  return e;
}

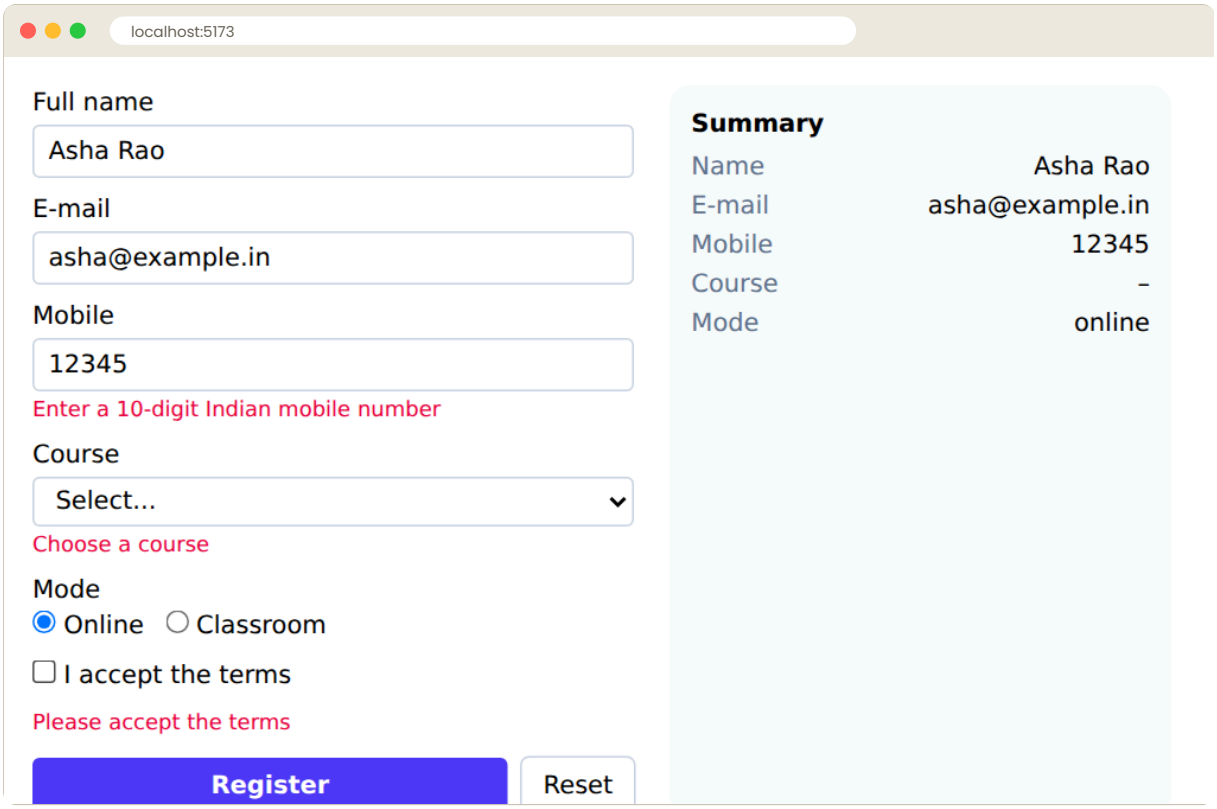
function Field({ label, error, children }) {
  const id = useId();
  return (
    <div className="space-y-0.5">
      <label htmlFor={id} className="block font-medium">{label}</label>
      {children(id, error ? `${id}-err` : undefined)}
      {error && <p id={`${id}-err`} role="alert" className="text-xs text-rose-600">{error}</p>}
    </div>
  );
}

export default function Registration() {
  const [form, setForm] = useState(INITIAL);
  const [tried, setTried] = useState(false);
  const [done, setDone] = useState(null);
  const errors = validate(form);
  const err = f => (tried ? errors[f] : undefined);
  const input = "w-full rounded border border-slate-300 px-2 py-1";

  function handleChange(e) {
    const { name, value, type, checked } = e.target;
    setForm(f => ({ ...f, [name]: type === "checkbox" ? checked : value }));
  }
  function handleSubmit(e) {
    e.preventDefault();
    setTried(true);
    if (Object.keys(errors).length === 0) setDone(form);
  }
  function handleReset() { setForm(INITIAL); setTried(false); setDone(null); }

  return (
    <div className="grid w-[640px] grid-cols-[1.2fr_1fr] gap-5 text-sm">
```

```
      <form onSubmit={handleSubmit} noValidate className="space-y-2">
        <Field label="Full name" error={err("name")}>{(id, d) => <input id={id} aria-describedby={d}
aria-invalid={!!d} name="name" value={form.name} onChange={handleChange} className={input} />}</Field>
        <Field label="E-mail" error={err("email")}>{(id, d) => <input id={id} aria-describedby={d}
aria-invalid={!!d} name="email" type="email" value={form.email} onChange={handleChange} className=
{input} />}</Field>
        <Field label="Mobile" error={err("phone")}>{(id, d) => <input id={id} aria-describedby={d}
aria-invalid={!!d} name="phone" inputMode="numeric" value={form.phone} onChange={handleChange}
className={input} />}</Field>
        <Field label="Course" error={err("course")}>{(id, d) => (
          <select id={id} aria-describedby={d} name="course" value={form.course} onChange=
{handleChange} className={input}>
            <option value="">Select...</option>{COURSES.map(c => <option key={c}>{c}</option>)}
          </select>)}</Field>
        <fieldset className="flex gap-3"><legend className="font-medium">Mode</legend>
          {["online", "classroom"].map(m => <label key={m} className="capitalize"><input type="radio"
name="mode" value={m} checked={form.mode === m} onChange={handleChange} /> {m}</label>)}
        </fieldset>
        <label className="block"><input type="checkbox" name="terms" checked={form.terms} onChange=
{handleChange} /> I accept the terms</label>
        {err("terms") && <p role="alert" className="text-xs text-rose-600">{errors.terms}</p>}
        <div className="flex gap-2 pt-1">
          <button className="flex-1 rounded bg-indigo-600 py-1.5 font-semibold text-
white">Register</button>
          <button type="button" onClick={handleReset} className="rounded px-3 ring-1 ring-slate-
300">Reset</button>
        </div>
      </form>
      <aside className="rounded-xl bg-slate-50 p-3">
        <h3 className="mb-1 font-bold">Summary</h3>
        <dl className="space-y-0.5">
          {Object.entries({ Name: form.name, "E-mail": form.email, Mobile: form.phone, Course:
form.course, Mode: form.mode }).map(([k, v]) => (
            <div key={k} className="flex justify-between gap-2"><dt className="text-slate-500">{k}
</dt><dd className="truncate font-medium">{v || "—"}</dd></div>
          ))}
        </dl>
        {done && <p className="mt-2 rounded bg-emerald-100 p-2 text-emerald-800">Registered
{done.name} for {done.course}!</p>}
      </aside>
    </div>
  );
}
```



AFTER FILLING SOME FIELDS AND CLICKING REGISTER: ERRORS FOR THE REST, LIVE SUMMARY ON THE RIGHT

Controlled inputs keep the value in state (`value` + `onChange`; `checked` for checkboxes) and suit interactive forms; uncontrolled inputs keep it in the DOM and are read with `FormData`. Submit with `onSubmit` + `preventDefault` (or the React 19 `action` prop), derive validation errors from values, show them when fields are touched, and connect labels and errors with `useId` and ARIA.

LEVEL 2

Intermediate level

Outcome: handle effects and data fetching, style apps, master hooks and context, add routing, use TanStack Query and React Hook Form, and type everything with TypeScript.

7 MODULES · 16 DAYS MINIMUM

Module 09 · Side effects and useEffect	**2 days**
Module 10 · Styling React applications	**1 day**
Module 11 · Hooks in depth and custom hooks	**3 days**
Module 12 · Context API and state management patterns	**2 days**
Module 13 · Routing with React Router	**3 days**
Module 14 · Working with APIs and forms at scale	**3 days**
Module 15 · TypeScript with React	**2 days**

MODULE 09 · INTERMEDIATE

Side effects and useEffect

Minimum duration **2 days** | Level **Intermediate**

Components must be pure while rendering, yet real apps need to fetch data, start timers, listen to the window and talk to browser APIs. useEffect is how a component synchronises with such external systems after rendering. This module covers what side effects are, useEffect syntax, the three forms of the dependency array, cleanup functions and why effects run twice in StrictMode; common uses (fetching with loading and error states, timers, window listeners, localStorage, the document title); data-fetching pitfalls (race conditions and AbortController); when you do not need an effect at all and how infinite loops happen; and useLayoutEffect. Data examples fetch real JSON files over HTTP from a local server in Chromium.

IN THIS MODULE

9.1 Effects
9.2 Common uses
9.3 Data fetching pitfalls
9.4 You might not need an effect
9.5 useLayoutEffect (overview)

9.1 Effects

9.1.1 What a side effect is

A **side effect** is anything a function does besides computing its return value: changing something outside itself, or depending on something that can change from outside. In React, rendering must be pure (Module 5), so side effects live in two places: **event handlers**, for effects caused by a specific user action (submit a form, add to cart), and **effects** (`useEffect`), for effects caused by **rendering itself**: whenever this component is on screen, it should be connected to, or in sync with, something outside React.

Side effect	Where it belongs	Why
save a form when the user clicks Submit	event handler	caused by a specific interaction
fetch the profile of the user being viewed	effect (or a data library)	needed because the component is displayed
start a timer for a clock	effect, with cleanup	runs while the component is on screen
listen to window resize	effect, with cleanup	external system subscription
compute a filtered list	**neither**: compute during render	it is not a side effect at all

9.1.2 useEffect syntax

`useEffect(setup, dependencies?)` registers a function that React runs **after** the component has rendered and the browser has painted the screen. The setup function can return a **cleanup** function. Effects run

after commit, so they can safely read the DOM.

THE SHAPE OF AN EFFECT JavaScript

```
import { useEffect } from "react";

useEffect(() => {
  // setup: runs after render (connect, subscribe, fetch, start a timer)
  return () => {
    // cleanup (optional): runs before the next setup and when the component unmounts
  };
}, [dep1, dep2]);   // dependencies: re-run setup only when one of these changes
```

APP.JSX JavaScript

```
import { useState, useEffect } from "react";

export default function App() {
  const [count, setCount] = useState(0);
  console.log("1. render, count =", count);
  useEffect(() => {
    console.log("2. effect, after the DOM shows:", document.querySelector("button").textContent);
  });
  return <button onClick={() => setCount(c => c + 1)}>Count {count}</button>;
}
```

BROWSER CONSOLE: EFFECTS RUN AFTER RENDERING

```
1. render, count = 0
2. effect, after the DOM shows: Count 0
--- click ---
1. render, count = 1
2. effect, after the DOM shows: Count 1
```

9.1.3 The dependency array: none, empty, with values

The second argument controls **when** the effect re-runs. React compares each dependency with its value from the previous render (with `Object.is`) and re-runs the effect only if something changed. Every reactive value the effect reads (props, state, values computed from them) must be listed; the ESLint rule `react-hooks/exhaustive-deps` checks this for you.

Form	Runs after	Use for
`useEffect(fn)` (no array)	every render	rarely needed; logging, measuring
`useEffect(fn, [])`	the first render only (mount)	connect once: subscriptions, one-time fetch
`useEffect(fn, [a, b])`	the first render and whenever `a` or `b` changes	synchronise with values: fetch for this id, title for this count

APP.JSX

JavaScript

```
import { useState, useEffect } from "react";

export default function App() {
  const [a, setA] = useState(0);
  const [b, setB] = useState(0);
  useEffect(() => { console.log("  every render"); });
  useEffect(() => { console.log("  [] : mount only"); }, []);
  useEffect(() => { console.log("  [a]: a is now", a); }, [a]);
  return (
    <>
      <button id="a" onClick={() => setA(a + 1)}>a</button>
      <button id="b" onClick={() => setB(b + 1)}>b</button>
    </>
  );
}
```

BROWSER CONSOLE

```
  every render
  [] : mount only
  [a]: a is now 0
--- click a ---
  every render
  [a]: a is now 1
--- click b ---
  every render
```

9.1.4 Cleanup functions

If an effect connects to something, its cleanup must disconnect. React runs the cleanup **before re-running the effect** (with the old values) and when the component **unmounts**. Forgetting cleanup causes leaks: timers that keep running after the component is gone, duplicated listeners, or responses applied to the wrong screen.

APP.JSX

JavaScript

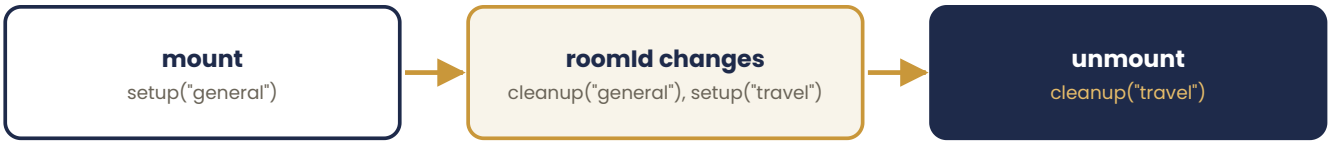
```
import { useState, useEffect } from "react";

function ChatRoom({ roomId }) {
  useEffect(() => {
    console.log("connect to", roomId);
    return () => console.log("disconnect from", roomId);
  }, [roomId]);
  return <h2>Welcome to {roomId}</h2>;
}

export default function App() {
  const [room, setRoom] = useState("general");
  const [show, setShow] = useState(true);
  return (
    <>
      <button id="travel" onClick={() => setRoom("travel")}>Switch to travel</button>
      <button id="close" onClick={() => setShow(false)}>Close chat</button>
      {show && <ChatRoom roomId={room} />}
    </>
  );
}
```

BROWSER CONSOLE

```
connect to general
--- switch room ---
disconnect from general
connect to travel
--- close (unmount) ---
disconnect from travel
```



THE LIFECYCLE OF ONE EFFECT

9.1.5 Why effects run twice in StrictMode during development

In development, `<StrictMode>` mounts every component, immediately runs the cleanup, and mounts it again: **setup → cleanup → setup**. This stress test reveals effects without proper cleanup (a connection opened twice and never closed). If your effect is written correctly, the extra cycle is invisible to the user. It does not happen in production. Do not "fix" it with refs that skip the second run; add the missing cleanup instead.

APP.JSX — JavaScript

```
import { useEffect } from "react";

export default function App() {
  useEffect(() => {
    console.log("setup: connect");
    return () => console.log("cleanup: disconnect");
  }, []);
  return <p>Chat</p>;
}
```

BROWSER CONSOLE WITH <StrictMode> (development)

```
setup: connect
cleanup: disconnect
setup: connect
```

BROWSER CONSOLE WITHOUT StrictMode (what production does)

```
setup: connect
```

9.2 Common uses

9.2.1 Fetching data with loading and error states

A fetching effect needs three pieces of state: the **data**, a **loading** flag and an **error**. The effect starts the request and sets state when it finishes; the component renders whichever state applies (Module 7). The effect function itself cannot be `async` (it must return a cleanup or nothing), so define an async function inside and call it.

APP.JSX

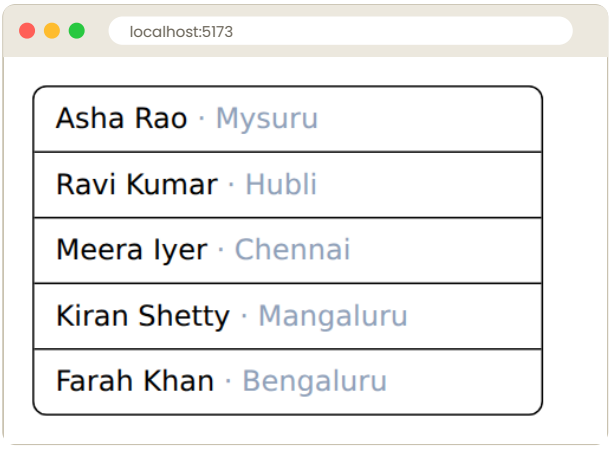
JavaScript

```
import { useState, useEffect } from "react";

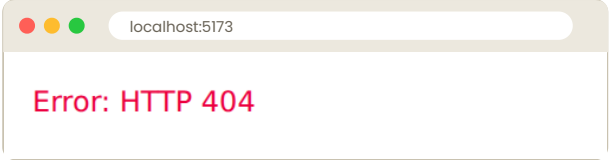
export default function App() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function load() {
      try {
        const res = await fetch("users.json");              // a real HTTP request
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        setUsers(await res.json());
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }
    load();
  }, []);                                                   // once, on mount

  if (loading) return <p className="text-slate-500">Loading users...</p>;
  if (error) return <p className="text-rose-600">Error: {error}</p>;
  return (
    <ul className="w-72 divide-y rounded-lg border">
      {users.map(u => <li key={u.id} className="px-3 py-1.5">{u.name} <span className="text-slate-
400">· {u.city}</span></li>)}
    </ul>
  );
}
```



THE DATA ARRIVED FROM USERS.JSON OVER HTTP AND WAS RENDERED



THE SAME COMPONENT WITH A MISTYPED URL: THE 404 BECOMES THE ERROR STATE

TIP

For real applications prefer a data-fetching library (TanStack Query, SWR) or your framework's data loading (React Router loaders, Next.js Server Components). They add caching, deduplication, retries and background refresh, and avoid the pitfalls in Section 9.3 (Module 14).

9.2.2 Timers and intervals

Start the timer in the effect and clear it in the cleanup. Use the **updater form** of the setter inside the interval, so the effect does not depend on the current value and does not need to restart every tick.

APP.JSX

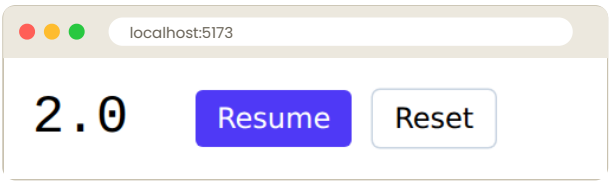
JavaScript

```
import { useState, useEffect } from "react";

export default function App() {
  const [seconds, setSeconds] = useState(0);
  const [running, setRunning] = useState(true);

  useEffect(() => {
    if (!running) return;                                  // no timer while paused
    const id = setInterval(() => setSeconds(s => s + 1), 100);   // 100 ms so the demo is quick
    return () => clearInterval(id);                        // stop it on pause or unmount
  }, [running]);

  return (
    <div className="flex items-center gap-3">
      <span className="w-20 font-mono text-3xl">{(seconds / 10).toFixed(1)}</span>
      <button onClick={() => setRunning(r => !r)} className="rounded bg-indigo-600 px-3 py-1 text-
white">{running ? "Pause" : "Resume"}</button>
      <button onClick={() => setSeconds(0)} className="rounded px-3 py-1 ring-1 ring-slate-
300">Reset</button>
    </div>
  );
}
```



A STOPWATCH AFTER ABOUT TWO SECONDS, THEN PAUSED

WATCH OUT

`setInterval(() => setSeconds(seconds + 1), 1000)` with `[]` dependencies is a classic bug: the callback is a closure over the first render, so it keeps setting 0 + 1 = 1 forever. Use `setSeconds(s => s + 1)`.

9.2.3 Event listeners on window

Listeners on `window` or `document` (resize, scroll, keydown, online/offline) are outside React's tree, so they are attached in an effect and **removed in the cleanup with the same function reference**.

APP.JSX

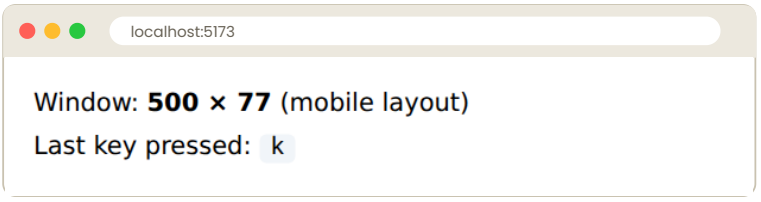
JavaScript

```
import { useState, useEffect } from "react";

export default function App() {
  const [size, setSize] = useState({ w: window.innerWidth, h: window.innerHeight });
  const [lastKey, setLastKey] = useState("(none)");

  useEffect(() => {
    const onResize = () => setSize({ w: window.innerWidth, h: window.innerHeight });
    const onKey = e => setLastKey(e.key);
    window.addEventListener("resize", onResize);
    window.addEventListener("keydown", onKey);
    return () => {
      window.removeEventListener("resize", onResize);
      window.removeEventListener("keydown", onKey);
    };
  }, []);

  return (
    <div className="space-y-1 text-sm">
      <p>Window: <b>{size.w} × {size.h}</b> ({size.w < 640 ? "mobile" : "desktop"} layout)</p>
      <p>Last key pressed: <kbd className="rounded bg-slate-100 px-1.5 font-mono">{lastKey}</kbd></p>
    </div>
  );
}
```



AFTER THE WINDOW WAS RESIZED TO 500 PX AND THE "K" KEY PRESSED

9.2.4 Syncing with localStorage

To remember state across page reloads, read `localStorage` once for the **initial** state (pass a function to `useState` so it runs only on the first render, "lazy initialisation") and write it back in an effect whenever the state changes.

APP.JSX

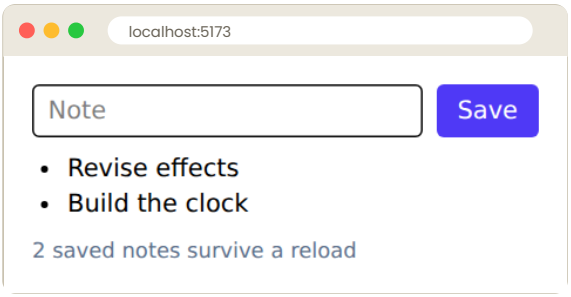
JavaScript

```
import { useState, useEffect } from "react";

export default function App() {
  const [notes, setNotes] = useState(() => {
    const saved = localStorage.getItem("notes");            // read once
    return saved ? JSON.parse(saved) : [];
  });
  const [text, setText] = useState("");

  useEffect(() => {
    localStorage.setItem("notes", JSON.stringify(notes));    // write on every change
  }, [notes]);

  return (
    <div className="w-72 space-y-2 text-sm">
      <form onSubmit={e => { e.preventDefault(); setNotes([...notes, text]); setText(""); }}
className="flex gap-2">
        <input value={text} onChange={e => setText(e.target.value)} className="flex-1 rounded border
px-2 py-1" placeholder="Note" />
        <button className="rounded bg-indigo-600 px-3 text-white">Save</button>
      </form>
      <ul className="list-disc pl-5">{notes.map((n, i) => <li key={i}>{n}</li>)}</ul>
      <p className="text-xs text-slate-500">{notes.length} saved notes survive a reload</p>
    </div>
  );
}
```



TWO NOTES SAVED, THEN THE PAGE WAS RELOADED: THEY CAME BACK FROM LOCALSTORAGE

9.2.5 Updating the document title

The browser tab title is outside React, so it is synchronised with an effect that depends on the values it shows. React 19 also lets you render `<title>` anywhere in your components and hoists it into the document head, which is simpler for static titles.

APP.JSX

JavaScript

```
import { useState, useEffect } from "react";

export default function App() {
  const [unread, setUnread] = useState(0);
  useEffect(() => {
    document.title = unread > 0 ? `(${unread}) Inbox` : "Inbox";
    console.log("document.title =", document.title);
  }, [unread]);
  return <button onClick={() => setUnread(n => n + 1)}>New message</button>;
}
```

BROWSER CONSOLE

```
document.title = Inbox
document.title = (1) Inbox
document.title = (2) Inbox
document.title = (3) Inbox
```

9.3 Data fetching pitfalls

9.3.1 Race conditions and ignoring stale responses

When an effect fetches data for a value that changes quickly (a search query, a selected user), requests can finish **out of order**. If the user selects user 1 and then user 2, and the response for 1 is slower, it arrives last and overwrites the correct data. The fix is a flag set by the cleanup: the old effect's cleanup marks its response as stale, so it is ignored.

APP.JSX

JavaScript

```
import { useState, useEffect } from "react";

const delays = { 1: 600, 2: 100, 3: 300 };                  // user 1's server is slow
async function fetchUser(id) {
  await new Promise(r => setTimeout(r, delays[id]));        // simulate network time
  const res = await fetch(`user-${id}.json`);               // then a real request
  return res.json();
}

function Profile({ userId, safe }) {
  const [user, setUser] = useState(null);
  useEffect(() => {
    let ignore = false;
    fetchUser(userId).then(data => {
      if (safe && ignore) return;                           // the fix: drop stale responses
      setUser(data);
    });
    return () => { ignore = true; };                        // cleanup marks this request stale
  }, [userId, safe]);
  return <p className="text-sm">{safe ? "With ignore flag" : "Without fix"}: showing <b>{user ?
user.name : "..."}</b></p>;
}

export default function App() {
  const [id, setId] = useState(1);
  return (
    <div className="space-y-1">
      <div className="flex gap-1">{[1, 2, 3].map(n => <button key={n} id={"u" + n} onClick={() =>
setId(n)} className="rounded bg-slate-200 px-2">User {n}</button>)}</div>
      <p className="text-sm">Selected: user {id}</p>
      <Profile userId={id} safe={false} />
      <Profile userId={id} safe={true} />
    </div>
  );
}
```



USER 2 WAS CLICKED RIGHT AFTER LOAD: USER 1'S SLOW RESPONSE ARRIVED LAST AND OVERWROTE THE UNPROTECTED PROFILE

9.3.2 Cancelling requests with AbortController

Ignoring a stale response still lets the request finish and waste bandwidth. `AbortController` actually cancels it: pass `controller.signal` to `fetch` and call `controller.abort()` in the cleanup. The aborted fetch rejects with an `AbortError`, which you ignore.

CANCELLING A FETCH IN THE CLEANUP JavaScript

```
useEffect(() => {
  const controller = new AbortController();
  fetch(`/api/search?q=${query}`, { signal: controller.signal })
    .then(res => res.json())
    .then(setResults)
    .catch(err => {
      if (err.name !== "AbortError") setError(err.message);   // aborts are expected
    });
  return () => controller.abort();                            // cancel when query changes or on
unmount
}, [query]);
```

APP.JSX JavaScript

```
import { useEffect } from "react";

export default function App() {
  useEffect(() => {
    const controller = new AbortController();
    fetch("users.json", { signal: controller.signal })
      .then(r => r.json())
      .then(d => console.log("loaded", d.length, "users"))
      .catch(err => console.log("caught:", err.name, "-", err.message));
    controller.abort();                       // abort immediately, as a cleanup would
  }, []);
  return <p>Abort demo</p>;
}
```

BROWSER CONSOLE: AN ABORTED REQUEST

```
caught: AbortError - signal is aborted without reason
```

9.4 You might not need an effect

9.4.1 Deriving data during render

Effects are an escape hatch for **external** systems. A very common mistake is using an effect to compute state from other state: it renders once with stale data, then the effect sets state and renders again.

Compute derived values directly while rendering instead; for expensive calculations, use `useMemo` (Module 11).

DERIVE INSTEAD OF SYNCING JavaScript

```
// BAD: redundant state + effect = an extra render with a stale value
const [fullName, setFullName] = useState("");
useEffect(() => { setFullName(firstName + " " + lastName); }, [firstName, lastName]);

// GOOD: just calculate it
const fullName = firstName + " " + lastName;

// GOOD: an expensive calculation, cached between renders
const visibleTodos = useMemo(() => filterTodos(todos, filter), [todos, filter]);
```

APP.JSX JavaScript

```
import { useState, useEffect } from "react";

export default function App() {
  const [first, setFirst] = useState("Asha");
  const [fullFromEffect, setFull] = useState("");
  useEffect(() => { setFull(first + " Rao"); }, [first]);
  const fullDerived = first + " Rao";
  console.log("render  | effect version:", JSON.stringify(fullFromEffect), "| derived:",
JSON.stringify(fullDerived));
  return <button onClick={() => setFirst("Ravi")}>rename</button>;
}
```

BROWSER CONSOLE: THE EFFECT VERSION IS ALWAYS ONE RENDER BEHIND

```
render  | effect version: "" | derived: "Asha Rao"
render  | effect version: "Asha Rao" | derived: "Asha Rao"
--- rename ---
render  | effect version: "Asha Rao" | derived: "Ravi Rao"
render  | effect version: "Ravi Rao" | derived: "Ravi Rao"
```

9.4.2 Handling user events in event handlers instead of effects

If code should run **because the user did something**, put it in that event handler, not in an effect that watches state. Effects that react to "the state changed" cannot tell *why* it changed, so they fire at the wrong times (for example a "Added to cart!" notification showing again after a page reload, because the cart was restored from storage).

EFFECTS VERSUS EVENT HANDLERS JavaScript

```
// BAD: runs whenever product.isInCart becomes true, including restoring from storage
useEffect(() => {
  if (product.isInCart) showNotification(`Added ${product.name} to the cart!`);
}, [product]);

// GOOD: runs exactly when the user clicks
function handleBuyClick() {
  addToCart(product);
  showNotification(`Added ${product.name} to the cart!`);
}
```

9.4.3 Infinite loop causes and fixes

Cause	What happens	Fix
`useEffect(() => setCount(count + 1))` with no array	set state → render → effect → set state … forever	add dependencies; ask whether the effect is needed at all
an object or array created during render as a dependency	it is a new reference every render, so the effect always re-runs	move it inside the effect, depend on primitive fields, or `useMemo`
a function defined in the component as a dependency	a new function every render	define it inside the effect, or `useCallback` (Module 11)
setting state to a new object with the same content	never equal by reference	compare before setting, or store primitives

APP.JSX

JavaScript

```
import { useState, useEffect } from "react";

export default function App() {
  const [runs, setRuns] = useState(0);
  const options = { page: 1 };                    // new object every render
  useEffect(() => {
    if (runs < 5) setRuns(r => r + 1);            // guard only so the demo stops
    console.log("effect ran, runs =", runs);
  }, [options]);                                  // always "changed"
  return <p>{runs}</p>;
}
```

BROWSER CONSOLE: AN OBJECT DEPENDENCY RE-RUNS THE EFFECT EVERY RENDER

```
effect ran, runs = 0
effect ran, runs = 1
effect ran, runs = 2
effect ran, runs = 3
effect ran, runs = 4
effect ran, runs = 5
```

9.5 useLayoutEffect (overview)

`useLayoutEffect` has the same signature as `useEffect` but runs **synchronously after the DOM is updated and before the browser paints**. Use it only when you must **measure** layout and change something before the user sees it, such as positioning a tooltip based on its own height, to avoid a visible flicker. Because it blocks painting, prefer `useEffect` in all other cases.

APP.JSX

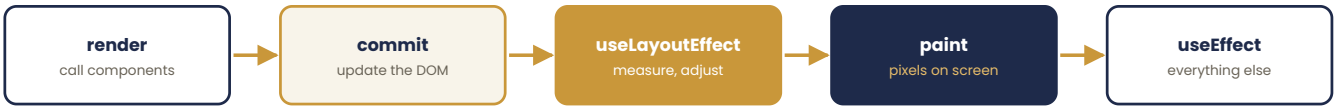
JavaScript

```
import { useEffect, useLayoutEffect } from "react";

export default function App() {
  useEffect(() => console.log("3. useEffect (after paint)"), []);
  useLayoutEffect(() => console.log("2. useLayoutEffect (before paint), height =",
document.querySelector("p").offsetHeight), []);
  console.log("1. render");
  return <p>Measure me</p>;
}
```

BROWSER CONSOLE: ORDER OF EXECUTION

1. render
3. useEffect (after paint)
2. useLayoutEffect (before paint), height = 24



WHERE EACH EFFECT RUNS

PRACTICE CHECKPOINT

A **user directory** that fetches users from an API with loading, error and retry states, a search box, and a **live clock** with proper cleanup.

USERDIRECTORY.JSX: SAMPLE SOLUTION (AS APP.JSX)

JavaScript

```
import { useState, useEffect } from "react";

function Clock() {
  const [now, setNow] = useState(() => new Date(2026, 9, 11, 9, 41, 7));     // fixed start for the
screenshot
  useEffect(() => {
    const id = setInterval(() => setNow(d => new Date(d.getTime() + 1000)), 1000);
    return () => clearInterval(id);                                          // cleanup
  }, []);
  return <span className="font-mono">{now.toLocaleTimeString("en-IN")}</span>;
}

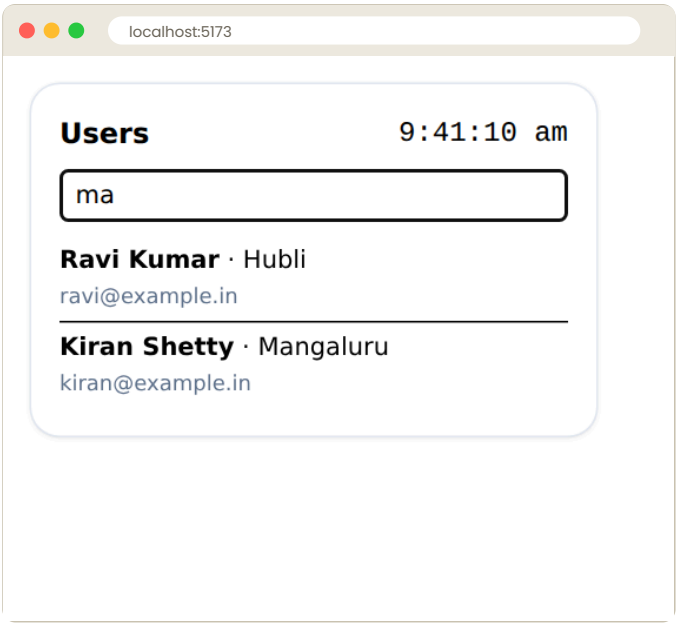
function useUsers(url, attempt) {
  const [state, setState] = useState({ status: "loading", users: [], error: null });
  useEffect(() => {
    const controller = new AbortController();
    setState(s => ({ ...s, status: "loading" }));
    fetch(url, { signal: controller.signal })
      .then(res => { if (!res.ok) throw new Error(`Server replied ${res.status}`); return res.json();
})
      .then(users => setState({ status: "success", users, error: null }))
      .catch(err => { if (err.name !== "AbortError") setState({ status: "error", users: [], error:
err.message }); });
    return () => controller.abort();
  }, [url, attempt]);                                          // retry = bump attempt
  return state;
}

export function UserDirectory({ url }) {
  const [attempt, setAttempt] = useState(0);
  const [query, setQuery] = useState("");
  const { status, users, error } = useUsers(url, attempt);
  const visible = users.filter(u => (u.name + u.city).toLowerCase().includes(query.toLowerCase()));
// derived

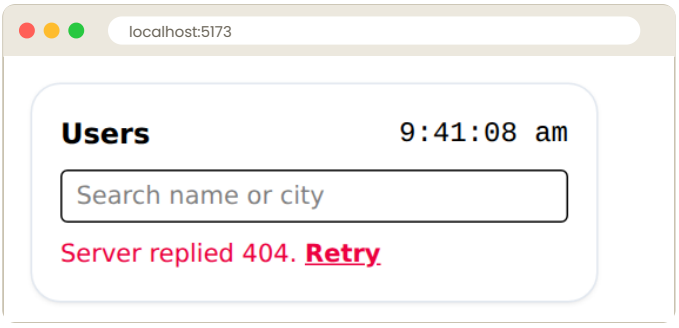
  return (
    <div className="w-80 rounded-2xl bg-white p-4 shadow ring-1 ring-slate-200">
      <div className="mb-2 flex items-center justify-between"><h2 className="font-bold">Users</h2>
<Clock /></div>
      <input value={query} onChange={e => setQuery(e.target.value)} placeholder="Search name or city"
             className="mb-2 w-full rounded border px-2 py-1 text-sm" disabled={status !== "success"}
/>
      {status === "loading" && <p className="text-sm text-slate-500">Loading...</p>}
```

```
      {status === "error" && (
        <p className="text-sm text-rose-600">{error}. <button onClick={() => setAttempt(a => a + 1)}
className="font-semibold underline">Retry</button></p>
      )}
      {status === "success" && (visible.length === 0
        ? <p className="text-sm text-slate-500">No users match "{query}".</p>
        : <ul className="divide-y text-sm">{visible.map(u => <li key={u.id} className="py-1"><b>
{u.name}</b> · {u.city}<br /><span className="text-xs text-slate-500">{u.email}</span></li>)}</ul>)}
    </div>
  );
}

export default function App() {
  return <UserDirectory url="users.json" />;
}
```



USERS LOADED OVER HTTP, FILTERED BY "MA" WHILE THE CLOCK TICKS



WITH A WRONG URL: THE ERROR STATE WITH A RETRY BUTTON (EACH RETRY BUMPS ATTEMPT AND RE-RUNS THE EFFECT)

Use `useEffect` to synchronise a component with external systems after rendering: list every reactive value in the dependency array, and return a cleanup that undoes the setup (clear timers, remove listeners, abort requests). StrictMode's extra setup-cleanup-setup checks your cleanup. Derive data during render and run interaction logic in event handlers; reach for an effect only when there is something external to sync with.

MODULE 10 · INTERMEDIATE

Styling React applications

Minimum duration **1 day** | Level **Intermediate**

React has no built-in styling system, so teams choose among several approaches. This module compares them with working examples: plain CSS files imported into components, inline style objects and conditional classes with clsx; CSS Modules for locally scoped class names; Tailwind CSS with React and its component patterns; CSS-in-JS libraries such as styled-components and Emotion; component libraries (shadcn/ui, Material UI, Chakra UI, Ant Design); and icons and animation. CSS files and CSS Modules were bundled with esbuild exactly as Vite does, and icons come from the real react-icons package.

IN THIS MODULE

10.1 Plain CSS and inline styles
10.2 CSS Modules
10.3 Tailwind CSS with React
10.4 CSS-in-JS (overview)
10.5 Component libraries
10.6 Icons and animation

10.1 Plain CSS and inline styles

10.1.1 Importing CSS files

With Vite (and any modern bundler) a component can `import "./Button.css"`. The bundler adds the stylesheet to the page; in production all imported CSS is combined, minified and loaded once. Use `className` in JSX. The catch: every class is **global**, so two components that both define `.title` affect each other. Teams prevent clashes with naming conventions such as BEM (`.card__title--active`).

CARD.CSS

CSS

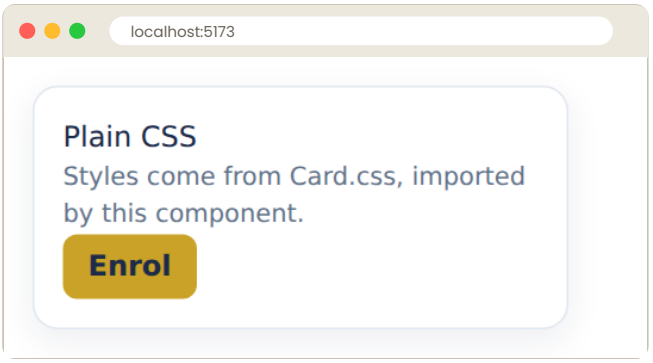
```
.card {
  max-width: 300px;
  padding: 16px;
  border-radius: 14px;
  background: #fff;
  border: 1px solid #e2e8f0;
  box-shadow: 0 6px 20px rgb(28 43 74 / 0.08);
  font-family: system-ui, sans-serif;
}
.card__title { margin: 0; color: #1c2b4a; }
.card__text { color: #64748b; font-size: 14px; }
.card__btn { border: 0; border-radius: 8px; padding: 6px 14px; font-weight: 600; }
.card__btn--primary { background: #c9a227; color: #1c2b4a; }
```

APP.JSX

JavaScript

```
import "./Card.css";

export default function App() {
  return (
    <div className="card">
      <h3 className="card__title">Plain CSS</h3>
      <p className="card__text">Styles come from Card.css, imported by this component.</p>
      <button className="card__btn card__btn--primary">Enrol</button>
    </div>
  );
}
```



THE IMPORTED STYLESHEET APPLIED TO THE COMPONENT

10.1.2 Inline style objects

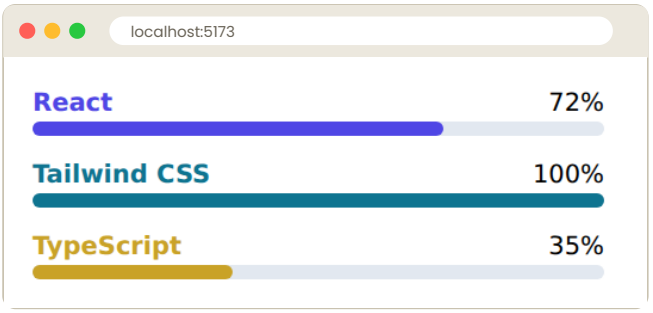
The `style` prop (Module 4) takes an object with camelCase properties. Inline styles are scoped to the element and can use any JavaScript value, which makes them right for **dynamic** values: positions, sizes and colours computed from data or user input. They cannot express hover, focus, media queries or pseudo-elements, and they are verbose, so they are rarely the main styling method. A good modern pattern combines classes with CSS custom properties set inline:

APP.JSX

JavaScript

```
const courses = [
  { name: "React", progress: 72, colour: "#4f46e5" },
  { name: "Tailwind CSS", progress: 100, colour: "#0e7490" },
  { name: "TypeScript", progress: 35, colour: "#c9a227" },
];

export default function App() {
  return (
    <div className="w-80 space-y-3">
      {courses.map(c => (
        <div key={c.name} style={{ "--accent": c.colour }}>
          <div className="flex justify-between text-sm"><b style={{ color: "var(--accent)" }}>{c.name}
</b>{c.progress}%</div>
          <div className="h-2 rounded-full bg-slate-200">
            <div className="h-2 rounded-full bg-(--accent)" style={{ width: `${c.progress}%` }} />
          </div>
        </div>
      ))}
    </div>
  );
}
```



INLINE VALUES FOR WHAT VARIES PER ITEM (COLOUR, WIDTH), CLASSES FOR EVERYTHING ELSE

10.1.3 Conditional classes with clsx

Joining class names with template literals and ternaries gets messy when several conditions apply. **clsx** (and the older **classnames**) builds a class string from strings, objects and arrays, dropping falsy values. The package is tiny; here a five-line equivalent is defined so the example runs without installing anything:

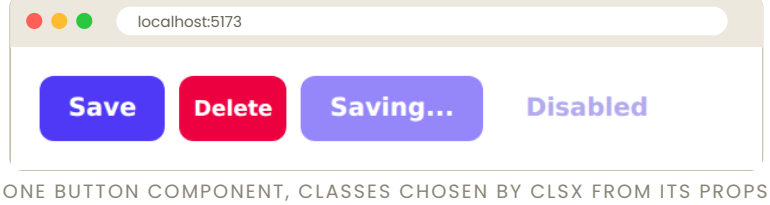
APP.JSX

JavaScript

```
// npm install clsx   →   import clsx from "clsx";   (same behaviour as this mini version)
function clsx(...args) {
  return args.flat(Infinity).flatMap(a =>
    !a ? [] : typeof a === "object" ? Object.keys(a).filter(k => a[k]) : [a]).join(" ");
}

function Button({ variant = "primary", size = "md", loading, disabled, children }) {
  return (
    <button disabled={disabled || loading}
      className={clsx(
        "rounded-lg font-semibold",
        { primary: "bg-indigo-600 text-white", danger: "bg-rose-600 text-white", ghost: "text-indigo-
700" }[variant],
        size === "sm" ? "px-2 py-1 text-xs" : "px-4 py-2 text-sm",
        { "cursor-wait opacity-60": loading, "opacity-40": disabled },
      )}>
      {loading ? "Saving..." : children}
    </button>
  );
}

export default function App() {
  console.log(clsx("a", { b: true, c: false }, ["d", null, ["e"]], 0 && "f"));
  return (
    <div className="flex gap-2">
      <Button>Save</Button>
      <Button variant="danger" size="sm">Delete</Button>
      <Button loading>Save</Button>
      <Button variant="ghost" disabled>Disabled</Button>
    </div>
  );
}
```



ONE BUTTON COMPONENT, CLASSES CHOSEN BY CLSX FROM ITS PROPS

BROWSER CONSOLE: WHAT clsx RETURNED

```
"a b d e"
```

10.2 CSS Modules

10.2.1 Locally scoped class names

A **CSS Module** is a CSS file named `*.module.css`. Importing it returns an object that maps your class names to **unique generated names**, so classes are scoped to the component that imports them and can never clash. You write normal CSS (including `:hover` and media queries) and use `styles.title` in JSX. Vite supports CSS Modules with no configuration.

BUTTON.MODULE.CSS CSS

```css
.button {
  border: 0;
  border-radius: 10px;
  padding: 8px 16px;
  font-weight: 600;
  background: #4f46e5;
  color: white;
}
.button:hover { background: #4338ca; }
.secondary {
  background: white;
  color: #1e293b;
  box-shadow: inset 0 0 0 1px #cbd5e1;
}
```

CARD.MODULE.CSS CSS

```css
.button {            /* same class name as in Button.module.css: no clash */
  font-size: 12px;
  color: #b45309;
  background: #fef3c7;
  border-radius: 999px;
  padding: 2px 10px;
}
```

BUTTON.JSX JavaScript

```jsx
import styles from "./Button.module.css";

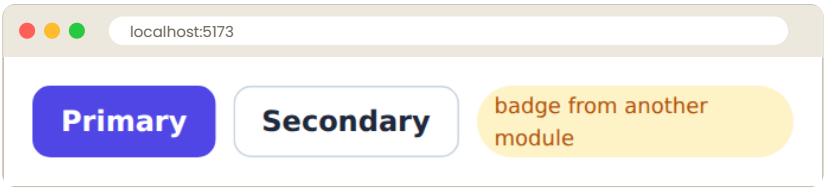
export default function Button({ secondary, children }) {
  const cls = secondary ? `${styles.button} ${styles.secondary}` : styles.button;
  return <button className={cls}>{children}</button>;
}
```

APP.JSX

JavaScript

```
import Button from "./Button.jsx";
import card from "./Card.module.css";

export default function App() {
  console.log("Card.module.css exports:", card);
  return (
    <div style={{ display: "flex", gap: 10, alignItems: "center" }}>
      <Button>Primary</Button>
      <Button secondary>Secondary</Button>
      <span className={card.button}>badge from another module</span>
    </div>
  );
}
```



TWO MODULES BOTH DEFINE .BUTTON; EACH COMPONENT GETS ITS OWN

BROWSER CONSOLE: THE GENERATED CLASS NAMES

```
Button.module.css: {"button":"Button_button","secondary":"Button_secondary"}
Card.module.css: {"button":"Card_button"}
```

RENDERED DOM

```
<button class="Button_button">Primary</button>
```

	Plain CSS	CSS Modules
File name	`Card.css`	`Card.module.css`
Import	`import "./Card.css"`	`import styles from "./Card.module.css"`
Use	`className="title"`	`className={styles.title}`
Scope	global	local to the importing component
Global escape hatch	n/a	`:global(.some-class) { }`

NOTE

esbuild (used to build these examples) names generated classes `Button_button`; Vite's default produces names such as `_button_1x2yz_1`. The principle is identical: a file-specific prefix or hash makes each class unique.

10.3 Tailwind CSS with React

10.3.1 Setup and component patterns

Tailwind CSS (covered in depth in the LBF Tailwind book) fits React naturally: utility classes go straight into `className`, and repetition is removed by **components**, not by CSS classes. Setup in a Vite project takes two steps:

Adding Tailwind CSS v4 to a Vite React project

```
$ npm install tailwindcss @tailwindcss/vite
# vite.config.js:  plugins: [react(), tailwindcss()]
# src/index.css:   @import "tailwindcss";
```

Component patterns that keep Tailwind code clean: keep variant classes in an object map (complete class names, so the scanner finds them), accept a `className` prop for one-off adjustments, and use `tailwind-merge` to resolve conflicts when merging (Tailwind book, Module 15).

APP.JSX

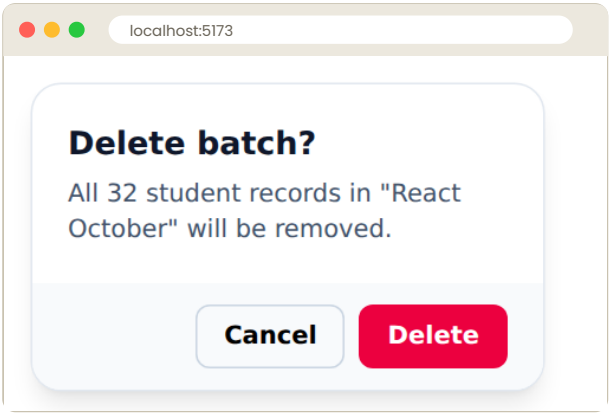
JavaScript

```
const VARIANTS = {
  primary: "bg-indigo-600 text-white hover:bg-indigo-500",
  outline: "ring-1 ring-inset ring-slate-300 hover:bg-slate-50",
  danger: "bg-rose-600 text-white hover:bg-rose-500",
};

function Button({ variant = "primary", className = "", ...props }) {
  return <button className={`rounded-lg px-4 py-2 text-sm font-semibold ${VARIANTS[variant]}
${className}`} {...props} />;
}

function Card({ title, children, footer }) {
  return (
    <article className="w-72 overflow-hidden rounded-2xl bg-white shadow-lg ring-1 ring-slate-200">
      <div className="p-5">
        <h3 className="text-lg font-bold text-slate-900">{title}</h3>
        <div className="mt-1 text-sm text-slate-600">{children}</div>
      </div>
      {footer && <div className="flex justify-end gap-2 bg-slate-50 px-5 py-3">{footer}</div>}
    </article>
  );
}

export default function App() {
  return (
    <Card title="Delete batch?" footer={<><Button variant="outline">Cancel</Button><Button
variant="danger">Delete</Button></>}>
      All 32 student records in "React October" will be removed.
    </Card>
  );
}
```



TAILWIND CLASSES INSIDE REUSABLE REACT COMPONENTS

10.4 CSS-in-JS (overview)

10.4.1 styled-components and Emotion

CSS-in-JS libraries let you write real CSS inside JavaScript, attached to components, with props available for dynamic styles. **styled-components** and **Emotion** are the best known. They generate unique class names at runtime and inject a style tag. Their downsides are a runtime cost and poor compatibility with React Server Components (styles must be collected on the server), so new projects increasingly prefer CSS Modules, Tailwind or zero-runtime libraries (vanilla-extract, Panda CSS, StyleX).

STYLED-COMPONENTS (REPRESENTATIVE: THE PACKAGE IS NOT INSTALLED IN THIS BUILD) JavaScript

```
import styled from "styled-components";

const Button = styled.button`
  padding: 8px 16px;
  border: 0;
  border-radius: 8px;
  font-weight: 600;
  color: white;
  background: ${props => (props.$danger ? "#e11d48" : "#4f46e5")};
  &:hover { opacity: 0.9; }
`;

export default function App() {
  return (
    <>
      <Button>Save</Button>
      <Button $danger>Delete</Button>
    </>
  );
}
```

Approach	Scoping	Dynamic styles	Runtime cost	Server Components
Plain CSS	global	classes / CSS variables	none	yes
CSS Modules	local	classes / CSS variables	none	yes
Tailwind CSS	utilities (no naming)	conditional classes	none	yes
styled-components / Emotion	local	props in CSS	yes	limited
Inline styles	element	any JS value	small	yes

10.5 Component libraries

Component libraries provide ready-made, accessible, themed components (buttons, dialogs, menus, date pickers, data tables), so teams build products faster. They differ in how much control they give over the look.

10.5.1 shadcn/ui

shadcn/ui copies component source code into your project (built on Radix UI primitives and Tailwind CSS), so you own and edit every line instead of installing a dependency. It is very popular for new React and Next.js projects (Tailwind book, Module 15).

SHADCN/UI (REPRESENTATIVE) JavaScript

```
npx shadcn@latest init
npx shadcn@latest add button dialog

import { Button } from "@/components/ui/button";
<Button variant="outline" size="sm">Cancel</Button>
```

10.5.2 Material UI

Material UI (MUI) implements Google's Material Design: a large, mature set of components with a theming system, used widely in enterprise dashboards. MUI X adds advanced data grids, date pickers and charts.

MATERIAL UI (REPRESENTATIVE) JavaScript

```
npm install @mui/material @emotion/react @emotion/styled

import Button from "@mui/material/Button";
import TextField from "@mui/material/TextField";

<TextField label="E-mail" variant="outlined" size="small" />
<Button variant="contained" color="primary">Enrol</Button>
```

10.5.3 Chakra UI and Ant Design (overview)

Library	Character	Good for
shadcn/ui	copied source, Radix + Tailwind, fully customisable	products with their own design
Material UI	Material Design, huge component set, themeable	enterprise apps, dashboards
Chakra UI	style props on components (`<Box p={4} bg="gray.50">`), accessible	fast prototyping, startups
Ant Design	enterprise design language from Ant Group, rich tables and forms	admin systems, data-heavy apps
Mantine, **HeroUI**	modern full-featured kits	teams wanting batteries included

10.6 Icons and animation

10.6.1 Icon libraries (Lucide, React Icons)

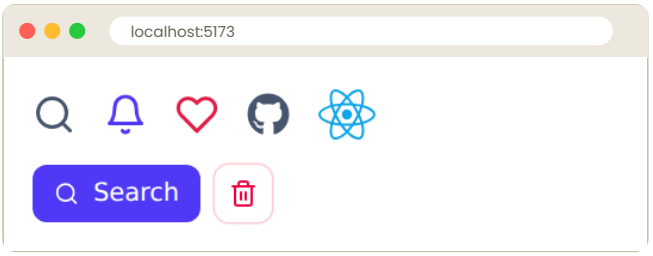
Icon packages export each icon as a React component that renders an inline SVG using `currentColor`, so it takes the text colour and can be sized with CSS or a `size` prop. **Lucide** (`lucide-react`) is a consistent modern set; **React Icons** (`react-icons`) bundles many popular sets (Font Awesome, Material, Heroicons, Feather…) under one API. Only the icons you import end up in the bundle. The example uses the real `react-icons` package:

APP.JSX

JavaScript

```
import { FiSearch, FiBell, FiHeart, FiTrash2 } from "react-icons/fi";     // Feather icons
import { FaGithub, FaReact } from "react-icons/fa";                       // Font Awesome

export default function App() {
  return (
    <div className="space-y-3">
      <div className="flex items-center gap-4 text-2xl text-slate-600">
        <FiSearch /><FiBell className="text-indigo-600" /><FiHeart color="#e11d48" /><FaGithub />
<FaReact size={32} color="#0ea5e9" />
      </div>
      <div className="flex gap-2">
        <button className="inline-flex items-center gap-2 rounded-lg bg-indigo-600 px-3 py-1.5 text-sm
text-white"><FiSearch aria-hidden="true" /> Search</button>
        <button aria-label="Delete" className="rounded-lg p-2 text-rose-600 ring-1 ring-rose-200">
<FiTrash2 /></button>
      </div>
    </div>
  );
}
```



ICONS FROM THE REACT-ICONS PACKAGE, COLOURED AND SIZED LIKE TEXT

TIP

Icon-only buttons need an accessible name (`aria-label`); decorative icons next to text should be hidden from screen readers with `aria-hidden="true"` .

10.6.2 Animations with Motion (Framer Motion)

Simple transitions are best done with CSS (Tailwind's `transition` utilities, keyframes). For animations tied to React state, such as elements animating **out** when they are removed, layout animations when a list reorders, gestures and springs, use **Motion** (formerly Framer Motion, package `motion`). You replace an element with its `motion.` version and describe states; `AnimatePresence` keeps removed elements mounted until their exit animation finishes.

MOTION (REPRESENTATIVE: THE PACKAGE IS NOT INSTALLED IN THIS BUILD) JavaScript

```
import { motion, AnimatePresence } from "motion/react";

function TodoList({ todos }) {
  return (
    <ul>
      <AnimatePresence>
        {todos.map(t => (
          <motion.li key={t.id} layout
            initial={{ opacity: 0, y: -8 }}
            animate={{ opacity: 1, y: 0 }}
            exit={{ opacity: 0, x: 40 }}
            transition={{ duration: 0.2 }}>
            {t.text}
          </motion.li>
        ))}
      </AnimatePresence>
    </ul>
  );
}
```

Without a library, CSS can animate elements **in** when they mount. The following runs for real: each new item fades and slides in using a keyframe animation in an imported stylesheet.

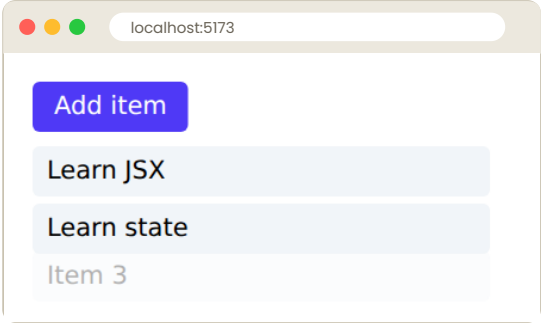
ANIM.CSS CSS

```
@keyframes fade-in {
  from { opacity: 0; transform: translateY(-8px); }
  to { opacity: 1; transform: none; }
}
.fade-in { animation: fade-in 600ms ease-out both; }
```

APP.JSX JavaScript

```
import { useState } from "react";
import "./anim.css";

export default function App() {
  const [items, setItems] = useState(["Learn JSX", "Learn state"]);
  return (
    <div className="w-64 space-y-2 text-sm">
      <button onClick={() => setItems([...items, `Item ${items.length + 1}`])} className="rounded bg-
indigo-600 px-3 py-1 text-white">Add item</button>
      <ul className="space-y-1">{items.map(i => <li key={i} className="fade-in rounded bg-slate-100
px-2 py-1">{i}</li>)}</ul>
    </div>
  );
}
```



THE NEW ITEM PAUSED 120 MS INTO ITS 600 MS ENTRY ANIMATION (THE OLDER ITEMS HAVE FINISHED THEIRS)

PRACTICE CHECKPOINT

Style the to-do app twice, once with **CSS Modules** and once with **Tailwind CSS**, and add an **animated list** with Motion.

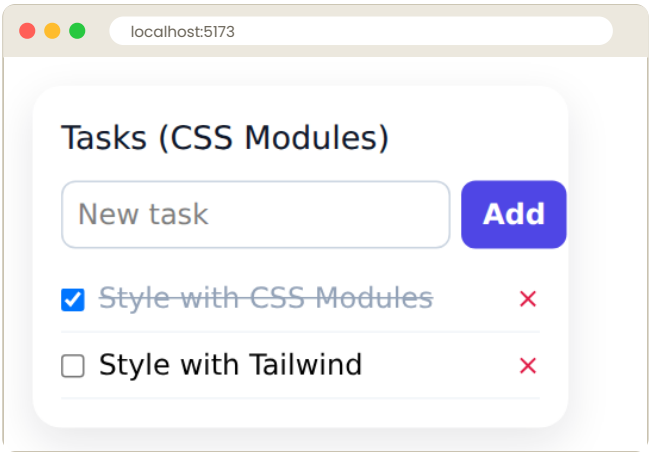
TODO.MODULE.CSS CSS

```css
.app { width: 300px; padding: 16px; border-radius: 16px; background: #fff; box-shadow: 0 8px 24px rgb(15 23 42 / 0.1); font-family: system-ui, sans-serif; }
.title { margin: 0 0 10px; font-size: 18px; color: #0f172a; }
.form { display: flex; gap: 6px; margin-bottom: 10px; }
.input { flex: 1; padding: 6px 8px; border: 1px solid #cbd5e1; border-radius: 8px; }
.add { padding: 0 12px; border: 0; border-radius: 8px; background: #4f46e5; color: #fff; font-weight: 600; }
.list { list-style: none; margin: 0; padding: 0; }
.item { display: flex; align-items: center; gap: 8px; padding: 6px 0; border-bottom: 1px solid #f1f5f9; animation: slide 300ms ease-out both; }
.done { color: #94a3b8; text-decoration: line-through; }
.delete { margin-left: auto; border: 0; background: none; color: #e11d48; font-size: 16px; cursor: pointer; }
@keyframes slide { from { opacity: 0; transform: translateX(-10px); } to { opacity: 1; transform: none; } }
```

TODOCSSMODULES.JSX: SAMPLE SOLUTION (AS APP.JSX) JavaScript

```jsx
import { useState } from "react";
import s from "./Todo.module.css";

export default function App() {
  const [todos, setTodos] = useState([{ id: 1, text: "Style with CSS Modules", done: true }, { id: 2, text: "Style with Tailwind", done: false }]);
  const [text, setText] = useState("");
  const add = e => { e.preventDefault(); if (text.trim()) { setTodos([...todos, { id: Date.now(), text, done: false }]); setText(""); } };
  const toggle = id => setTodos(todos.map(t => (t.id === id ? { ...t, done: !t.done } : t)));
  const remove = id => setTodos(todos.filter(t => t.id !== id));
  return (
    <div className={s.app}>
      <h2 className={s.title}>Tasks (CSS Modules)</h2>
      <form onSubmit={add} className={s.form}>
        <input value={text} onChange={e => setText(e.target.value)} className={s.input} placeholder="New task" />
        <button className={s.add}>Add</button>
      </form>
      <ul className={s.list}>
        {todos.map(t => (
          <li key={t.id} className={s.item}>
            <input type="checkbox" checked={t.done} onChange={() => toggle(t.id)} />
            <span className={t.done ? s.done : undefined}>{t.text}</span>
            <button onClick={() => remove(t.id)} className={s.delete} aria-label="Delete">&times;</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```



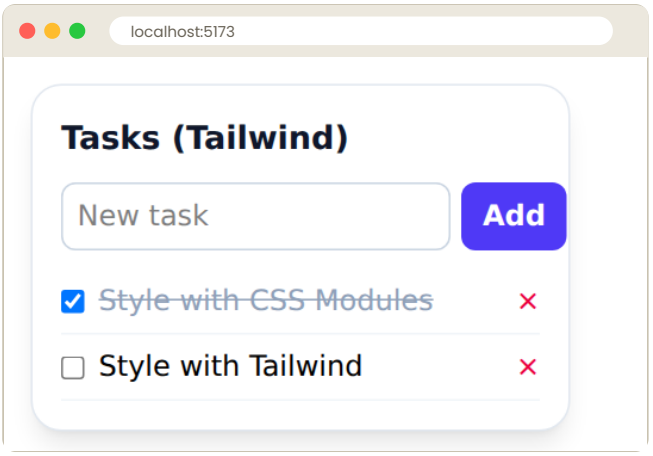
VERSION 1: CSS MODULES

TODOTAILWIND.JSX: SAMPLE SOLUTION (AS APP.JSX)

JavaScript

```
import { useState } from "react";

export default function App() {
  const [todos, setTodos] = useState([{ id: 1, text: "Style with CSS Modules", done: true }, { id: 2,
text: "Style with Tailwind", done: false }]);
  const [text, setText] = useState("");
  const add = e => { e.preventDefault(); if (text.trim()) { setTodos([...todos, { id: Date.now(),
text, done: false }]); setText(""); } };
  const toggle = id => setTodos(todos.map(t => (t.id === id ? { ...t, done: !t.done } : t)));
  const remove = id => setTodos(todos.filter(t => t.id !== id));
  return (
    <div className="w-[300px] rounded-2xl bg-white p-4 shadow-lg ring-1 ring-slate-200">
      <h2 className="mb-2.5 text-lg font-bold text-slate-900">Tasks (Tailwind)</h2>
      <form onSubmit={add} className="mb-2.5 flex gap-1.5">
        <input value={text} onChange={e => setText(e.target.value)} className="flex-1 rounded-lg
border border-slate-300 px-2 py-1.5" placeholder="New task" />
        <button className="rounded-lg bg-indigo-600 px-3 font-semibold text-white">Add</button>
      </form>
      <ul>
        {todos.map(t => (
          <li key={t.id} className="flex items-center gap-2 border-b border-slate-100 py-1.5
starting:opacity-0 starting:-translate-x-2 transition duration-300">
            <input type="checkbox" checked={t.done} onChange={() => toggle(t.id)} />
            <span className={t.done ? "text-slate-400 line-through" : ""}>{t.text}</span>
            <button onClick={() => remove(t.id)} className="ml-auto text-rose-600" aria-
label="Delete">&times;</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```



VERSION 2: TAILWIND CSS, WITH AN ENTRY TRANSITION USING THE STARTING: VARIANT

For the Motion version, wrap the list in `<AnimatePresence>` and turn each `<li>` into `<motion.li key={t.id} layout initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0, x: 40 }}>` as shown in Section 10.6.2; deleted tasks then slide out instead of disappearing.

Choose one main styling approach per project: CSS Modules for scoped hand-written CSS, Tailwind for utility classes inside components, or a component library for speed. Use inline styles and CSS variables for truly dynamic values, clsx for conditional class lists, an icon package for SVG icons, and Motion when animations must follow React state (especially exits).

MODULE 11 · INTERMEDIATE

Hooks in depth and custom hooks

Minimum duration **3 days** | Level **Intermediate**

Beyond useState and useEffect, React provides hooks for references, complex state, performance and concurrent rendering, and lets you build your own. This module covers useRef for DOM access and for mutable values that do not re-render, refs versus state, passing ref as a prop (React 19) and useImperativeHandle; useReducer with reducers and actions, and when to prefer it; useMemo and useCallback and when memoisation helps; useId, useTransition and useDeferredValue, useSyncExternalStore and useEffectEvent (stable in React 19.2); and custom hooks: extracting logic, naming rules, and six practical examples. Every hook was run in React 19.2 in Chromium.

IN THIS MODULE

11.1 useRef
11.2 useReducer
11.3 Performance hooks
11.4 Other hooks
11.5 Custom hooks

11.1 useRef

11.1.1 Accessing DOM elements (focus, scroll, measure)

`useRef(initial)` returns a plain object `{ current: initial }` that stays the same for the whole life of the component. Pass it to an element's `ref` attribute and React sets `ref.current` to the DOM node after it is mounted. You then call DOM methods such as `focus()`, `scrollIntoView()` or `getBoundingClientRect()`, typically in event handlers or effects (never while rendering).

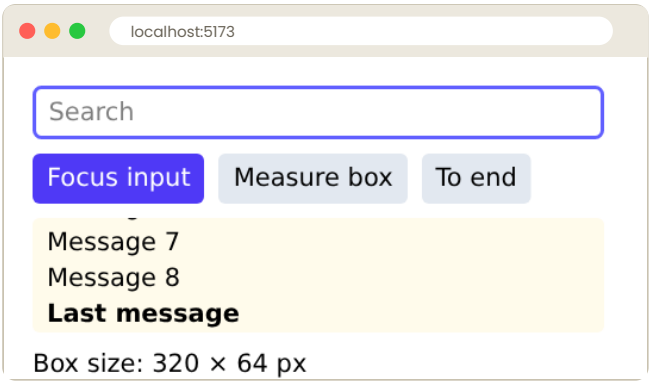
APP.JSX

JavaScript

```
import { useRef, useState } from "react";

export default function App() {
  const inputRef = useRef(null);
  const boxRef = useRef(null);
  const endRef = useRef(null);
  const [size, setSize] = useState("");

  return (
    <div className="w-80 space-y-2 text-sm">
      <input ref={inputRef} placeholder="Search" className="w-full rounded border px-2 py-1
focus:outline-indigo-500" />
      <div className="flex gap-2">
        <button onClick={() => inputRef.current.focus()} className="rounded bg-indigo-600 px-2 py-1
text-white">Focus input</button>
        <button onClick={() => { const r = boxRef.current.getBoundingClientRect();
setSize(`${Math.round(r.width)} × ${Math.round(r.height)} px`); }}
                className="rounded bg-slate-200 px-2 py-1">Measure box</button>
        <button onClick={() => endRef.current.scrollIntoView({ block: "nearest" })} className="rounded
bg-slate-200 px-2 py-1">To end</button>
      </div>
      <div ref={boxRef} className="h-16 overflow-auto rounded bg-amber-50 p-2">
        {Array.from({ length: 8 }, (_, i) => <p key={i}>Message {i + 1}</p>)}
        <p ref={endRef} className="font-semibold">Last message</p>
      </div>
      <p>Box size: {size}</p>
    </div>
  );
}
```



AFTER CLICKING ALL THREE BUTTONS: INPUT FOCUSED (RING), BOX MEASURED, LIST SCROLLED TO THE LAST MESSAGE

11.1.2 Storing mutable values that do not trigger re-renders

A ref is also a box for **any** value that must survive re-renders but should **not** cause one when it changes: a timer id, the previous value of a prop, a flag, a count of renders. Changing `ref.current` is instant and silent.

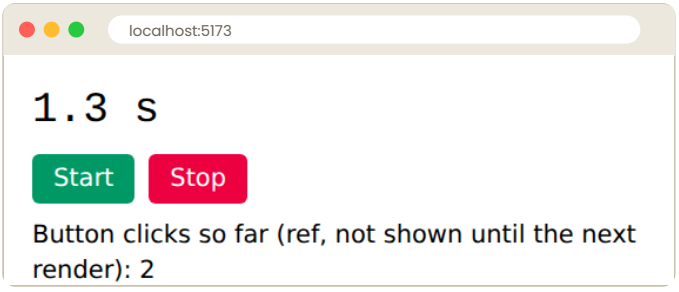
APP.JSX

JavaScript

```
import { useRef, useState } from "react";

export default function App() {
  const [elapsed, setElapsed] = useState(0);
  const timerRef = useRef(null);            // the interval id: needed later, never displayed
  const clicksRef = useRef(0);              // counted silently

  function start() {
    if (timerRef.current) return;           // already running
    timerRef.current = setInterval(() => setElapsed(e => e + 1), 100);
  }
  function stop() {
    clearInterval(timerRef.current);
    timerRef.current = null;
  }
  return (
    <div className="space-y-2 text-sm">
      <p className="font-mono text-2xl">{(elapsed / 10).toFixed(1)} s</p>
      <div className="flex gap-2">
        <button onClick={() => { clicksRef.current++; start(); }} className="rounded bg-emerald-600
px-3 py-1 text-white">Start</button>
        <button onClick={() => { clicksRef.current++; stop(); }} className="rounded bg-rose-600 px-3
py-1 text-white">Stop</button>
      </div>
      <p>Button clicks so far (ref, not shown until the next render): {clicksRef.current}</p>
    </div>
  );
}
```



START, START AGAIN (IGNORED), THEN STOP AFTER ABOUT A SECOND

11.1.3 Refs vs state

	State (useState)	Ref (useRef)
Changing it	re-renders the component	does not re-render
Read during render	yes (that is its purpose)	should not (value may be stale; StrictMode-unsafe)
Updates	asynchronous snapshot per render	mutable, immediate
Use for	anything shown on screen	DOM nodes, timer ids, previous values, instance variables
Returns	`[value, setValue]`	`{ current: value }`

APP.JSX

JavaScript

```
import { useRef, useState } from "react";

export default function App() {
  const [stateCount, setStateCount] = useState(0);
  const refCount = useRef(0);
  console.log("render: state =", stateCount, "| ref =", refCount.current);
  return (
    <>
      <button id="r" onClick={() => { refCount.current++; console.log("  ref is now",
refCount.current, "(no render)"); }}>ref +1</button>
      <button id="s" onClick={() => setStateCount(stateCount + 1)}>state +1</button>
    </>
  );
}
```

BROWSER CONSOLE

```
render: state = 0 | ref = 0
  ref is now 1 (no render)
  ref is now 2 (no render)
--- now a state update ---
render: state = 1 | ref = 2
```

11.1.4 Passing ref as a prop to components (React 19)

A `ref` on a **custom component** needs that component to put it on one of its DOM elements. In React 19, `ref` is simply a **prop**: destructure it and pass it on. (Before React 19 this required wrapping the component in `forwardRef`, which still works but is no longer needed.)

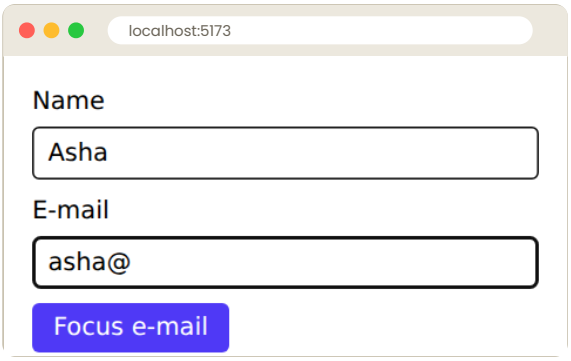
APP.JSX

JavaScript

```
import { useRef } from "react";

function TextInput({ label, ref, ...props }) {            // React 19: ref is a normal prop
  return (
    <label className="block text-sm">
      {label}
      <input ref={ref} {...props} className="mt-1 block w-full rounded border px-2 py-1" />
    </label>
  );
}

export default function App() {
  const emailRef = useRef(null);
  return (
    <form className="w-72 space-y-2" onSubmit={e => e.preventDefault()}>
      <TextInput label="Name" defaultValue="Asha" />
      <TextInput label="E-mail" ref={emailRef} placeholder="you@example.in" />
      <button type="button" onClick={() => { emailRef.current.focus(); emailRef.current.value =
"asha@"; }}
              className="rounded bg-indigo-600 px-3 py-1 text-sm text-white">Focus e-mail</button>
    </form>
  );
}
```



THE PARENT FOCUSED AN INPUT THAT LIVES INSIDE THE CHILD COMPONENT

THE LEGACY FORWARDREF FORM (FOR OLDER CODE BASES)

JavaScript

```
import { forwardRef } from "react";                // React 18 and earlier
const TextInput = forwardRef(function TextInput({ label, ...props }, ref) {
  return <input ref={ref} {...props} />;
});
```

11.1.5 useImperativeHandle (overview)

Sometimes a component should expose a small **custom API** rather than its whole DOM node, for example a video player offering only `play()` and `pause()`. `useImperativeHandle(ref, () => ({ ... }))` defines what the parent's ref receives. Use it rarely: most things should be props.

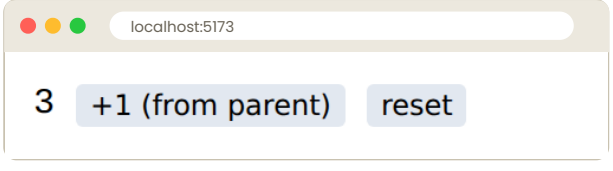
APP.JSX

JavaScript

```
import { useRef, useImperativeHandle, useState } from "react";

function Counter({ ref }) {
  const [n, setN] = useState(0);
  useImperativeHandle(ref, () => ({          // the only things the parent can call
    increment: () => setN(x => x + 1),
    reset: () => setN(0),
  }), []);
  return <span className="font-mono text-xl">{n}</span>;
}

export default function App() {
  const counterRef = useRef(null);
  return (
    <div className="flex items-center gap-3">
      <Counter ref={counterRef} />
      <button onClick={() => counterRef.current.increment()} className="rounded bg-slate-200 px-2">+1
(from parent)</button>
      <button onClick={() => counterRef.current.reset()} className="rounded bg-slate-200 px-
2">reset</button>
    </div>
  );
}
```

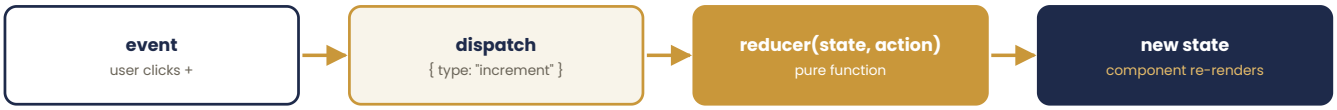


AFTER THREE INCREMENTS CALLED THROUGH THE HANDLE

11.2 useReducer

11.2.1 Reducers and actions

`useReducer` moves state update logic out of event handlers into one pure function, the **reducer**: `(state, action) => newState`. Components describe **what happened** by dispatching an **action** object (by convention with a `type`); the reducer decides **how state changes**. It returns `[state, dispatch]`.



THE USEREDUCER CYCLE

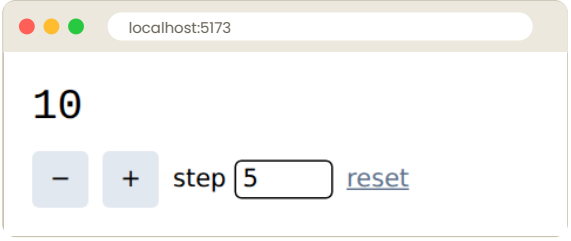
APP.JSX

JavaScript

```
import { useReducer } from "react";

function counterReducer(state, action) {
  switch (action.type) {
    case "increment": return { ...state, count: state.count + state.step };
    case "decrement": return { ...state, count: state.count - state.step };
    case "setStep":   return { ...state, step: action.step };
    case "reset":     return { count: 0, step: 1 };
    default: throw new Error("Unknown action: " + action.type);
  }
}

export default function App() {
  const [state, dispatch] = useReducer(counterReducer, { count: 0, step: 1 });
  return (
    <div className="space-y-2 text-sm">
      <p className="font-mono text-2xl">{state.count}</p>
      <div className="flex items-center gap-2">
        <button onClick={() => dispatch({ type: "decrement" })} className="size-8 rounded bg-slate-
200">−</button>
        <button onClick={() => dispatch({ type: "increment" })} className="size-8 rounded bg-slate-
200">+</button>
        <label>step <input type="number" value={state.step} min={1} onChange={e => dispatch({ type:
"setStep", step: Number(e.target.value) })} className="w-14 rounded border px-1" /></label>
        <button onClick={() => dispatch({ type: "reset" })} className="text-slate-500
underline">reset</button>
      </div>
    </div>
  );
}
```



STEP SET TO 5, THEN + THREE TIMES AND − ONCE

Because a reducer is a pure function, it can be tested without React at all:

REDUCER.TEST.JS (RUN WITH NODE.JS) main.js

```
function counterReducer(state, action) {
  switch (action.type) {
    case "increment": return { ...state, count: state.count + state.step };
    case "setStep":   return { ...state, step: action.step };
    default: throw new Error("Unknown action: " + action.type);
  }
}
let s = { count: 0, step: 1 };
s = counterReducer(s, { type: "setStep", step: 10 });
s = counterReducer(s, { type: "increment" });
s = counterReducer(s, { type: "increment" });
console.log(s);
try { counterReducer(s, { type: "oops" }); } catch (e) { console.log(e.message); }
```

OUTPUT

```
{ count: 20, step: 10 }
Unknown action: oops
```

11.2.2 When to choose useReducer over useState

Prefer useState when	Prefer useReducer when
state is a few independent values	several values change together, or the next state depends on several of them
updates are simple (set, toggle, increment)	there are many kinds of update (add, remove, edit, undo, reset…)
logic fits in one handler	the same logic is spread across many handlers
	you want to test update logic separately, or log every action for debugging

Both are equally capable; `useReducer` is a code organisation choice. It is also the foundation of Redux (Module 16): same reducer and action pattern, but app-wide.

11.2.3 Managing complex state logic

A task manager with several operations shows the benefit: every way the list can change is in one place, handlers only dispatch, and an action log can be added in one line for debugging.

APP.JSX

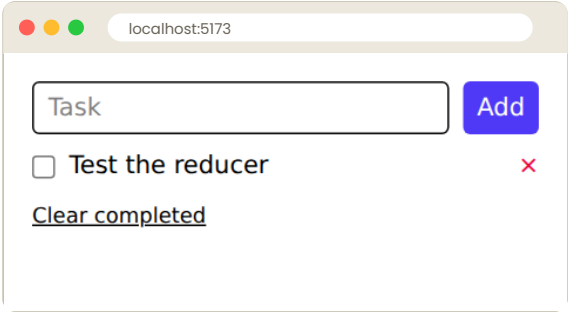
JavaScript

```
import { useReducer, useState } from "react";

function tasksReducer(tasks, action) {
  switch (action.type) {
    case "added":   return [...tasks, { id: action.id, text: action.text, done: false }];
    case "toggled": return tasks.map(t => (t.id === action.id ? { ...t, done: !t.done } : t));
    case "edited":  return tasks.map(t => (t.id === action.id ? { ...t, text: action.text } : t));
    case "deleted": return tasks.filter(t => t.id !== action.id);
    case "cleared": return tasks.filter(t => !t.done);
    default: throw new Error("Unknown action " + action.type);
  }
}

function loggingReducer(state, action) {           // wraps the reducer to log every action
  const next = tasksReducer(state, action);
  console.log(action.type, JSON.stringify(action), "→", next.length, "tasks");
  return next;
}

let nextId = 3;
export default function App() {
  const [tasks, dispatch] = useReducer(loggingReducer, [{ id: 1, text: "Learn reducers", done: true },
{ id: 2, text: "Write actions", done: false }]);
  const [text, setText] = useState("");
  return (
    <div className="w-72 space-y-2 text-sm">
      <form className="flex gap-2" onSubmit={e => { e.preventDefault(); dispatch({ type: "added", id:
nextId++, text }); setText(""); }}>
        <input value={text} onChange={e => setText(e.target.value)} className="flex-1 rounded border
px-2 py-1" placeholder="Task" />
        <button className="rounded bg-indigo-600 px-2 text-white">Add</button>
      </form>
      {tasks.map(t => (
        <div key={t.id} className="flex items-center gap-2">
          <input type="checkbox" checked={t.done} onChange={() => dispatch({ type: "toggled", id: t.id
})} />
          <span className={t.done ? "flex-1 text-slate-400 line-through" : "flex-1"}>{t.text}</span>
          <button onClick={() => dispatch({ type: "deleted", id: t.id })} className="text-rose-
600">&times;</button>
        </div>
      ))}
      <button onClick={() => dispatch({ type: "cleared" })} className="text-xs underline">Clear
completed</button>
    </div>
  );
}
```



AFTER ADDING A TASK, TICKING "WRITE ACTIONS" AND CLEARING COMPLETED

BROWSER CONSOLE: THE ACTION LOG

```
added {"type":"added","id":3,"text":"Test the reducer"} → 3 tasks
toggled {"type":"toggled","id":2} → 3 tasks
cleared {"type":"cleared"} → 1 tasks
```

11.3 Performance hooks

11.3.1 useMemo for expensive calculations

`useMemo(calculate, deps)` caches the result of a calculation between renders and recomputes it only when a dependency changes. Use it for genuinely **expensive** work (filtering or sorting thousands of items, heavy maths), or to keep an object's identity stable when it is passed to a memoised child or used as an effect dependency.

APP.JSX — JavaScript

```javascript
import { useMemo, useState } from "react";

const numbers = Array.from({ length: 200000 }, (_, i) => (i * 7919) % 100003);

function slowStats(list, min) {
  const t = performance.now();
  const filtered = list.filter(n => n >= min);
  const sum = filtered.reduce((a, b) => a + b, 0);
  console.log(`  computed stats for min=${min} (${(performance.now() - t) > 0 ? "work done" : ""})`);
  return { count: filtered.length, sum };
}

export default function App() {
  const [min, setMin] = useState(50000);
  const [dark, setDark] = useState(false);
  const stats = useMemo(() => slowStats(numbers, min), [min]);   // only when min changes
  console.log("render, dark =", dark, "| count =", stats.count);
  return (
    <div>
      <button id="theme" onClick={() => setDark(d => !d)}>toggle theme</button>
      <button id="min" onClick={() => setMin(m => m + 10000)}>raise min</button>
    </div>
  );
}
```

BROWSER CONSOLE

```
  computed stats for min=50000 (work done)
render, dark = false | count = 100000
--- toggle theme (unrelated) ---
render, dark = true | count = 100000
--- raise min ---
  computed stats for min=60000 (work done)
render, dark = true | count = 80001
```

Toggling the theme re-rendered the component but reused the cached stats; changing `min` recomputed them.

11.3.2 useCallback for stable function references

Every render creates new function objects for handlers. Usually that is fine. It matters when the function is passed to a child wrapped in `memo` (which skips re-rendering when props are unchanged) or used as an

effect dependency: a new function each time defeats both. `useCallback(fn, deps)` returns the **same function** until a dependency changes. It is exactly `useMemo(() => fn, deps)`.

APP.JSX

JavaScript

```
import { memo, useCallback, useState } from "react";

const Button = memo(function Button({ onClick, label }) {
  console.log("  render Button:", label);
  return <button onClick={onClick}>{label}</button>;
});

export default function App() {
  const [count, setCount] = useState(0);
  const [other, setOther] = useState(0);
  const inline = () => setCount(c => c + 1);                    // new every render
  const stable = useCallback(() => setCount(c => c + 1), []);   // same every render
  console.log("render App");
  return (
    <>
      <Button onClick={inline} label="inline handler" />
      <Button onClick={stable} label="useCallback handler" />
      <button id="other" onClick={() => setOther(o => o + 1)}>unrelated update</button>
    </>
  );
}
```

BROWSER CONSOLE

```
render App
  render Button: inline handler
  render Button: useCallback handler
--- unrelated update ---
render App
  render Button: inline handler
```

11.3.3 When memoisation helps and when it does not

Helps	Does not help (just adds code)
an expensive calculation that runs on many renders	cheap calculations (string joins, small arrays)
a value or function passed to a `memo` child	props passed to normal (non-memo) components, which re-render anyway
an object or function used as an effect dependency	values that change on every render anyway (deps always differ)
measured slowness in the Profiler	guesswork; memoise after measuring

NOTE

The **React Compiler** (Module 18) inserts this memoisation automatically at build time. In projects that enable it, manual `useMemo`, `useCallback` and `memo` are rarely needed. Learn them anyway: you will read them in existing code and need them where the compiler is not used.

11.4 Other hooks

11.4.1 useId

`useId()` returns a unique id string, stable for the life of the component and identical in server and client rendering. Use it to connect labels, descriptions and ARIA attributes in reusable components (Module 8). Do not use it for list keys; keys come from data.

BROWSER CONSOLE

```
Name → id _r_0_
E-mail → id _r_1_
Phone → id _r_2_
```

11.4.2 useTransition and useDeferredValue (introduced; detailed in Module 17)

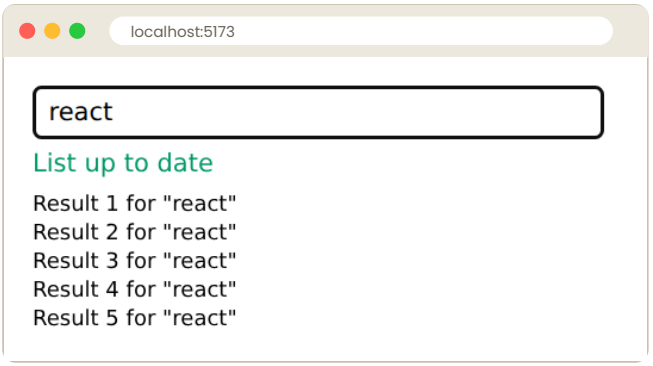
Some updates are **urgent** (typing must feel instant) and some are not (re-rendering a long filtered list). `useTransition` marks a state update as a non-urgent **transition** that React may interrupt to keep the UI responsive, and provides an `isPending` flag. `useDeferredValue(value)` gives a copy of a value that "lags behind" during heavy renders, so the urgent part (the input) updates first.

APP.JSX JavaScript

```
import { useDeferredValue, useState, memo } from "react";

const SlowList = memo(function SlowList({ text }) {
  const items = [];
  for (let i = 0; i < 200; i++) {
    const start = performance.now();
    while (performance.now() - start < 0.3) {}                // simulate a slow item (0.3 ms each)
    items.push(<li key={i}>Result {i + 1} for "{text}"</li>);
  }
  return <ul className="h-20 overflow-hidden text-xs">{items}</ul>;
});

export default function App() {
  const [text, setText] = useState("");
  const deferred = useDeferredValue(text);                    // may lag behind text
  return (
    <div className="w-80 space-y-1 text-sm">
      <input value={text} onChange={e => setText(e.target.value)} placeholder="Type fast..."
className="w-full rounded border px-2 py-1" />
      <p className={text !== deferred ? "text-amber-600" : "text-emerald-600"}>{text !== deferred ?
"Updating list..." : "List up to date"}</p>
      <SlowList text={deferred} />
    </div>
  );
}
```



AFTER TYPING "REACT": THE INPUT UPDATED IMMEDIATELY AND THE SLOW LIST CAUGHT UP AFTERWARDS

11.4.3 useSyncExternalStore (overview)

`useSyncExternalStore(subscribe, getSnapshot)` subscribes a component to a store that lives **outside React** (a browser API, a third-party state library) safely under concurrent rendering. State libraries such as Redux and Zustand use it internally. A typical direct use is reading a browser value that changes over time:

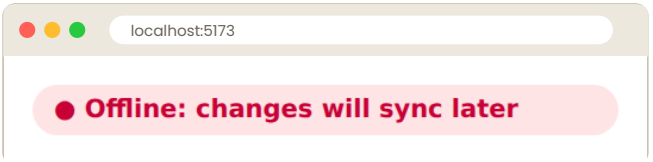
APP.JSX

JavaScript

```
import { useSyncExternalStore } from "react";

function subscribe(callback) {                    // how React listens for changes
  window.addEventListener("online", callback);
  window.addEventListener("offline", callback);
  return () => {
    window.removeEventListener("online", callback);
    window.removeEventListener("offline", callback);
  };
}

export default function App() {
  const isOnline = useSyncExternalStore(subscribe, () => navigator.onLine);
  return (
    <p className={`rounded-full px-3 py-1 text-sm font-semibold ${isOnline ? "bg-emerald-100 text-
emerald-700" : "bg-rose-100 text-rose-700"}`}>
      {isOnline ? "● Online" : "● Offline: changes will sync later"}
    </p>
  );
}
```



CHROMIUM SWITCHED TO OFFLINE MODE: THE COMPONENT UPDATED THROUGH THE SUBSCRIPTION

11.4.4 useEffectEvent (overview)

Sometimes an effect needs to read the **latest** value of something without **re-running** when it changes. Example: a chat room effect should reconnect when `roomId` changes, but not when the notification `theme` changes, yet the "connected" message must use the current theme. `useEffectEvent` (stable since React 19.2) wraps such logic in an **effect event**: it always sees the latest props and state, and is not a dependency.

APP.JSX

JavaScript

```
import { useEffect, useEffectEvent, useState } from "react";

function ChatRoom({ roomId, theme }) {
  const onConnected = useEffectEvent(() => {
    console.log(`  connected to ${roomId}, notify with ${theme} theme`);   // always the latest theme
  });
  useEffect(() => {
    console.log("connect", roomId);
    const t = setTimeout(onConnected, 50);           // e.g. when the connection opens
    return () => { clearTimeout(t); console.log("disconnect", roomId); };
  }, [roomId]);                                      // theme is NOT a dependency
  return <p>{roomId}</p>;
}

export default function App() {
  const [room, setRoom] = useState("general");
  const [theme, setTheme] = useState("light");
  return (
    <>
      <button id="theme" onClick={() => setTheme("dark")}>dark theme</button>
      <button id="room" onClick={() => setRoom("travel")}>travel room</button>
      <ChatRoom roomId={room} theme={theme} />
    </>
  );
}
```

BROWSER CONSOLE

```
connect general
  connected to general, notify with light theme
--- change theme: no reconnect ---
--- change room ---
disconnect general
connect travel
  connected to travel, notify with dark theme
```

11.5 Custom hooks

11.5.1 Extracting reusable logic

When two components need the same **stateful logic** (not the same UI), extract it into a **custom hook**: a function whose name starts with `use` and that calls other hooks. Each component that calls a custom hook gets its **own independent state**; hooks share logic, not state.

APP.JSX

JavaScript

```
import { useState } from "react";

function useCounter(initial = 0, step = 1) {              // a custom hook
  const [count, setCount] = useState(initial);
  const increment = () => setCount(c => c + step);
  const reset = () => setCount(initial);
  return { count, increment, reset };
}

function Likes() {
  const { count, increment } = useCounter(0);
  return <button onClick={increment} className="rounded bg-rose-50 px-3 py-1 ring-1 ring-rose-200">♥
{count}</button>;
}

function Score() {
  const { count, increment, reset } = useCounter(100, 10);
  return <span className="space-x-2"><b>Score {count}</b><button onClick={increment}
className="rounded bg-slate-200 px-2">+10</button><button onClick={reset} className="text-xs
underline">reset</button></span>;
}

export default function App() {
  return <div className="flex items-center gap-6"><Likes /><Likes /><Score /></div>;
}
```



THREE COMPONENTS, ONE HOOK, INDEPENDENT STATE (FIRST LIKES CLICKED TWICE, SCORE ONCE)

11.5.2 Naming rules (use prefix)

- A custom hook's name **must** start with `use` followed by a capital letter (`useFetch`, `useOnlineStatus`). This tells React and the linter to apply the Rules of Hooks inside it and wherever it is called.
- A function that does not call any hooks should **not** start with `use`; make it a normal function (`formatPrice`, `getSorted`).
- Name hooks after their **purpose**, not their implementation: `useChatRoom(roomId)` rather than `useEffectWithInterval`.
- Return whatever is convenient: a single value, an array like `useState` (`[value, toggle]`), or an object for many values.

11.5.3 Examples: useFetch, useLocalStorage, useDebounce, useToggle, useWindowSize, useOnlineStatus

Six hooks that appear in almost every project. They are defined in one `hooks.js` file and used together by the demo component, which was run for real:

HOOKS.JS

JavaScript

```
import { useEffect, useState, useSyncExternalStore } from "react";

export function useToggle(initial = false) {
```

```
  const [on, setOn] = useState(initial);
  return [on, () => setOn(o => !o)];
}

export function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved !== null ? JSON.parse(saved) : initial;
  });
  useEffect(() => { localStorage.setItem(key, JSON.stringify(value)); }, [key, value]);
  return [value, setValue];
}

export function useDebounce(value, delay = 300) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(t);                  // restart the timer on every change
  }, [value, delay]);
  return debounced;
}

export function useFetch(url) {
  const [state, setState] = useState({ data: null, loading: true, error: null });
  useEffect(() => {
    const controller = new AbortController();
    setState(s => ({ ...s, loading: true }));
    fetch(url, { signal: controller.signal })
      .then(r => { if (!r.ok) throw new Error("HTTP " + r.status); return r.json(); })
      .then(data => setState({ data, loading: false, error: null }))
      .catch(err => { if (err.name !== "AbortError") setState({ data: null, loading: false, error:
err.message }); });
    return () => controller.abort();
  }, [url]);
  return state;
}

export function useWindowSize() {
  const [size, setSize] = useState({ width: window.innerWidth, height: window.innerHeight });
  useEffect(() => {
    const onResize = () => setSize({ width: window.innerWidth, height: window.innerHeight });
    window.addEventListener("resize", onResize);
    return () => window.removeEventListener("resize", onResize);
  }, []);
  return size;
}

function subscribe(cb) {
  window.addEventListener("online", cb);
  window.addEventListener("offline", cb);
  return () => { window.removeEventListener("online", cb); window.removeEventListener("offline", cb);
};
}
export function useOnlineStatus() {
  return useSyncExternalStore(subscribe, () => navigator.onLine);
}
```

APP.JSX

JavaScript

```
import { useState } from "react";
import { useToggle, useLocalStorage, useDebounce, useFetch, useWindowSize, useOnlineStatus } from
"./hooks.js";

export default function App() {
  const [dark, toggleDark] = useToggle(false);
  const [name, setName] = useLocalStorage("name", "Asha");
  const [query, setQuery] = useState("");
  const debounced = useDebounce(query, 300);
  const { data: users, loading } = useFetch("users.json");
  const { width } = useWindowSize();
  const online = useOnlineStatus();
  const visible = (users ?? []).filter(u => u.name.toLowerCase().includes(debounced.toLowerCase()));

  return (
    <div className={`w-96 space-y-2 rounded-xl p-4 text-sm ${dark ? "bg-slate-900 text-slate-100" :
"bg-white ring-1 ring-slate-200"}`}>
      <div className="flex justify-between">
        <span>{online ? "● online" : "○ offline"} · {width}px wide</span>
        <button onClick={toggleDark} className="underline">{dark ? "Light" : "Dark"} mode</button>
      </div>
      <label className="block">Your name (saved): <input value={name} onChange={e =>
setName(e.target.value)} className="rounded border px-1 text-slate-900" /></label>
      <input value={query} onChange={e => setQuery(e.target.value)} placeholder="Search users
(debounced)" className="w-full rounded border px-2 py-1 text-slate-900" />
      <p className="text-xs opacity-70">typed: "{query}" · searching for: "{debounced}"</p>
      {loading ? <p>Loading...</p> : <ul className="list-disc pl-5">{visible.map(u => <li key={u.id}>
{u.name}</li>)}</ul>}
    </div>
  );
}
```



ALL SIX HOOKS WORKING TOGETHER: DARK MODE TOGGLED, A SEARCH TYPED AND DEBOUNCED, USERS FETCHED OVER HTTP

Hook	Returns	Built on
`useToggle`	`[on, toggle]`	useState
`useLocalStorage`	`[value, setValue]`, persisted	useState (lazy init) + useEffect
`useDebounce`	the value, after it stops changing for *delay* ms	useState + useEffect with cleanup
`useFetch`	`{ data, loading, error }`	useState + useEffect + AbortController

Hook	Returns	Built on
`useWindowSize`	`{ width, height }`	useState + useEffect (window listener)
`useOnlineStatus`	`true` / `false`	useSyncExternalStore

PRACTICE CHECKPOINT

A **shopping cart rebuilt with useReducer**, a **search with useDebounce**, and **three custom hooks** used across components.

HOOKS.JS JavaScript

```
import { useEffect, useState, useSyncExternalStore } from "react";

export function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  return [on, () => setOn(o => !o)];
}

export function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved !== null ? JSON.parse(saved) : initial;
  });
  useEffect(() => { localStorage.setItem(key, JSON.stringify(value)); }, [key, value]);
  return [value, setValue];
}

export function useDebounce(value, delay = 300) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(t);                  // restart the timer on every change
  }, [value, delay]);
  return debounced;
}

export function useFetch(url) {
  const [state, setState] = useState({ data: null, loading: true, error: null });
  useEffect(() => {
    const controller = new AbortController();
    setState(s => ({ ...s, loading: true }));
    fetch(url, { signal: controller.signal })
      .then(r => { if (!r.ok) throw new Error("HTTP " + r.status); return r.json(); })
      .then(data => setState({ data, loading: false, error: null }))
      .catch(err => { if (err.name !== "AbortError") setState({ data: null, loading: false, error:
err.message }); });
    return () => controller.abort();
  }, [url]);
  return state;
}

export function useWindowSize() {
  const [size, setSize] = useState({ width: window.innerWidth, height: window.innerHeight });
  useEffect(() => {
    const onResize = () => setSize({ width: window.innerWidth, height: window.innerHeight });
    window.addEventListener("resize", onResize);
    return () => window.removeEventListener("resize", onResize);
  }, []);
  return size;
```

```
}

function subscribe(cb) {
  window.addEventListener("online", cb);
  window.addEventListener("offline", cb);
  return () => { window.removeEventListener("online", cb); window.removeEventListener("offline", cb);
};
}
export function useOnlineStatus() {
  return useSyncExternalStore(subscribe, () => navigator.onLine);
}
```

APP.JSX: SAMPLE SOLUTION

JavaScript

```
import { useReducer, useState, useEffect } from "react";
import { useDebounce, useLocalStorage, useToggle } from "./hooks.js";

const PRODUCTS = [
  { id: 1, name: "Mechanical keyboard", price: 3499 }, { id: 2, name: "Wireless mouse", price: 899 },
  { id: 3, name: "USB-C hub", price: 1599 }, { id: 4, name: "Laptop stand", price: 1299 }, { id: 5,
name: "Webcam", price: 2199 },
];

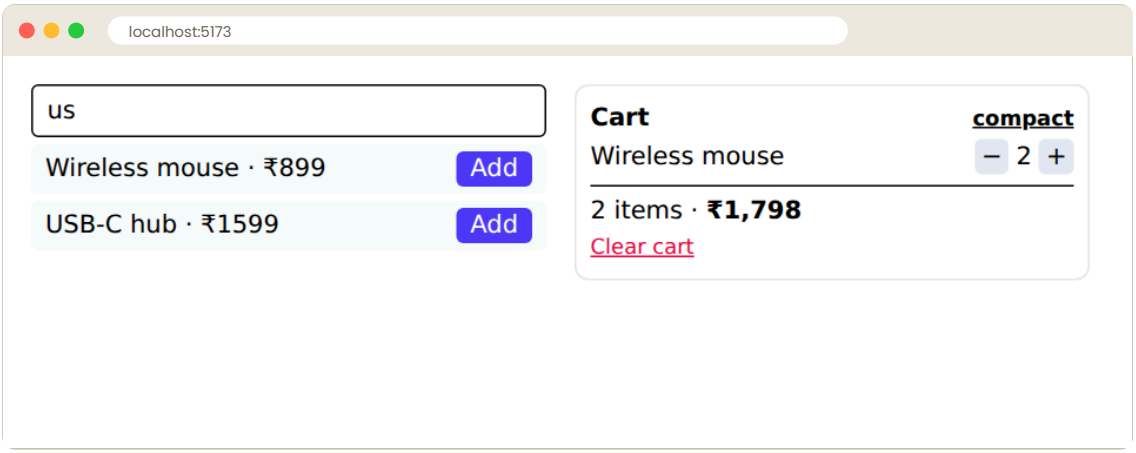
function cartReducer(cart, action) {
  switch (action.type) {
    case "add": {
      const found = cart.find(i => i.id === action.product.id);
      return found ? cart.map(i => (i.id === found.id ? { ...i, qty: i.qty + 1 } : i)) : [...cart, {
...action.product, qty: 1 }];
    }
    case "changeQty": return cart.map(i => (i.id === action.id ? { ...i, qty: i.qty + action.delta } :
i)).filter(i => i.qty > 0);
    case "remove": return cart.filter(i => i.id !== action.id);
    case "clear": return [];
    default: throw new Error("Unknown action " + action.type);
  }
}

function ProductSearch({ onAdd }) {                          // uses useDebounce
  const [q, setQ] = useState("");
  const query = useDebounce(q, 250);
  const results = PRODUCTS.filter(p => p.name.toLowerCase().includes(query.toLowerCase()));
  return (
    <div className="space-y-1">
      <input value={q} onChange={e => setQ(e.target.value)} placeholder="Search products"
className="w-full rounded border px-2 py-1" />
      {results.map(p => (
        <div key={p.id} className="flex items-center justify-between rounded bg-slate-50 px-2 py-1">
          <span>{p.name} · &#8377;{p.price}</span>
          <button onClick={() => onAdd(p)} aria-label={`Add ${p.name}`} className="rounded bg-indigo-
600 px-2 text-white">Add</button>
        </div>
      ))}
    </div>
  );
}

function Cart({ cart, dispatch }) {
  const [compact, toggleCompact] = useToggle(false);          // uses useToggle
  const total = cart.reduce((s, i) => s + i.price * i.qty, 0);
  return (
    <div className="rounded-lg border border-slate-200 p-2">
      <div className="flex justify-between font-semibold"><span>Cart</span><button onClick=
{toggleCompact} className="text-xs underline">{compact ? "details" : "compact"}</button></div>
```

```
      {cart.length === 0 && <p className="text-slate-400">Empty</p>}
      {!compact && cart.map(i => (
        <div key={i.id} className="flex items-center justify-between py-0.5">
          <span className="truncate">{i.name}</span>
          <span className="flex items-center gap-1">
            <button onClick={() => dispatch({ type: "changeQty", id: i.id, delta: -1 })} aria-label=
{`Decrease ${i.name}`} className="rounded bg-slate-200 px-1">−</button>{i.qty}
            <button onClick={() => dispatch({ type: "changeQty", id: i.id, delta: 1 })}
className="rounded bg-slate-200 px-1">+</button>
          </span>
        </div>
      ))}
      <p className="mt-1 border-t pt-1">{cart.reduce((n, i) => n + i.qty, 0)} items · <b>&#8377;
{total.toLocaleString("en-IN")}</b></p>
      {cart.length > 0 && <button onClick={() => dispatch({ type: "clear" })} className="text-xs text-
rose-600 underline">Clear cart</button>}
    </div>
  );
}

export default function App() {
  const [saved, setSaved] = useLocalStorage("cart", []);      // uses useLocalStorage
  const [cart, dispatch] = useReducer(cartReducer, saved);
  useEffect(() => { setSaved(cart); }, [cart, setSaved]);
  return (
    <div className="grid w-[600px] grid-cols-2 gap-4 text-sm">
      <ProductSearch onAdd={p => dispatch({ type: "add", product: p })} />
      <Cart cart={cart} dispatch={dispatch} />
    </div>
  );
}
```



SEARCHED "US", ADDED THE HUB AND THE MOUSE TWICE, THEN REDUCED THE HUB TO ZERO (REMOVED)

`useRef` holds DOM nodes and silent mutable values; in React 19 `ref` is a normal prop. `useReducer` centralises complex updates in a pure reducer driven by actions. `useMemo` / `useCallback` cache values and functions when it matters (or let the React Compiler do it). `useId`, `useDeferredValue`, `useSyncExternalStore` and `useEffectEvent` solve specific problems, and custom `useX` hooks package reusable stateful logic.

MODULE 12 · INTERMEDIATE

Context API and state management patterns

Minimum duration **2 days** Level **Intermediate**

Props carry data one level at a time; context lets a component make data available to its whole subtree. This module explains prop drilling and when it is fine; createContext, providing values with the React 19 <Context> provider, consuming with useContext and use, default values and nested contexts; combining context with useReducer behind custom provider components and hooks; the classic use cases (theme, authenticated user, language, cart); the re-rendering pitfalls of context and how to split and memoise values; and a decision guide for where each kind of state should live. All examples were run with React 19.2 in Chromium, including render counts.

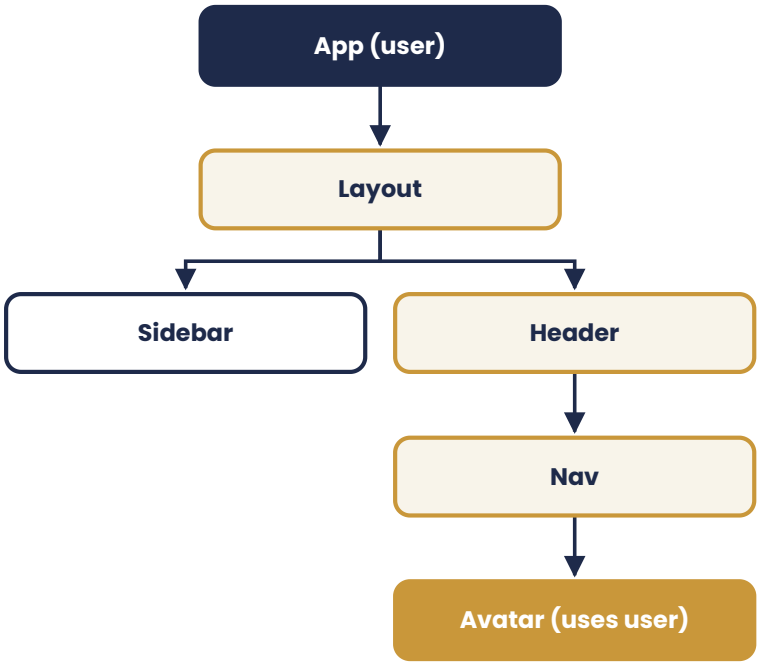
IN THIS MODULE

12.1 Prop drilling
12.2 Context
12.3 Context with reducers
12.4 Common use cases
12.5 Context pitfalls
12.6 Choosing where state lives

12.1 Prop drilling

12.1.1 The problem and when it is acceptable

Prop drilling means passing a prop through several intermediate components that do not use it themselves, only so that a deeply nested component can receive it. It works, but every component in the chain must know about data it does not care about, and adding or renaming the prop touches many files.



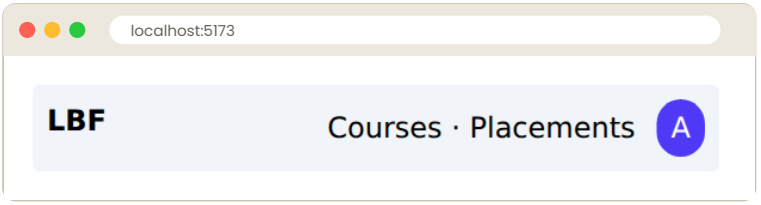
PROP DRILLING: USER PASSES THROUGH LAYOUT, HEADER AND NAV (CREAM) JUST TO REACH AVATAR

APP.JSX

JavaScript

```
function Avatar({ user }) { return <span className="rounded-full bg-indigo-600 px-2 py-1 text-white">
{user.name[0]}</span>; }
function Nav({ user }) { return <nav className="flex items-center gap-3">Courses · Placements <Avatar
user={user} /></nav>; }
function Header({ user }) { return <header className="flex justify-between rounded bg-slate-100 p-2">
<b>LBF</b><Nav user={user} /></header>; }
function Layout({ user }) { return <div className="w-96"><Header user={user} /></div>; }

export default function App() {
  const user = { name: "Asha" };
  return <Layout user={user} />;                // user drilled through 3 components
}
```



IT WORKS, BUT LAYOUT, HEADER AND NAV ALL CARRY A PROP THEY NEVER USE

Drilling through **one or two** levels is perfectly fine and is often clearer than context, because the data flow is explicit. Before reaching for context, also consider **composition**: pass the finished element down as `children`, so intermediate components do not need the data at all.

AVOIDING DRILLING WITH COMPOSITION (ELEMENT PROPS)

JavaScript

```
// Composition: Layout and Header never see `user`
function App() {
  const user = { name: "Asha" };
  return (
    <Layout header={<Header nav={<Nav avatar={<Avatar user={user} />} />} />} />
  );
}
```

12.2 Context

12.2.1 createContext

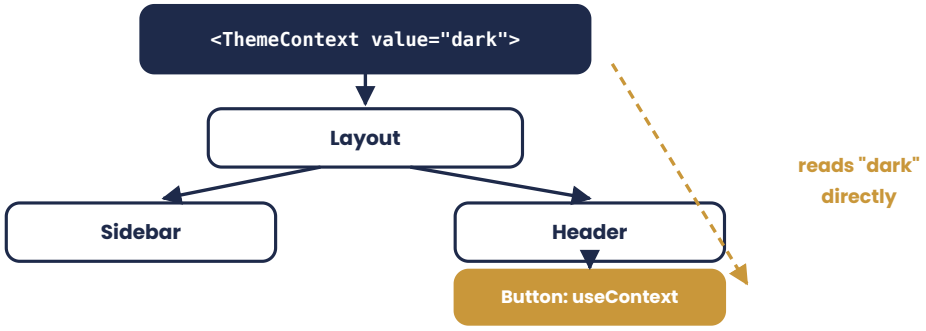
Context lets a parent make a value available to **every component below it**, however deep, without passing props. Three steps: **create** the context (usually in its own module, exported), **provide** a value somewhere up the tree, and **consume** it in any descendant.

THEMECONTEXT.JS

JavaScript

```
import { createContext } from "react";

export const ThemeContext = createContext("light");   // "light" is the default value
```



A PROVIDER MAKES THE VALUE AVAILABLE TO ALL DESCENDANTS

12.2.2 Providing values with <Context> provider

Wrap part of the tree in the context itself, with a `value` prop. In React 19 you render `<ThemeContext value="dark">` directly; older code writes `<ThemeContext.Provider value="dark">`, which still works. The value can be anything: a string, an object, state and its setter. When the value changes, every component that reads it re-renders.

12.2.3 Consuming context with useContext and use

`useContext(SomeContext)` returns the value from the **nearest provider above** the component. React 19 also adds `use(SomeContext)`, which does the same but, unlike hooks, may be called inside conditions and loops (it can also read promises, Module 17).

APP.JSX

JavaScript

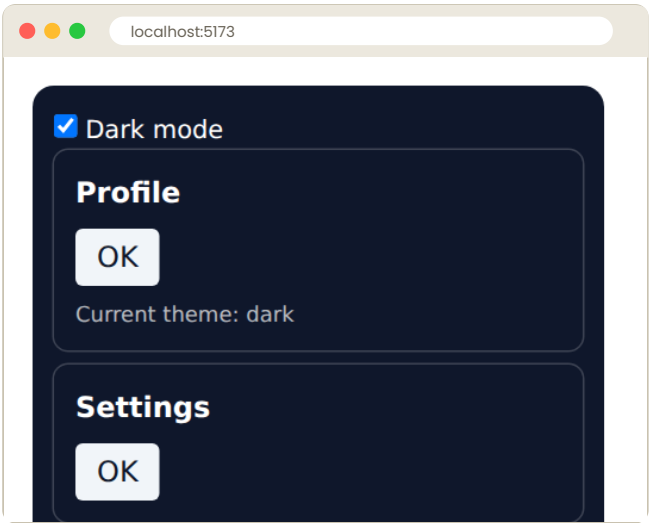
```
import { createContext, useContext, use, useState } from "react";

const ThemeContext = createContext("light");

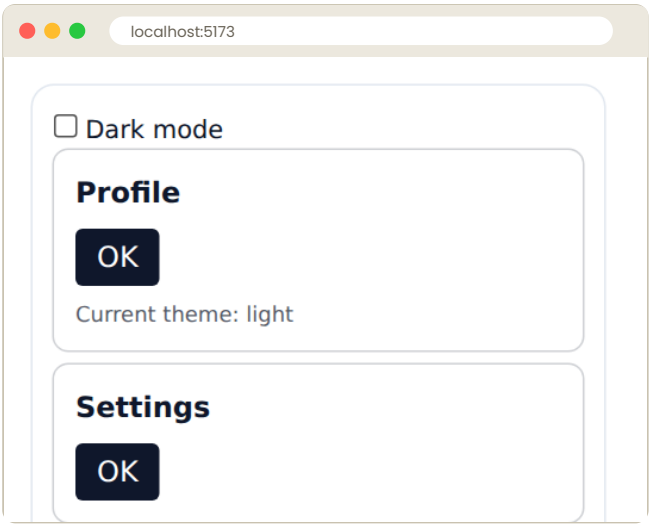
function Button({ children }) {
  const theme = useContext(ThemeContext);                    // nearest provider
  return <button className={`rounded px-3 py-1 ${theme === "dark" ? "bg-slate-100 text-slate-900" :
"bg-slate-900 text-white"}`}>{children}</button>;
}

function Panel({ title, showHint }) {
  let hint = null;
  if (showHint) {
    const theme = use(ThemeContext);                         // use() may be called conditionally
    hint = <p className="text-xs opacity-70">Current theme: {theme}</p>;
  }
  return <section className="space-y-2 rounded-lg p-3 ring-1 ring-current/20"><h3 className="font-
semibold">{title}</h3><Button>OK</Button>{hint}</section>;
}

export default function App() {
  const [theme, setTheme] = useState("dark");
  return (
    <ThemeContext value={theme}>
      <div className={`w-80 space-y-2 rounded-xl p-3 ${theme === "dark" ? "bg-slate-900 text-white" :
"bg-white text-slate-900 ring-1 ring-slate-200"}`}>
        <label className="text-sm"><input type="checkbox" checked={theme === "dark"} onChange={e =>
setTheme(e.target.checked ? "dark" : "light")} /> Dark mode</label>
        <Panel title="Profile" showHint />
        <Panel title="Settings" />
      </div>
    </ThemeContext>
  );
}
```



DARK THEME PROVIDED ONCE AT THE TOP AND READ BY DEEPLY NESTED BUTTONS



AFTER UNTICKING DARK MODE: ONE STATE CHANGE AT THE TOP UPDATED EVERY CONSUMER

12.2.4 Default values

The argument to `createContext(default)` is used **only** when a component reads the context with **no provider above it**. It is a fallback, not an initial state: changing it later does nothing. A useful pattern is a default of `null` plus a custom hook that throws a clear error if someone forgets the provider (Section 12.3.2).

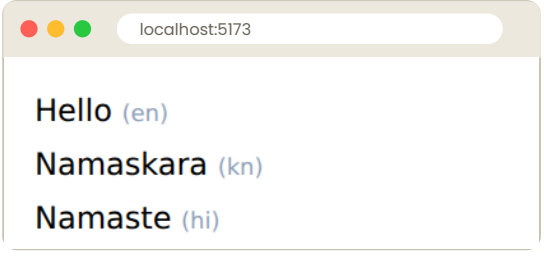
APP.JSX JavaScript

```
import { createContext, useContext } from "react";

const LanguageContext = createContext("en");           // fallback

function Greeting() {
  const lang = useContext(LanguageContext);
  const text = { en: "Hello", kn: "Namaskara", hi: "Namaste" }[lang];
  return <p>{text} <span className="text-xs text-slate-400">({lang})</span></p>;
}

export default function App() {
  return (
    <div className="space-y-1">
      <Greeting />                                      {/* no provider: default "en" */}
      <LanguageContext value="kn"><Greeting /></LanguageContext>
      <LanguageContext value="hi"><Greeting /></LanguageContext>
    </div>
  );
}
```



THE FIRST GREETING HAS NO PROVIDER ABOVE IT AND USES THE DEFAULT

12.2.5 Multiple and nested contexts

An app usually has several contexts (theme, user, language), each with its own provider; stack them at the top. A provider can also be **nested** inside another provider of the same context to **override** the value for a subtree: the nearest one wins.

APP.JSX

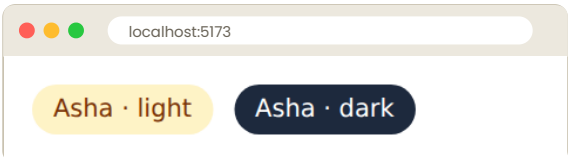
JavaScript

```
import { createContext, useContext } from "react";

const ThemeContext = createContext("light");
const UserContext = createContext(null);

function Badge() {
  const theme = useContext(ThemeContext);
  const user = useContext(UserContext);
  return (
    <span className={`rounded-full px-3 py-1 text-sm ${theme === "dark" ? "bg-slate-800 text-white" :
"bg-amber-100 text-amber-900"}`}>
      {user.name} · {theme}
    </span>
  );
}

export default function App() {
  return (
    <UserContext value={{ name: "Asha" }}>
      <ThemeContext value="light">
        <div className="flex items-center gap-3">
          <Badge />
          <ThemeContext value="dark">                {/* override for this subtree */}
            <Badge />
          </ThemeContext>
        </div>
      </ThemeContext>
    </UserContext>
  );
}
```



TWO CONTEXTS READ BY ONE COMPONENT; THE INNER PROVIDER OVERRIDES THE THEME

12.3 Context with reducers

12.3.1 Scaling state with context + useReducer

A powerful, dependency-free pattern for medium-sized app state: keep the state in a `useReducer` at the top, and provide **two** contexts, one for the **state** and one for **dispatch**. Any component can read the state or dispatch actions, without prop drilling. Splitting them means components that only dispatch do not re-render when the state changes (Section 12.5).

12.3.2 Custom provider components and custom hooks

Hide the contexts behind a **provider component** and **custom hooks**. Consumers then import only `useTasks()` and `useTasksDispatch()`, never the raw context, and the hooks can throw a helpful error

when used outside the provider.

TASKSCONTEXT.JSX JavaScript

```
import { createContext, useContext, useReducer } from "react";

const TasksContext = createContext(null);
const TasksDispatchContext = createContext(null);

function tasksReducer(tasks, action) {
  switch (action.type) {
    case "added":   return [...tasks, { id: Date.now(), text: action.text, done: false }];
    case "toggled": return tasks.map(t => (t.id === action.id ? { ...t, done: !t.done } : t));
    case "deleted": return tasks.filter(t => t.id !== action.id);
    default: throw new Error("Unknown action " + action.type);
  }
}

export function TasksProvider({ children }) {
  const [tasks, dispatch] = useReducer(tasksReducer, [
    { id: 1, text: "Read about context", done: true },
    { id: 2, text: "Build the provider", done: false },
  ]);
  return (
    <TasksContext value={tasks}>
      <TasksDispatchContext value={dispatch}>{children}</TasksDispatchContext>
    </TasksContext>
  );
}

export function useTasks() {
  const ctx = useContext(TasksContext);
  if (ctx === null) throw new Error("useTasks must be used inside <TasksProvider>");
  return ctx;
}

export function useTasksDispatch() {
  const ctx = useContext(TasksDispatchContext);
  if (ctx === null) throw new Error("useTasksDispatch must be used inside <TasksProvider>");
  return ctx;
}
```

APP.JSX

JavaScript

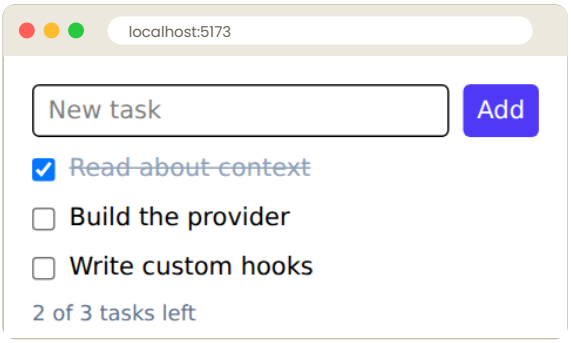
```
import { useState } from "react";
import { TasksProvider, useTasks, useTasksDispatch } from "./TasksContext.jsx";

function AddTask() {                                    // only dispatches
  const dispatch = useTasksDispatch();
  const [text, setText] = useState("");
  return (
    <form className="flex gap-2" onSubmit={e => { e.preventDefault(); dispatch({ type: "added", text
}); setText(""); }}>
      <input value={text} onChange={e => setText(e.target.value)} placeholder="New task"
className="flex-1 rounded border px-2 py-1" />
      <button className="rounded bg-indigo-600 px-2 text-white">Add</button>
    </form>
  );
}

function TaskList() {                                   // reads and dispatches
  const tasks = useTasks();
  const dispatch = useTasksDispatch();
  return tasks.map(t => (
    <label key={t.id} className="flex items-center gap-2">
      <input type="checkbox" checked={t.done} onChange={() => dispatch({ type: "toggled", id: t.id })}
/>
      <span className={t.done ? "text-slate-400 line-through" : ""}>{t.text}</span>
    </label>
  ));
}

function Summary() {                                    // reads only
  const tasks = useTasks();
  return <p className="text-xs text-slate-500">{tasks.filter(t => !t.done).length} of {tasks.length}
tasks left</p>;
}

export default function App() {
  return (
    <TasksProvider>
      <div className="w-72 space-y-2 text-sm"><AddTask /><TaskList /><Summary /></div>
    </TasksProvider>
  );
}
```



THREE COMPONENTS SHARE TASK STATE THROUGH THE PROVIDER, WITH NO PROPS PASSED

BROWSER CONSOLE: THE CUSTOM HOOK EXPLAINS THE MISTAKE

```
Uncaught useTasks must be used inside <TasksProvider>
Warning: An error occurred in the <Summary> component.

Consider adding an error boundary to your tree to customize error handling behavior.
Visit https://react.dev/link/error-boundaries to learn more about error boundaries.
```

12.4 Common use cases

Context suits values that are needed by **many components at different depths** and change **rarely**:

12.4.1 Theme (light/dark)

A theme context provides the current theme and a toggle; components read it to choose classes, or the provider sets a class on the root element so CSS handles the rest (see the checkpoint). Persist the choice in localStorage.

12.4.2 Authenticated user

An auth context provides the current user (or `null`), plus `login` and `logout` functions. Headers show the avatar, protected pages check for a user, and API helpers read the token. The provider is the only place that talks to the authentication API.

12.4.3 Language

A language (locale) context provides the current language and a translation function. Libraries such as react-i18next and FormatJS (react-intl) are built on this exact idea and add plural rules, number and date formatting.

APP.JSX

JavaScript

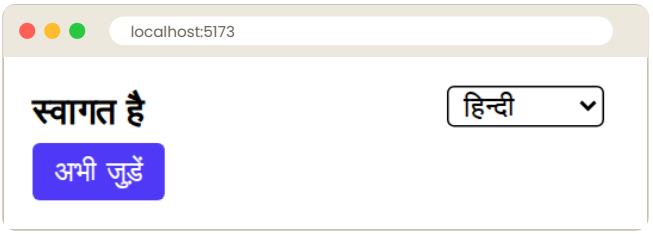
```
import { createContext, useContext, useState } from "react";

const messages = {
  en: { welcome: "Welcome", courses: "Courses", enrol: "Enrol now" },
  kn: { welcome: "ಸ್ವಾಗತ", courses: "ಕೋರ್ಸ್‌ಗಳು", enrol: "ಈಗಲೇ ಸೇರಿ" },
  hi: { welcome: "स्वागत है", courses: "पाठ्यक्रम", enrol: "अभी जुड़ें" },
};
const I18nContext = createContext(null);

function I18nProvider({ children }) {
  const [lang, setLang] = useState("en");
  const t = key => messages[lang][key] ?? key;
  return <I18nContext value={{ lang, setLang, t }}>{children}</I18nContext>;
}
const useI18n = () => useContext(I18nContext);

function Hero() {
  const { t } = useI18n();
  return <div><h2 className="text-xl font-bold">{t("welcome")}</h2><button className="mt-1 rounded bg-
indigo-600 px-3 py-1 text-white">{t("enrol")}</button></div>;
}
function LanguagePicker() {
  const { lang, setLang } = useI18n();
  return <select value={lang} onChange={e => setLang(e.target.value)} className="rounded border px-1">
<option value="en">English</option><option value="kn">ಕನ್ನಡ</option><option value="hi">हिन्दी</option>
</select>;
}

export default function App() {
  return <I18nProvider><div className="flex w-80 items-start justify-between"><Hero /><LanguagePicker
/></div></I18nProvider>;
}
```



AFTER CHOOSING हिन्दी: EVERY TRANSLATED TEXT UPDATED

12.4.4 Shopping cart

A cart context (state + dispatch, as in Section 12.3) lets the product cards add items, the header show the count, and the checkout page list the items, all without drilling. Large stores with many updates often move the cart to a global store library instead (Module 16).

12.5 Context pitfalls

12.5.1 Unnecessary re-renders

When a provider's `value` changes (by `Object.is`), **every** component that reads that context re-renders, even if it uses only a part of the value that did not change. A common accidental cause is passing a **new object** on every render: `value={{ user, login }}` creates a new object each time the provider renders, so all consumers re-render even if nothing changed.

APP.JSX

JavaScript

```
import { createContext, useContext, useState, memo } from "react";

const AppContext = createContext(null);

const ThemeLabel = memo(function ThemeLabel() {
  const { theme } = useContext(AppContext);              // only cares about theme
  console.log("  render ThemeLabel");
  return <span>{theme}</span>;
});

export default function App() {
  const [theme] = useState("light");
  const [count, setCount] = useState(0);
  console.log("render App, count =", count);
  return (
    <AppContext value={{ theme, count }}>                {/* one context for everything */}
      <ThemeLabel />
      <button onClick={() => setCount(c => c + 1)}>count +1</button>
    </AppContext>
  );
}
```

BROWSER CONSOLE: ThemeLabel RE-RENDERS ALTHOUGH THEME DID NOT CHANGE

```
  render ThemeLabel
render App, count = 0
--- click count +1 ---
  render ThemeLabel
render App, count = 1
```

12.5.2 Splitting contexts and memoising values

Two fixes, often combined: **split** one large context into several that change independently (theme, user, cart; or state versus dispatch), and **memoise** object values with `useMemo` (and functions with `useCallback`) so the value only changes when its contents do.

APP.JSX

JavaScript

```
import { createContext, useContext, useMemo, useState, memo } from "react";

const ThemeContext = createContext(null);
const CountContext = createContext(null);

const ThemeLabel = memo(function ThemeLabel() {
  const { theme } = useContext(ThemeContext);
  console.log("  render ThemeLabel");
  return <span>{theme}</span>;
});

export default function App() {
  const [theme, setTheme] = useState("light");
  const [count, setCount] = useState(0);
  const themeValue = useMemo(() => ({ theme, setTheme }), [theme]);   // stable until theme changes
  console.log("render App, count =", count);
  return (
    <ThemeContext value={themeValue}>
      <CountContext value={count}>
        <ThemeLabel />
        <button onClick={() => setCount(c => c + 1)}>count +1</button>
      </CountContext>
    </ThemeContext>
  );
}
```

BROWSER CONSOLE: AFTER SPLITTING AND MEMOISING, ThemeLabel IS SKIPPED

```
  render ThemeLabel
render App, count = 0
--- click count +1 ---
render App, count = 1
```

NOTE

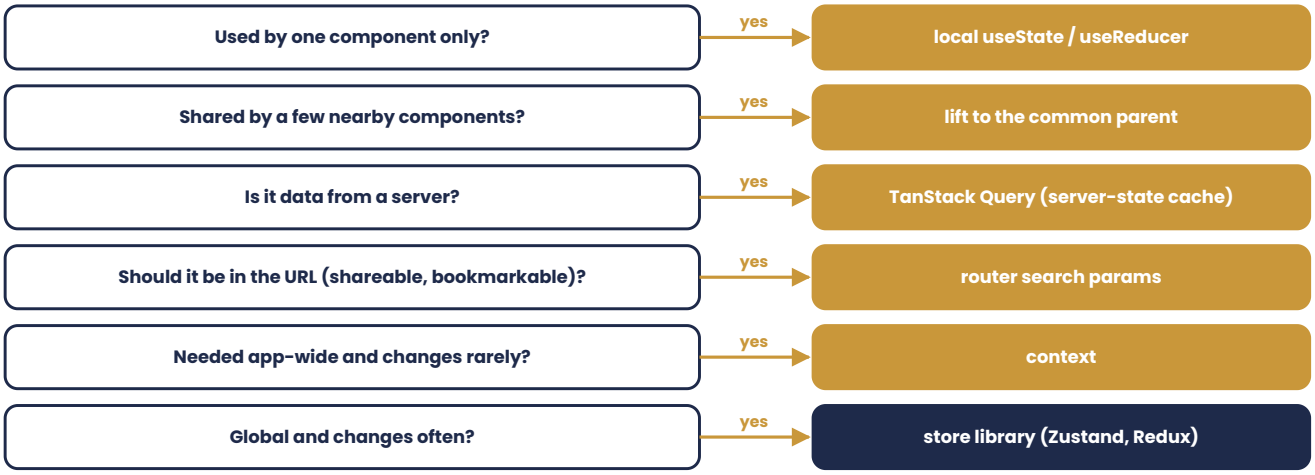
Context is a **transport** mechanism, not a state manager. For state that changes very frequently and is read by many components (a live editor, a large cart with many interactions), a store library with fine-grained subscriptions (Zustand, Redux Toolkit, Jotai, Module 16) re-renders far less.

12.6 Choosing where state lives

12.6.1 Local, lifted, context, server and global store state

Kind of state	Example	Where it lives	Tool
Local UI state	is this dropdown open? input text	the component that uses it	`useState`, `useReducer`
Shared by siblings	selected tab, filter used by list and counter	closest common parent (lifted)	props and callbacks
App-wide, rarely changing	theme, logged-in user, language	a provider near the root	context
Server state	products, students, orders fetched from an API	a cache in front of the server	TanStack Query, SWR, framework loaders (Module 14)

Kind of state	Example	Where it lives	Tool
Complex global client state	cart with many interactions, editor document	an external store	Redux Toolkit, Zustand (Module 16)
URL state	current page, search query, filters to share	the URL	React Router (Module 13)
Form state	values and errors of a big form	a form library	React Hook Form (Module 14)



DECIDING WHERE A PIECE OF STATE SHOULD LIVE (ASK FROM THE TOP)

PRACTICE CHECKPOINT

An app-wide **theme toggle** and an **authentication context** with login and logout, consumed through **custom hooks**.

THEMECONTEXT.JSX JavaScript

```
import { createContext, useContext, useEffect, useMemo, useState } from "react";

const ThemeContext = createContext(null);

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState(() => localStorage.getItem("theme") ?? "light");
  useEffect(() => { localStorage.setItem("theme", theme); }, [theme]);
  const value = useMemo(() => ({ theme, toggle: () => setTheme(t => (t === "light" ? "dark" :
"light")) }), [theme]);
  return <ThemeContext value={value}><div className={theme === "dark" ? "dark" : ""}>{children}</div>
</ThemeContext>;
}

export function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error("useTheme must be used inside <ThemeProvider>");
  return ctx;
}
```

AUTHCONTEXT.JSX JavaScript

```
import { createContext, useCallback, useContext, useMemo, useState } from "react";

const AuthContext = createContext(null);
const fakeApi = (email, password) =>                          // stands in for POST /api/login
  new Promise((resolve, reject) => setTimeout(() =>
    password === "react19" ? resolve({ name: email.split("@")[0], email }) : reject(new Error("Wrong
e-mail or password")), 200));

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [error, setError] = useState(null);
  const [pending, setPending] = useState(false);

  const login = useCallback(async (email, password) => {
    setPending(true); setError(null);
    try { setUser(await fakeApi(email, password)); }
    catch (e) { setError(e.message); }
    finally { setPending(false); }
  }, []);
  const logout = useCallback(() => setUser(null), []);

  const value = useMemo(() => ({ user, error, pending, login, logout }), [user, error, pending, login,
logout]);
  return <AuthContext value={value}>{children}</AuthContext>;
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be used inside <AuthProvider>");
  return ctx;
}
```

APP.JSX: SAMPLE SOLUTION JavaScript

```
import { useState } from "react";
import { ThemeProvider, useTheme } from "./ThemeContext.jsx";
import { AuthProvider, useAuth } from "./AuthContext.jsx";

function Header() {
  const { theme, toggle } = useTheme();
  const { user, logout } = useAuth();
  return (
    <header className="flex items-center justify-between border-b border-slate-200 px-4 py-2
dark:border-slate-700">
      <b>LBF Portal</b>
      <div className="flex items-center gap-3 text-sm">
        <button onClick={toggle} aria-label="Toggle theme">{theme === "dark" ? "☀ Light" : "☾ Dark"}
</button>
        {user && <><span>Hi, {user.name}</span><button onClick={logout} className="underline">Log
out</button></>}
      </div>
    </header>
  );
}

function LoginForm() {
  const { login, error, pending } = useAuth();
  const [email, setEmail] = useState("asha@example.in");
  const [password, setPassword] = useState("");
  return (
    <form onSubmit={e => { e.preventDefault(); login(email, password); }} className="mx-auto w-64
```

```
space-y-2 py-4 text-sm">
      <input value={email} onChange={e => setEmail(e.target.value)} className="w-full rounded border
px-2 py-1 text-slate-900" />
      <input type="password" value={password} onChange={e => setPassword(e.target.value)}
placeholder="Password (react19)" className="w-full rounded border px-2 py-1 text-slate-900" />
      <button disabled={pending} className="w-full rounded bg-indigo-600 py-1 text-white
disabled:opacity-50">{pending ? "Signing in..." : "Log in"}</button>
      {error && <p role="alert" className="text-rose-500">{error}</p>}
    </form>
  );
}

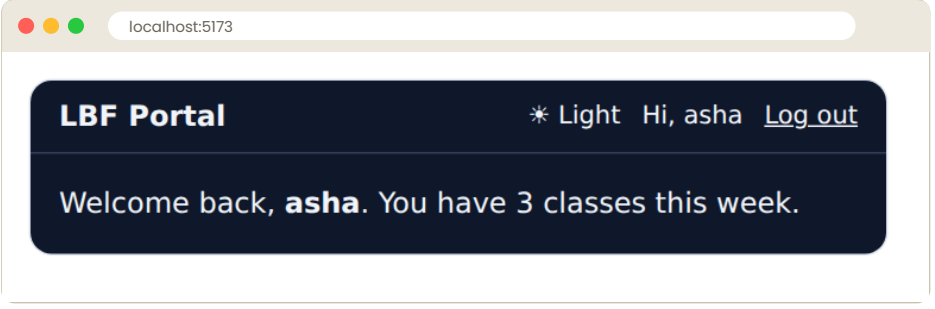
function Dashboard() {
  const { user } = useAuth();
  return <p className="p-4">Welcome back, <b>{user.name}</b>. You have 3 classes this week.</p>;
}

function Main() {
  const { user } = useAuth();
  return user ? <Dashboard /> : <LoginForm />;
}

export default function App() {
  return (
    <ThemeProvider>
      <AuthProvider>
        <div className="w-[480px] rounded-xl bg-white text-slate-900 ring-1 ring-slate-200 dark:bg-
slate-900 dark:text-slate-100">
          <Header />
          <Main />
        </div>
      </AuthProvider>
    </ThemeProvider>
  );
}
```



A WRONG PASSWORD FIRST: THE AUTH CONTEXT REPORTS THE ERROR



THEN THE RIGHT PASSWORD AND THE DARK THEME: BOTH CONTEXTS UPDATED THE WHOLE APP

Context passes a value to a whole subtree: `createContext`, provide with `<Ctx value={...}>`, read with `useContext` or `use`. Wrap it in a provider component and custom hooks, combine it with `useReducer` for shared state, split contexts and memoise values to avoid needless re-renders, and keep each kind of state where it belongs: local, lifted, context, server cache, URL or store.

MODULE 13 · INTERMEDIATE

Routing with React Router

Minimum duration **3 days** | Level **Intermediate**

A single-page application has one HTML page, yet users expect many "pages" with their own URLs, working back and forward buttons and shareable links. Routing maps URLs to screens. This module teaches React Router v7: declarative routes with BrowserRouter, Routes and Route; the data router created with createBrowserRouter; navigation with Link, NavLink and useNavigate; redirects; URL parameters and query strings; nested, layout, index and 404 routes; loaders, actions, pending UI and error elements; protected routes; code splitting with lazy and Suspense; and an overview of framework mode.

IN THIS MODULE

13.1 Routing basics
13.2 Navigation
13.3 Dynamic routing
13.4 Data APIs
13.5 Protected routes
13.6 Code splitting routes
13.7 Framework mode (overview)

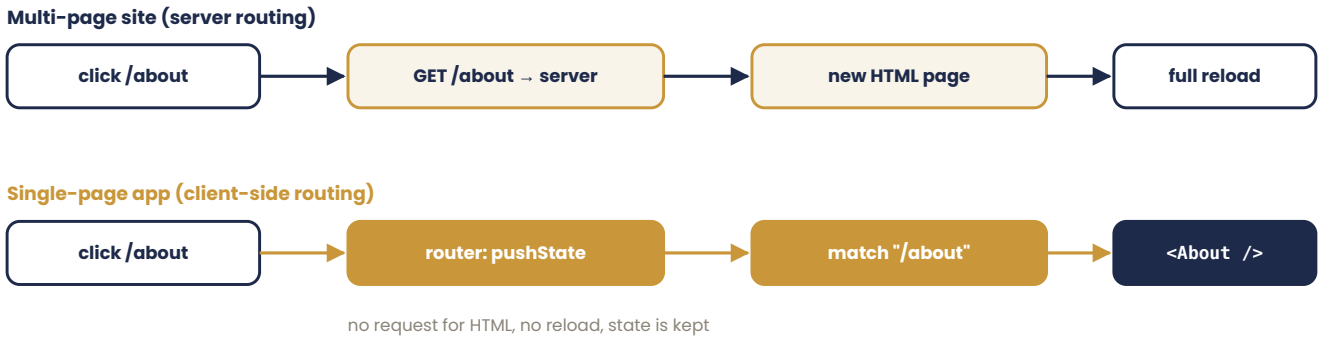
HOW THE EXAMPLES WERE RUN

The React Router package is not installed in the offline build used for these notes. Every code listing below is written exactly as you type it for **React Router 7** (imports from `"react-router"`). To produce the screenshots, the same code was bundled against a small stand-in module that implements the same components and hooks, and run in Chromium. The address bar in each screenshot shows the URL the app was on.

13.1 Routing basics

13.1.1 Client-side routing

In a traditional multi-page website every link asks the **server** for a new HTML page: the screen goes blank, everything reloads and all JavaScript state is lost. In a **single-page application** the server sends one HTML page once. After that a **client-side router** intercepts link clicks, changes the URL with the browser History API (`history.pushState`) and renders the matching component, without a page reload. The back and forward buttons keep working because the router listens to the browser's `popstate` event.



SERVER ROUTING RELOADS THE PAGE; CLIENT-SIDE ROUTING SWAPS COMPONENTS

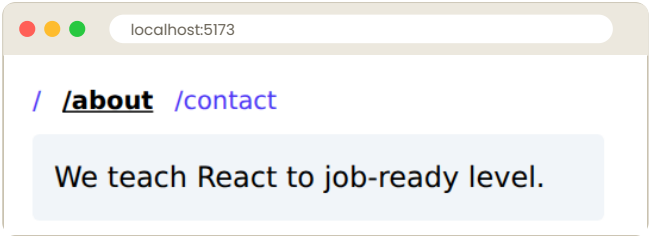
The idea fits in a few lines. The component below keeps the current path in state, and its links change that state instead of loading a page. A real router adds URL syncing, nesting, parameters and much more, but the core is the same: **path in, component out**.

APP.JSX: A ROUTER IN 20 LINES (IDEA ONLY) JavaScript

```
import { useState } from "react";

const pages = {
  "/": () => <p>Welcome to the home page.</p>,
  "/about": () => <p>We teach React to job-ready level.</p>,
  "/contact": () => <p>Write to info@learnbeingforward.in</p>,
};

export default function App() {
  const [path, setPath] = useState("/");
  const Page = pages[path] ?? (() => <p>404: no page here</p>);
  return (
    <div className="w-80 space-y-2">
      <nav className="flex gap-3 text-sm">
        {Object.keys(pages).map((p) => (
          <a key={p} href={p} className={p === path ? "font-bold underline" : "text-indigo-600"}
             onClick={(e) => { e.preventDefault(); setPath(p); }}>{p}</a>   // no page load
        ))}
      </nav>
      <div className="rounded bg-slate-100 p-3"><Page /></div>
    </div>
  );
}
```



CLICKING "/ABOUT" SWAPPED THE COMPONENT WITHOUT A RELOAD

13.1.2 Installing React Router

Terminal (typical output)

```
$ npm install react-router

added 3 packages, and audited 160 packages in 2s
found 0 vulnerabilities
```

Since version 7 everything ships in one package, `react-router`. Older tutorials install `react-router-dom`; in v7 that package still exists but only re-exports `react-router`, so new projects import from `"react-router"`. React Router can be used in three **modes**, each adding features on top of the previous one:

Mode	Set up with	Adds	Use when
Declarative	`<BrowserRouter>`, `<Routes>`, `<Route>`	URL matching, links, params, nesting	simple apps, or adding routing to an existing app
Data	`createBrowserRouter` + `<RouterProvider>`	loaders, actions, pending states, error elements	most new single-page apps
Framework	the React Router Vite plugin (`routes.ts`)	file-based routes, server rendering, type safety	full-stack apps (Section 13.7)

13.1.3 Declarative routes with BrowserRouter, Routes and Route

`<BrowserRouter>` wraps the app once (usually in `main.jsx`) and keeps the UI in sync with the URL. Inside it, `<Routes>` looks at the current URL and renders the **one** `<Route>` whose `path` matches best, showing that route's `element`. Everything outside `<Routes>` (the header below) is shown on every page.

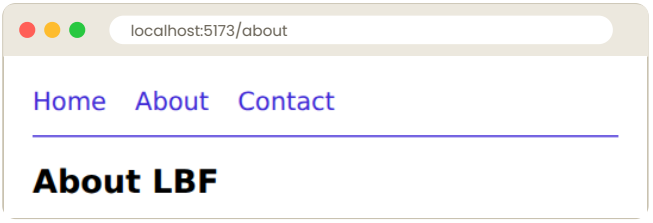
APP.JSX

JavaScript

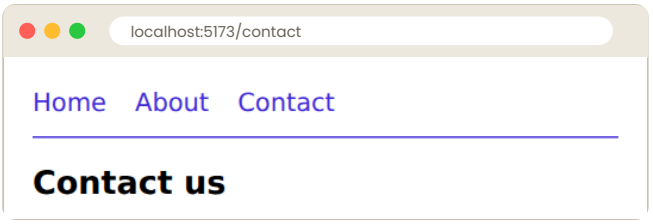
```
import { BrowserRouter, Routes, Route, Link } from "react-router";

function Home() { return <h2 className="text-lg font-semibold">Home</h2>; }
function About() { return <h2 className="text-lg font-semibold">About LBF</h2>; }
function Contact() { return <h2 className="text-lg font-semibold">Contact us</h2>; }

export default function App() {
  return (
    <BrowserRouter>
      <header className="mb-3 flex gap-4 border-b pb-2 text-sm text-indigo-700">
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
        <Link to="/contact">Contact</Link>
      </header>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </BrowserRouter>
  );
}
```



THE APP OPENED AT /ABOUT: ROUTES RENDERED THE ABOUT ELEMENT



AFTER CLICKING "CONTACT": THE URL AND THE SCREEN CHANGED, WITH NO RELOAD

> **TIP**
> In a real project `<BrowserRouter>` usually goes in `main.jsx` around `<App />`, so that every component, including `App` itself, can use the router hooks.

13.1.4 Data router setup with createBrowserRouter

The **data router** describes routes as an array of plain objects, outside the component tree. Each route object can carry a `loader` (load data before rendering), an `action` (handle form submissions), an `errorElement` and `children`. The router object is passed to `<RouterProvider>`, which replaces `<BrowserRouter>`. Sections 13.4 and the checkpoint use this mode.

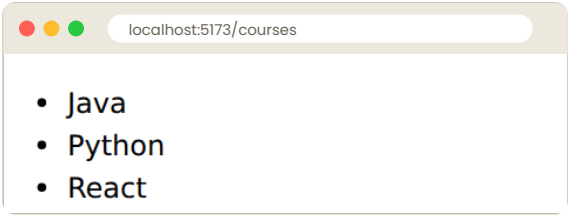
APP.JSX JavaScript

```
import { createBrowserRouter, RouterProvider, Link, useLoaderData } from "react-router";

function Courses() {
  const courses = useLoaderData();                    // data from the loader below
  return <ul className="list-disc pl-5">{courses.map((c) => <li key={c}>{c}</li>)}</ul>;
}

const router = createBrowserRouter([
  { path: "/", element: <p>Home. <Link className="text-indigo-600" to="/courses">See courses</Link>
</p> },
  { path: "/courses", element: <Courses />, loader: () => ["Java", "Python", "React"] },
]);

export default function App() {
  return <RouterProvider router={router} />;
}
```



THE LOADER RAN FIRST, THEN COURSES RENDERED WITH ITS DATA

13.2 Navigation

13.2.1 Link and NavLink (active styles)

Never use a plain `<a href>` for internal pages: it reloads the whole app. `<Link to="...">` renders a real `<a>` (so right-click, open in new tab and screen readers still work) but navigates on the client. `<NavLink>` is a Link that knows whether it is **active**: its `className` (or `style`, or children) can be a function receiving `{ isActive }`, and it sets `aria-current="page"` on the active link. A NavLink stays active on child URLs too (`/courses/react` keeps "Courses" active); add `end` to match exactly.

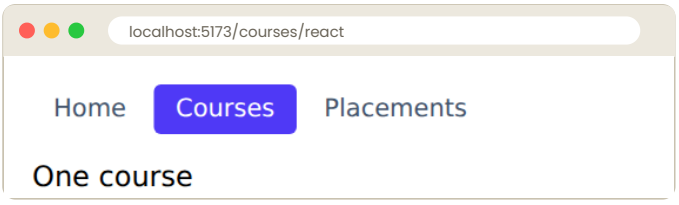
APP.JSX

JavaScript

```
import { BrowserRouter, Routes, Route, NavLink } from "react-router";

const linkClass = ({ isActive }) =>
  isActive ? "rounded bg-indigo-600 px-3 py-1 text-white" : "rounded px-3 py-1 text-slate-600
hover:bg-slate-100";

export default function App() {
  return (
    <BrowserRouter>
      <nav className="mb-3 flex gap-1 text-sm">
        <NavLink to="/" end className={linkClass}>Home</NavLink>
        <NavLink to="/courses" className={linkClass}>Courses</NavLink>
        <NavLink to="/placements" className={linkClass}>Placements</NavLink>
      </nav>
      <Routes>
        <Route path="/" element={<p>Home page</p>} />
        <Route path="/courses" element={<p>All courses</p>} />
        <Route path="/courses/:slug" element={<p>One course</p>} />
        <Route path="/placements" element={<p>Placement record</p>} />
      </Routes>
    </BrowserRouter>
  );
}
```



AT /COURSES/REACT THE "COURSES" NAVLINK IS STILL ACTIVE; "HOME" IS NOT, THANKS TO END

RENDERED DOM: THE NAV (excerpt of #root)

```
<nav class="mb-3 flex gap-1 text-sm"><a href="/" class="rounded px-3 py-1 text-slate-600 hover:bg-
slate-100">Home</a><a href="/courses" class="rounded bg-indigo-600 px-3 py-1 text-white" aria-
current="page">Courses</a><a href="/placements" class="rounded px-3 py-1 text-slate-600 hover:bg-
slate-100">Placements</a></nav><p>One course</p>
```

13.2.2 Programmatic navigation with useNavigate

Sometimes navigation must happen in code: after a form is saved, after login, or after a timer. `useNavigate()` returns a `navigate` function. `navigate("/path")` pushes a new entry onto the history stack; `navigate("/path", { replace: true })` replaces the current entry (so Back does not return to it); `navigate(-1)` goes back one step; and `navigate(to, { state })` passes data that the next page reads with `useLocation().state`.

APP.JSX

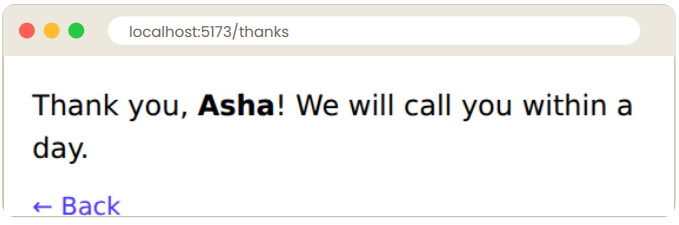
JavaScript

```
import { BrowserRouter, Routes, Route, useNavigate, useLocation } from "react-router";
import { useState } from "react";

function Enrol() {
  const [name, setName] = useState("");
  const navigate = useNavigate();
  function handleSubmit(e) {
    e.preventDefault();
    // ...save the enrolment here...
    navigate("/thanks", { state: { name } });            // go there in code
  }
  return (
    <form onSubmit={handleSubmit} className="flex gap-2">
      <input className="rounded border px-2 py-1" placeholder="Your name" value={name} onChange={(e)
=> setName(e.target.value)} />
      <button className="rounded bg-indigo-600 px-3 py-1 text-white">Enrol</button>
    </form>
  );
}

function Thanks() {
  const { state } = useLocation();                       // data sent with navigate()
  const navigate = useNavigate();
  return (
    <div className="space-y-2">
      <p>Thank you, <b>{state?.name}</b>! We will call you within a day.</p>
      <button className="text-sm text-indigo-600" onClick={() => navigate(-1)}>← Back</button>
    </div>
  );
}

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Enrol />} />
        <Route path="/thanks" element={<Thanks />} />
      </Routes>
    </BrowserRouter>
  );
}
```



TYPED A NAME AND CLICKED ENROL: THE HANDLER NAVIGATED TO /THANKS AND PASSED THE NAME AS STATE

13.2.3 Redirects

A redirect sends the user from one URL to another automatically. In declarative mode render `<Navigate to="/new" replace />`: as soon as it renders, it navigates. Use `replace` so the old URL does not stay in history (otherwise Back would bounce straight back to the redirect). In data mode a loader or action can `return redirect("/login")`, which happens **before** anything renders.

APP.JSX

JavaScript

```
import { BrowserRouter, Routes, Route, Navigate } from "react-router";

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<p>Home page</p>} />
        <Route path="/courses" element={<p>All courses (the new URL)</p>} />
        <Route path="/programs" element={<Navigate to="/courses" replace />} />   {/* old URL */}
      </Routes>
    </BrowserRouter>
  );
}
```



THE APP WAS OPENED AT THE OLD URL /PROGRAMS AND ENDED UP AT /COURSES

REDIRECTING FROM A LOADER

JavaScript

```
import { redirect } from "react-router";

// data mode: decide before rendering
export async function dashboardLoader() {
  const user = await getCurrentUser();
  if (!user) return redirect("/login");     // nothing of the dashboard renders
  return user;
}
```

13.3 Dynamic routing

13.3.1 URL parameters with useParams

A segment that starts with a colon is a **URL parameter**: the route `/products/:id` matches `/products/1`, `/products/42` and so on. Inside the element, `useParams()` returns an object of the parameters, here `{ id: "42" }`. Parameters are always **strings**; convert with `Number()` before comparing with numeric ids. Several parameters are allowed: `/courses/:courseId/lessons/:lessonId`.

PRODUCTS.JS

JavaScript

```
export const products = [
  { id: 1, name: "Wireless mouse", price: 799, category: "accessories" },
  { id: 2, name: "Mechanical keyboard", price: 3499, category: "accessories" },
  { id: 3, name: "27-inch monitor", price: 15999, category: "displays" },
  { id: 4, name: "Laptop stand", price: 1299, category: "accessories" },
  { id: 5, name: "Portable monitor", price: 11499, category: "displays" },
];
```

APP.JSX

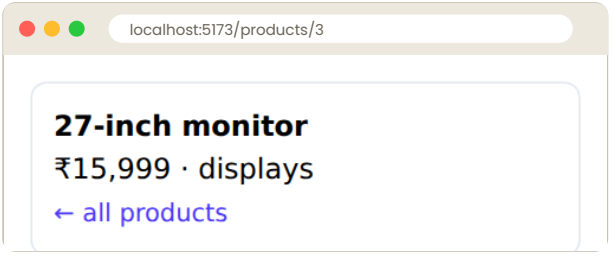
JavaScript

```
import { BrowserRouter, Routes, Route, Link, useParams } from "react-router";
import { products } from "./products.js";

function ProductList() {
  return (
    <ul className="space-y-1">
      {products.map((p) => <li key={p.id}><Link className="text-indigo-600" to={`/products/${p.id}`}>
{p.name}</Link></li>)}
    </ul>
  );
}

function ProductDetail() {
  const { id } = useParams();                              // "3" (a string)
  const product = products.find((p) => p.id === Number(id));
  if (!product) return <p>No product with id {id}.</p>;
  return (
    <div className="rounded-lg p-3 ring-1 ring-slate-200">
      <h2 className="font-semibold">{product.name}</h2>
      <p>₹{product.price.toLocaleString("en-IN")} · {product.category}</p>
      <Link className="text-sm text-indigo-600" to="/products">← all products</Link>
    </div>
  );
}

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/products" element={<ProductList />} />
        <Route path="/products/:id" element={<ProductDetail />} />
      </Routes>
    </BrowserRouter>
  );
}
```



CLICKED “27-INCH MONITOR” IN THE LIST: THE DETAIL PAGE READ ID = "3" FROM THE URL

13.3.2 Query strings with useSearchParams

The part after `?` is the **query string**: optional settings such as search terms, filters, sort order and page numbers, for example `/products?category=displays&sort=price`. `useSearchParams()` works like `useState` for the query string: it returns a `URLSearchParams` object and a setter. Keeping filters in the URL means the filtered view can be bookmarked, shared and restored with the Back button.

PRODUCTS.JS JavaScript

```
export const products = [
  { id: 1, name: "Wireless mouse", price: 799, category: "accessories" },
  { id: 2, name: "Mechanical keyboard", price: 3499, category: "accessories" },
  { id: 3, name: "27-inch monitor", price: 15999, category: "displays" },
  { id: 4, name: "Laptop stand", price: 1299, category: "accessories" },
  { id: 5, name: "Portable monitor", price: 11499, category: "displays" },
];
```

APP.JSX JavaScript

```
import { BrowserRouter, Routes, Route, useSearchParams } from "react-router";
import { products } from "./products.js";

function Products() {
  const [searchParams, setSearchParams] = useSearchParams();
  const category = searchParams.get("category") ?? "all";     // read
  const sort = searchParams.get("sort") ?? "name";

  let list = category === "all" ? products : products.filter((p) => p.category === category);
  list = [...list].sort((a, b) => (sort === "price" ? a.price - b.price :
a.name.localeCompare(b.name)));

  function update(key, value) {
    setSearchParams((prev) => { prev.set(key, value); return prev; });   // write, keep other keys
  }
  return (
    <div className="w-80 space-y-2 text-sm">
      <div className="flex gap-2">
        {["all", "accessories", "displays"].map((c) => (
          <button key={c} onClick={() => update("category", c)}
                  className={`rounded px-2 py-1 ${c === category ? "bg-indigo-600 text-white" : "bg-
slate-100"}`}>{c}</button>
        ))}
        <button onClick={() => update("sort", "price")} className="ml-auto rounded bg-slate-100 px-2
py-1">sort by price</button>
      </div>
      <ul>{list.map((p) => <li key={p.id}>{p.name} · ₹{p.price}</li>)}</ul>
    </div>
  );
}

export default function App() {
  return <BrowserRouter><Routes><Route path="/products" element={<Products />} /></Routes>
</BrowserRouter>;
}
```

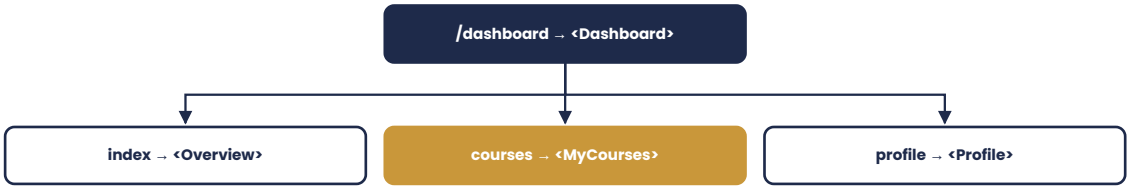


CLICKED “DISPLAYS”, THEN “SORT BY PRICE”: BOTH VALUES ARE NOW IN THE URL

13.3.3 Nested routes and Outlet

Routes can be **nested**: a child route's path continues its parent's path, and the child is rendered **inside** the parent wherever the parent places an `<Outlet />`. This mirrors how real screens are built: a

dashboard frame with a sidebar stays on screen while only the inner panel changes.



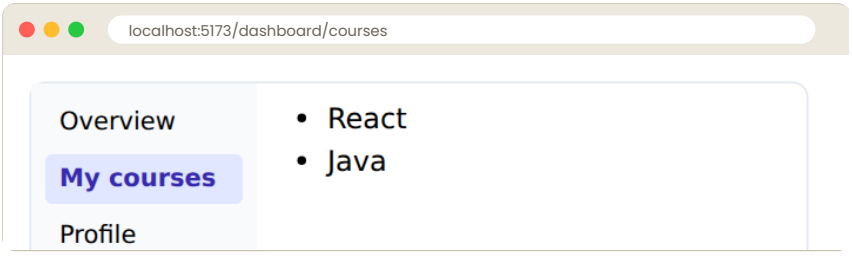
URL /DASHBOARD/COURSES RENDERS DASHBOARD WITH MYCOURSES IN ITS OUTLET

APP.JSX JavaScript

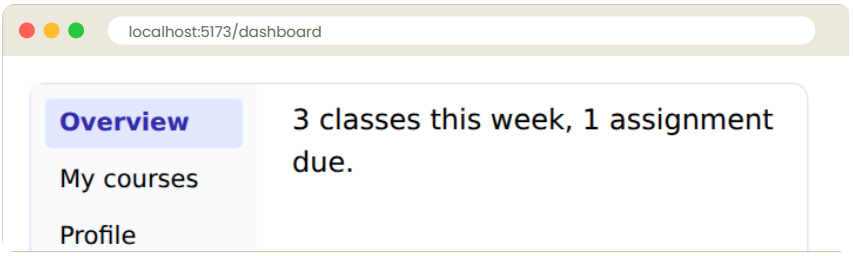
```
import { BrowserRouter, Routes, Route, NavLink, Outlet } from "react-router";

function Dashboard() {
  const item = ({ isActive }) => `block rounded px-2 py-1 ${isActive ? "bg-indigo-100 font-semibold
text-indigo-800" : ""}`;
  return (
    <div className="flex w-[440px] gap-3 rounded-lg ring-1 ring-slate-200">
      <aside className="w-32 space-y-1 bg-slate-50 p-2 text-sm">
        <NavLink to="/dashboard" end className={item}>Overview</NavLink>
        <NavLink to="/dashboard/courses" className={item}>My courses</NavLink>
        <NavLink to="/dashboard/profile" className={item}>Profile</NavLink>
      </aside>
      <main className="flex-1 p-2">
        <Outlet />               {/* the matching child route renders here */}
      </main>
    </div>
  );
}

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/dashboard" element={<Dashboard />}>
          <Route index element={<p>3 classes this week, 1 assignment due.</p>} />
          <Route path="courses" element={<ul className="list-disc pl-5"><li>React</li><li>Java</li>
</ul>} />
          <Route path="profile" element={<p>Asha Rao · asha@example.in</p>} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}
```



AT /DASHBOARD/COURSES: DASHBOARD (SIDEBAR) STAYS, MYCOURSES FILLS THE OUTLET



AT /DASHBOARD: THE INDEX ROUTE FILLS THE OUTLET

Child paths are written **without** a leading slash (`path="courses"`, not `"/courses"`): they are relative to the parent. A child path that starts with `/` would have to repeat the full parent path.

13.3.4 Layout routes

A **layout route** is a parent route **without a path**. It does not add anything to the URL; it only wraps its children in shared UI such as a header and footer. Use it to give a group of pages the same frame, and a different frame (or none) to others, for example a marketing layout for public pages and no layout for the login page.

A PATHLESS LAYOUT ROUTE JavaScript

```
<Routes>
  <Route element={<MainLayout />}>            {/* no path: a layout route */}
    <Route path="/" element={<Home />} />
    <Route path="/courses" element={<Courses />} />
  </Route>
  <Route path="/login" element={<Login />} />  {/* outside the layout: no header */}
</Routes>

function MainLayout() {
  return (
    <>
      <Header />
      <Outlet />
      <Footer />
    </>
  );
}
```

13.3.5 Index routes

An **index route** (`<Route index element={...} />`) is the default child: it renders in the parent's `Outlet` when the URL matches the parent **exactly**. Without one, `/dashboard` would show the sidebar with an empty panel. An index route has no `path` and cannot have children.

13.3.6 404 not-found routes

The path `*` matches **anything** that no other route matched, so it is used for a “not found” page. Put it last among its siblings (the real router ranks routes automatically, but placing it last keeps the intent obvious). A `*` route nested inside a layout keeps the layout around the 404 message.

APP.JSX

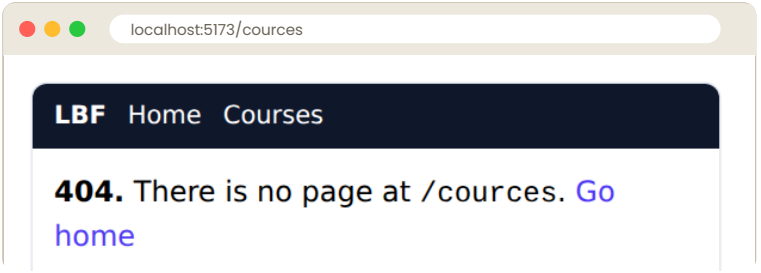
JavaScript

```
import { BrowserRouter, Routes, Route, Link, Outlet, useLocation } from "react-router";

function MainLayout() {
  return (
    <div className="w-96 overflow-hidden rounded-lg ring-1 ring-slate-200">
      <header className="flex gap-3 bg-slate-900 px-3 py-2 text-sm text-white"><b>LBF</b><Link
to="/">Home</Link><Link to="/courses">Courses</Link></header>
      <main className="p-3"><Outlet /></main>
      <footer className="bg-slate-100 px-3 py-1 text-xs text-slate-500">© Learn Being Forward</footer>
    </div>
  );
}

function NotFound() {
  const { pathname } = useLocation();
  return <p><b>404.</b> There is no page at <code>{pathname}</code>. <Link className="text-indigo-600"
to="/">Go home</Link></p>;
}

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route element={<MainLayout />}>                    {/* layout route */}
          <Route index element={<p>Welcome to LBF.</p>} />  {/* index route */}
          <Route path="courses" element={<p>Java · Python · React</p>} />
          <Route path="*" element={<NotFound />} />          {/* catch-all */}
        </Route>
      </Routes>
    </BrowserRouter>
  );
}
```

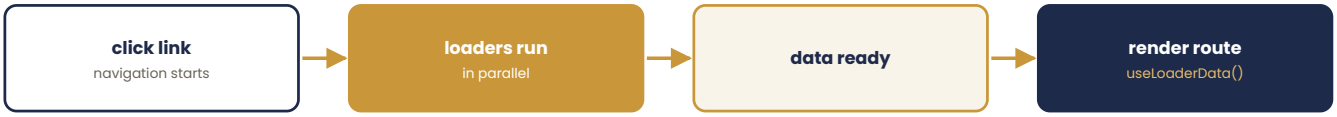


A MISTYPED URL: THE * ROUTE RENDERED NOTFOUND INSIDE THE LAYOUT

13.4 Data APIs

13.4.1 Loaders for fetching route data

With `useEffect` fetching (Module 9) a page renders first, shows a spinner, then fetches: and nested components often fetch one after another (a “waterfall”). A **loader** is a function attached to a route that the data router calls **before** rendering the route, as soon as navigation starts. Loaders of nested routes run in **parallel**. The component reads the result with `useLoaderData()`, so it never renders without its data. A loader receives `{ params, request }`.



THE DATA ROUTER LOADS FIRST, RENDERS SECOND

APP.JSX JavaScript

```
import { createBrowserRouter, RouterProvider, Link, useLoaderData } from "react-router";

async function studentsLoader() {
  const res = await fetch("/students.json");              // runs before the page renders
  return res.json();
}

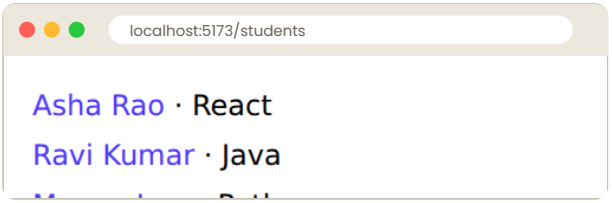
async function studentLoader({ params }) {
  const res = await fetch(`/student-${params.id}.json`);
  if (!res.ok) throw new Response("Student not found", { status: 404 });
  return res.json();
}

function Students() {
  const students = useLoaderData();
  return (
    <ul className="space-y-1">
      {students.map((s) => <li key={s.id}><Link className="text-indigo-600" to={`/students/${s.id}`}>
{s.name}</Link> · {s.course}</li>)}
    </ul>
  );
}

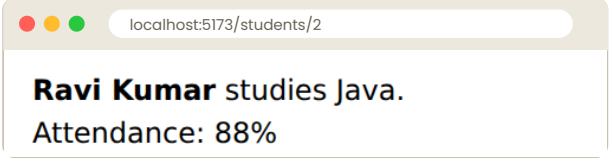
function Student() {
  const s = useLoaderData();
  return <p><b>{s.name}</b> studies {s.course}. Attendance: {s.attendance}%</p>;
}

const router = createBrowserRouter([
  { path: "/students", element: <Students />, loader: studentsLoader },
  { path: "/students/:id", element: <Student />, loader: studentLoader },
]);

export default function App() {
  return <RouterProvider router={router} />;
}
```



THE LIST CAME FROM THE LOADER (A REAL HTTP REQUEST FOR STUDENTS.JSON)



AFTER CLICKING "RAVI KUMAR": THE DETAIL LOADER FETCHED STUDENT-2.JSON BEFORE RENDERING

13.4.2 Actions for form submissions

An **action** handles data **mutations** for a route: creating, updating and deleting. Render the router's `<Form method="post">` instead of `<form>`: on submit it calls the matching route's action with a `request` whose `formData()` holds the fields (by their `name`). After the action finishes, the router **revalidates**: it runs the loaders again, so the page shows fresh data automatically. Whatever the action returns (for example validation errors) is available through `useActionData()`.

APP.JSX

JavaScript

```
import { createBrowserRouter, RouterProvider, Form, useLoaderData, useActionData } from "react-
router";

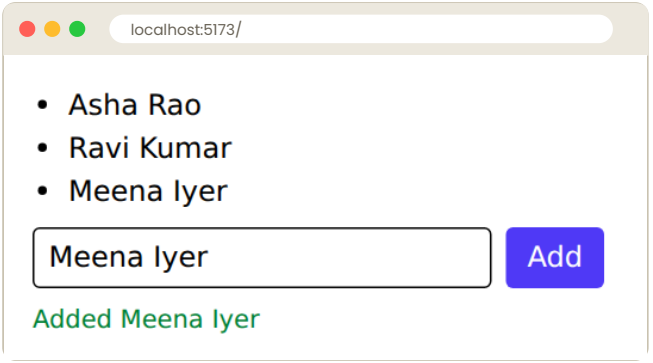
const db = ["Asha Rao", "Ravi Kumar"];                        // stands in for a server
const wait = (ms) => new Promise((r) => setTimeout(r, ms));

async function loader() { await wait(100); return [...db]; }

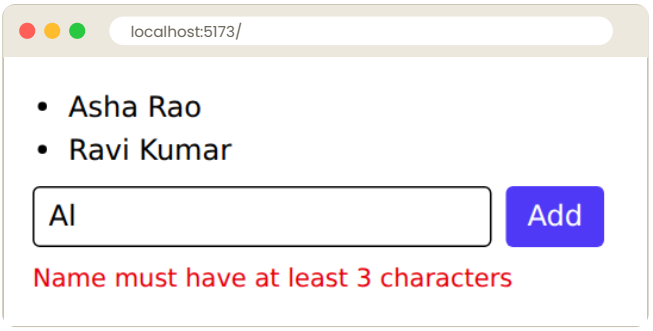
async function action({ request }) {
  const form = await request.formData();
  const name = form.get("name").trim();
  if (name.length < 3) return { error: "Name must have at least 3 characters" };
  await wait(100);
  db.push(name);                                              // e.g. a POST to the API
  return { ok: `Added ${name}` };
}

function Students() {
  const students = useLoaderData();
  const result = useActionData();
  return (
    <div className="w-80 space-y-2">
      <ul className="list-disc pl-5">{students.map((s) => <li key={s}>{s}</li>)}</ul>
      <Form method="post" className="flex gap-2">
        <input name="name" className="flex-1 rounded border px-2 py-1" placeholder="New student" />
        <button className="rounded bg-indigo-600 px-3 py-1 text-white">Add</button>
      </Form>
      {result?.error && <p className="text-sm text-red-600">{result.error}</p>}
      {result?.ok && <p className="text-sm text-green-700">{result.ok}</p>}
    </div>
  );
}

const router = createBrowserRouter([{ path: "/", element: <Students />, loader, action }]);
export default function App() { return <RouterProvider router={router} />; }
```



SUBMITTED “MEENA IYER”: THE ACTION SAVED IT, THEN THE LOADER RE-RAN AND THE LIST UPDATED



SUBMITTED "AL": THE ACTION RETURNED AN ERROR, SHOWN THROUGH USEACTIONDATA

13.4.3 Pending UI with useNavigation

While loaders or an action are running, the old page stays on screen. `useNavigation()` tells you what is going on: `navigation.state` is `"idle"`, `"loading"` (loaders running for the next page) or `"submitting"` (an action is running). Use it to dim the page, show a progress bar, or disable a submit button so it cannot be clicked twice.

APP.JSX

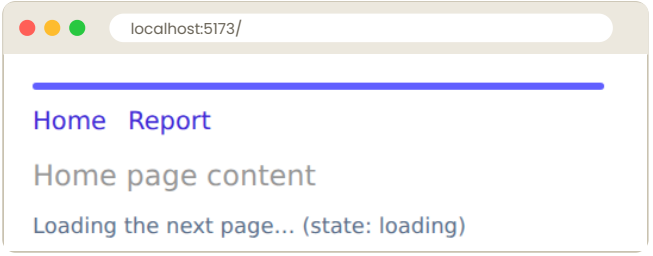
JavaScript

```
import { createBrowserRouter, RouterProvider, Link, Outlet, useNavigation } from "react-router";

const slow = (value) => new Promise((r) => setTimeout(() => r(value), 3000));

function Root() {
  const navigation = useNavigation();
  const busy = navigation.state !== "idle";
  return (
    <div className="w-80 space-y-2">
      <div className={`h-1 rounded ${busy ? "bg-indigo-500" : "bg-transparent"}`} />   {/* progress
bar */}
      <nav className="flex gap-3 text-sm text-indigo-700"><Link to="/">Home</Link><Link
to="/report">Report</Link></nav>
      <div className={busy ? "opacity-40" : ""}><Outlet /></div>
      {busy && <p className="text-xs text-slate-500">Loading the next page… (state:
{navigation.state})</p>}
    </div>
  );
}

const router = createBrowserRouter([
  { path: "/", element: <Root />, children: [
    { index: true, element: <p>Home page content</p> },
    { path: "report", element: <p>Report</p>, loader: () => slow("ok") },   // takes 3 s
  ] },
]);
export default function App() { return <RouterProvider router={router} />; }
```



CLICKED "REPORT": DURING THE 3-SECOND LOADER THE OLD PAGE IS DIMMED AND THE PENDING STATE IS SHOWN

13.4.4 Error elements

If a loader, an action or a component throws, the router renders the route's `errorElement` instead of its `element`. If that route has none, the error **bubbles up** to the nearest parent route that has one, so a single `errorElement` on the root route catches everything, and nested ones keep the surrounding layout visible. Inside the error element, `useRouteError()` returns what was thrown; `isRouteErrorResponse(error)` tells whether it was a thrown `Response` (with `status` and `statusText`) rather than a JavaScript error.

APP.JSX

JavaScript

```
import { createBrowserRouter, RouterProvider, Outlet, useLoaderData, useRouteError,
isRouteErrorResponse } from "react-router";

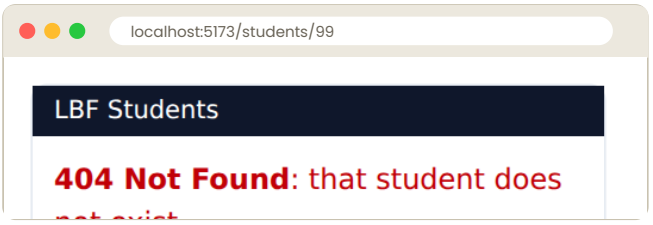
function Layout() {
  return <div className="w-80 rounded-lg ring-1 ring-slate-200"><header className="bg-slate-900 px-3
py-1 text-sm text-white">LBF Students</header><div className="p-3"><Outlet /></div></div>;
}

async function studentLoader({ params }) {
  const res = await fetch(`/student-${params.id}.json`);
  if (!res.ok) throw new Response("Not Found", { status: 404, statusText: "Not Found" });
  return res.json();
}

function Student() { const s = useLoaderData(); return <p>{s.name}</p>; }

function StudentError() {
  const error = useRouteError();
  if (isRouteErrorResponse(error)) {
    return <p className="text-red-700"><b>{error.status} {error.statusText}</b>: that student does not
exist.</p>;
  }
  return <p className="text-red-700">Something went wrong: {error.message}</p>;
}

const router = createBrowserRouter([
  { path: "/", element: <Layout />, children: [
    { path: "students/:id", element: <Student />, loader: studentLoader, errorElement: <StudentError
/> },
  ] },
]);
export default function App() { return <RouterProvider router={router} /> }
```



OPENED /STUDENTS/99: THE LOADER THREW A 404 RESPONSE AND THE NESTED ERRORELEMENT REPLACED ONLY THE INNER PART

13.5 Protected routes

13.5.1 Guarding routes by authentication and role

Some pages are only for logged-in users (orders, profile), and some only for certain roles (an admin panel). A common pattern is a **guard component** used as a layout route: it checks the user (from the

auth context of Module 12) and either renders `<Outlet />` or redirects with `<Navigate>`. Passing the attempted location in `state` lets the login page send the user back where they wanted to go.

AUTH.JSX JavaScript

```
import { createContext, useContext, useState } from "react";

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const login = (name, role = "student") => setUser({ name, role });
  const logout = () => setUser(null);
  return <AuthContext value={{ user, login, logout }}>{children}</AuthContext>;
}
export const useAuth = () => useContext(AuthContext);
```

APP.JSX JavaScript

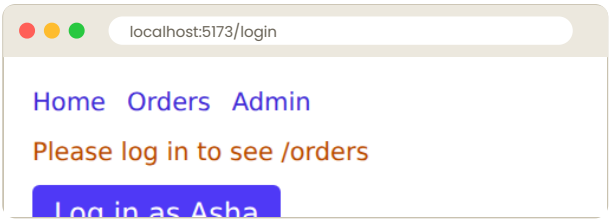
```
import { BrowserRouter, Routes, Route, Navigate, Outlet, Link, useLocation, useNavigate } from "react-
router";
import { AuthProvider, useAuth } from "./auth.jsx";

function RequireAuth({ role }) {
  const { user } = useAuth();
  const location = useLocation();
  if (!user) return <Navigate to="/login" replace state={{ from: location.pathname }} />;
  if (role && user.role !== role) return <p className="text-red-700">403: only {role}s can see this
page.</p>;
  return <Outlet />;
}

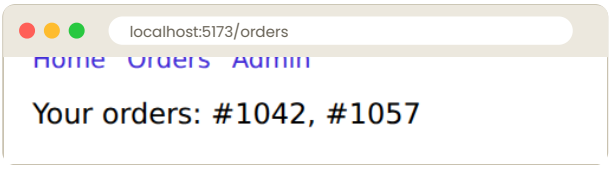
function Login() {
  const { login } = useAuth();
  const navigate = useNavigate();
  const from = useLocation().state?.from ?? "/";
  return (
    <div className="space-y-2">
      {from !== "/" && <p className="text-sm text-amber-700">Please log in to see {from}</p>}
      <button className="rounded bg-indigo-600 px-3 py-1 text-white"
              onClick={() => { login("Asha"); navigate(from, { replace: true }); }}>Log in as
Asha</button>
    </div>
  );
}

export default function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <nav className="mb-2 flex gap-3 text-sm text-indigo-700"><Link to="/">Home</Link><Link
to="/orders">Orders</Link><Link to="/admin">Admin</Link></nav>
        <Routes>
          <Route path="/" element={<p>Public home page</p>} />
          <Route path="/login" element={<Login />} />
          <Route element={<RequireAuth />}>                  {/* any logged-in user */}
            <Route path="/orders" element={<p>Your orders: #1042, #1057</p>} />
          </Route>
          <Route element={<RequireAuth role="admin" />}>      {/* admins only */}
            <Route path="/admin" element={<p>Admin panel</p>} />
          </Route>
        </Routes>
      </BrowserRouter>
```

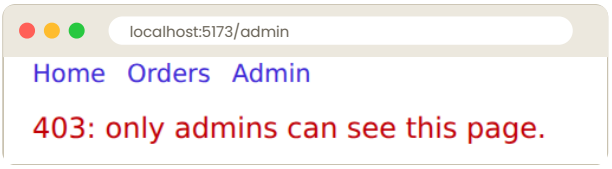
```
    </AuthProvider>
  );
}
```



CLICKED “ORDERS” WHILE LOGGED OUT: THE GUARD REDIRECTED TO /LOGIN AND REMEMBERED WHERE WE WERE GOING



CLICKED “LOG IN AS ASHA”: THE LOGIN PAGE SENT US BACK TO /ORDERS



LOGGED IN AS A STUDENT, THEN OPENED “ADMIN”: THE ROLE CHECK REFUSED

WATCH OUT

Route guards only control what the **user interface** shows. Anyone can read the JavaScript bundle and call your API directly, so the **server** must check the token and the role on every request. Never treat a hidden page as security.

13.6 Code splitting routes

13.6.1 lazy() and Suspense

By default the bundler puts every page into one JavaScript file, so the first visit downloads the admin panel, the reports and every other page, even if the user only opens the home page. **Code splitting** moves each lazily loaded page into its own file (a **chunk**) that is downloaded the first time it is needed. `lazy(() => import("./Reports.jsx"))` creates a component that loads its code on first render; while it loads, the nearest `<Suspense fallback>` above it shows the fallback.

REPORTS.JSX JavaScript

```
export default function Reports() {
  return <p>Monthly report: 128 enrolments, 94% attendance.</p>;
}
```

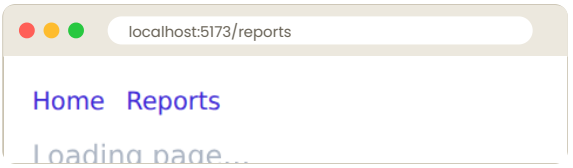
APP.JSX

JavaScript

```
import { BrowserRouter, Routes, Route, Link } from "react-router";
import { lazy, Suspense } from "react";

// The import starts only when <Reports /> first renders. The delay simulates a slow network.
const Reports = lazy(() => new Promise((r) => setTimeout(r, 3000)).then(() =>
import("./Reports.jsx")));

export default function App() {
  return (
    <BrowserRouter>
      <nav className="mb-2 flex gap-3 text-sm text-indigo-700"><Link to="/">Home</Link><Link
to="/reports">Reports</Link></nav>
      <Suspense fallback={<p className="animate-pulse text-slate-500">Loading page…</p>}>
        <Routes>
          <Route path="/" element={<p>Home page</p>} />
          <Route path="/reports" element={<Reports />} />
        </Routes>
      </Suspense>
    </BrowserRouter>
  );
}
```



RIGHT AFTER CLICKING “REPORTS”: ITS CODE IS STILL DOWNLOADING, SO SUSPENSE SHOWS THE FALLBACK

In a real project you write simply `lazy(() => import("./pages/Reports.jsx"))`, and Vite produces a separate file such as `Reports-[hash].js` in `dist/assets`. Split by route first: it is the natural boundary, and users accept a short wait when they move to a new page. In data mode, a route object can also use the `lazy` property to load its component, loader and action together.

DATA MODE: ROUTE-LEVEL LAZY LOADING

JavaScript

```
const router = createBrowserRouter([
  { path: "/", element: <Home /> },
  { path: "/reports", lazy: () => import("./routes/reports.jsx") },   // exports Component, loader...
]);
```

13.7 Framework mode (overview)

In **framework mode** React Router becomes a full framework (it is the successor of Remix). Routes are listed in `app/routes.ts`, and each route module exports its component, `loader`, `action` and metadata. The framework adds **server-side rendering**, static pre-rendering, automatic code splitting per route and generated TypeScript types for params and loader data. It is a peer of Next.js, which Module 21 covers in depth.

FRAMEWORK MODE (REPRESENTATIVE) JavaScript

```
// app/routes.ts
import { type RouteConfig, index, route, layout } from "@react-router/dev/routes";

export default [
  index("routes/home.tsx"),
  layout("routes/shop-layout.tsx", [
    route("products", "routes/products.tsx"),
    route("products/:id", "routes/product.tsx"),
  ]),
  route("*", "routes/not-found.tsx"),
] satisfies RouteConfig;

// app/routes/product.tsx
export async function loader({ params }) {          // can run on the server
  return getProduct(params.id);
}
export default function Product({ loaderData }) {
  return <h1>{loaderData.name}</h1>;
}
```

PRACTICE CHECKPOINT

A multi-page store with Home, Products, Product Detail (dynamic), Cart, Login and a protected Orders page, with nested layouts and a 404 page.

STORE.JSX JavaScript

```
import { createContext, useContext, useState } from "react";

export const products = [
  { id: 1, name: "Wireless mouse", price: 799 },
  { id: 2, name: "Mechanical keyboard", price: 3499 },
  { id: 3, name: "27-inch monitor", price: 15999 },
];
export const rupees = (n) => "₹" + n.toLocaleString("en-IN");

const StoreContext = createContext(null);

export function StoreProvider({ children }) {
  const [cart, setCart] = useState([]);              // [{ id, qty }]
  const [user, setUser] = useState(null);
  const add = (id) => setCart((c) => c.some((i) => i.id === id)
    ? c.map((i) => (i.id === id ? { ...i, qty: i.qty + 1 } : i))
    : [...c, { id, qty: 1 }]);
  const value = { cart, add, clear: () => setCart([]), user, login: (name) => setUser({ name }),
logout: () => setUser(null) };
  return <StoreContext value={value}>{children}</StoreContext>;
}
export const useStore = () => useContext(StoreContext);
```

PAGES.JSX JavaScript

```
import { Link, NavLink, Outlet, Navigate, useParams, useNavigate, useLocation } from "react-router";
import { products, rupees, useStore } from "./store.jsx";

export function ShopLayout() {
  const { cart, user, logout } = useStore();
  const count = cart.reduce((n, i) => n + i.qty, 0);
  const nav = ({ isActive }) => (isActive ? "font-semibold text-amber-300" : "");
```

```
  return (
    <div className="w-[540px] overflow-hidden rounded-lg ring-1 ring-slate-200">
      <header className="flex items-center gap-4 bg-slate-900 px-3 py-2 text-sm text-white">
        <b>LBF Store</b>
        <NavLink to="/" end className={nav}>Home</NavLink>
        <NavLink to="/products" className={nav}>Products</NavLink>
        <NavLink to="/orders" className={nav}>Orders</NavLink>
        <NavLink to="/cart" className={nav}>Cart ({count})</NavLink>
        <span className="ml-auto">{user ? <button onClick={logout}>Log out {user.name}</button> :
<Link to="/login">Log in</Link>}</span>
      </header>
      <main className="p-3"><Outlet /></main>
    </div>
  );
}

export function Home() {
  return <p>Welcome! Browse our <Link className="text-indigo-600" to="/products">products</Link>.</p>;
}

export function Products() {
  return (
    <ul className="divide-y">
      {products.map((p) => (
        <li key={p.id} className="flex justify-between py-1">
          <Link className="text-indigo-600" to={`/products/${p.id}`}>{p.name}</Link><span>
{rupees(p.price)}</span>
        </li>
      ))}
    </ul>
  );
}

export function ProductDetail() {
  const { id } = useParams();
  const { add } = useStore();
  const product = products.find((p) => p.id === Number(id));
  if (!product) return <NotFound />;
  return (
    <div className="space-y-2">
      <h2 className="text-lg font-semibold">{product.name}</h2>
      <p>{rupees(product.price)}</p>
      <button className="rounded bg-indigo-600 px-3 py-1 text-white" onClick={() =>
add(product.id)}>Add to cart</button>
    </div>
  );
}

export function Cart() {
  const { cart } = useStore();
  if (cart.length === 0) return <p>Your cart is empty.</p>;
  const rows = cart.map((i) => ({ ...products.find((p) => p.id === i.id), qty: i.qty }));
  const total = rows.reduce((s, r) => s + r.price * r.qty, 0);
  return (
    <div className="space-y-1">
      {rows.map((r) => <p key={r.id}>{r.name} × {r.qty} = {rupees(r.price * r.qty)}</p>)}
      <p className="border-t pt-1 font-semibold">Total {rupees(total)}</p>
    </div>
  );
}

export function RequireAuth() {
  const { user } = useStore();
  const location = useLocation();
```

```
  return user ? <Outlet /> : <Navigate to="/login" replace state={{ from: location.pathname }} />;
}

export function Login() {
  const { login } = useStore();
  const navigate = useNavigate();
  const from = useLocation().state?.from ?? "/";
  return (
    <form className="flex gap-2" onSubmit={(e) => { e.preventDefault(); login(e.target.name.value);
navigate(from, { replace: true }); }}>
      <input name="name" className="rounded border px-2 py-1" placeholder="Name" required />
      <button className="rounded bg-indigo-600 px-3 py-1 text-white">Log in</button>
    </form>
  );
}

export function Orders() {
  const { user } = useStore();
  return <p>{user.name}, you have 2 past orders: #1042 (₹4,298) and #1057 (₹799).</p>;
}

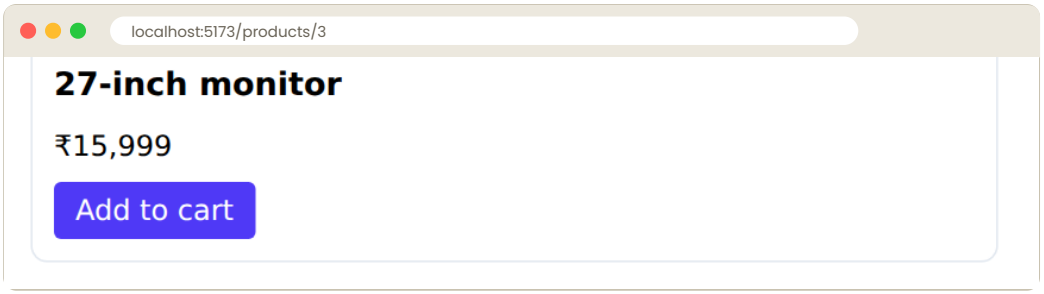
export function NotFound() {
  return <p><b>404.</b> Page not found. <Link className="text-indigo-600" to="/">Back to the
shop</Link></p>;
}
```

APP.JSX: SAMPLE SOLUTION JavaScript

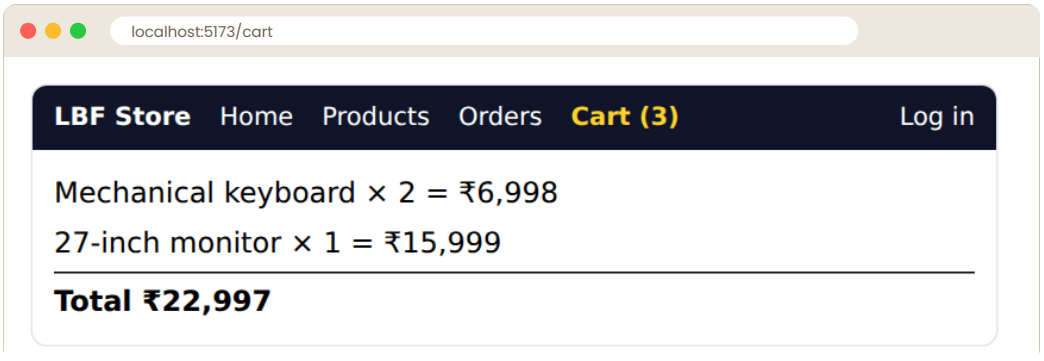
```
import { createBrowserRouter, RouterProvider } from "react-router";
import { StoreProvider } from "./store.jsx";
import { ShopLayout, Home, Products, ProductDetail, Cart, Login, Orders, RequireAuth, NotFound } from
"./pages.jsx";

const router = createBrowserRouter([
  {
    path: "/",
    element: <ShopLayout />,                    // nested layout for every page
    children: [
      { index: true, element: <Home /> },
      { path: "products", element: <Products /> },
      { path: "products/:id", element: <ProductDetail /> },
      { path: "cart", element: <Cart /> },
      { path: "login", element: <Login /> },
      { element: <RequireAuth />, children: [   // protected group
        { path: "orders", element: <Orders /> },
      ] },
      { path: "*", element: <NotFound /> },
    ],
  },
]);

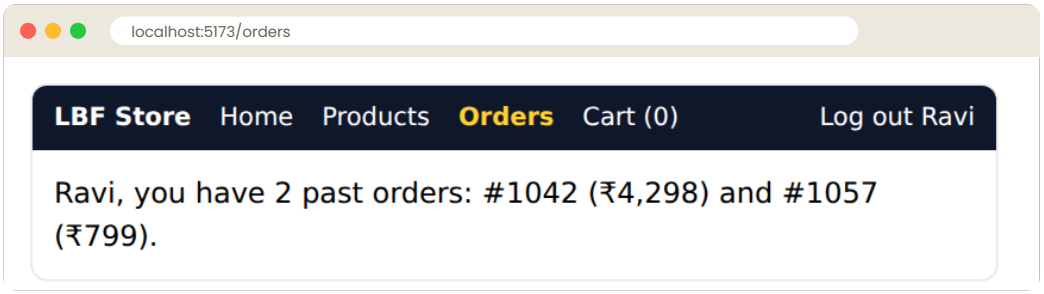
export default function App() {
  return (
    <StoreProvider>
      <RouterProvider router={router} />
    </StoreProvider>
  );
}
```



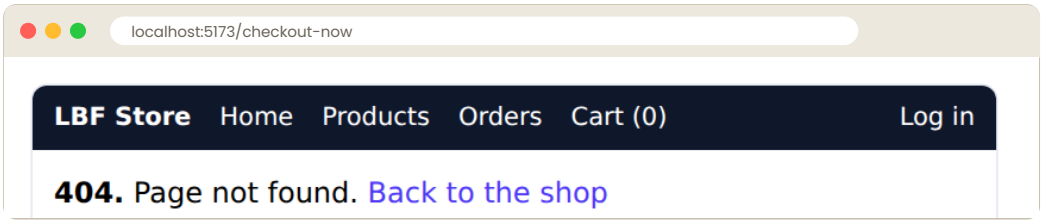
PRODUCTS, THEN “MECHANICAL KEYBOARD”, ADD TO CART TWICE, THEN “27-INCH MONITOR” AND ADD TO CART: THE CART COUNT IS 3



THEN “CART”: LINE TOTALS AND THE ORDER TOTAL



“ORDERS” WHILE LOGGED OUT REDIRECTED TO /LOGIN; AFTER LOGGING IN AS RAVI THE ORDERS PAGE OPENED



A URL THAT MATCHES NOTHING: THE 404 PAGE INSIDE THE SHOP LAYOUT

Client-side routing maps URLs to components without reloading. Declarative mode uses `<BrowserRouter>`, `<Routes>` and `<Route>`; data mode uses `createBrowserRouter` with loaders, actions, `useNavigation` and `errorElement`. Navigate with `Link`, `NavLink` (active styles), `useNavigate` and `<Navigate>`. Read `useParams` and `useSearchParams`; nest routes with `<Outlet />`, layout, index and `*` routes; guard pages with a wrapper route (and always check on the server); split pages with `lazy` and `Suspense`.

MODULE 14 · INTERMEDIATE

Working with APIs and forms at scale

Minimum duration **3 days** | Level **Intermediate**

Real applications talk to a back end all the time and collect data through many forms. This module covers REST and HTTP methods, fetch versus Axios, an API service layer with environment variables, authentication headers and CORS errors; server state with TanStack Query (queries, caching, stale time, mutations, invalidation, optimistic updates, pagination and DevTools); large forms with React Hook Form, validation rules and Zod schemas, including controlled third-party inputs; and image uploads with FormData and previews.

IN THIS MODULE

14.1 API communication
14.2 TanStack Query (React Query)
14.3 Forms with React Hook Form
14.4 File uploads

HOW THE EXAMPLES WERE RUN

TanStack Query, React Hook Form, Zod and Axios are not installed in the offline build used for these notes. The listings are written exactly as for the real packages (TanStack Query 5, React Hook Form 7, Zod 4). The screenshots were produced by bundling the same code against small stand-in modules with the same API and running it in Chromium. Where an example needs a server, a small in-memory module (`api.js`, shown once) plays the role of a REST API with realistic delays.

14.1 API communication

14.1.1 REST basics: GET, POST, PUT, PATCH, DELETE

Most front ends talk to a **REST API**: the server exposes **resources** at URLs (`/students`, `/students/7`) and the HTTP **method** says what to do with them. Data travels as JSON. The server answers with a **status code** that tells the client how the request went, and the client must check it: `fetch` only rejects on network failures, not on 404 or 500.

Method	Meaning	Example	Typical success status
`GET`	read (never changes data)	`GET /students`, `GET /students/7`	200 OK
`POST`	create a new resource	`POST /students` with a JSON body	201 Created
`PUT`	replace a resource completely	`PUT /students/7` with all fields	200 OK
`PATCH`	change some fields	`PATCH /students/7` with `{"course":"React"}`	200 OK
`DELETE`	remove a resource	`DELETE /students/7`	204 No Content

Status range	Meaning	Common codes
2xx	success	200 OK, 201 Created, 204 No Content
3xx	redirection	301 Moved Permanently, 304 Not Modified
4xx	the client made a mistake	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 422 Unprocessable
5xx	the server failed	500 Internal Server Error, 503 Service Unavailable

14.1.2 fetch vs Axios

`fetch` is built into every browser and Node. **Axios** is a popular library that wraps HTTP requests with conveniences. Both are fine; many teams start with fetch plus a tiny wrapper (next section), which removes most of the differences.

	fetch	Axios
Install	built in	`npm install axios` (about 13 kB gzipped)
JSON body	`JSON.stringify(data)` and a `Content-Type` header	pass the object; Axios serialises it
Reading JSON	`await res.json()`	already parsed in `res.data`
HTTP errors (404, 500)	do **not** reject: check `res.ok`	reject automatically
Base URL, shared headers	write a wrapper	`axios.create({ baseURL, headers })`
Interceptors	write a wrapper	built in (add tokens, handle 401 globally)
Cancel	`AbortController`	`AbortController` (signal option)

THE SAME POST REQUEST WITH FETCH AND WITH AXIOS JavaScript

```
// fetch
const res = await fetch("/api/students", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name: "Asha", course: "React" }),
});
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const created = await res.json();

// Axios
import axios from "axios";
const { data: created2 } = await axios.post("/api/students", { name: "Asha", course: "React" });
```

14.1.3 API service layer and environment variables

Do not scatter `fetch` calls across components. Put every request in an **API service layer**: one module that knows the base URL, adds headers, checks status codes and turns errors into useful exceptions. Components then call meaningful functions such as `studentsApi.list()`, and if the back end changes only one file changes. The base URL differs between development and production, so it comes from an

environment variable. In Vite, variables in `.env` files that start with `VITE_` are exposed to the browser code as `import.meta.env.VITE_...`.

.env files

```
# .env.development
VITE_API_URL=http://localhost:3000

# .env.production
VITE_API_URL=https://api.learnbeingforward.in
```

APICLIENT.JS JavaScript

```
const BASE_URL = import.meta.env?.VITE_API_URL ?? "";      // "" = same origin

export class ApiError extends Error {
  constructor(status, message) { super(message); this.status = status; }
}

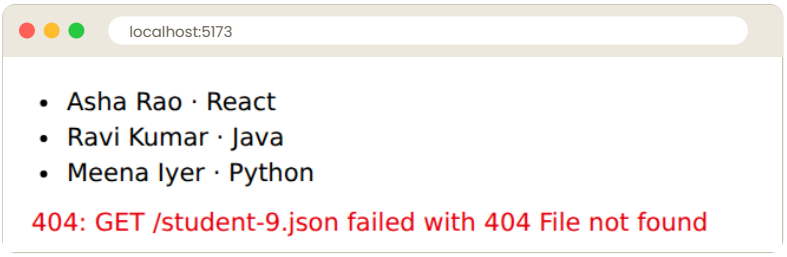
export async function request(path, { method = "GET", body, token } = {}) {
  const headers = { Accept: "application/json" };
  if (body !== undefined) headers["Content-Type"] = "application/json";
  if (token) headers.Authorization = `Bearer ${token}`;
  const res = await fetch(BASE_URL + path, { method, headers, body: body === undefined ? undefined :
JSON.stringify(body) });
  if (!res.ok) throw new ApiError(res.status, `${method} ${path} failed with ${res.status}
${res.statusText}`);
  return res.status === 204 ? null : res.json();
}

export const studentsApi = {
  list: () => request("/students.json"),
  get: (id) => request(`/student-${id}.json`),
};
```

APP.JSX JavaScript

```
import { useEffect, useState } from "react";
import { studentsApi, ApiError } from "./apiClient.js";

export default function App() {
  const [students, setStudents] = useState([]);
  const [error, setError] = useState("");
  useEffect(() => {
    studentsApi.list().then(setStudents);
    studentsApi.get(9).catch((e) => setError(e instanceof ApiError ? `${e.status}: ${e.message}` :
e.message));
  }, []);
  return (
    <div className="w-96 space-y-2 text-sm">
      <ul className="list-disc pl-5">{students.map((s) => <li key={s.id}>{s.name} · {s.course}</li>)}
</ul>
      {error && <p className="text-red-600">{error}</p>}
    </div>
  );
}
```



THE LIST LOADED THROUGH THE SERVICE LAYER; THE MISSING STUDENT BECAME A CLEAR APIERROR

14.1.4 Authentication headers and tokens

After login the server usually returns a **token** (often a JWT). Every later request proves who the user is by sending it in the `Authorization` header: `Authorization: Bearer <token>`. Keep that logic in the service layer, not in components. When the server answers `401 Unauthorized` the token is missing or expired: log the user out, or refresh the token and retry. Where to keep the token safely is discussed in Module 22 (security).

ADDING THE TOKEN TO EVERY REQUEST JavaScript

```
// apiClient.js (continued): the token comes from the auth store
let token = null;
export const setToken = (t) => { token = t; };

export async function authRequest(path, options = {}) {
  try {
    return await request(path, { ...options, token });
  } catch (e) {
    if (e.status === 401) {
      setToken(null);
      window.dispatchEvent(new Event("auth:logout"));   // the AuthProvider listens and shows the
login page
    }
    throw e;
  }
}

// Axios equivalent: an interceptor adds the header to every request
api.interceptors.request.use((config) => {
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

BROWSER CONSOLE: HEADERS SENT WITH THE REQUEST

```
accept: application/json
authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI3In0.abc
```

14.1.5 CORS errors

Browsers apply the **same-origin policy**: JavaScript on `http://localhost:5173` may not read responses from a different **origin** (scheme + host + port), such as `http://localhost:3000`, unless that server allows it with **CORS** headers (Cross-Origin Resource Sharing). Without them the request may even reach the server, but the browser blocks your code from reading the answer and logs an error like the one below. CORS is fixed on the **server**, never in React.

A TYPICAL CORS ERROR IN THE BROWSER CONSOLE main.js

```
Access to fetch at 'http://localhost:3000/students' from origin 'http://localhost:5173'
has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on
the requested resource.
```

Fix	Where	How
Allow the origin	the API server	send `Access-Control-Allow-Origin: http://localhost:5173` (Express: `app.use(cors({ origin: "http://localhost:5173" }))`)
Proxy in development	`vite.config.js`	`server: { proxy: { "/api": "http://localhost:3000" } }`: the browser talks to the same origin, Vite forwards
Same origin in production	hosting	serve the app and the API from one domain, or put both behind a reverse proxy

14.2 TanStack Query (React Query)

14.2.1 Why server state is different from client state

Client state (is the menu open, what is typed in the input) belongs to the browser and is always up to date. **Server state** (the list of students) lives on a server: your copy is only a **snapshot** that may already be out of date, other users can change it, and fetching it is slow and can fail. Managing it with `useState` and `useEffect` means hand-writing loading flags, error handling, caching, de-duplication of identical requests, refetching when data gets old, and updating the list after a change. **TanStack Query** does all of that for you.

Client state	Server state
owned by the browser	owned by the server, shared by users
always current	a snapshot that becomes stale
instant to read and write	slow, asynchronous, can fail
useState, useReducer, context, Zustand	TanStack Query, SWR, RTK Query, router loaders

TWO KINDS OF STATE, TWO KINDS OF TOOLS

API.JS (USED BY THE EXAMPLES IN THIS SECTION)

JavaScript

```
// api.js: an in-memory stand-in for a REST API (JSON Server, Express...)
const wait = (ms) => new Promise((r) => setTimeout(r, ms));
let nextId = 4;
let students = [
  { id: 1, name: "Asha Rao", course: "React", email: "asha@example.in" },
  { id: 2, name: "Ravi Kumar", course: "Java", email: "ravi@example.in" },
  { id: 3, name: "Meena Iyer", course: "Python", email: "meena@example.in" },
];
export const calls = { count: 0 };
export const settings = { fail: false };

export async function getStudents() {
  calls.count++;
  await wait(1500);
  return students.map((s) => ({ ...s }));
}
export async function addStudent(data) {
  await wait(1500);
  if (settings.fail) throw new Error("500 Internal Server Error");
  const s = { id: nextId++, ...data };
  students = [...students, s];
  return s;
}
export async function deleteStudent(id) {
  await wait(1500);
  if (settings.fail) throw new Error("500 Internal Server Error");
  students = students.filter((s) => s.id !== id);
}
export async function getPage(page, size = 3) {
  await wait(1500);
  const all = Array.from({ length: 10 }, (_, i) => ({ id: i + 1, name: `Student ${i + 1}` }));
  return { items: all.slice((page - 1) * size, page * size), hasMore: page * size < all.length };
}
```

14.2.2 QueryClient and useQuery

Create one `QueryClient` (it holds the cache) and provide it with `<QueryClientProvider>` at the top of the app. Then any component calls `useQuery({ queryKey, queryFn })`. The **query key** is an array that identifies the data (`["students"]`, `["students", 7]`); the **query function** returns a promise. The hook returns the data plus status flags: `isPending` (no data yet), `isError` and `error`, and `isFetching` (a request is in flight, including background refreshes).

APP.JSX

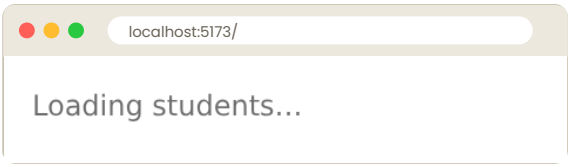
JavaScript

```
import { QueryClient, QueryClientProvider, useQuery } from "@tanstack/react-query";
import { getStudents } from "./api.js";

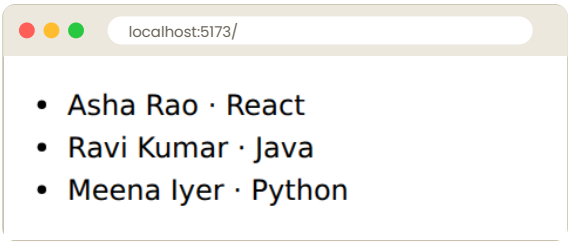
const queryClient = new QueryClient();

function Students() {
  const { data, isPending, isError, error } = useQuery({
    queryKey: ["students"],
    queryFn: getStudents,
  });
  if (isPending) return <p className="animate-pulse">Loading students…</p>;
  if (isError) return <p className="text-red-600">Error: {error.message}</p>;
  return <ul className="list-disc pl-5">{data.map((s) => <li key={s.id}>{s.name} · {s.course}</li>)}
</ul>;
}

export default function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <Students />
    </QueryClientProvider>
  );
}
```



RIGHT AFTER MOUNTING: THE QUERY IS PENDING (THE STAND-IN API TAKES 1.5 S)



ONCE THE REQUEST FINISHED: DATA ARRIVED AND THE LIST RENDERED; NO USESTATE OR USEEFFECT WAS WRITTEN

14.2.3 Caching, stale time and refetching

Results are **cached by query key**. When a second component (or the same one, mounted again) asks for `["students"]`, it gets the cached data **immediately**. Whether TanStack Query also refetches depends on `staleTime`: data is **fresh** for that long and is not refetched; after that it is **stale** and is refetched in the background on mount, on window focus and on reconnect, while the old data stays on screen. The default `staleTime` is 0 (always refetch in the background); unused data is removed from the cache after `gcTime` (5 minutes by default).

APP.JSX

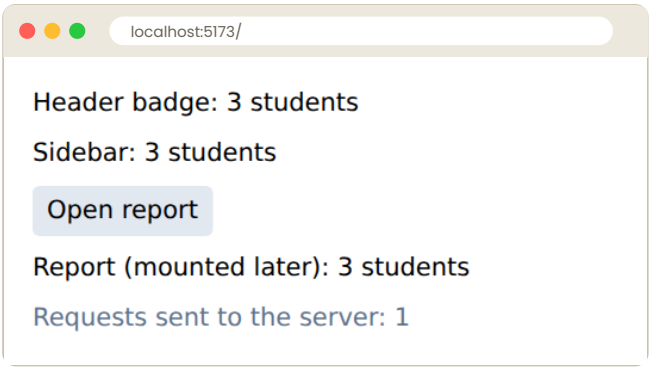
JavaScript

```
import { QueryClient, QueryClientProvider, useQuery } from "@tanstack/react-query";
import { useState } from "react";
import { getStudents, calls } from "./api.js";

const queryClient = new QueryClient({ defaultOptions: { queries: { staleTime: 60_000 } } });  // fresh
for 1 minute

function StudentCount({ label }) {
  const { data, isFetching } = useQuery({ queryKey: ["students"], queryFn: getStudents });
  return <p>{label}: {data ? `${data.length} students` : "loading…"} {isFetching && "(fetching)"}</p>;
}

export default function App() {
  const [show, setShow] = useState(false);
  return (
    <QueryClientProvider client={queryClient}>
      <div className="w-80 space-y-2 text-sm">
        <StudentCount label="Header badge" />
        <StudentCount label="Sidebar" />                {/* same key: ONE request, shared */}
        <button className="rounded bg-slate-200 px-2 py-1" onClick={() => setShow(true)}>Open
report</button>
        {show && <StudentCount label="Report (mounted later)" />}   {/* served from the cache */}
        <p className="text-slate-500">Requests sent to the server: {calls.count}</p>
      </div>
    </QueryClientProvider>
  );
}
```



TWO COMPONENTS, ONE REQUEST; THE REPORT MOUNTED LATER GOT THE DATA FROM THE CACHE INSTANTLY, WITHOUT A NEW REQUEST

Option	Default	Meaning
`staleTime`	0	how long data counts as fresh (no refetch); set it per query or as a default
`gcTime`	5 minutes	how long unused data stays in the cache
`refetchOnWindowFocus`	true	refetch stale queries when the user returns to the tab
`retry`	3 (in the browser)	how many times a failed query is retried, with growing delays
`enabled`	true	set to false to wait, e.g. `enabled: !!userId` for dependent queries
`select`	-	transform or pick part of the data for this component

14.2.4 useMutation and invalidating queries

Reads use `useQuery`; writes (create, update, delete) use `useMutation`. Call `mutation.mutate(variables)`; the hook tracks `isPending`, `isError` and `isSuccess`. After a successful write the cached list is out of date, so tell the client: `queryClient.invalidateQueries({ queryKey: ["students"] })` marks every query whose key starts with `["students"]` as stale and refetches the ones on screen.

APP.JSX

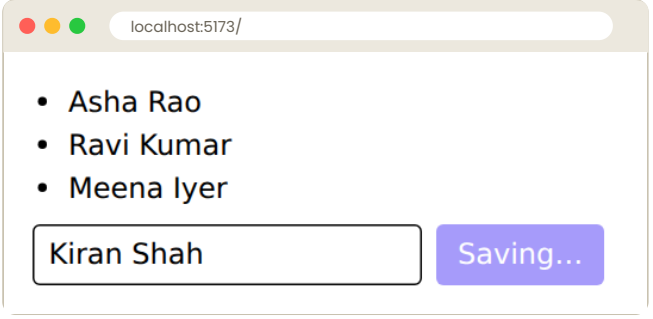
JavaScript

```
import { QueryClient, QueryClientProvider, useQuery, useMutation, useQueryClient } from
"@tanstack/react-query";
import { useState } from "react";
import { getStudents, addStudent } from "./api.js";

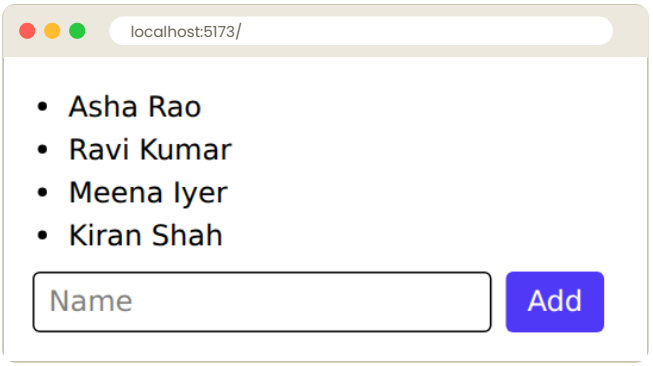
function AddStudent() {
  const [name, setName] = useState("");
  const queryClient = useQueryClient();
  const mutation = useMutation({
    mutationFn: addStudent,
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["students"] });   // refetch the list
      setName("");
    },
  });
  return (
    <form className="flex gap-2" onSubmit={(e) => { e.preventDefault(); mutation.mutate({ name,
course: "React" }); }}>
      <input className="flex-1 rounded border px-2 py-1" value={name} onChange={(e) =>
setName(e.target.value)} placeholder="Name" />
      <button disabled={mutation.isPending} className="rounded bg-indigo-600 px-3 py-1 text-white
disabled:opacity-50">
        {mutation.isPending ? "Saving…" : "Add"}
      </button>
    </form>
  );
}

function Students() {
  const { data = [], isFetching } = useQuery({ queryKey: ["students"], queryFn: getStudents });
  return <ul className="list-disc pl-5">{data.map((s) => <li key={s.id}>{s.name}</li>)}{isFetching &&
<li className="text-slate-400">refreshing…</li>}</ul>;
}

const queryClient = new QueryClient();
export default function App() {
  return <QueryClientProvider client={queryClient}><div className="w-80 space-y-2"><Students />
<AddStudent /></div></QueryClientProvider>;
}
```



WHILE SAVING: THE BUTTON IS DISABLED AND SHOWS THE PENDING STATE



AFTER SUCCESS: THE LIST WAS INVALIDATED, REFETCHED AND NOW CONTAINS KIRAN SHAH

14.2.5 Optimistic updates

An **optimistic update** changes the screen **before** the server confirms, so the app feels instant, and undoes the change if the server fails. With TanStack Query: in `onMutate` cancel running fetches, save the current data, write the expected new data with `setQueryData`, and return the saved copy as **context**; in `onError` put the saved copy back; in `onSettled` invalidate to sync with the server either way.

APP.JSX JavaScript

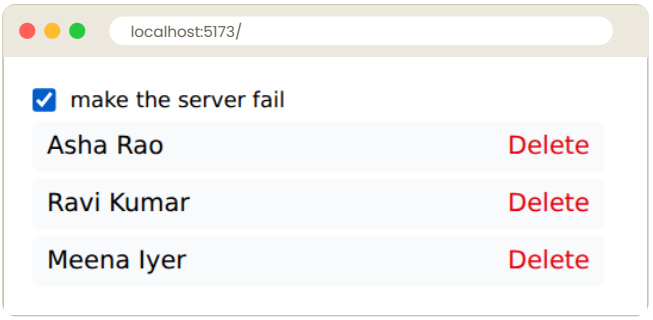
```
import { QueryClient, QueryClientProvider, useQuery, useMutation, useQueryClient } from
"@tanstack/react-query";
import { getStudents, deleteStudent, settings } from "./api.js";

function Students() {
  const queryClient = useQueryClient();
  const { data = [] } = useQuery({ queryKey: ["students"], queryFn: getStudents });
  const remove = useMutation({
    mutationFn: deleteStudent,
    onMutate: async (id) => {
      await queryClient.cancelQueries({ queryKey: ["students"] });
      const previous = queryClient.getQueryData(["students"]);
      queryClient.setQueryData(["students"], (old) => old.filter((s) => s.id !== id));  // instant
      return { previous };                                          // context for onError
    },
    onError: (err, id, context) => {
      queryClient.setQueryData(["students"], context.previous);   // roll back
    },
    onSettled: () => queryClient.invalidateQueries({ queryKey: ["students"] }),
  });
  return (
    <div className="w-80 space-y-1 text-sm">
      <label className="flex gap-2 text-xs"><input type="checkbox" onChange={(e) => (settings.fail =
e.target.checked)} /> make the server fail</label>
      {data.map((s) => (
        <p key={s.id} className="flex justify-between rounded bg-slate-50 px-2 py-1">{s.name}
          <button className="text-red-600" onClick={() => remove.mutate(s.id)}>Delete</button></p>
      ))}
      {remove.isError && <p className="text-red-600">Could not delete: {remove.error.message}.
Restored.</p>}
    </div>
  );
}

const queryClient = new QueryClient();
export default function App() { return <QueryClientProvider client={queryClient}><Students />
</QueryClientProvider>; }
```



A MOMENT AFTER CLICKING DELETE ON RAVI KUMAR: HE IS ALREADY GONE, ALTHOUGH THE SERVER NEEDS 1.5 S TO ANSWER



WITH “MAKE THE SERVER FAIL” TICKED: THE ROW VANISHED, THE REQUEST FAILED, AND ONERROR RESTORED IT

14.2.6 Pagination and infinite queries

Put the page number in the query key: `["students", "page", page]`. Each page is cached separately, so going back is instant. Without care, the list would flash a loading state on every page change; `placeholderData: keepPreviousData` keeps showing the previous page (with `isPlaceholderData` true) until the next one arrives. For “load more” and infinite scrolling, `useInfiniteQuery` stores all loaded pages and gives you `fetchNextPage` and `hasNextPage`.

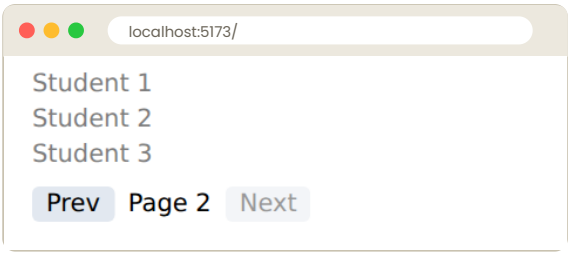
APP.JSX

JavaScript

```
import { QueryClient, QueryClientProvider, useQuery, keepPreviousData } from "@tanstack/react-query";
import { useState } from "react";
import { getPage } from "./api.js";

function StudentPages() {
  const [page, setPage] = useState(1);
  const { data, isPending, isPlaceholderData } = useQuery({
    queryKey: ["students", "page", page],
    queryFn: () => getPage(page),
    placeholderData: keepPreviousData,                    // no flash between pages
  });
  if (isPending) return <p>Loading…</p>;
  return (
    <div className="w-72 space-y-2 text-sm">
      <ul className={isPlaceholderData ? "opacity-50" : ""}>{data.items.map((s) => <li key={s.id}>
{s.name}</li>)}</ul>
      <div className="flex items-center gap-2">
        <button className="rounded bg-slate-200 px-2 disabled:opacity-40" disabled={page === 1}
onClick={() => setPage((p) => p - 1)}>Prev</button>
        <span>Page {page}</span>
        <button className="rounded bg-slate-200 px-2 disabled:opacity-40" disabled={isPlaceholderData
|| !data.hasMore} onClick={() => setPage((p) => p + 1)}>Next</button>
      </div>
    </div>
  );
}

const queryClient = new QueryClient();
export default function App() { return <QueryClientProvider client={queryClient}><StudentPages />
</QueryClientProvider>; }
```



JUST AFTER CLICKING NEXT: PAGE 1 STAYS VISIBLE (DIMMED) WHILE PAGE 2 LOADS



THEN PAGE 2 ARRIVED AND REPLACED IT

USEINFINITEQUERY FOR "LOAD MORE" JavaScript

```
const { data, fetchNextPage, hasNextPage, isFetchingNextPage } = useInfiniteQuery({
  queryKey: ["students", "infinite"],
  queryFn: ({ pageParam }) => getPage(pageParam),
  initialPageParam: 1,
  getNextPageParam: (lastPage, allPages) => (lastPage.hasMore ? allPages.length + 1 : undefined),
});

const students = data?.pages.flatMap((p) => p.items) ?? [];
// <button onClick={() => fetchNextPage()} disabled={!hasNextPage || isFetchingNextPage}>Load
more</button>
```

14.2.7 React Query DevTools

The DevTools panel shows every query in the cache with its key, status (fresh, stale, fetching, inactive), data and timing, and lets you refetch or invalidate queries by hand. It is only included in development builds.

ADDING THE DEVTOOLS JavaScript

```
// npm install @tanstack/react-query-devtools
import { ReactQueryDevtools } from "@tanstack/react-query-devtools";

<QueryClientProvider client={queryClient}>
  <App />
  <ReactQueryDevtools initialIsOpen={false} />   {/* a floating button opens the panel */}
</QueryClientProvider>
```

14.3 Forms with React Hook Form

14.3.1 useForm, register, handleSubmit

Controlled inputs (Module 8) re-render the form on every keystroke and need a state variable, a handler and an error message per field. **React Hook Form** keeps field values in refs (uncontrolled inputs), so typing does not re-render the whole form, and it handles validation, errors and submission. `useForm()` returns `register`, which connects an input by name (it returns `name`, `ref`, `onChange` and `onBlur` to spread onto the input); `handleSubmit(onValid)`, which validates and then calls your function with all values; and `formState` with `errors` and `isSubmitting`.

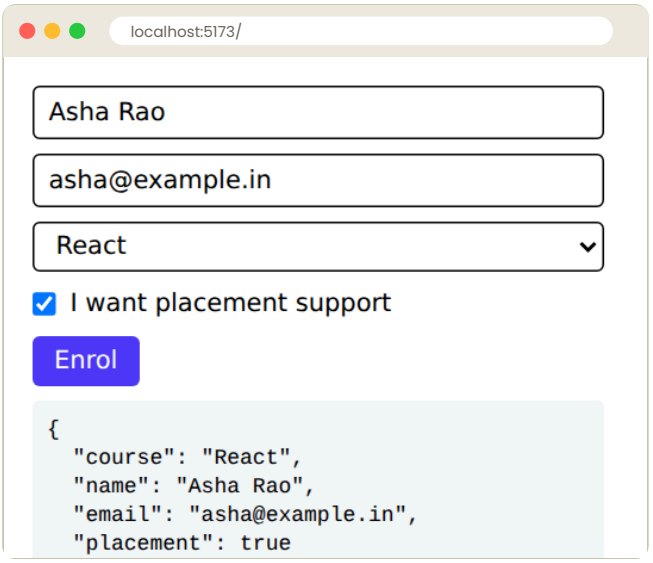
APP.JSX

JavaScript

```
import { useForm } from "react-hook-form";
import { useState } from "react";

export default function App() {
  const { register, handleSubmit } = useForm({ defaultValues: { course: "React" } });
  const [saved, setSaved] = useState(null);
  const onSubmit = (data) => setSaved(data);            // called with every field, by name

  return (
    <form onSubmit={handleSubmit(onSubmit)} className="w-80 space-y-2 text-sm">
      <input {...register("name")} placeholder="Name" className="w-full rounded border px-2 py-1" />
      <input {...register("email")} placeholder="Email" className="w-full rounded border px-2 py-1" />
      <select {...register("course")} className="w-full rounded border px-2 py-1">
        <option>React</option><option>Java</option><option>Python</option>
      </select>
      <label className="flex gap-2"><input type="checkbox" {...register("placement")} /> I want
placement support</label>
      <button className="rounded bg-indigo-600 px-3 py-1 text-white">Enrol</button>
      {saved && <pre className="rounded bg-slate-100 p-2 text-xs">{JSON.stringify(saved, null, 2)}
</pre>}
    </form>
  );
}
```



FILLED IN AND SUBMITTED: HANDLESUBMIT PASSED ONE OBJECT WITH EVERY FIELD

14.3.2 Validation rules and error display

Pass rules as the second argument of `register`: `required`, `minLength`, `maxLength`, `min`, `max`, `pattern` and a custom `validate` function. Each rule can carry its message. Errors appear in `formState.errors.fieldName`; by default validation runs on submit and then again on every change of a field that has an error. `handleSubmit` only calls your function when everything is valid, and focuses the first invalid field.

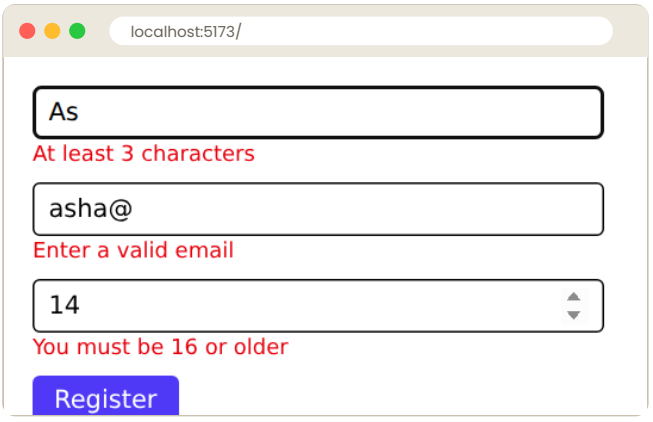
APP.JSX

JavaScript

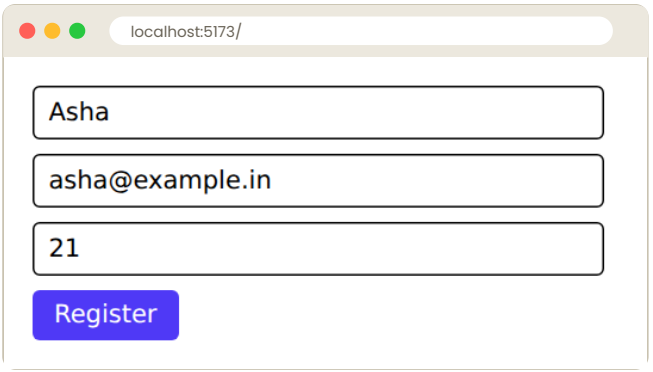
```
import { useForm } from "react-hook-form";

export default function App() {
  const { register, handleSubmit, formState: { errors, isSubmitting } } = useForm();
  const onSubmit = async (data) => { await new Promise((r) => setTimeout(r, 1000)); };

  return (
    <form onSubmit={handleSubmit(onSubmit)} noValidate className="w-80 space-y-2 text-sm">
      <div>
        <input placeholder="Name" className="w-full rounded border px-2 py-1"
          {...register("name", { required: "Name is required", minLength: { value: 3, message: "At
least 3 characters" } })} />
        {errors.name && <p className="text-xs text-red-600">{errors.name.message}</p>}
      </div>
      <div>
        <input placeholder="Email" className="w-full rounded border px-2 py-1"
          {...register("email", { required: "Email is required", pattern: { value: /^\S+@\S+\.\S+$/,
message: "Enter a valid email" } })} />
        {errors.email && <p className="text-xs text-red-600">{errors.email.message}</p>}
      </div>
      <div>
        <input type="number" placeholder="Age" className="w-full rounded border px-2 py-1"
          {...register("age", { valueAsNumber: true, min: { value: 16, message: "You must be 16 or
older" } })} />
        {errors.age && <p className="text-xs text-red-600">{errors.age.message}</p>}
      </div>
      <button disabled={isSubmitting} className="rounded bg-indigo-600 px-3 py-1 text-white
disabled:opacity-50">
        {isSubmitting ? "Sending…" : "Register"}
      </button>
    </form>
  );
}
```



SUBMITTED WITH A SHORT NAME, A BAD EMAIL AND AGE 14: EACH RULE'S MESSAGE IS SHOWN



FIXED AND SUBMITTED AGAIN: ERRORS CLEARED AND ISSUBMITTING DISABLED THE BUTTON

14.3.3 Schema validation with Zod

For bigger forms, describe the data once as a **Zod schema** and let it validate everything. A schema states the type and the rules of every field; `schema.safeParse(value)` returns either `{ success: true, data }` or the list of problems. With `zodResolver(schema)` (from `@hookform/resolvers`) React Hook Form uses the schema instead of per-field rules. The same schema can validate API responses, and in TypeScript `z.infer<typeof schema>` gives the matching type for free (Module 15).

ZOD-BASICS.JS JavaScript

```
import { z } from "zod";

const Student = z.object({
  name: z.string().trim().min(3, "Name needs at least 3 characters"),
  email: z.string().email("Enter a valid email"),
  age: z.number().int().min(16),
  course: z.enum(["React", "Java", "Python"]),
});

console.log(Student.safeParse({ name: "  Asha Rao ", email: "asha@example.in", age: 21, course:
"React" }));

const bad = Student.safeParse({ name: "As", email: "asha@", age: 14.5, course: "C" });
console.log("success:", bad.success);
for (const issue of bad.error.issues) console.log(issue.path.join("."), "->", issue.message);
```

BROWSER CONSOLE

```
{success: true, data: {name: 'Asha Rao', email: 'asha@example.in', age: 21, course: 'React'}}
success: false
name -> Name needs at least 3 characters
email -> Enter a valid email
age -> Invalid input: expected int, received number
course -> Invalid option: expected one of "React"|"Java"|"Python"
```

Notice that `trim()` also **transformed** the name: the parsed `data` is cleaned, not just checked. Now the same idea inside a form, including a cross-field rule (password confirmation) with `refine`:

APP.JSX

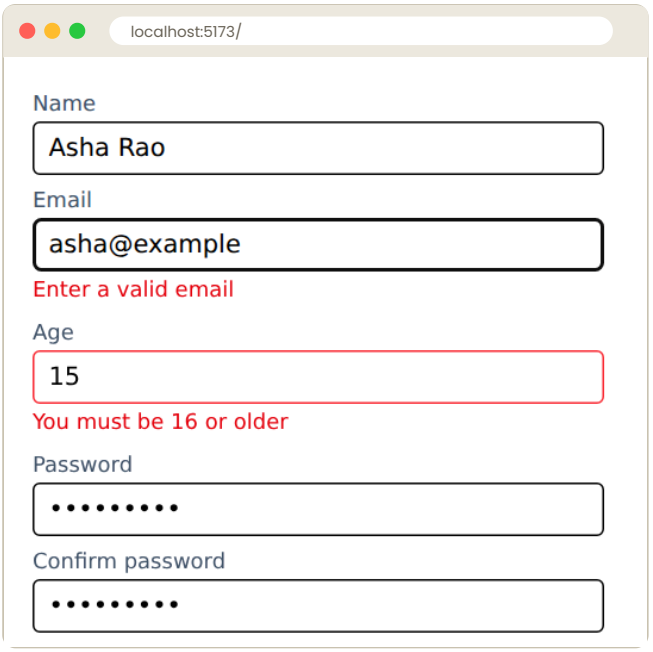
JavaScript

```
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";

const schema = z.object({
  name: z.string().trim().min(3, "At least 3 characters"),
  email: z.string().email("Enter a valid email"),
  age: z.coerce.number().min(16, "You must be 16 or older"),
  password: z.string().min(8, "Use 8 or more characters"),
  confirm: z.string(),
}).refine((d) => d.password === d.confirm, { message: "Passwords do not match", path: ["confirm"] });

function Field({ label, error, ...props }) {
  return (
    <label className="block">
      <span className="text-xs text-slate-600">{label}</span>
      <input className={`w-full rounded border px-2 py-1 ${error ? "border-red-500" : ""}`} {...props}
/>
      {error && <span className="text-xs text-red-600">{error.message}</span>}
    </label>
  );
}

export default function App() {
  const { register, handleSubmit, formState: { errors, isSubmitSuccessful } } = useForm({ resolver:
zodResolver(schema) });
  return (
    <form onSubmit={handleSubmit((data) => console.log(data))} className="w-80 space-y-1 text-sm">
      <Field label="Name" error={errors.name} {...register("name")} />
      <Field label="Email" error={errors.email} {...register("email")} />
      <Field label="Age" error={errors.age} {...register("age")} />
      <Field label="Password" type="password" error={errors.password} {...register("password")} />
      <Field label="Confirm password" type="password" error={errors.confirm} {...register("confirm")}
/>
      <button className="mt-1 rounded bg-indigo-600 px-3 py-1 text-white">Create account</button>
      {isSubmitSuccessful && <p className="text-green-700">Account created.</p>}
    </form>
  );
}
```



SUBMITTED WITH A MISMATCHED CONFIRMATION: THE SCHEMA FOUND EVERY PROBLEM, INCLUDING THE CROSS-FIELD ONE

14.3.4 Controlled third-party inputs

`register` needs a real input element with `onChange` and a ref. Many components from UI libraries (date pickers, selects, sliders, star ratings) are **controlled**: they take `value` and `onChange(newValue)` instead. Wrap them in `<Controller>`: it registers the field and hands your component a `field` object with `value`, `onChange`, `onBlur` and `ref`, plus `fieldState.error`.

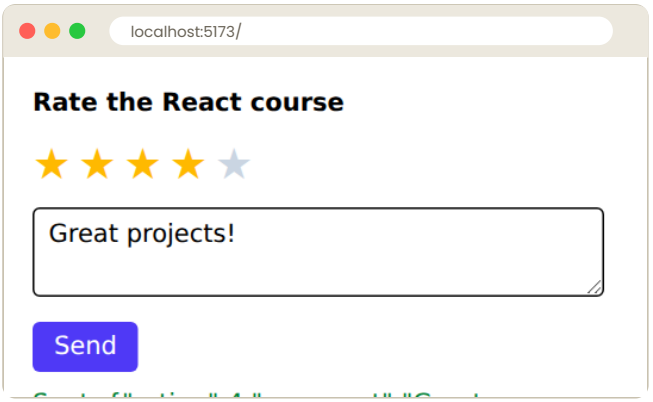
APP.JSX

JavaScript

```
import { useForm, Controller } from "react-hook-form";
import { useState } from "react";

// A controlled component, like one from a UI library: value + onChange(newValue)
function StarRating({ value = 0, onChange }) {
  return (
    <div className="flex gap-1 text-2xl">
      {[1, 2, 3, 4, 5].map((n) => (
        <button type="button" key={n} aria-label={`${n} stars`} onClick={() => onChange(n)}
                className={n <= value ? "text-amber-400" : "text-slate-300"}>★</button>
      ))}
    </div>
  );
}

export default function App() {
  const { control, register, handleSubmit } = useForm({ defaultValues: { rating: 0, comment: "" } });
  const [sent, setSent] = useState(null);
  return (
    <form onSubmit={handleSubmit(setSent)} className="w-80 space-y-2 text-sm">
      <p className="font-semibold">Rate the React course</p>
      <Controller
        name="rating"
        control={control}
        rules={{ min: { value: 1, message: "Pick at least one star" } }}
        render={({ field, fieldState }) => (
          <>
            <StarRating value={field.value} onChange={field.onChange} />
            {fieldState.error && <p className="text-xs text-red-600">{fieldState.error.message}</p>}
          </>
        )}
      />
      <textarea {...register("comment")} placeholder="Comment" className="w-full rounded border px-2
py-1" />
      <button className="rounded bg-indigo-600 px-3 py-1 text-white">Send</button>
      {sent && <p className="text-green-700">Sent: {JSON.stringify(sent)}</p>}
    </form>
  );
}
```



CLICKED THE FOURTH STAR, TYPED A COMMENT AND SENT: THE CONTROLLED RATING BECAME A NORMAL FORM VALUE

14.4 File uploads

14.4.1 Uploading images with FormData and previews

A file input gives you `File` objects in `e.target.files`. To show a preview before uploading, create a temporary URL with `URL.createObjectURL(file)` and use it as an `<img src>`; release it with `URL.revokeObjectURL` when it is no longer needed (an effect cleanup is the right place). To upload, put the file in a **FormData** object and send it as the request body: do **not** set the `Content-Type` header yourself, because the browser must add the multipart boundary.

APP.JSX

JavaScript

```
import { useEffect, useState } from "react";

export default function App() {
  const [file, setFile] = useState(null);
  const [preview, setPreview] = useState("");
  const [error, setError] = useState("");

  useEffect(() => {
    if (!file) return;
    const url = URL.createObjectURL(file);                  // temporary local URL
    setPreview(url);
    return () => URL.revokeObjectURL(url);                   // free it later
  }, [file]);

  function handleChange(e) {
    const f = e.target.files[0];
    if (!f) return;
    if (!f.type.startsWith("image/")) return setError("Please choose an image");
    if (f.size > 2 * 1024 * 1024) return setError("Max size is 2 MB");
    setError("");
    setFile(f);
  }

  async function upload() {
    const body = new FormData();
    body.append("avatar", file);                            // field name expected by the server
    body.append("studentId", "7");
    // await fetch("/api/upload", { method: "POST", body }); // no Content-Type header!
    console.log([...body.entries()].map(([k, v]) => `${k}=${v instanceof File ? v.name + " (" + v.size
+ " bytes)" : v}`).join(", "));
  }

  return (
    <div className="w-80 space-y-2 text-sm">
      <input type="file" accept="image/*" onChange={handleChange} />
      {error && <p className="text-red-600">{error}</p>}
      {preview && <img src={preview} alt="Preview" className="h-24 w-24 rounded-full object-cover
ring-2 ring-indigo-500" />}
      {file && <p>{file.name} · {(file.size / 1024).toFixed(1)} kB <button onClick={upload}
className="ml-2 rounded bg-indigo-600 px-2 text-white">Upload</button></p>}
    </div>
  );
}
```



AN IMAGE WAS CHOSEN: THE PREVIEW COMES FROM A LOCAL OBJECT URL, BEFORE ANY UPLOAD

BROWSER CONSOLE: THE FORMDATA THAT WOULD BE SENT

```
avatar=asha.svg (235 bytes), studentId=7
```

PRACTICE CHECKPOINT

A full CRUD Student Management front end using TanStack Query against a REST API (or JSON Server), with React Hook Form + Zod forms and optimistic updates.

The solution keeps three concerns apart: `api.js` (the REST calls; here an in-memory stand-in with the same functions you would write against JSON Server), `useStudents.js` (all server-state logic as custom hooks) and the components (only UI).

API.JS JavaScript

```
// api.js: replace the bodies with fetch calls to JSON Server (http://localhost:3000/students)
const wait = (ms) => new Promise((r) => setTimeout(r, ms));
let nextId = 4;
let db = [
  { id: 1, name: "Asha Rao", email: "asha@example.in", course: "React" },
  { id: 2, name: "Ravi Kumar", email: "ravi@example.in", course: "Java" },
  { id: 3, name: "Meena Iyer", email: "meena@example.in", course: "Python" },
];
export const list = async () => { await wait(300); return db.map((s) => ({ ...s })); };
export const create = async (s) => { await wait(300); const n = { id: nextId++, ...s }; db = [...db,
n]; return n; };
export const update = async (s) => { await wait(300); db = db.map((x) => (x.id === s.id ? { ...s } :
x)); return s; };
export const remove = async (id) => { await wait(300); db = db.filter((x) => x.id !== id); };
```

USESTUDENTS.JS JavaScript

```
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
import * as api from "./api.js";

const KEY = ["students"];

export const useStudents = () => useQuery({ queryKey: KEY, queryFn: api.list });

// one helper for create / update / delete, all optimistic
function useOptimistic(mutationFn, apply) {
  const qc = useQueryClient();
  return useMutation({
    mutationFn,
    onMutate: async (input) => {
      await qc.cancelQueries({ queryKey: KEY });
      const previous = qc.getQueryData(KEY);
      qc.setQueryData(KEY, (old = []) => apply(old, input));
      return { previous };
    },
    onError: (_e, _input, ctx) => qc.setQueryData(KEY, ctx.previous),
    onSettled: () => qc.invalidateQueries({ queryKey: KEY }),
  });
}

export const useCreateStudent = () => useOptimistic(api.create, (old, s) => [...old, { ...s, id:
"temp" }]);
export const useUpdateStudent = () => useOptimistic(api.update, (old, s) => old.map((x) => (x.id ===
s.id ? s : x)));
export const useDeleteStudent = () => useOptimistic(api.remove, (old, id) => old.filter((x) => x.id
!== id));
```

STUDENTFORM.JSX JavaScript

```
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";

export const studentSchema = z.object({
  name: z.string().trim().min(3, "At least 3 characters"),
  email: z.string().email("Enter a valid email"),
  course: z.enum(["React", "Java", "Python"], "Choose a course"),
});

export function StudentForm({ initial, onSave, onCancel }) {
  const { register, handleSubmit, reset, formState: { errors } } = useForm({
    resolver: zodResolver(studentSchema),
    defaultValues: initial ?? { name: "", email: "", course: "React" },
  });
  const submit = (data) => { onSave(data); if (!initial) reset(); };
  return (
    <form onSubmit={handleSubmit(submit)} className="grid grid-cols-[1fr_1fr_90px_auto] items-start
gap-2 text-sm">
      <div><input {...register("name")} placeholder="Name" className="w-full rounded border px-2 py-1"
/>
        {errors.name && <p className="text-xs text-red-600">{errors.name.message}</p>}</div>
      <div><input {...register("email")} placeholder="Email" className="w-full rounded border px-2 py-
1" />
        {errors.email && <p className="text-xs text-red-600">{errors.email.message}</p>}</div>
      <select {...register("course")} className="rounded border px-1 py-1"><option>React</option>
<option>Java</option><option>Python</option></select>
      <div className="flex gap-1">
        <button className="rounded bg-indigo-600 px-2 py-1 text-white">{initial ? "Save" : "Add"}
</button>
        {onCancel && <button type="button" onClick={onCancel} className="rounded bg-slate-200 px-2 py-
1">x</button>}
      </div>
    </form>
  );
}
```

APP.JSX: SAMPLE SOLUTION JavaScript

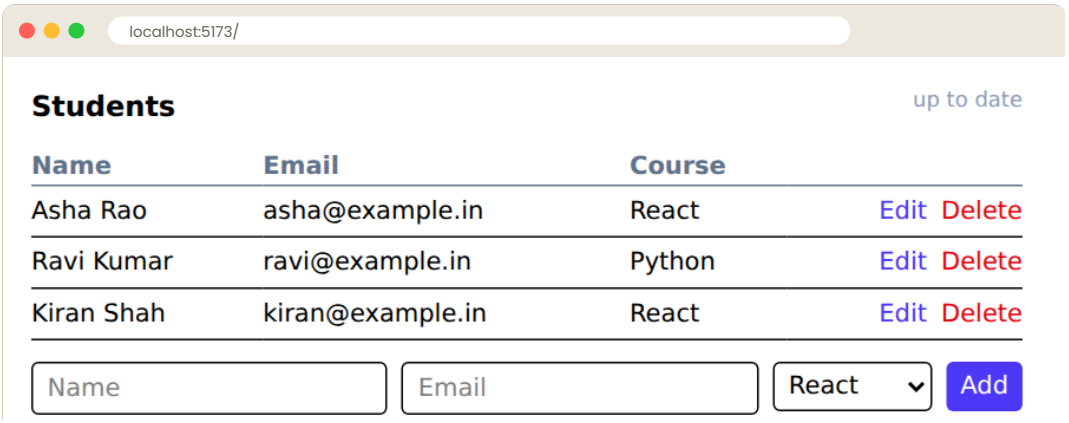
```
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { useState } from "react";
import { useStudents, useCreateStudent, useUpdateStudent, useDeleteStudent } from "./useStudents.js";
import { StudentForm } from "./StudentForm.jsx";

function StudentTable() {
  const { data: students = [], isPending, isFetching } = useStudents();
  const create = useCreateStudent();
  const update = useUpdateStudent();
  const remove = useDeleteStudent();
  const [editing, setEditing] = useState(null);

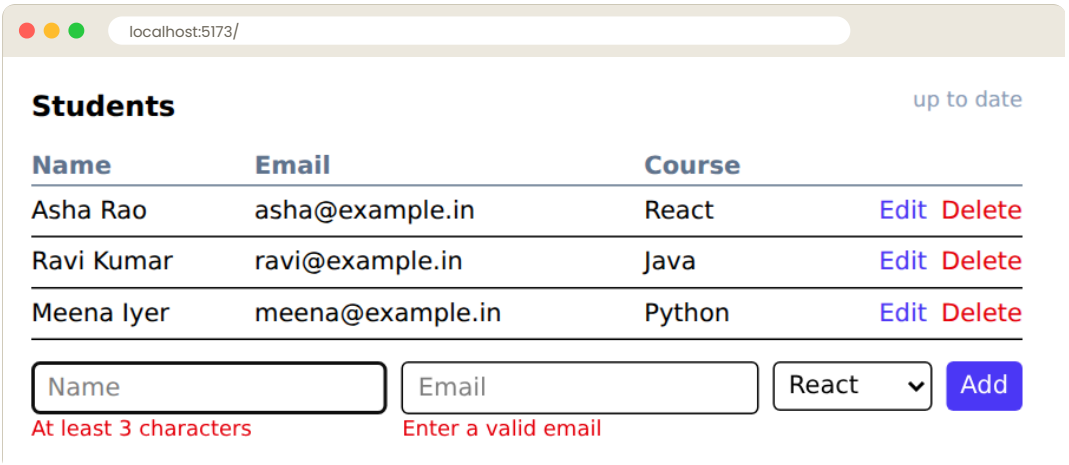
  if (isPending) return <p>Loading students…</p>;
  return (
    <div className="w-[560px] space-y-3">
      <h2 className="flex justify-between font-semibold">Students <span className="text-xs font-normal
text-slate-400">{isFetching ? "syncing…" : "up to date"}</span></h2>
      <table className="w-full text-left text-sm">
        <thead><tr className="border-b text-slate-500"><th>Name</th><th>Email</th><th>Course</th><th
/></tr></thead>
        <tbody>
          {students.map((s) => editing === s.id ? (
```

```
            <tr key={s.id}><td colSpan={4} className="py-1">
              <StudentForm initial={s} onSave={(d) => { update.mutate({ ...d, id: s.id });
setEditing(null); }} onCancel={() => setEditing(null)} />
            </td></tr>
          ) : (
            <tr key={s.id} className="border-b">
              <td className="py-1">{s.name}</td><td>{s.email}</td><td>{s.course}</td>
              <td className="space-x-2 text-right">
                <button className="text-indigo-600" onClick={() => setEditing(s.id)}>Edit</button>
                <button className="text-red-600" onClick={() => remove.mutate(s.id)}>Delete</button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
      <StudentForm onSave={(d) => create.mutate(d)} />
    </div>
  );
}

const queryClient = new QueryClient();
export default function App() {
  return <QueryClientProvider client={queryClient}><StudentTable /></QueryClientProvider>;
}
```



ADDED KIRAN SHAH, EDITED RAVI'S COURSE TO PYTHON AND DELETED MEENA: EACH CHANGE APPEARED INSTANTLY, THEN SYNCED



SUBMITTING THE ADD FORM EMPTY: ZOD ERRORS UNDER EACH FIELD

Talk to REST APIs through one service layer (base URL from `VITE_` variables, auth header, status checks); CORS is fixed on the server. Treat server data as server state: TanStack Query gives you `useQuery` with caching by key and `staleTime`, `useMutation` with `invalidateQueries`, optimistic updates via `onMutate` / `onError` / `onSettled`, and pagination with `keepPreviousData`. Build large forms with React Hook Form (`register`, `handleSubmit`, `formState.errors`, `Controller`) and one Zod schema through `zodResolver`. Preview uploads with object URLs and send files in `FormData`.

MODULE 15 · INTERMEDIATE

TypeScript with React

Minimum duration **2 days** | Level **Intermediate**

TypeScript is the default for professional React projects: it catches wrong props, missing null checks and typos while you type, and makes refactoring safe. This module sets up a Vite React + TypeScript project and types every part of a React app: props (with interfaces or types, optional props and defaults, children, and props extended from HTML elements), hooks (useState with unions and null, useRef for DOM elements, useReducer with discriminated union actions, typed context with a custom hook), events, API data with types inferred from Zod schemas, and generic components such as a typed List and Select.

IN THIS MODULE

15.1 Setup
15.2 Typing components
15.3 Typing hooks
15.4 Typing events
15.5 Typing API data
15.6 Generic components

HOW THE EXAMPLES WERE CHECKED

Every **TYPE CHECK** box is the real output of the TypeScript 6 compiler (`tsc --strict`) on the listing above it. Because the official `@types/react` package is not installed in the offline build, React's types came from a small compatible declaration file; the error messages are produced by the compiler itself. The apps were then bundled with esbuild (which strips the types) and run in Chromium.

15.1 Setup

15.1.1 Vite React + TypeScript template

Create a TypeScript project with the `react-ts` template. Vite compiles TypeScript by simply **removing** the types (very fast); type **checking** is done by `tsc`, which runs in your editor all the time and in `npm run build` before the bundle is produced. A build therefore fails on type errors, so they never reach production.

Terminal (typical output and files)

```
$ npm create vite@latest student-app -- --template react-ts
$ cd student-app && npm install
$ npm run dev

student-app/
├── src/
│   ├── App.tsx
│   ├── main.tsx
│   └── vite-env.d.ts        types for import.meta.env and asset imports
├── tsconfig.json            references the two configs below
├── tsconfig.app.json        settings for src/ (strict: true)
├── tsconfig.node.json       settings for vite.config.ts
└── package.json             "build": "tsc -b && vite build"
```

TSCONFIG.APP.JSON (THE IMPORTANT PARTS) JavaScript

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "moduleResolution": "bundler",
    "jsx": "react-jsx",                 // the automatic JSX runtime
    "strict": true,                     // the most important line
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true,
    "noEmit": true                      // Vite produces the output, tsc only checks
  },
  "include": ["src"]
}
```

To add TypeScript to an existing JavaScript project: install `typescript`, `@types/react` and `@types/react-dom`, add a `tsconfig.json`, and rename files one at a time from `.jsx` to `.tsx`. Setting `"allowJs": true` lets both kinds of file live together during the migration.

15.1.2 .tsx files

Files that contain JSX use the extension `.tsx`; files with only TypeScript (types, utilities, API calls) use `.ts`. One JSX-related difference: in a `.tsx` file the old type-assertion syntax `<User>data` would look like a JSX tag, so always write `data as User`. A generic arrow function also needs a hint: `const first = <T,>(items: T[]) => items[0];` (the comma tells the parser it is not a tag).

Extension	Contains	Examples
`.tsx`	components (JSX)	`App.tsx`, `StudentCard.tsx`
`.ts`	no JSX: types, hooks without JSX, API calls, utilities	`types.ts`, `api.ts`, `useDebounce.ts`
`.d.ts`	type declarations only	`vite-env.d.ts`, declarations for untyped packages

15.2 Typing components

15.2.1 Typing props with interfaces or types

A component is a function, so its props are typed like any parameter: describe them with an `interface` or a `type` and annotate the destructured object. TypeScript then checks every place the component is used: missing props, misspelt props and wrong value types are all errors. Both forms work; a common convention is `interface` for props objects and `type` for unions and combinations. The older `React.FC<Props>` style is no longer recommended.

APP.TSX TypeScript

```ts
interface StudentCardProps {
  name: string;
  course: "React" | "Java" | "Python";       // a union of allowed strings
  attendance: number;
}

// the same with a type alias:
// type StudentCardProps = { name: string; course: "React" | "Java" | "Python"; attendance: number };

function StudentCard({ name, course, attendance }: StudentCardProps) {
  return (
    <div className="w-60 rounded-lg p-3 ring-1 ring-slate-200">
      <p className="font-semibold">{name}</p>
      <p className="text-sm text-slate-600">{course} · attendance {attendance}%</p>
    </div>
  );
}

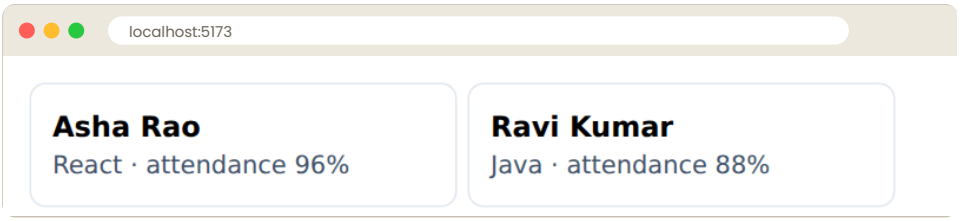
export default function App() {
  return (
    <div className="flex gap-2">
      <StudentCard name="Asha Rao" course="React" attendance={96} />
      <StudentCard name="Ravi Kumar" course="Java" attendance={88} />
    </div>
  );
}
```

TSC ON APP.TSX main.js

```
$ npx tsc --noEmit
```

TYPE CHECK (TSC --STRICT)

```
(no output: every file type-checks)
```



CORRECT USAGE: NO TYPE ERRORS, AND THE APP RENDERS

Now three typical mistakes in the JSX that uses the component. Each one is caught before the code ever runs:

APP.TSX (WRONG USAGE) TypeScript

```ts
export default function App() {
  return (
    <div className="flex gap-2">
      <StudentCard name="Asha Rao" course="React" attendance={96} />
      <StudentCard name="Ravi Kumar" course="C#" attendance="88" />
      <StudentCard name="Meena Iyer" attendence={91} />
    </div>
  );
}
```

TSC ON APP.TSX (WRONG USAGE) main.js

```
$ npx tsc --noEmit
```

TYPE CHECK (TSC --STRICT)

```
App.tsx(23,38): error TS2322: Type '"C#"' is not assignable to type '"React" | "Java" | "Python"'.
App.tsx(23,50): error TS2322: Type 'string' is not assignable to type 'number'.
App.tsx(24,38): error TS2322: Type '{ name: string; attendence: number; }' is not assignable to type
'IntrinsicAttributes & StudentCardProps'.
  Property 'attendence' does not exist on type 'IntrinsicAttributes & StudentCardProps'. Did you mean
'attendance'?
```

Read the errors: line 23 has a `course` value outside the union and a string where a number is required; line 24 misspells `attendance`, and TypeScript even suggests the correct name. (Once the typo is fixed it will also report that the required `course` prop is missing.) In the editor these appear as red underlines with the same messages.

15.2.2 Optional props and defaults

Mark a prop optional with `?`. Inside the component its type includes `undefined`, so give it a **default value** in the destructuring: TypeScript then knows it is always defined.

APP.TSX TypeScript

```tsx
interface BadgeProps {
  label: string;
  tone?: "info" | "success" | "danger";     // optional
  count?: number;
}

function Badge({ label, tone = "info", count }: BadgeProps) {   // default for tone
  const colours = { info: "bg-sky-100 text-sky-800", success: "bg-green-100 text-green-800", danger:
"bg-red-100 text-red-800" };
  return (
    <span className={`rounded-full px-2 py-0.5 text-sm ${colours[tone]}`}>
      {label}{count !== undefined && ` (${count})`}      {/* count may be undefined */}
    </span>
  );
}

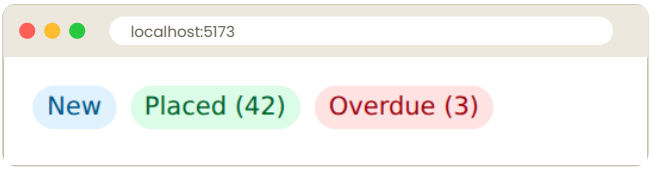
export default function App() {
  return (
    <div className="flex gap-2">
      <Badge label="New" />
      <Badge label="Placed" tone="success" count={42} />
      <Badge label="Overdue" tone="danger" count={3} />
    </div>
  );
}
```

TSC ON APP.TSX main.js

```
$ npx tsc --noEmit
```

TYPE CHECK (TSC --STRICT)

```
(no output: every file type-checks)
```



OPTIONAL PROPS FALL BACK TO THEIR DEFAULTS

15.2.3 Typing children with ReactNode

`children` is just another prop. Type it as `ReactNode`, which accepts anything React can render: elements, strings, numbers, arrays, `null` and `undefined`. Use `ReactElement` only if you really need a single element. `PropsWithChildren<P>` is a shortcut that adds an optional `children` to `P`.

APP.TSX TypeScript

```tsx
import type { ReactNode } from "react";

interface PanelProps {
  title: string;
  footer?: ReactNode;          // any renderable content, optional
  children: ReactNode;         // required content
}

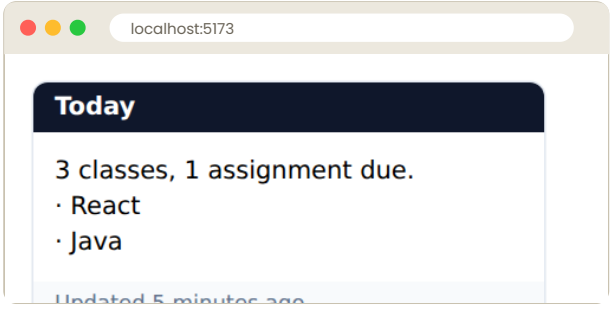
function Panel({ title, footer, children }: PanelProps) {
  return (
    <section className="w-72 overflow-hidden rounded-lg ring-1 ring-slate-200">
      <h2 className="bg-slate-900 px-3 py-1 text-sm font-semibold text-white">{title}</h2>
      <div className="p-3 text-sm">{children}</div>
      {footer && <div className="bg-slate-50 px-3 py-1 text-xs text-slate-500">{footer}</div>}
    </section>
  );
}

export default function App() {
  return (
    <Panel title="Today" footer={<span>Updated 5 minutes ago</span>}>
      <p>3 classes, 1 assignment due.</p>
      {["React", "Java"].map((c) => <p key={c}>· {c}</p>)}
    </Panel>
  );
}
```

TSC ON APP.TSX main.js

```
$ npx tsc --noEmit
```

TYPE CHECK (TSC --STRICT)
(no output: every file type-checks)



ELEMENTS, TEXT AND ARRAYS ARE ALL VALID REACTNODE CHILDREN

15.2.4 Extending HTML element props (ComponentProps<"button">)

A reusable `Button` or `Input` should accept everything the native element accepts (`onClick`, `disabled`, `type`, `aria-*`…) plus a few props of its own. `ComponentProps<"button">` is the type of all props a `<button>` accepts; combine it with your own props using `&` and pass the rest through with `...rest`. Use `Omit<...>` to remove a native prop you want to redefine.

APP.TSX TypeScript

```tsx
import type { ComponentProps } from "react";

type ButtonProps = ComponentProps<"button"> & {
  variant?: "primary" | "outline";
};

function Button({ variant = "primary", className = "", ...rest }: ButtonProps) {
  const styles = variant === "primary" ? "bg-indigo-600 text-white" : "ring-1 ring-indigo-600 text-
indigo-700";
  return <button className={`rounded px-3 py-1 text-sm disabled:opacity-40 ${styles} ${className}`}
{...rest} />;
}

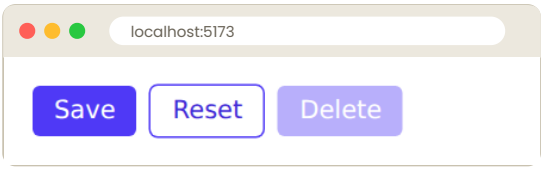
export default function App() {
  return (
    <div className="flex gap-2">
      <Button onClick={() => alert("Saved")}>Save</Button>          {/* native onClick */}
      <Button variant="outline" type="reset">Reset</Button>          {/* native type */}
      <Button disabled aria-label="Delete student">Delete</Button>  {/* native disabled, aria */}
    </div>
  );
}
```

TSC ON APP.TSX main.js

```
$ npx tsc --noEmit
```

TYPE CHECK (TSC --STRICT)

```
(no output: every file type-checks)
```



ONE COMPONENT, ALL NATIVE BUTTON PROPS AVAILABLE AND TYPE-CHECKED

TSC AFTER CHANGING TYPE="RESET" TO "REST" AND VARIANT TO "GHOST" main.js

```
$ npx tsc --noEmit
```

TYPE CHECK (TSC --STRICT)

```
App.tsx(16,15): error TS2322: Type '"ghost"' is not assignable to type '"primary" | "outline" |
undefined'.
App.tsx(16,31): error TS2820: Type '"rest"' is not assignable to type '"button" | "submit" | "reset" |
undefined'. Did you mean '"reset"'?
```

15.3 Typing hooks

15.3.1 useState with unions and null

`useState` **infers** the type from the initial value: `useState(0)` is a number, `useState("")` a string. When the initial value does not show the full type, pass it explicitly as a type argument: `useState<User | null>(null)` for "nothing yet", `useState<Student[]>([])` for an empty array (otherwise it would be `never[]`), or a union of string literals for a status.

APP.TSX (WITH FOUR MISTAKES) TypeScript

```tsx
import { useState } from "react";

interface User { name: string; email: string }
type Status = "idle" | "loading" | "error";

export default function App() {
  const [count, setCount] = useState(0);                    // inferred: number
  const [user, setUser] = useState<User | null>(null);      // explicit: User or null
  const [tags, setTags] = useState<string[]>([]);           // explicit: string[]
  const [status, setStatus] = useState<Status>("idle");

  return (
    <div className="space-y-1 text-sm">
      <p>{user.name}</p>
      <button onClick={() => setCount("1")}>+</button>
      <button onClick={() => setStatus("done")}>finish</button>
      <button onClick={() => setTags([...tags, 42])}>tag</button>
    </div>
  );
}
```

TSC ON APP.TSX main.js

```
$ npx tsc --noEmit
```

TYPE CHECK (TSC --STRICT)

```
App.tsx(14,11): error TS18047: 'user' is possibly 'null'.
App.tsx(15,39): error TS2345: Argument of type 'string' is not assignable to parameter of type
'SetStateAction<number>'.
App.tsx(16,40): error TS2345: Argument of type '"done"' is not assignable to parameter of type
'SetStateAction<Status>'.
App.tsx(17,39): error TS2322: Type 'string | number' is not assignable to type 'string'.
  Type 'number' is not assignable to type 'string'.
App.tsx(17,48): error TS2322: Type 'number' is not assignable to type 'string'.
```

The first error is the most valuable one: `user` can be `null`, and reading `user.name` would crash at runtime. **Narrow** the type first; inside the check TypeScript knows `user` is a `User`:

APP.TSX

TypeScript

```
import { useState } from "react";

interface User { name: string; email: string }

export default function App() {
  const [user, setUser] = useState<User | null>(null);
  return (
    <div className="space-y-2 text-sm">
      {user ? <p>Hello, {user.name} ({user.email})</p> : <p>Not logged in</p>}   {/* narrowed */}
      <p>Optional chaining also works: {user?.name ?? "guest"}</p>
      <button className="rounded bg-indigo-600 px-2 text-white" onClick={() => setUser({ name: "Asha",
email: "asha@example.in" })}>Log in</button>
    </div>
  );
}
```

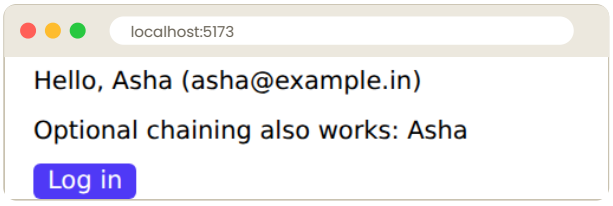
TSC ON APP.TSX

main.js

```
$ npx tsc --noEmit
```

TYPE CHECK (TSC --STRICT)

(no output: every file type-checks)



AFTER CLICKING LOG IN: THE NARROWED BRANCH RENDERS

15.3.2 useRef for DOM elements

For a ref attached to an element, give the element type and start with `null`: `useRef<HTMLInputElement>(null)`. React fills `ref.current` after rendering, so its type is `HTMLInputElement | null` and you must check it (or use `?.`) before using it. Element types follow a pattern: `HTMLInputElement`, `HTMLButtonElement`, `HTMLDivElement`, `HTMLFormElement`, `HTMLCanvasElement`. A ref used as a mutable box (a timer id) is typed by its value: `useRef<number | null>(null)`.

APP.TSX TypeScript

```tsx
import { useEffect, useRef } from "react";

export default function App() {
  const inputRef = useRef<HTMLInputElement>(null);       // element type + null
  const timer = useRef<number | null>(null);             // a mutable box

  useEffect(() => {
    inputRef.current?.focus();                           // may be null: use ?.
    timer.current = window.setTimeout(() => {
      if (inputRef.current) inputRef.current.placeholder = "Still waiting…";
    }, 300);
    return () => { if (timer.current !== null) clearTimeout(timer.current); };
  }, []);

  return <input ref={inputRef} placeholder="Search students" className="w-64 rounded border px-2 py-1
outline-indigo-500" />;
}
```

TSC ON APP.TSX main.js

```
$ npx tsc --noEmit
```

TYPE CHECK (TSC --STRICT)

```
(no output: every file type-checks)
```



THE INPUT WAS FOCUSED THROUGH THE TYPED REF, AND ITS PLACEHOLDER CHANGED BY THE TIMER

15.3.3 useReducer with discriminated union actions

Type a reducer with a state type and an action type. Describe the actions as a **discriminated union**: every action has a `type` field with a different string literal, and each variant lists exactly the data it carries. Inside `switch (action.type)` TypeScript narrows `action` to the matching variant, so `action.student` is only available where it exists, and `dispatch` refuses unknown types or missing data.

APP.TSX TypeScript

```tsx
import { useReducer } from "react";

interface Student { id: number; name: string }
interface State { students: Student[]; selectedId: number | null }

type Action =
  | { type: "added"; student: Student }
  | { type: "removed"; id: number }
  | { type: "selected"; id: number | null };

function reducer(state: State, action: Action): State {
  switch (action.type) {
    case "added":                          // here action is { type: "added"; student: Student }
      return { ...state, students: [...state.students, action.student] };
    case "removed":
      return { ...state, students: state.students.filter((s) => s.id !== action.id) };
    case "selected":
      return { ...state, selectedId: action.id };
  }
}

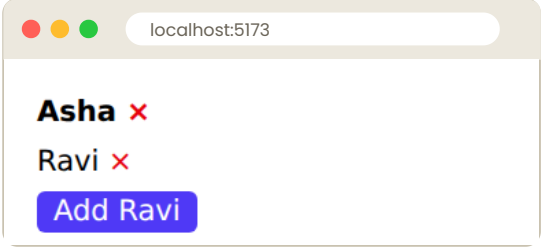
export default function App() {
  const [state, dispatch] = useReducer(reducer, { students: [{ id: 1, name: "Asha" }], selectedId:
null });
  return (
    <div className="space-y-1 text-sm">
      {state.students.map((s) => (
        <p key={s.id} className={s.id === state.selectedId ? "font-bold" : ""} onClick={() =>
dispatch({ type: "selected", id: s.id })}>
          {s.name} <button className="text-red-600" onClick={() => dispatch({ type: "removed", id:
s.id })}>×</button>
        </p>
      ))}
      <button className="rounded bg-indigo-600 px-2 text-white" onClick={() => dispatch({ type:
"added", student: { id: Date.now(), name: "Ravi" } })}>Add Ravi</button>
    </div>
  );
}
```

TSC ON APP.TSX main.js

```
$ npx tsc --noEmit
```

TYPE CHECK (TSC --STRICT)

```
(no output: every file type-checks)
```



ADDED RAVI AND SELECTED ASHA: EVERY DISPATCH WAS TYPE-CHECKED

TSC AFTER MISSPELLING "ADDED" AND FORGETTING THE ID OF "REMOVED"

main.js

```
$ npx tsc --noEmit
```

TYPE CHECK (TSC --STRICT)

```
App.tsx(28,77): error TS2345: Argument of type '{ type: "removed"; }' is not assignable to parameter
of type 'Action'.
  Property 'id' is missing in type '{ type: "removed"; }' but required in type '{ type: "removed"; id:
number; }'.
App.tsx(31,91): error TS2322: Type '"add"' is not assignable to type '"added" | "removed" |
"selected"'.
```

TIP

Because every `case` returns, TypeScript knows the switch is **exhaustive**. If you add a new action variant and forget to handle it, the reducer no longer returns a `State` on every path and `tsc` reports an error, pointing straight at the missing case.

15.3.4 Typed context and custom hooks

Context needs a type for its value. A popular pattern starts the context with `null` (there is no sensible default) and wraps `useContext` in a custom hook that throws if no provider is found. The hook's return type is then the non-null value, so components never have to check for `null`.

AUTH.TSX

TypeScript

```
import { createContext, useContext, useState, type ReactNode } from "react";

export interface AuthValue {
  user: { name: string } | null;
  login: (name: string) => void;
  logout: () => void;
}

const AuthContext = createContext<AuthValue | null>(null);

export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<AuthValue["user"]>(null);
  const value: AuthValue = { user, login: (name) => setUser({ name }), logout: () => setUser(null) };
  return <AuthContext value={value}>{children}</AuthContext>;
}

export function useAuth(): AuthValue {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be used inside <AuthProvider>");
  return ctx;                                  // narrowed: AuthValue, never null
}
```

APP.TSX

TypeScript

```tsx
import { AuthProvider, useAuth } from "./auth.tsx";

function Header() {
  const { user, login, logout } = useAuth();   // fully typed, no null checks
  return (
    <header className="flex w-72 items-center justify-between rounded bg-slate-900 px-3 py-2 text-sm
text-white">
      <b>LBF</b>
      {user ? <button onClick={logout}>Log out {user.name}</button> : <button onClick={() =>
login("Asha")}>Log in</button>}
    </header>
  );
}

export default function App() {
  return <AuthProvider><Header /></AuthProvider>;
}
```

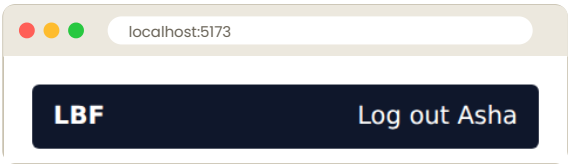
TSC ON AUTH.TSX, APP.TSX

main.js

```
$ npx tsc --noEmit
```

TYPE CHECK (TSC --STRICT)

```
(no output: every file type-checks)
```



AFTER CLICKING LOG IN: THE TYPED CONTEXT UPDATED THE HEADER

15.4 Typing events

15.4.1 ChangeEvent, FormEvent, MouseEvent

When a handler is written **inline** in JSX (`onChange={(e) => ...}`), TypeScript infers the event type for you. When the handler is a separate function, annotate the parameter. Event types are generic over the element: `ChangeEvent<HTMLInputElement>` gives `e.target.value` as a string and `e.target.checked` as a boolean. If you are unsure of the type, write the handler inline, hover over `e` in the editor, and copy the type.

Event	Type	Used with
change	`ChangeEvent<HTMLInputElement>` (or `HTMLSelectElement`, `HTMLTextAreaElement`)	`onChange`
submit	`FormEvent<HTMLFormElement>`	`onSubmit`
click	`MouseEvent<HTMLButtonElement>`	`onClick`, `onMouseEnter`
key	`KeyboardEvent<HTMLInputElement>`	`onKeyDown`
handler type	`ChangeEventHandler<HTMLInputElement>`	typing a prop that receives a handler

APP.TSX TypeScript

```
import { useState, type ChangeEvent, type FormEvent, type MouseEvent } from "react";

export default function App() {
  const [form, setForm] = useState({ name: "", course: "React", agree: false });
  const [log, setLog] = useState("");

  function handleChange(e: ChangeEvent<HTMLInputElement | HTMLSelectElement>) {
    const { name, value } = e.target;
    const checked = e.target instanceof HTMLInputElement && e.target.type === "checkbox" ?
e.target.checked : undefined;
    setForm((f) => ({ ...f, [name]: checked ?? value }));
  }
  function handleSubmit(e: FormEvent<HTMLFormElement>) {
    e.preventDefault();
    setLog(`submitted ${JSON.stringify(form)}`);
  }
  function handleClear(e: MouseEvent<HTMLButtonElement>) {
    setLog(`cleared with ${e.shiftKey ? "Shift+" : ""}click`);
    setForm({ name: "", course: "React", agree: false });
  }

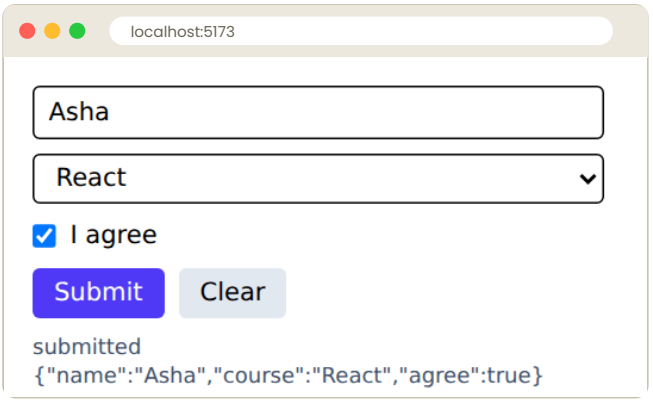
  return (
    <form onSubmit={handleSubmit} className="w-80 space-y-2 text-sm">
      <input name="name" value={form.name} onChange={handleChange} placeholder="Name" className="w-
full rounded border px-2 py-1" />
      <select name="course" value={form.course} onChange={handleChange} className="w-full rounded
border px-2 py-1">
        <option>React</option><option>Java</option>
      </select>
      <label className="flex gap-2"><input type="checkbox" name="agree" checked={form.agree} onChange=
{handleChange} /> I agree</label>
      <div className="flex gap-2">
        <button className="rounded bg-indigo-600 px-3 py-1 text-white">Submit</button>
        <button type="button" onClick={handleClear} className="rounded bg-slate-200 px-3 py-
1">Clear</button>
      </div>
      <p className="text-xs text-slate-600">{log}</p>
    </form>
  );
}
```

TSC ON APP.TSX main.js

```
$ npx tsc --noEmit
```

TYPE CHECK (TSC --STRICT)

```
(no output: every file type-checks)
```



TYPED CHANGE, SUBMIT AND CLICK HANDLERS WORKING TOGETHER

15.5 Typing API data

15.5.1 Response types and Zod-inferred types

`res.json()` returns `Promise<any>`: TypeScript has no idea what the server sent. Writing `as Student[]` silences the compiler but proves nothing; if the server changes, the app breaks at runtime with no warning. The robust approach is to **validate at the boundary**: describe the response with a Zod schema, parse the data with it, and let TypeScript **infer** the type from the schema with `z.infer`. One definition gives both the runtime check and the static type, so they can never disagree.

API.TS — TypeScript

```ts
import { z } from "zod";

export const StudentSchema = z.object({
  id: z.number(),
  name: z.string(),
  course: z.enum(["React", "Java", "Python"]),
  attendance: z.number().min(0).max(100),
});
export type Student = z.infer<typeof StudentSchema>;
// = { id: number; name: string; course: "React" | "Java" | "Python"; attendance: number }

const StudentList = z.array(StudentSchema);

export async function fetchStudents(url: string): Promise<Student[]> {
  const res = await fetch(url);
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  const json: unknown = await res.json();          // unknown, not any
  return StudentList.parse(json);                  // throws if the shape is wrong
}
```

APP.TSX TypeScript

```tsx
import { useEffect, useState } from "react";
import { fetchStudents, type Student } from "./api.ts";

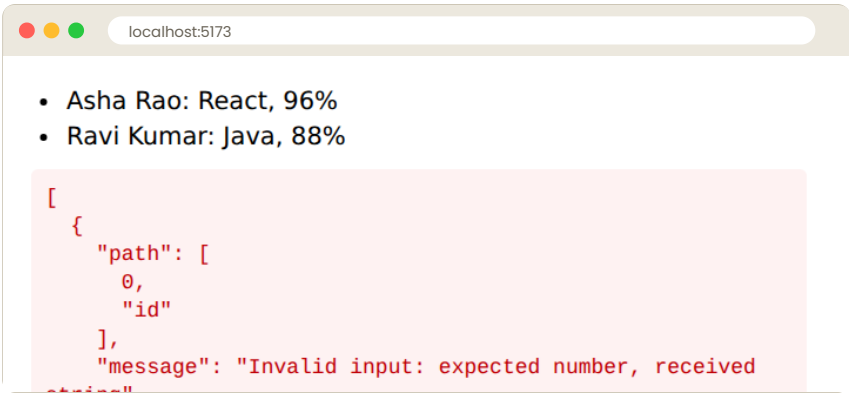
export default function App() {
  const [good, setGood] = useState<Student[]>([]);
  const [error, setError] = useState("");
  useEffect(() => {
    fetchStudents("/students.json").then(setGood);
    fetchStudents("/students-v2.json").catch((e: Error) => setError(e.message.slice(0, 160)));
  }, []);
  return (
    <div className="w-[440px] space-y-2 text-sm">
      <ul className="list-disc pl-5">{good.map((s) => <li key={s.id}>{s.name}: {s.course},
{s.attendance}%</li>)}</ul>
      {error && <pre className="whitespace-pre-wrap rounded bg-red-50 p-2 text-xs text-red-700">
{error}</pre>}
    </div>
  );
}
```

TSC ON API.TS, APP.TSX main.js

```
$ npx tsc --noEmit
```

TYPE CHECK (TSC --STRICT)

```
(no output: every file type-checks)
```



VALID DATA RENDERED; THE CHANGED RESPONSE (ID BECAME A STRING) WAS REJECTED AT THE BOUNDARY

Because `Student` comes from the schema, a typo such as `s.atendance` or comparing `s.course === "C#"` is a compile error, and adding a field to the schema updates the type everywhere:

TSC AFTER A TYPO AND AN IMPOSSIBLE COMPARISON main.js

```
$ npx tsc --noEmit
```

TYPE CHECK (TSC --STRICT)

```
App.tsx(13,81): error TS2367: This comparison appears to be unintentional because the types '"React" |
"Java" | "Python"' and '"C#"' have no overlap.
App.tsx(13,144): error TS2551: Property 'atendance' does not exist on type '{ id: number; name:
string; course: "React" | "Java" | "Python"; attendance: number; }'. Did you mean 'attendance'?
```

15.6 Generic components

15.6.1 A typed List<T> or Select<T> component

A **generic component** works with any item type while keeping that type connected between its props. Declare a type parameter `<T,>`: the `items` are `T[]`, and `renderItem` receives a `T`. When you use the component, TypeScript **infers** `T` from the items, so inside `renderItem` the item has the right type without any annotation.

LIST.TSX TypeScript

```
import type { ReactNode } from "react";

interface ListProps<T> {
  items: T[];
  getKey: (item: T) => string | number;
  renderItem: (item: T) => ReactNode;
  empty?: ReactNode;
}

export function List<T,>({ items, getKey, renderItem, empty = "Nothing here" }: ListProps<T>) {
  if (items.length === 0) return <p className="text-slate-400">{empty}</p>;
  return <ul className="divide-y">{items.map((item) => <li key={getKey(item)} className="py-1">
{renderItem(item)}</li>)}</ul>;
}
```

SELECT.TSX TypeScript

```
interface SelectProps<T> {
  options: T[];
  value: T;
  onChange: (value: T) => void;
  getLabel: (option: T) => string;
}

export function Select<T,>({ options, value, onChange, getLabel }: SelectProps<T>) {
  return (
    <select className="rounded border px-2 py-1" value={options.indexOf(value)} onChange={(e) =>
onChange(options[Number(e.target.value)])}>
      {options.map((o, i) => <option key={i} value={i}>{getLabel(o)}</option>)}
    </select>
  );
}
```

APP.TSX TypeScript

```tsx
import { useState } from "react";
import { List } from "./List.tsx";
import { Select } from "./Select.tsx";

interface Course { code: string; title: string; weeks: number }
const courses: Course[] = [
  { code: "RCT", title: "React", weeks: 10 },
  { code: "JAV", title: "Java Full Stack", weeks: 16 },
];
const mentors = ["Anil", "Priya", "Farah"];

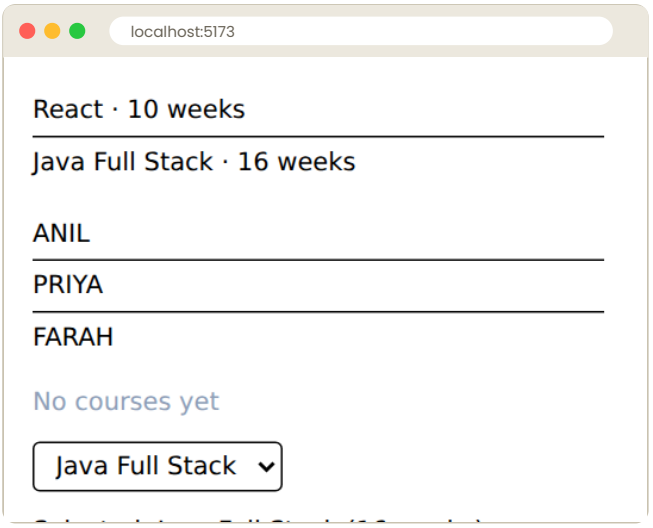
export default function App() {
  const [course, setCourse] = useState(courses[0]);
  return (
    <div className="w-80 space-y-3 text-sm">
      <List items={courses} getKey={(c) => c.code}
            renderItem={(c) => <span>{c.title} · {c.weeks} weeks</span>} />   {/* c: Course */}
      <List items={mentors} getKey={(m) => m} renderItem={(m) => m.toUpperCase()} />   {/* m: string
*/}
      <List items={[] as Course[]} getKey={(c) => c.code} renderItem={(c) => c.title} empty="No
courses yet" />
      <Select options={courses} value={course} onChange={setCourse} getLabel={(c) => c.title} />
      <p>Selected: {course.title} ({course.weeks} weeks)</p>
    </div>
  );
}
```

TSC ON LIST.TSX, SELECT.TSX, APP.TSX main.js

```
$ npx tsc --noEmit
```

TYPE CHECK (TSC --STRICT)

(no output: every file type-checks)



ONE LIST COMPONENT RENDERING COURSE OBJECTS, STRINGS AND AN EMPTY LIST; THE TYPED SELECT RETURNED A COURSE

TSC AFTER USING FIELDS THAT COURSE DOES NOT HAVE

main.js

```
$ npx tsc --noEmit
```

TYPE CHECK (TSC --STRICT)

```
App.tsx(17,41): error TS2339: Property 'name' does not exist on type 'Course'.
App.tsx(17,58): error TS2339: Property 'toUpperCase' does not exist on type 'number'.
```

PRACTICE CHECKPOINT

Convert the Student Management front end to TypeScript in strict mode with fully typed props, hooks, events and API responses.

The sample solution below is the plain-React version of the student manager (state in a typed reducer and context, API responses validated with Zod), written in strict TypeScript. The type check of all five files comes first, then the running app.

TYPES.TS

TypeScript

```ts
import { z } from "zod";

export const COURSES = ["React", "Java", "Python"] as const;

export const StudentSchema = z.object({
  id: z.number(),
  name: z.string().min(3),
  email: z.string().email(),
  course: z.enum(COURSES),
});
export type Student = z.infer<typeof StudentSchema>;
export type NewStudent = Omit<Student, "id">;
export const StudentListSchema = z.array(StudentSchema);
```

API.TS

TypeScript

```ts
import { StudentListSchema, type Student, type NewStudent } from "./types.ts";

const wait = (ms: number) => new Promise<void>((r) => setTimeout(r, ms));
let db: unknown = [
  { id: 1, name: "Asha Rao", email: "asha@example.in", course: "React" },
  { id: 2, name: "Ravi Kumar", email: "ravi@example.in", course: "Java" },
];

export async function listStudents(): Promise<Student[]> {
  await wait(200);
  return StudentListSchema.parse(db);                // validate what "the server" sent
}
export async function createStudent(data: NewStudent): Promise<Student> {
  await wait(200);
  const list = StudentListSchema.parse(db);
  const student: Student = { id: Math.max(0, ...list.map((s) => s.id)) + 1, ...data };
  db = [...list, student];
  return student;
}
```

STORE.TSX TypeScript

```
import { createContext, useContext, useEffect, useReducer, type ReactNode } from "react";
import type { Student, NewStudent } from "./types.ts";
import { listStudents, createStudent } from "./api.ts";

interface State { students: Student[]; status: "loading" | "ready" | "error"; error: string | null }
type Action =
  | { type: "loaded"; students: Student[] }
  | { type: "added"; student: Student }
  | { type: "failed"; error: string };

function reducer(state: State, action: Action): State {
  switch (action.type) {
    case "loaded": return { students: action.students, status: "ready", error: null };
    case "added": return { ...state, students: [...state.students, action.student] };
    case "failed": return { ...state, status: "error", error: action.error };
  }
}

interface StudentsValue extends State { add: (data: NewStudent) => Promise<void> }
const StudentsContext = createContext<StudentsValue | null>(null);

export function StudentsProvider({ children }: { children: ReactNode }) {
  const [state, dispatch] = useReducer(reducer, { students: [], status: "loading", error: null });
  useEffect(() => {
    listStudents()
      .then((students) => dispatch({ type: "loaded", students }))
      .catch((e: unknown) => dispatch({ type: "failed", error: e instanceof Error ? e.message :
String(e) }));
  }, []);
  const add = async (data: NewStudent) => dispatch({ type: "added", student: await createStudent(data)
});
  return <StudentsContext value={{ ...state, add }}>{children}</StudentsContext>;
}

export function useStudents(): StudentsValue {
  const ctx = useContext(StudentsContext);
  if (!ctx) throw new Error("useStudents must be used inside <StudentsProvider>");
  return ctx;
}
```

STUDENTFORM.TSX TypeScript

```
import { useState, type ChangeEvent, type FormEvent } from "react";
import { COURSES, StudentSchema, type NewStudent } from "./types.ts";

interface StudentFormProps { onSave: (data: NewStudent) => Promise<void> }
type Errors = Partial<Record<keyof NewStudent, string>>;

const empty: NewStudent = { name: "", email: "", course: "React" };

export function StudentForm({ onSave }: StudentFormProps) {
  const [values, setValues] = useState<NewStudent>(empty);
  const [errors, setErrors] = useState<Errors>({});

  function handleChange(e: ChangeEvent<HTMLInputElement | HTMLSelectElement>) {
    setValues((v) => ({ ...v, [e.target.name]: e.target.value }));
  }
  async function handleSubmit(e: FormEvent<HTMLFormElement>) {
    e.preventDefault();
    const result = StudentSchema.safeParse({ ...values, id: 0 });
    if (!result.success) {
```

```
      const next: Errors = {};
      for (const issue of result.error.issues) next[issue.path[0] as keyof NewStudent] ??=
issue.message;
      setErrors(next);
      return;
    }
    setErrors({});
    await onSave(values);
    setValues(empty);
  }
  return (
    <form onSubmit={handleSubmit} className="grid grid-cols-[1fr_1fr_90px_auto] items-start gap-2
text-sm">
      <div><input name="name" value={values.name} onChange={handleChange} placeholder="Name"
className="w-full rounded border px-2 py-1" />
        {errors.name && <p className="text-xs text-red-600">{errors.name}</p>}</div>
      <div><input name="email" value={values.email} onChange={handleChange} placeholder="Email"
className="w-full rounded border px-2 py-1" />
        {errors.email && <p className="text-xs text-red-600">{errors.email}</p>}</div>
      <select name="course" value={values.course} onChange={handleChange} className="rounded border
px-1 py-1">
        {COURSES.map((c) => <option key={c}>{c}</option>)}
      </select>
      <button className="rounded bg-indigo-600 px-3 py-1 text-white">Add</button>
    </form>
  );
}
```

APP.TSX: SAMPLE SOLUTION TypeScript

```
import { StudentsProvider, useStudents } from "./store.tsx";
import { StudentForm } from "./StudentForm.tsx";
import type { Student } from "./types.ts";

function StudentRow({ student }: { student: Student }) {
  return <tr className="border-b"><td className="py-1">{student.name}</td><td>{student.email}</td><td>
{student.course}</td></tr>;
}

function StudentManager() {
  const { students, status, error, add } = useStudents();
  if (status === "loading") return <p>Loading…</p>;
  if (status === "error") return <p className="text-red-600">{error}</p>;
  return (
    <div className="w-[540px] space-y-3">
      <table className="w-full text-left text-sm">
        <thead><tr className="border-b text-slate-500"><th>Name</th><th>Email</th><th>Course</th></tr>
</thead>
        <tbody>{students.map((s) => <StudentRow key={s.id} student={s} />)}</tbody>
      </table>
      <StudentForm onSave={add} />
    </div>
  );
}

export default function App() {
  return <StudentsProvider><StudentManager /></StudentsProvider>;
}
```

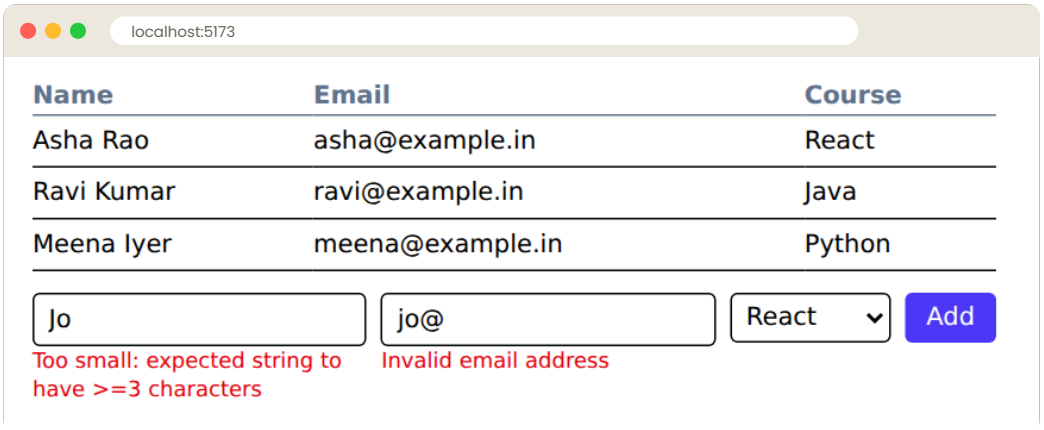
TSC ON TYPES.TS, API.TS, STORE.TSX, STUDENTFORM.TSX, APP.TSX

main.js

```
$ npx tsc --noEmit
```

TYPE CHECK (TSC --STRICT)

```
(no output: every file type-checks)
```



ADDED MEENA IYER (PYTHON); THEN A SECOND ATTEMPT WITH A BAD EMAIL WAS STOPPED BY THE SCHEMA

Use the `react-ts` template with `strict: true`; `tsc` checks, Vite strips types. Type props with an interface or type, mark optional props with `?` and give defaults, type children as `ReactNode`, and extend native props with `ComponentProps<"button">`. Give `useState` a type argument for unions, `null` and empty arrays, and narrow before use; type DOM refs as `useRef<HTMLInputElement>(null)`; model reducer actions as a discriminated union; create context with `null` and a throwing custom hook. Annotate separate handlers with `ChangeEvent`, `FormEvent` and `MouseEvent`. Treat API data as `unknown`, validate it with Zod and infer the type with `z.infer`. Generic components (`List<T,>`) keep item types connected.

LEVEL 3

Advanced level

Outcome: manage global state, use React 19 Actions and Suspense, optimise performance, apply advanced patterns, test thoroughly and build full-stack apps with Next.js.

7 MODULES · 20 DAYS MINIMUM

Module 16 · Global state management	**3 days**
Module 17 · Modern React: Actions, Suspense and concurrent features	**3 days**
Module 18 · Performance and React internals	**2 days**
Module 19 · Advanced patterns and application architecture	**2 days**
Module 20 · Testing React applications	**2 days**
Module 21 · Full-stack React with Next.js	**4 days**
Module 22 · Deployment, projects and interview preparation	**4 days**

MODULE 16 · ADVANCED

Global state management

Minimum duration **3 days** | Level **Advanced**

In a large app many distant components read and change the same client state: the cart, the logged-in user, preferences, a document being edited. Context works, but every consumer re-renders on every change and the update logic spreads across components. Dedicated state libraries keep this state in one predictable place. This module recaps server state versus client state, then teaches Redux Toolkit (store, slices, useSelector and useDispatch, async thunks, an overview of RTK Query and the DevTools) and Zustand (stores, selectors that avoid re-renders, persistence), surveys atom-based libraries and MobX, and compares the options.

IN THIS MODULE

16.1 When you need a state library
16.2 Redux Toolkit
16.3 Zustand
16.4 Other options (overview)
16.5 Choosing a solution

HOW THE EXAMPLES WERE RUN

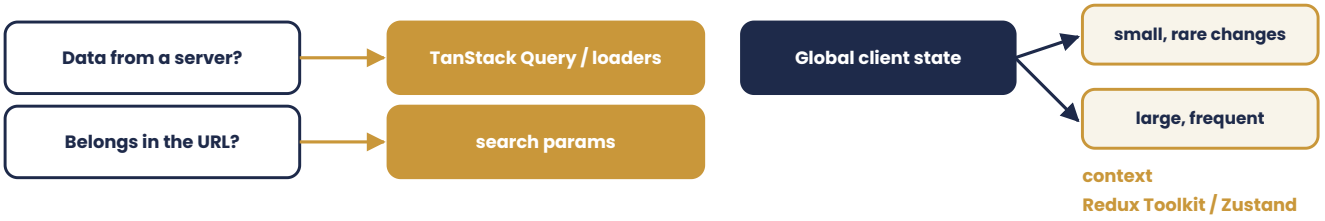
Redux Toolkit, React-Redux and Zustand are not installed in the offline build used for these notes. Listings are written exactly as for Redux Toolkit 2 / React-Redux 9 and Zustand 5; the screenshots come from the same code bundled against small stand-ins with the same API and run in Chromium.

16.1 When you need a state library

16.1.1 Server state vs client state recap

First remove what does **not** belong in a global store. Data that comes from the server (students, products, orders) is **server state**: TanStack Query or router loaders already cache, share and refresh it (Module 14). URL state (filters, page) belongs in the URL. What remains is **global client state**: data created in the browser that many distant components need, such as the cart before checkout, the current user and token, theme and language, or a multi-step wizard. Often that is small enough for context. Reach for a library when:

- the state changes **often** and many components read **different parts** of it (context re-renders every consumer on every change);
- the update logic is **complex** (many actions, rules that span several pieces of state) and you want it in one testable place;
- you want **tooling**: time-travel debugging, action logs, persistence, middleware;
- a big team needs one **consistent pattern** for shared state.



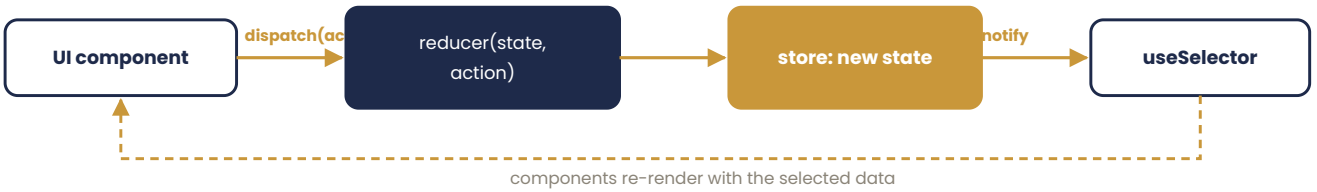
Only what is left after removing server and URL state needs a global store.

CHOOSING A HOME FOR A PIECE OF STATE

16.2 Redux Toolkit

16.2.1 Redux principles: store, actions, reducers, single source of truth

Redux is built on three principles. **Single source of truth**: all global state lives in one object, the **store**. **State is read-only**: the only way to change it is to **dispatch an action**, a plain object describing what happened, such as `{ type: "cart/itemAdded", payload: 3 }`. **Changes are made by pure functions**: a **reducer** takes the current state and an action and returns the next state. Because every change goes through dispatch, each one can be logged, replayed and inspected. **Redux Toolkit** (RTK) is the official, modern way to write Redux; it removes almost all of the old boilerplate.



THE ONE-WAY REDUX DATA FLOW

16.2.2 configureStore

`configureStore` creates the store. Pass an object of slice reducers: each key becomes a top-level field of the state (`state.cart`, `state.auth`). It also sets up the thunk middleware for async logic, checks for accidental mutations in development, and connects to the Redux DevTools. Make the store available with `<Provider store={store}>` from React-Redux.

STORE.JS AND MAIN.JSX JavaScript

```
// app/store.js
import { configureStore } from "@reduxjs/toolkit";
import cartReducer from "../features/cart/cartSlice.js";
import authReducer from "../features/auth/authSlice.js";

export const store = configureStore({
  reducer: {
    cart: cartReducer,        // state.cart
    auth: authReducer,        // state.auth
  },
});

// main.jsx
import { Provider } from "react-redux";
createRoot(document.getElementById("root")).render(
  <Provider store={store}><App /></Provider>
);
```

16.2.3 createSlice

A **slice** is the state and logic for one feature. `createSlice` takes a name, the initial state and an object of **case reducers**, and generates the reducer plus one **action creator** per case. Inside a case reducer you may write "mutating" code such as `state.items.push(...)`: RTK uses the Immer library to turn it into a safe immutable update. (Either mutate the draft or return a new value, never both.)

STORE-DEMO.JS JavaScript

```
import { configureStore } from "@reduxjs/toolkit";
import cartReducer, { itemAdded, itemRemoved } from "./cartSlice.js";

const store = configureStore({ reducer: { cart: cartReducer } });

const action = itemAdded({ id: 2, name: "Keyboard", price: 3499 });
console.log("action object:", action);
store.dispatch(action);
store.dispatch(itemAdded({ id: 2, name: "Keyboard", price: 3499 }));
store.dispatch(itemAdded({ id: 1, name: "Mouse", price: 799 }));
console.log("after 3 adds:", JSON.stringify(store.getState()));
store.dispatch(itemRemoved(1));
console.log("after removing id 1:", JSON.stringify(store.getState()));
```

BROWSER CONSOLE

```
after 3 adds: {"cart":{"items":[{"id":2,"name":"Keyboard","price":3499,"qty":2},
{"id":1,"name":"Mouse","price":799,"qty":1}]}}
after removing id 1: {"cart":{"items":[{"id":2,"name":"Keyboard","price":3499,"qty":2}]}}
action object: {type: 'cart/itemAdded', payload: {id: 2, name: 'Keyboard', price: 3499}}
```

FEATURES/CART/CARTSLICE.JS (USED ABOVE) JavaScript

```
import { createSlice } from "@reduxjs/toolkit";

const cartSlice = createSlice({
  name: "cart",
  initialState: { items: [] },              // [{ id, name, price, qty }]
  reducers: {
    itemAdded(state, action) {
      const existing = state.items.find((i) => i.id === action.payload.id);
      if (existing) existing.qty += 1;      // "mutating" is safe here (Immer)
      else state.items.push({ ...action.payload, qty: 1 });
    },
    itemRemoved(state, action) {
      state.items = state.items.filter((i) => i.id !== action.payload);
    },
    cleared(state) {
      state.items = [];
    },
  },
});

export const { itemAdded, itemRemoved, cleared } = cartSlice.actions;
export default cartSlice.reducer;
```

16.2.4 useSelector and useDispatch with React-Redux

`useSelector(selector)` reads data from the store: the selector receives the whole state and returns the part the component needs. The component re-renders only when that **selected value** changes (compared with `===`), so select as little as possible, and never return a new object or array from a

selector on every call. `useDispatch()` returns `dispatch` for sending actions. Selectors are often defined next to the slice and reused.

APP.JSX JavaScript

```
import { configureStore } from "@reduxjs/toolkit";
import { Provider, useSelector, useDispatch } from "react-redux";
import { useRef } from "react";
import cartReducer, { itemAdded, itemRemoved, cleared } from "./cartSlice.js";

const store = configureStore({ reducer: { cart: cartReducer } });

// selectors
const selectItems = (state) => state.cart.items;
const selectCount = (state) => state.cart.items.reduce((n, i) => n + i.qty, 0);
const selectTotal = (state) => state.cart.items.reduce((s, i) => s + i.qty * i.price, 0);

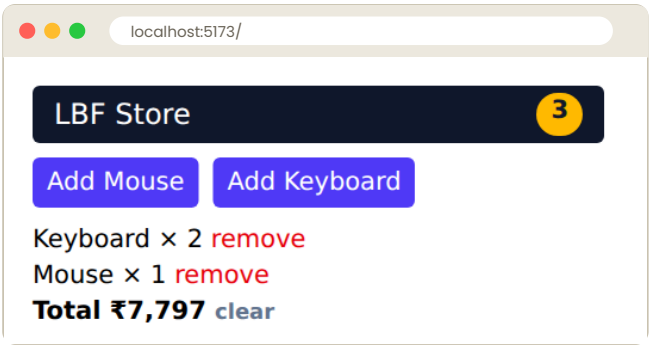
const products = [{ id: 1, name: "Mouse", price: 799 }, { id: 2, name: "Keyboard", price: 3499 }];

function CartBadge() {
  const count = useSelector(selectCount);              // re-renders only when count changes
  return <span className="rounded-full bg-amber-400 px-2 text-sm font-bold text-slate-900">{count}
</span>;
}

function ProductList() {
  const dispatch = useDispatch();
  return products.map((p) => (
    <button key={p.id} onClick={() => dispatch(itemAdded(p))} className="mr-2 rounded bg-indigo-600
px-2 py-1 text-sm text-white">Add {p.name}</button>
  ));
}

function CartView() {
  const items = useSelector(selectItems);
  const total = useSelector(selectTotal);
  const dispatch = useDispatch();
  return (
    <div className="text-sm">
      {items.map((i) => <p key={i.id}>{i.name} × {i.qty} <button className="text-red-600" onClick={()
=> dispatch(itemRemoved(i.id))}>remove</button></p>)}
      <p className="font-semibold">Total ₹{total.toLocaleString("en-IN")} <button className="text-xs
text-slate-500" onClick={() => dispatch(cleared())}>clear</button></p>
    </div>
  );
}

export default function App() {
  return (
    <Provider store={store}>
      <div className="w-80 space-y-2">
        <header className="flex justify-between rounded bg-slate-900 px-3 py-1 text-white">LBF Store
<CartBadge /></header>
        <ProductList />
        <CartView />
      </div>
    </Provider>
  );
}
```



ADDED KEYBOARD TWICE AND MOUSE ONCE: THREE DISTANT COMPONENTS SHARE ONE STORE

16.2.5 Async logic with createAsyncThunk

Reducers must be pure and synchronous, so where does fetching go? A **thunk** is a function you dispatch instead of an action; it receives `dispatch` and `getState` and may do async work. `createAsyncThunk` wraps an async function and dispatches three actions for you: `pending` when it starts, `fulfilled` with the result, or `rejected` with the error. The slice handles them in `extraReducers`, typically to keep a `status` field.

STUDENTSSLICE.JS

JavaScript

```
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";

export const fetchStudents = createAsyncThunk("students/fetch", async () => {
  const res = await fetch("/students.json");
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();                          // becomes action.payload of "fulfilled"
});

const studentsSlice = createSlice({
  name: "students",
  initialState: { list: [], status: "idle", error: null },   // idle | loading | succeeded | failed
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchStudents.pending, (state) => { state.status = "loading"; })
      .addCase(fetchStudents.fulfilled, (state, action) => { state.status = "succeeded"; state.list =
action.payload; })
      .addCase(fetchStudents.rejected, (state, action) => { state.status = "failed"; state.error =
action.error.message; });
  },
});
export default studentsSlice.reducer;
```

APP.JSX

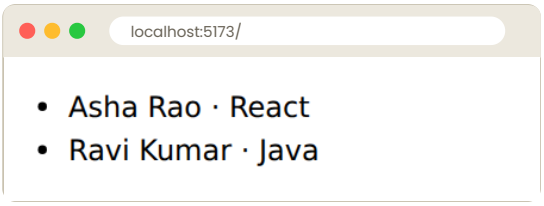
JavaScript

```
import { configureStore } from "@reduxjs/toolkit";
import { Provider, useSelector, useDispatch } from "react-redux";
import { useEffect } from "react";
import studentsReducer, { fetchStudents } from "./studentsSlice.js";

const store = configureStore({ reducer: { students: studentsReducer } });

function Students() {
  const dispatch = useDispatch();
  const { list, status, error } = useSelector((state) => state.students);
  useEffect(() => { if (status === "idle") dispatch(fetchStudents()); }, [status, dispatch]);
  if (status === "loading") return <p>Loading…</p>;
  if (status === "failed") return <p className="text-red-600">{error}</p>;
  return <ul className="list-disc pl-5">{list.map((s) => <li key={s.id}>{s.name} · {s.course}</li>)}
</ul>;
}

export default function App() {
  return <Provider store={store}><Students /></Provider>;
}
```



THE THUNK FETCHED STUDENTS.JSON; THE FULFILLED ACTION STORED THE LIST

16.2.6 RTK Query (overview)

RTK Query is Redux Toolkit's built-in data-fetching and caching layer, the Redux equivalent of TanStack Query. You describe endpoints once with `createApi`, and it generates hooks such as `useGetStudentsQuery` and `useAddStudentMutation` with caching, de-duplication and invalidation by **tags**. Choose it when the app already uses Redux; otherwise TanStack Query is the common choice.

RTK QUERY (REPRESENTATIVE)

JavaScript

```
import { createApi, fetchBaseQuery } from "@reduxjs/toolkit/query/react";

export const api = createApi({
  baseQuery: fetchBaseQuery({ baseUrl: "/api" }),
  tagTypes: ["Student"],
  endpoints: (build) => ({
    getStudents: build.query({ query: () => "/students", providesTags: ["Student"] }),
    addStudent: build.mutation({
      query: (body) => ({ url: "/students", method: "POST", body }),
      invalidatesTags: ["Student"],          // refetch getStudents afterwards
    }),
  }),
});
export const { useGetStudentsQuery, useAddStudentMutation } = api;

// store: reducer: { [api.reducerPath]: api.reducer }, middleware: (gDM) =>
gDM().concat(api.middleware)
// component: const { data = [], isLoading } = useGetStudentsQuery();
```

16.2.7 Redux DevTools

Install the **Redux DevTools** browser extension; `configureStore` connects to it automatically in development. Its panel lists every dispatched action in order with its payload, shows the state and the **diff** after each one, and lets you **time-travel**: jump back to any earlier action and watch the UI return to that moment. When a bug report says "the total was wrong after removing an item", you can replay exactly that sequence.

DevTools tab	Shows
Action list	every action with its type and time
Action	the payload of the selected action
State	the whole state tree after the action
Diff	only what this action changed
Slider / Jump	time-travel through the action history

16.3 Zustand

16.3.1 Creating a store

Zustand (German for "state") is a tiny state library with almost no boilerplate. `create` takes a function that receives `set` (and `get`) and returns the initial state **and** the actions that change it, all in one object. The result is a **hook**: there is no provider to add. Call `set` with the fields to change (they are merged into the state) or with a function of the current state.

CARTSTORE.JS

JavaScript

```
import { create } from "zustand";

export const useCartStore = create((set, get) => ({
  items: [],                                          // state
  add: (product) => set((state) => {                  // actions
    const existing = state.items.find((i) => i.id === product.id);
    return {
      items: existing
        ? state.items.map((i) => (i.id === product.id ? { ...i, qty: i.qty + 1 } : i))
        : [...state.items, { ...product, qty: 1 }],
    };
  }),
  remove: (id) => set((state) => ({ items: state.items.filter((i) => i.id !== id) })),
  clear: () => set({ items: [] }),
  total: () => get().items.reduce((s, i) => s + i.qty * i.price, 0),   // reads with get()
}));
```

APP.JSX

JavaScript

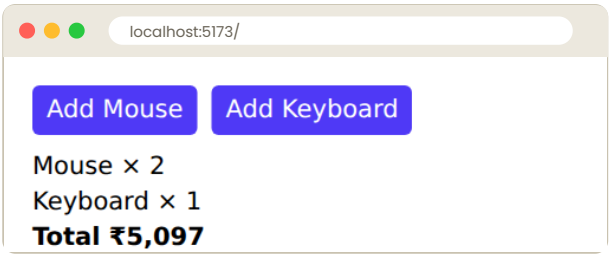
```
import { useCartStore } from "./cartStore.js";

const products = [{ id: 1, name: "Mouse", price: 799 }, { id: 2, name: "Keyboard", price: 3499 }];

function AddButtons() {
  const add = useCartStore((s) => s.add);
  return products.map((p) => <button key={p.id} onClick={() => add(p)} className="mr-2 rounded bg-
indigo-600 px-2 py-1 text-sm text-white">Add {p.name}</button>);
}

function Cart() {
  const items = useCartStore((s) => s.items);
  const total = useCartStore((s) => s.total());
  return (
    <div className="text-sm">
      {items.map((i) => <p key={i.id}>{i.name} × {i.qty}</p>)}
      <p className="font-semibold">Total ₹{total.toLocaleString("en-IN")}</p>
    </div>
  );
}

export default function App() {                          // no Provider needed
  return <div className="w-80 space-y-2"><AddButtons /><Cart /></div>;
}
```



THE SAME CART WITH ZUSTAND: ONE STORE FILE AND TWO HOOKS CALLS, NO PROVIDER

16.3.2 Selectors and avoiding re-renders

Like `useSelector`, a Zustand hook takes a **selector**, and the component re-renders only when the selected value changes. Calling the hook with no selector (`useStore()`) subscribes to the **whole** store, so the component re-renders on every change anywhere. When a selector must return several values as a new object or array, wrap it in `useShallow` so that it is compared field by field. The render counters below make the difference visible.

STORE.JS

JavaScript

```
import { create } from "zustand";
export const useStore = create((set) => ({
  count: 0,
  theme: "light",
  increment: () => set((s) => ({ count: s.count + 1 })),
  toggleTheme: () => set((s) => ({ theme: s.theme === "light" ? "dark" : "light" })),
}));
```

APP.JSX

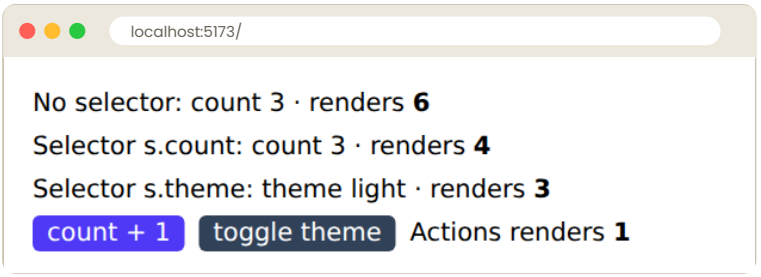
JavaScript

```
import { useRef } from "react";
import { useShallow } from "zustand/react/shallow";
import { useStore } from "./store.js";

function useRenderCount() { const r = useRef(0); r.current += 1; return r.current; }

function WholeStore() {
  const { count } = useStore();                              // subscribes to EVERYTHING
  return <p>No selector: count {count} · renders <b>{useRenderCount()}</b></p>;
}
function CountOnly() {
  const count = useStore((s) => s.count);                    // only count
  return <p>Selector s.count: count {count} · renders <b>{useRenderCount()}</b></p>;
}
function ThemeOnly() {
  const theme = useStore((s) => s.theme);                    // only theme
  return <p>Selector s.theme: theme {theme} · renders <b>{useRenderCount()}</b></p>;
}
function Actions() {
  const { increment, toggleTheme } = useStore(useShallow((s) => ({ increment: s.increment,
toggleTheme: s.toggleTheme })));
  return (
    <div className="flex gap-2">
      <button className="rounded bg-indigo-600 px-2 text-white" onClick={increment}>count + 1</button>
      <button className="rounded bg-slate-700 px-2 text-white" onClick={toggleTheme}>toggle
theme</button>
      <span>Actions renders <b>{useRenderCount()}</b></span>
    </div>
  );
}

export default function App() {
  return <div className="w-96 space-y-1 text-sm"><WholeStore /><CountOnly /><ThemeOnly /><Actions />
</div>;
}
```



AFTER 3 × “COUNT + 1” AND 2 × “TOGGLE THEME”: EACH COMPONENT RE-RENDERED ONLY FOR THE DATA IT SELECTED

The component without a selector rendered 6 times (once at the start plus all five changes); `CountOnly` 4 times (start + 3 count changes); `ThemeOnly` 3 times (start + 2 theme changes); and `Actions` only once, because `useShallow` saw that the two functions never changed.

16.3.3 Persist middleware

Middleware wraps the store creator to add behaviour. `persist` saves the state to `localStorage` (or any storage) after every change and restores it when the app starts, so the cart survives a page reload. Give it a unique `name` (the storage key) and, with `partialize`, choose which fields to save: never functions, and usually not temporary UI state.

CARTSTORE.JS

JavaScript

```
import { create } from "zustand";
import { persist, createJSONStorage } from "zustand/middleware";

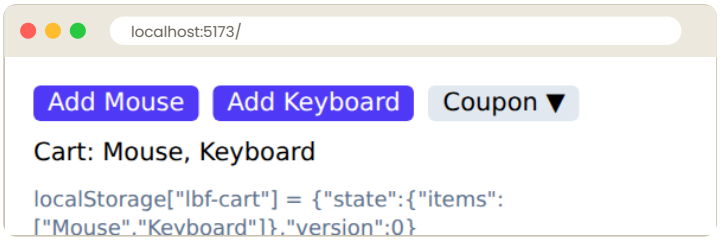
export const useCartStore = create(
  persist(
    (set) => ({
      items: [],
      couponOpen: false,                                    // temporary UI state
      add: (name) => set((s) => ({ items: [...s.items, name] })),
      toggleCoupon: () => set((s) => ({ couponOpen: !s.couponOpen })),
    }),
    {
      name: "lbf-cart",                                     // localStorage key
      storage: createJSONStorage(() => localStorage),
      partialize: (s) => ({ items: s.items }),              // save only items
    }
  )
);
```

APP.JSX

JavaScript

```
import { useCartStore } from "./cartStore.js";

export default function App() {
  const { items, add, couponOpen, toggleCoupon } = useCartStore();
  return (
    <div className="w-80 space-y-2 text-sm">
      <div className="flex gap-2">
        <button className="rounded bg-indigo-600 px-2 text-white" onClick={() => add("Mouse")}>Add
Mouse</button>
        <button className="rounded bg-indigo-600 px-2 text-white" onClick={() => add("Keyboard")}>Add
Keyboard</button>
        <button className="rounded bg-slate-200 px-2" onClick={toggleCoupon}>Coupon {couponOpen ? "▲"
: "▼"}</button>
      </div>
      <p>Cart: {items.join(", ") || "empty"}</p>
      <p className="text-xs text-slate-500">localStorage["lbf-cart"] = {localStorage.getItem("lbf-
cart")}</p>
    </div>
  );
}
```



ADDED TWO ITEMS, OPENED THE COUPON BOX, THEN RELOADED THE PAGE: THE ITEMS CAME BACK, THE UI FLAG DID NOT

16.4 Other options (overview)

16.4.1 Jotai and Recoil-style atoms

Atomic libraries split state into many tiny independent pieces called **atoms**, used much like `useState` but shared by every component that uses the same atom. **Derived atoms** compute values from other

atoms and update automatically. Components re-render only when the atoms they use change. **Jotai** is the popular, maintained library in this style; **Recoil**, which introduced it at Meta, has been archived and should not be used in new projects.

JOTAI (REPRESENTATIVE) JavaScript

```
import { atom, useAtom, useAtomValue } from "jotai";

const itemsAtom = atom([]);                                  // a piece of state
const countAtom = atom((get) => get(itemsAtom).length);      // derived, read-only

function AddButton() {
  const [items, setItems] = useAtom(itemsAtom);              // like useState, but shared
  return <button onClick={() => setItems([...items, "Mouse"])}>Add</button>;
}
function Badge() {
  const count = useAtomValue(countAtom);                     // re-renders only when count changes
  return <span>{count}</span>;
}
```

16.4.2 MobX

MobX uses **observable** objects and classes: you change state by plain mutation (`store.items.push(x)`), and components wrapped in `observer` automatically re-render when any observable they **read** during render changes. It feels natural to developers from object-oriented backgrounds and needs little code, at the cost of more "magic" and less explicit data flow than Redux.

MOBX (REPRESENTATIVE) JavaScript

```
import { makeAutoObservable } from "mobx";
import { observer } from "mobx-react-lite";

class CartStore {
  items = [];
  constructor() { makeAutoObservable(this); }     // fields become observable, methods actions
  add(item) { this.items.push(item); }            // plain mutation
  get count() { return this.items.length; }       // computed value
}
const cart = new CartStore();

const Badge = observer(() => <span>{cart.count}</span>);   // tracks what it reads
```

16.5 Choosing a solution

16.5.1 Comparison of Context, Redux Toolkit and Zustand

	Context + useReducer	Redux Toolkit	Zustand
Install	built into React	`@reduxjs/toolkit` + `react-redux`	`zustand` (about 1 kB)
Boilerplate	low	medium (slices, store, provider)	very low (one `create` call)
Provider needed	yes	yes, one `<Provider>`	no
Re-renders	every consumer on every change (unless split)	only components whose selected value changed	only components whose selected value changed

	Context + useReducer	Redux Toolkit	Zustand
Async logic	in components or custom hooks	`createAsyncThunk`, RTK Query	plain async functions in actions
DevTools and time travel	React DevTools only	excellent (Redux DevTools)	via the `devtools` middleware
Structure enforced	none	strong conventions (good for big teams)	flexible
Best for	theme, auth, locale; rarely changing data	large apps, many developers, complex rules	small to large apps wanting simplicity

TIP

A sensible default for a new app: **TanStack Query** for server state, the **URL** for filters and pages, **local state** wherever possible, **context** for theme and auth, and **Zustand** (or Redux Toolkit in a large team) for the remaining shared client state such as the cart.

PRACTICE CHECKPOINT

Implement the store's cart and auth state with Redux Toolkit, then rebuild the cart with Zustand and compare the code.

CARTSLICE.JS JavaScript

```
import { createSlice } from "@reduxjs/toolkit";

const cartSlice = createSlice({
  name: "cart",
  initialState: { items: [] },              // [{ id, name, price, qty }]
  reducers: {
    itemAdded(state, action) {
      const existing = state.items.find((i) => i.id === action.payload.id);
      if (existing) existing.qty += 1;      // "mutating" is safe here (Immer)
      else state.items.push({ ...action.payload, qty: 1 });
    },
    itemRemoved(state, action) {
      state.items = state.items.filter((i) => i.id !== action.payload);
    },
    cleared(state) {
      state.items = [];
    },
  },
});

export const { itemAdded, itemRemoved, cleared } = cartSlice.actions;
export const selectCount = (state) => state.cart.items.reduce((n, i) => n + i.qty, 0);
export const selectTotal = (state) => state.cart.items.reduce((s, i) => s + i.qty * i.price, 0);
export default cartSlice.reducer;
```

AUTHSLICE.JS JavaScript

```
import { createSlice } from "@reduxjs/toolkit";

const authSlice = createSlice({
  name: "auth",
  initialState: { user: null },
  reducers: {
    loggedIn(state, action) { state.user = { name: action.payload }; },
    loggedOut(state) { state.user = null; },
  },
});
export const { loggedIn, loggedOut } = authSlice.actions;
export const selectUser = (state) => state.auth.user;
export default authSlice.reducer;
```

STORE.JS JavaScript

```
import { configureStore } from "@reduxjs/toolkit";
import cartReducer from "./cartSlice.js";
import authReducer from "./authSlice.js";

export const store = configureStore({ reducer: { cart: cartReducer, auth: authReducer } });
```

APP.JSX: SAMPLE SOLUTION (REDUX TOOLKIT)

JavaScript

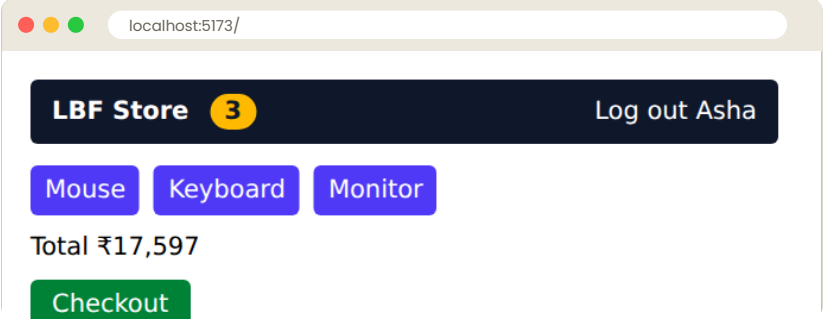
```
import { Provider, useSelector, useDispatch } from "react-redux";
import { store } from "./store.js";
import { itemAdded, cleared, selectCount, selectTotal } from "./cartSlice.js";
import { loggedIn, loggedOut, selectUser } from "./authSlice.js";

const products = [{ id: 1, name: "Mouse", price: 799 }, { id: 2, name: "Keyboard", price: 3499 }, {
id: 3, name: "Monitor", price: 15999 }];

function Header() {
  const user = useSelector(selectUser);
  const count = useSelector(selectCount);
  const dispatch = useDispatch();
  return (
    <header className="flex items-center gap-3 rounded bg-slate-900 px-3 py-2 text-sm text-white">
      <b>LBF Store</b><span className="rounded-full bg-amber-400 px-2 font-bold text-slate-900">
{count}</span>
      <span className="ml-auto">{user ? <button onClick={() => dispatch(loggedOut())}>Log out
{user.name}</button>
                                  : <button onClick={() => dispatch(loggedIn("Asha"))}>Log
in</button>}</span>
    </header>
  );
}

function Shop() {
  const dispatch = useDispatch();
  const total = useSelector(selectTotal);
  const user = useSelector(selectUser);
  return (
    <div className="space-y-2 text-sm">
      {products.map((p) => <button key={p.id} onClick={() => dispatch(itemAdded(p))} className="mr-2
rounded bg-indigo-600 px-2 py-1 text-white">{p.name}</button>)}
      <p>Total ₹{total.toLocaleString("en-IN")}</p>
      <button disabled={!user || total === 0} onClick={() => dispatch(cleared())} className="rounded
bg-green-700 px-3 py-1 text-white disabled:opacity-40">
        {user ? "Checkout" : "Log in to checkout"}
      </button>
    </div>
  );
}

export default function App() {
  return <Provider store={store}><div className="w-[420px] space-y-3"><Header /><Shop /></div>
</Provider>;
}
```



REDUX TOOLKIT VERSION: LOGGED IN AS ASHA AND ADDED A MONITOR AND TWO MICE

CARTSTORE.JS JavaScript

```
import { create } from "zustand";

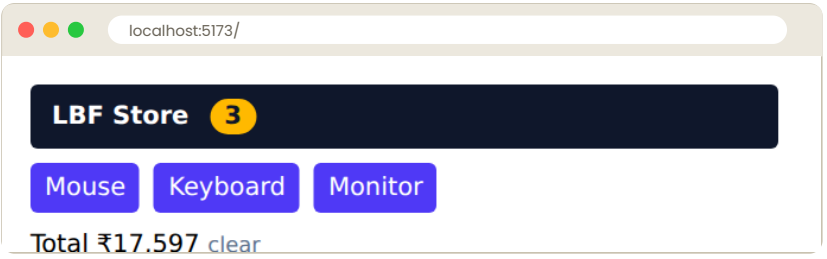
export const useCart = create((set) => ({
  items: [],
  add: (p) => set((s) => {
    const found = s.items.find((i) => i.id === p.id);
    return { items: found ? s.items.map((i) => (i.id === p.id ? { ...i, qty: i.qty + 1 } : i)) :
[...s.items, { ...p, qty: 1 }] };
  }),
  clear: () => set({ items: [] }),
}));
export const selectCount = (s) => s.items.reduce((n, i) => n + i.qty, 0);
export const selectTotal = (s) => s.items.reduce((t, i) => t + i.qty * i.price, 0);
```

APP.JSX: CART WITH ZUSTAND JavaScript

```
import { useCart, selectCount, selectTotal } from "./cartStore.js";

const products = [{ id: 1, name: "Mouse", price: 799 }, { id: 2, name: "Keyboard", price: 3499 }, {
id: 3, name: "Monitor", price: 15999 }];

export default function App() {
  const count = useCart(selectCount);
  const total = useCart(selectTotal);
  const add = useCart((s) => s.add);
  const clear = useCart((s) => s.clear);
  return (
    <div className="w-[420px] space-y-2 text-sm">
      <header className="flex gap-3 rounded bg-slate-900 px-3 py-2 text-white"><b>LBF Store</b><span
className="rounded-full bg-amber-400 px-2 font-bold text-slate-900">{count}</span></header>
      {products.map((p) => <button key={p.id} onClick={() => add(p)} className="mr-2 rounded bg-
indigo-600 px-2 py-1 text-white">{p.name}</button>)}
      <p>Total ₹{total.toLocaleString("en-IN")} <button onClick={clear} className="text-xs text-slate-
500">clear</button></p>
    </div>
  );
}
```



ZUSTAND VERSION OF THE CART: SAME BEHAVIOUR

Cart implementation	Files	Non-empty lines of state code	Provider
Redux Toolkit (cartSlice.js + store.js)	2	26	yes
Zustand (cartStore.js)	1	11	no

The comparison (line counts measured on the listings above) shows the usual trade-off: Zustand needs less code and no provider, while Redux Toolkit gives named actions, a strict structure and the best debugging tools, which pay off as the app and the team grow.

Keep server state in TanStack Query and URL state in the URL; use a library only for shared client state that changes often or has complex rules. Redux Toolkit: one store (`configureStore`), slices with case reducers and generated actions (`createSlice` , Immer-style updates), `useSelector` with small selectors and `useDispatch` , `createAsyncThunk` for async work, RTK Query for fetching and the DevTools for time travel. Zustand: one `create` call returns a hook with state and actions, selectors (and `useShallow`) prevent re-renders, `persist` saves to storage. Jotai offers atoms, MobX observables.

MODULE 17 · ADVANCED

Modern React: Actions, Suspense and concurrent features

Minimum duration **3 days** | Level **Advanced**

React 18 and 19 changed how React renders and how apps handle loading states and forms. Concurrent rendering lets urgent updates such as typing interrupt slow ones; Suspense declares loading states in one place; Actions turn async functions into transitions with built-in pending, error and optimistic states. This module covers useTransition, startTransition and useDeferredValue; Suspense boundaries for code and data and the use API; Actions with the form action prop, useActionState, useFormStatus and useOptimistic; and other React 19 features: ref as a prop with cleanup functions, document metadata in components, stylesheet and script support, Activity and improved error reporting.

IN THIS MODULE

17.1 Concurrent rendering
17.2 Suspense
17.3 Actions (React 19)
17.4 Other React 19 features
17.5 Keeping up with React

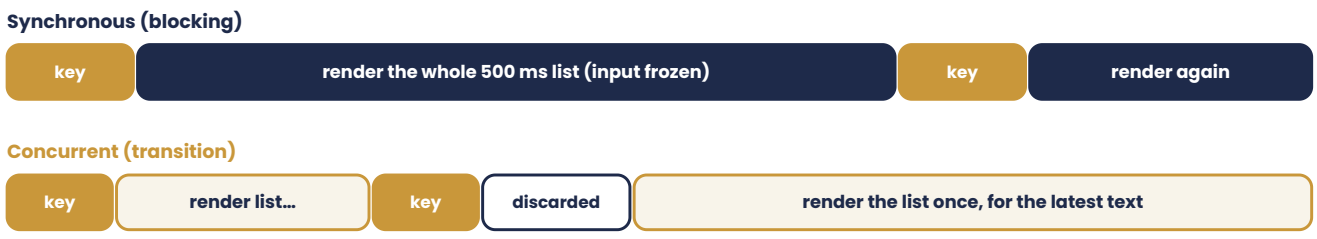
HOW THE EXAMPLES WERE RUN

Every example in this module uses only React 19.2 itself, so all of them ran unchanged in Chromium. Slow components are simulated with a busy loop so that the effect of each feature is visible in a screenshot.

17.1 Concurrent rendering

17.1.1 What concurrent rendering means

Before React 18, rendering was **synchronous**: once React started rendering an update it could not stop until it was done. If rendering a big list took 500 ms, the page froze for 500 ms and keystrokes waited. With **concurrent rendering** React can prepare a new UI in the background, **pause** it to handle something more urgent (a keystroke or click), and then continue or throw the unfinished work away. You opt in per update by marking it as a **transition**, which means "not urgent": typing into an input stays urgent, while re-rendering the results can wait.



Green: urgent input handled at once. Cream: interruptible background rendering.

TRANSITIONS LET URGENT UPDATES INTERRUPT SLOW RENDERING

17.1.2 useTransition and startTransition

`const [isPending, startTransition] = useTransition()`. Wrap a state update in `startTransition(() => setTab("reports"))` to mark it as non-urgent. While React renders it in the background the **old UI stays on screen and stays interactive**, and `isPending` is `true`, so you can show a subtle indicator instead of freezing. If another update arrives, the unfinished transition is abandoned. Outside components (or when you do not need `isPending`) use the standalone `startTransition` import.

APP.JSX

JavaScript

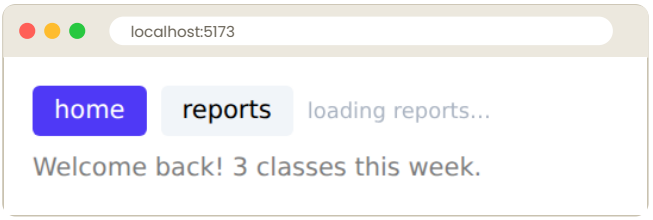
```
import { useState, useTransition } from "react";

function SlowRow({ n }) {
  const start = performance.now();
  while (performance.now() - start < 4) {}              // pretend each row is expensive (4 ms)
  return <li>Report row {n}</li>;
}

function SlowReports() {
  const rows = [];
  for (let i = 1; i <= 1000; i++) rows.push(<SlowRow key={i} n={i} />);   // 1000 x 4 ms = 4 s
  return <ul className="h-16 overflow-hidden text-xs">{rows}</ul>;
}

export default function App() {
  const [tab, setTab] = useState("home");
  const [isPending, startTransition] = useTransition();

  function selectTab(next) {
    startTransition(() => setTab(next));                // non-urgent: keep the old tab visible
  }
  return (
    <div className="w-80 space-y-2 text-sm">
      <nav className="flex items-center gap-2">
        {["home", "reports"].map((t) => (
          <button key={t} onClick={() => selectTab(t)}
                  className={`rounded px-3 py-1 ${tab === t ? "bg-indigo-600 text-white" : "bg-slate-
100"}`}>{t}</button>
        ))}
        {isPending && <span className="animate-pulse text-xs text-slate-500">loading reports…</span>}
      </nav>
      <div className={isPending ? "opacity-50" : ""}>
        {tab === "home" ? <p>Welcome back! 3 classes this week.</p> : <SlowReports />}
      </div>
    </div>
  );
}
```



JUST AFTER CLICKING “REPORTS”: THE HOME TAB STAYS VISIBLE (DIMMED) AND ISPENDING SHOWS AN INDICATOR WHILE 4 S OF RENDERING HAPPENS IN THE BACKGROUND

Without the transition the click would simply freeze the page for the whole render: no indicator, no response to further clicks, and the old tab would not even be dimmed, because the browser cannot paint until React finishes.

> **TIP**
> A transition does not make slow rendering faster; it keeps the app **responsive** while the slow work happens. To make rendering itself faster, see Module 18 (memoisation, virtualisation).

17.1.3 useDeferredValue

`useDeferredValue(value)` returns a copy of a value that "lags behind" during urgent updates. Typical use: the input updates the text immediately (urgent), while a slow results list renders with the **deferred** text in the background. Compare `text !== deferredText` to know that the results are stale and dim them. Use it when you receive a value (for example as a prop) and cannot wrap its update in `startTransition` yourself.

APP.JSX — JavaScript

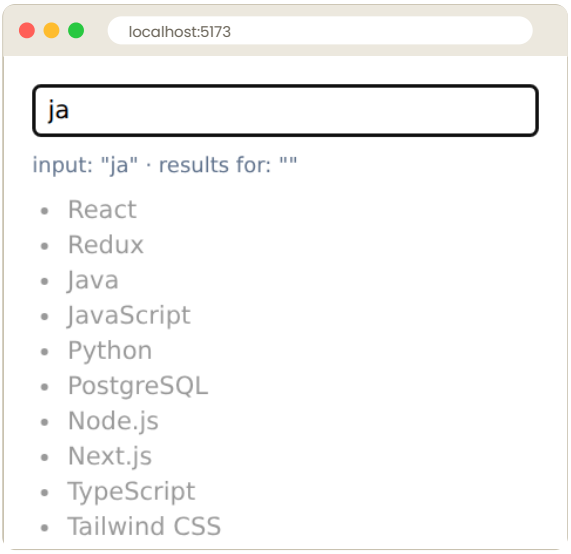
```
import { useDeferredValue, useState, memo } from "react";

const courses = ["React", "Redux", "Java", "JavaScript", "Python", "PostgreSQL", "Node.js", "Next.js",
"TypeScript", "Tailwind CSS"];

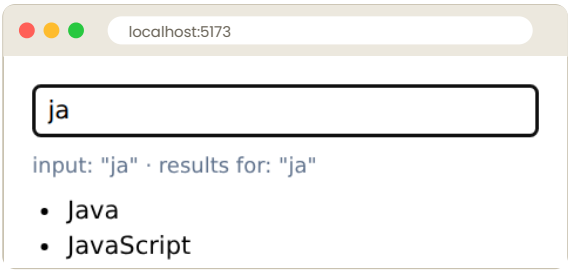
function SlowPart() {                                  // each part takes 5 ms to render
  const start = performance.now();
  while (performance.now() - start < 5) {}
  return null;
}

const SlowResults = memo(function SlowResults({ query }) {
  const found = courses.filter((c) => c.toLowerCase().includes(query.toLowerCase()));
  return (
    <ul className="list-disc pl-5">
      {found.map((c) => <li key={c}>{c}</li>)}
      {Array.from({ length: 300 }, (_, i) => <SlowPart key={i} />)}   {/* 300 x 5 ms = 1.5 s */}
    </ul>
  );
});

export default function App() {
  const [text, setText] = useState("");
  const deferredText = useDeferredValue(text);          // lags behind while typing
  const isStale = text !== deferredText;
  return (
    <div className="w-72 space-y-2 text-sm">
      <input value={text} onChange={(e) => setText(e.target.value)} placeholder="Search courses"
className="w-full rounded border px-2 py-1" />
      <p className="text-xs text-slate-500">input: "{text}" · results for: "{deferredText}"</p>
      <div className={isStale ? "opacity-40" : ""}><SlowResults query={deferredText} /></div>
    </div>
  );
}
```



JUST AFTER TYPING "JA": THE INPUT ALREADY SHOWS IT, WHILE THE LIST (DIMMED) STILL SHOWS THE OLD RESULTS AND RE-RENDERS IN THE BACKGROUND

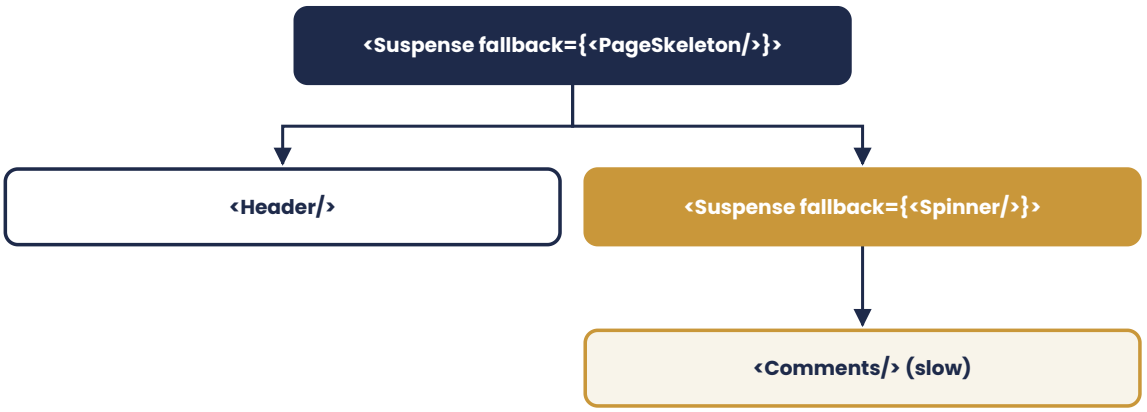


ABOUT TWO SECONDS LATER: THE DEFERRED VALUE CAUGHT UP AND THE RESULTS MATCH

17.2 Suspense

17.2.1 Suspense boundaries and fallbacks

`<Suspense fallback={<Spinner />}>` declares a **loading boundary**. When any component inside it **suspends** (its code or data is not ready yet), React shows the fallback in place of the whole boundary, then reveals the content when everything inside is ready. Boundaries can be nested: an outer one for the page shell and inner ones for slower sections, so the page appears **progressively** instead of waiting for the slowest part. Place boundaries where a loading state makes sense to the user.



NESTED BOUNDARIES: THE SLOW COMMENTS SECTION ONLY REPLACES ITSELF WITH A SPINNER

17.2.2 Suspense for code splitting with lazy

`lazy(() => import("./Chart.jsx"))` suspends the first time the component renders, while its code downloads (Module 13 used it for routes). The nested boundaries below show the page shell immediately, the fast widget after 0.5 s and the heavy chart after 2 s, each with its own fallback.

STATS.JSX JavaScript

```
export default function Stats() { return <p className="rounded bg-green-50 p-2">Enrolments: 128 · Attendance: 94%</p>; }
```

CHART.JSX JavaScript

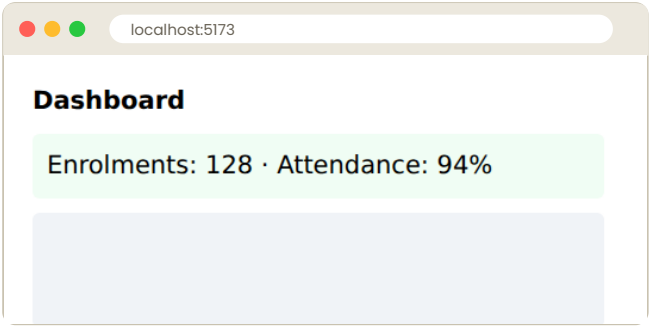
```
export default function Chart() { return <div className="flex h-16 items-end gap-1">{[40, 65, 50, 80, 72, 95].map((h, i) => <div key={i} className="w-6 bg-indigo-500" style={{ height: `${h}%` }} />)}</div>; }
```

APP.JSX JavaScript

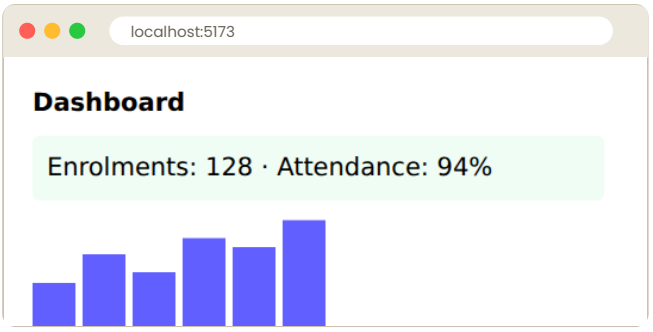
```
import { lazy, Suspense } from "react";

const delay = (ms) => (mod) => new Promise((r) => setTimeout(() => r(mod), ms));
const Stats = lazy(() => import("./Stats.jsx").then(delay(500)));    // small module
const Chart = lazy(() => import("./Chart.jsx").then(delay(2000)));   // heavy module

export default function App() {
  return (
    <div className="w-80 space-y-2 text-sm">
      <h2 className="font-semibold">Dashboard</h2>                        {/* shown at once */}
      <Suspense fallback={<p className="text-slate-400">Loading stats…</p>}>
        <Stats />
      </Suspense>
      <Suspense fallback={<div className="h-16 animate-pulse rounded bg-slate-200" />}>
        <Chart />
      </Suspense>
    </div>
  );
}
```



AT ABOUT 1 S: THE SHELL AND THE STATS ARE VISIBLE; THE CHART STILL SHOWS ITS SKELETON FALLBACK



AT ABOUT 3 S: THE CHART ARRIVED AND REPLACED ITS OWN FALLBACK ONLY

17.2.3 Suspense for data fetching with supporting libraries

Suspense can also wait for **data**, but only if the data source is Suspense-enabled: it must cache the request and **throw** (suspend) until the data is ready. Do not try to build this with `useEffect`. Use a library or framework that supports it: TanStack Query's `useSuspenseQuery`, Relay, SWR with `suspense: true`, React Router loaders, and Next.js Server Components (Module 21). With them, components simply read data as if it were already there, and the nearest boundary handles loading.

USESUSPENSEQUERY (REPRESENTATIVE) JavaScript

```
import { useSuspenseQuery } from "@tanstack/react-query";

function StudentList() {
  const { data } = useSuspenseQuery({ queryKey: ["students"], queryFn: getStudents });
  return <ul>{data.map((s) => <li key={s.id}>{s.name}</li>)}</ul>;   // data is never undefined
}

<ErrorBoundary fallback={<p>Could not load</p>}>
  <Suspense fallback={<Spinner />}>
    <StudentList />
  </Suspense>
</ErrorBoundary>
```

17.2.4 The use API for reading promises and context

React 19's `use(resource)` reads the value of a **promise** or a **context**. With a promise, the component suspends until it resolves (the nearest Suspense shows its fallback) and an error is thrown to the nearest error boundary if it rejects. Unlike hooks, `use` may be called inside conditions and loops. One rule matters: the promise must be **created outside the component** (in a parent, a loader, a cache or a Server Component) and passed down; a promise created during render would be new on every render and suspend forever.

APP.JSX

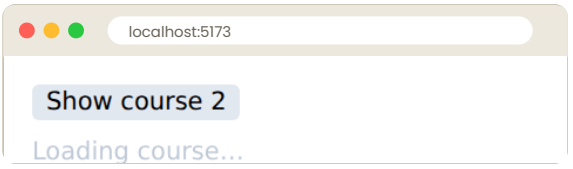
JavaScript

```
import { use, Suspense, useState } from "react";

function fetchCourse(id) {                                  // created by the parent, not during
render
  return new Promise((resolve) =>
    setTimeout(() => resolve({ id, title: id === 1 ? "React" : "Java Full Stack", weeks: id === 1 ? 10
: 16 }), 1500));
}

function CourseDetails({ coursePromise }) {
  const course = use(coursePromise);                        // suspends until resolved
  return <p><b>{course.title}</b>: {course.weeks} weeks</p>;
}

export default function App() {
  const [promise, setPromise] = useState(() => fetchCourse(1));
  return (
    <div className="w-72 space-y-2 text-sm">
      <button className="rounded bg-slate-200 px-2" onClick={() => setPromise(fetchCourse(2))}>Show
course 2</button>
      <Suspense fallback={<p className="animate-pulse text-slate-400">Loading course…</p>}>
        <CourseDetails coursePromise={promise} />
      </Suspense>
    </div>
  );
}
```



WHILE THE PROMISE IS PENDING, THE BOUNDARY SHOWS ITS FALLBACK



AFTER IT RESOLVES, USE() RETURNS THE VALUE AND COURSEDETAILS RENDERS

17.3 Actions (React 19)

17.3.1 Async transitions as Actions

In React 19 the function passed to `startTransition` may be **async**. Such a function is called an **Action**. While it runs, `isPending` stays `true` (no hand-written `loading` state), errors can be caught in the function or by an error boundary, and state updates inside it are applied together when it finishes. Actions are the foundation of the form features in the next sections.

APP.JSX

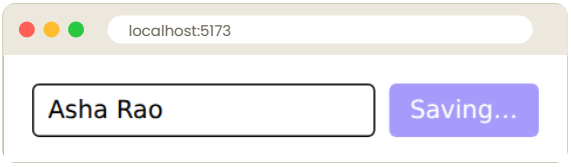
JavaScript

```
import { useState, useTransition } from "react";

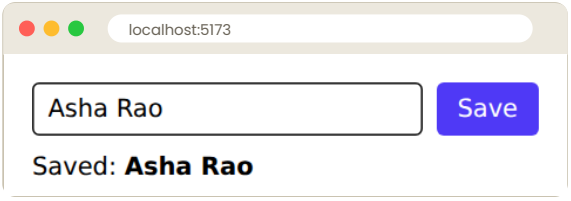
const saveName = (name) => new Promise((resolve, reject) =>
  setTimeout(() => (name.length < 3 ? reject(new Error("Name too short")) : resolve(name)), 1500));

export default function App() {
  const [name, setName] = useState("");
  const [saved, setSaved] = useState("");
  const [error, setError] = useState(null);
  const [isPending, startTransition] = useTransition();

  function handleSave() {
    startTransition(async () => {                    // an Action
      try {
        const result = await saveName(name);
        startTransition(() => { setSaved(result); setError(null); });
      } catch (e) {
        setError(e.message);
      }
    });
  }
  return (
    <div className="w-72 space-y-2 text-sm">
      <div className="flex gap-2">
        <input value={name} onChange={(e) => setName(e.target.value)} className="flex-1 rounded border
px-2 py-1" />
        <button onClick={handleSave} disabled={isPending} className="rounded bg-indigo-600 px-3 text-
white disabled:opacity-50">
          {isPending ? "Saving…" : "Save"}
        </button>
      </div>
      {error && <p className="text-red-600">{error}</p>}
      {saved && <p>Saved: <b>{saved}</b></p>}
    </div>
  );
}
```



WHILE THE ASYNC ACTION RUNS, ISPENDING DISABLES THE BUTTON: NO LOADING STATE WAS DECLARED



WHEN IT FINISHES, THE SAVED VALUE APPEARS AND ISPENDING RETURNS TO FALSE

Inside an async Action, state updates that happen **after** an `await` must be wrapped in another `startTransition` (as above) to stay part of the transition. This is a current limitation that the React team plans to remove; the hooks below handle it for you.

17.3.2 Form action prop

In React 19 a `<form>` accepts a **function** as its `action` prop. On submit React prevents the page reload, calls the function with the form's `FormData` inside a transition, and, for uncontrolled forms, **resets** the fields when it succeeds. No `onSubmit`, no `preventDefault`, no state per field: read the values by their `name` with `formData.get("name")`. A button can also override it with `formAction`.

APP.JSX

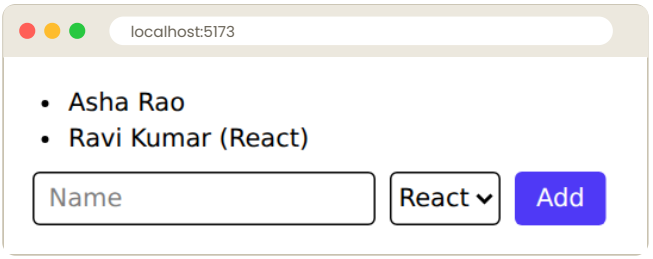
JavaScript

```
import { useState } from "react";

export default function App() {
  const [list, setList] = useState(["Asha Rao"]);

  async function addStudent(formData) {                    // receives FormData
    await new Promise((r) => setTimeout(r, 300));          // e.g. POST to the server
    setList((l) => [...l, `${formData.get("name")} (${formData.get("course")})`]);
  }

  return (
    <div className="w-80 space-y-2 text-sm">
      <ul className="list-disc pl-5">{list.map((s) => <li key={s}>{s}</li>)}</ul>
      <form action={addStudent} className="flex gap-2">
        <input name="name" required placeholder="Name" className="flex-1 rounded border px-2 py-1" />
        <select name="course" className="rounded border"><option>React</option><option>Java</option>
</select>
        <button className="rounded bg-indigo-600 px-3 text-white">Add</button>
      </form>
    </div>
  );
}
```



SUBMITTED “RAVI KUMAR”: THE ACTION READ THE FORMDATA, THE LIST UPDATED, AND REACT RESET THE FORM

17.3.3 useActionState

`const [state, formAction, isPending] = useActionState(action, initialState)` connects a form Action to a piece of **state**: whatever the action returns becomes the new state. The action receives the **previous state** and the `FormData`. It is ideal for forms that show a result or validation errors from the server, and `isPending` comes for free. (It replaces the short-lived `useFormState` from React DOM canaries.)

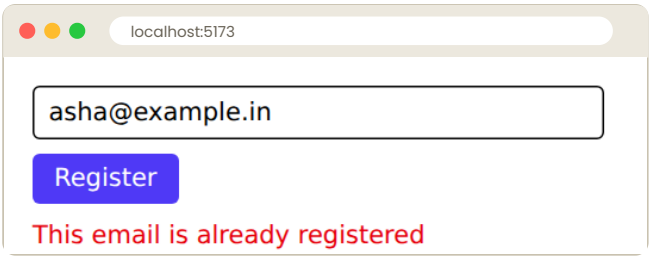
APP.JSX

JavaScript

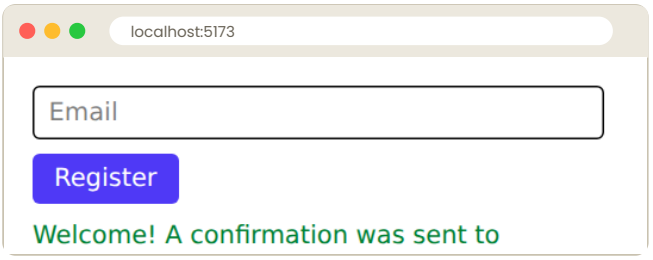
```
import { useActionState } from "react";

async function register(prevState, formData) {
  await new Promise((r) => setTimeout(r, 1200));          // pretend server round trip
  const email = formData.get("email");
  if (!email.includes("@")) return { error: "Enter a valid email", email };
  if (email === "asha@example.in") return { error: "This email is already registered", email };
  return { success: `Welcome! A confirmation was sent to ${email}.` };
}

export default function App() {
  const [state, formAction, isPending] = useActionState(register, { error: null });
  return (
    <form action={formAction} className="w-80 space-y-2 text-sm">
      <input name="email" defaultValue={state.email} placeholder="Email" className="w-full rounded
border px-2 py-1" />
      <button disabled={isPending} className="rounded bg-indigo-600 px-3 py-1 text-white
disabled:opacity-50">
        {isPending ? "Registering…" : "Register"}
      </button>
      {state.error && <p className="text-red-600">{state.error}</p>}
      {state.success && <p className="text-green-700">{state.success}</p>}
    </form>
  );
}
```



AN EXISTING EMAIL: THE RETURNED OBJECT BECAME THE NEW STATE AND THE ERROR IS SHOWN (THE VALUE WAS KEPT THROUGH DEFAULTVALUE)



A NEW EMAIL: THE SUCCESS STATE REPLACED THE ERROR

17.3.4 useFormStatus

`useFormStatus()` (from `react-dom`) lets a component **inside** a form read the status of the form's last submission: `pending`, plus the `data`, `method` and `action`. It is designed for reusable buttons and inputs that disable themselves while their form submits, without prop drilling. It only works in a component rendered **inside** the `<form>`, not in the component that renders the form.

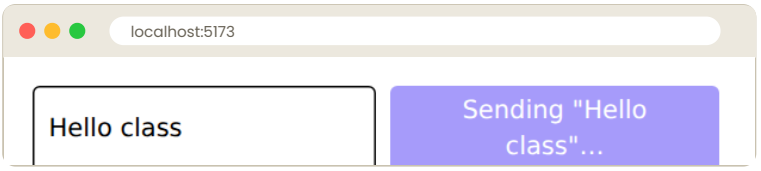
APP.JSX

JavaScript

```
import { useFormStatus } from "react-dom";

function SubmitButton({ children }) {
  const { pending, data } = useFormStatus();                // status of the parent form
  return (
    <button disabled={pending} className="rounded bg-indigo-600 px-3 py-1 text-white disabled:opacity-
50">
      {pending ? `Sending "${data.get("message")}"…` : children}
    </button>
  );
}

export default function App() {
  async function send(formData) { await new Promise((r) => setTimeout(r, 2000)); }
  return (
    <form action={send} className="flex w-96 gap-2 text-sm">
      <input name="message" className="flex-1 rounded border px-2 py-1" placeholder="Message" />
      <SubmitButton>Send</SubmitButton>                        {/* reusable, no props needed */}
    </form>
  );
}
```



DURING SUBMISSION THE REUSABLE SUBMITBUTTON DISABLES ITSELF AND EVEN READS THE SUBMITTED DATA

17.3.5 useOptimistic

`const [optimisticState, addOptimistic] = useOptimistic(state, updateFn)` shows a temporary, **optimistic** version of state while an Action is running. Call `addOptimistic(value)` inside the Action: the UI updates instantly through `updateFn(currentState, value)`. When the Action finishes, React drops the optimistic value and shows the real state again: the confirmed data on success, or the unchanged old data if it failed, which is an automatic rollback.

APP.JSX

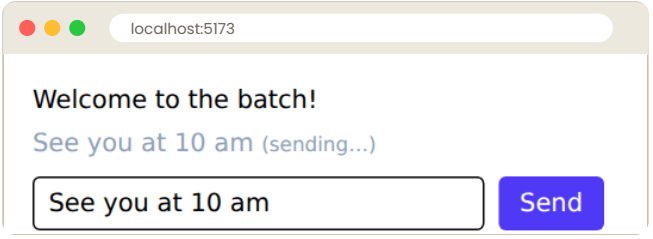
JavaScript

```
import { useOptimistic, useState, useRef } from "react";

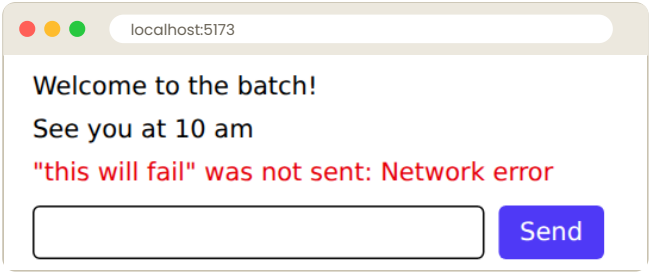
async function deliver(text) {
  await new Promise((r) => setTimeout(r, 2000));
  if (text.includes("fail")) throw new Error("Network error");
  return text;
}

export default function App() {
  const [messages, setMessages] = useState([{ text: "Welcome to the batch!" }]);
  const [optimistic, addOptimistic] = useOptimistic(messages, (state, text) => [...state, { text,
sending: true }]);
  const [error, setError] = useState("");

  async function send(formData) {
    const text = formData.get("message");
    addOptimistic(text);                                    // shown at once
    try {
      const sent = await deliver(text);
      setMessages((m) => [...m, { text: sent }]);           // the real state
      setError("");
    } catch (e) {
      setError(`"${text}" was not sent: ${e.message}`);      // optimistic item disappears
    }
  }
  return (
    <div className="w-80 space-y-1 text-sm">
      {optimistic.map((m, i) => <p key={i} className={m.sending ? "text-slate-400" : ""}>{m.text}
{m.sending && <small>(sending…)</small>}</p>)}
      {error && <p className="text-red-600">{error}</p>}
      <form action={send} className="flex gap-2 pt-1">
        <input name="message" className="flex-1 rounded border px-2 py-1" />
        <button className="rounded bg-indigo-600 px-3 text-white">Send</button>
      </form>
    </div>
  );
}
```



RIGHT AFTER SENDING: THE MESSAGE APPEARS IMMEDIATELY, MARKED AS SENDING



TWO SECONDS LATER IT IS CONFIRMED; A SECOND MESSAGE CONTAINING “FAIL” APPEARED OPTIMISTICALLY, THEN WAS ROLLED BACK WITH AN ERROR

17.4 Other React 19 features

17.4.1 ref as a prop and ref cleanup functions

In React 19 function components receive `ref` as an ordinary prop, so `forwardRef` is no longer needed (it still works, and will be deprecated). A **ref callback** may now return a **cleanup function**, which React calls when the element is removed, in the same way as an effect cleanup. That is the natural place to remove listeners or observers attached to the element.

APP.JSX

JavaScript

```
import { useRef, useState } from "react";

function TextInput({ ref, label, ...props }) {          // ref is just a prop now
  return <label className="block">{label} <input ref={ref} {...props} className="rounded border px-2
py-1" /></label>;
}

export default function App() {
  const inputRef = useRef(null);
  const [show, setShow] = useState(true);
  return (
    <div className="space-y-2 text-sm">
      <TextInput ref={inputRef} label="Name" />
      <button onClick={() => inputRef.current.focus()} className="mr-2 rounded bg-slate-200 px-
2">Focus</button>
      <button onClick={() => setShow((s) => !s)} className="rounded bg-slate-200 px-2">Toggle
box</button>
      {show && (
        <div className="h-8 w-40 bg-indigo-100" ref={(node) => {
          const observer = new ResizeObserver(() => console.log("box size:", node.offsetWidth, "x",
node.offsetHeight));
          observer.observe(node);
          console.log("ref attached");
          return () => { observer.disconnect(); console.log("ref cleanup: observer disconnected"); };
// React 19
        }} />
      )}
    </div>
  );
}
```

BROWSER CONSOLE: MOUNT, THEN TOGGLE THE BOX OFF

```
ref attached
box size: 160 x 32
ref cleanup: observer disconnected
```

17.4.2 Rendering <title> and <meta> in components

React 19 lets any component render `<title>`, `<meta>` and `<link>` tags. Wherever they appear in the tree, React **hoists** them into the document `<head>` and removes them when the component unmounts. Each page can therefore describe its own title and description without a library such as react-helmet.

APP.JSX

JavaScript

```
export default function App() {
  const course = { title: "React", summary: "Build modern user interfaces with React 19." };
  return (
    <article className="text-sm">
      <title>{`${course.title} course | Learn Being Forward`}</title>
      <meta name="description" content={course.summary} />
      <link rel="canonical" href="https://learnbeingforward.in/courses/react" />
      <h1 className="text-lg font-semibold">{course.title}</h1>
      <p>{course.summary}</p>
    </article>
  );
}
```

BROWSER CONSOLE: WHAT ENDED UP IN <HEAD>

```
document.title = React course | Learn Being Forward
in <head>: <meta name="description" content="Build modern user interfaces with React 19.">
in <head>: <link rel="canonical" href="https://learnbeingforward.in/courses/react">
```

The logging lines were appended to the listing only to inspect the document.

17.4.3 Stylesheet and script loading support

Components can also render `<link rel="stylesheet" href="..." precedence="default" />`. With a `precedence`, React inserts the stylesheet into the head in the right order, loads each URL only once even if many components render it, and (with Suspense) waits for it to load before revealing the content, so there is no flash of unstyled content. Similarly `<script async src="...">` rendered in a component is de-duplicated and placed in the head. New functions in `react-dom` such as `preload`, `preinit` and `prefetchDNS` tell the browser about resources early.

STYLESHEETS, SCRIPTS AND RESOURCE HINTS IN A COMPONENT

JavaScript

```
import { preload, preinit } from "react-dom";

function MapWidget() {
  preinit("https://maps.example.com/sdk.js", { as: "script" });   // start loading and run it early
  preload("/fonts/inter.woff2", { as: "font" });                   // just download it early
  return (
    <>
      <link rel="stylesheet" href="/map-widget.css" precedence="default" />  {/* loaded once, ordered
*/}
      <script async src="https://maps.example.com/sdk.js" />                {/* de-duplicated */}
      <div id="map" />
    </>
  );
}
```

17.4.4 <Activity> for hiding and preserving UI (overview)

`<Activity mode="visible" | "hidden">` (React 19.2) hides part of the UI **without unmounting it**. Hidden content keeps its state and DOM (typed text, scroll position), its effects are cleaned up while hidden and re-created when shown again, and React can even pre-render hidden content at low priority. Before Activity you had to choose between unmounting (losing state) and CSS hiding (effects keep running).

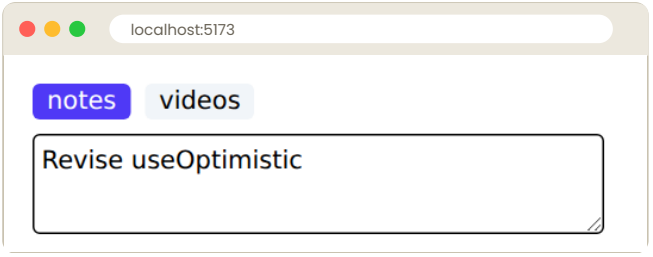
APP.JSX

JavaScript

```
import { Activity, useState } from "react";

function Notes() {
  return <textarea placeholder="Your notes…" className="h-14 w-full rounded border p-1" />;   // state
lives in the DOM/component
}

export default function App() {
  const [tab, setTab] = useState("notes");
  return (
    <div className="w-80 space-y-2 text-sm">
      <div className="flex gap-2">
        {["notes", "videos"].map((t) => <button key={t} onClick={() => setTab(t)} className={`rounded
px-2 ${tab === t ? "bg-indigo-600 text-white" : "bg-slate-100"}`}>{t}</button>)}
      </div>
      <Activity mode={tab === "notes" ? "visible" : "hidden"}><Notes /></Activity>
      <Activity mode={tab === "videos" ? "visible" : "hidden"}><p>Lesson videos…</p></Activity>
    </div>
  );
}
```



TYPED A NOTE, SWITCHED TO “VIDEOS” AND BACK: THE NOTE WAS PRESERVED BECAUSE NOTES WAS HIDDEN, NOT UNMOUNTED

17.4.5 Improved error reporting

React 19 reports errors more clearly. An error thrown during rendering and caught by an error boundary is logged **once** (React 18 logged it twice, plus a third message), uncaught errors go to `window.reportError`, and hydration mismatches show a single diff of what differed. Three new root options let you send errors to a monitoring service such as Sentry: `onCaughtError`, `onUncaughtError` and `onRecoverableError`.

ROOT ERROR CALLBACKS (MAIN.JSX)

JavaScript

```
createRoot(document.getElementById("root"), {
  onCaughtError: (error, info) => logToService("caught", error, info.componentStack),     // inside an
error boundary
  onUncaughtError: (error, info) => logToService("uncaught", error, info.componentStack), // no
boundary: app unmounts
  onRecoverableError: (error) => logToService("recovered", error),                        // React
recovered automatically
}).render(<App />);
```

17.5 Keeping up with React

17.5.1 Following react.dev and release notes

React evolves steadily, and new features usually appear first in the **canary** channel before a stable release. Good sources:

- **react.dev**: the official documentation, with a reference page for every API, plus the **blog** (react.dev/blog) for release posts and upgrade guides;
- **GitHub releases and CHANGELOG.md** of facebook/react for the exact list of changes in each version;
- **React Conf** talks (yearly) where major features such as the compiler and Actions are presented;
- **codemods**: `npx codemod react/19/migration-recipe` automates many upgrade steps; run the codemods, then fix the warnings that remain;
- release notes of the tools you use with React (Vite, Next.js, React Router, TanStack Query), since they adopt new React features quickly.

Version	Year	Headline features
16.8	2019	Hooks
18	2022	concurrent rendering, automatic batching, transitions, Suspense on the server
19	2024	Actions, useActionState, useFormStatus, useOptimistic, use, ref as a prop, document metadata, Server Components stable
19.1 / 19.2	2025	Owner stacks in development, Activity, useEffectEvent, performance tracks in Chrome DevTools

PRACTICE CHECKPOINT

Rebuild the student form with useActionState, useFormStatus and useOptimistic, and add a search list that stays responsive using useTransition.

APP.JSX: SAMPLE SOLUTION JavaScript

```
import { useActionState, useOptimistic, useState, useTransition, memo } from "react";
import { useFormStatus } from "react-dom";

// ----- fake server
const wait = (ms) => new Promise((r) => setTimeout(r, ms));
async function saveStudent(student) {
  await wait(1500);
  if (student.email.endsWith("@test.in")) throw new Error("Test addresses are not allowed");
  return student;
}

// ----- reusable submit button
function SubmitButton() {
  const { pending } = useFormStatus();
  return <button disabled={pending} className="rounded bg-indigo-600 px-3 py-1 text-white
disabled:opacity-50">{pending ? "Saving…" : "Add"}</button>;
}

// ----- slow search results
const SearchResults = memo(function SearchResults({ students, query }) {
  const start = performance.now();
  while (performance.now() - start < 300) {}              // simulate a heavy list
  const list = students.filter((s) => s.name.toLowerCase().includes(query.toLowerCase()));
  return <ul className="divide-y">{list.map((s) => <li key={s.email} className={`py-0.5 ${s.pending ?
"text-slate-400" : ""}`}>{s.name} · {s.email}{s.pending && " (saving…)"}</li>)}</ul>;
});

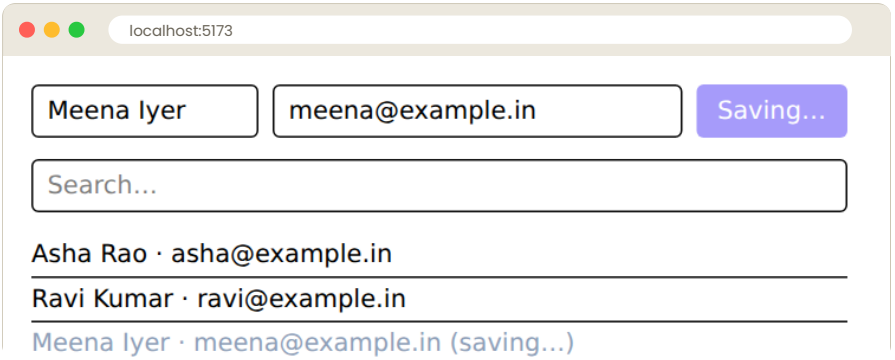
export default function App() {
  const [students, setStudents] = useState([{ name: "Asha Rao", email: "asha@example.in" }, { name:
```

```
"Ravi Kumar", email: "ravi@example.in" }]);
  const [optimistic, addOptimistic] = useOptimistic(students, (list, s) => [...list, { ...s, pending:
true }]);

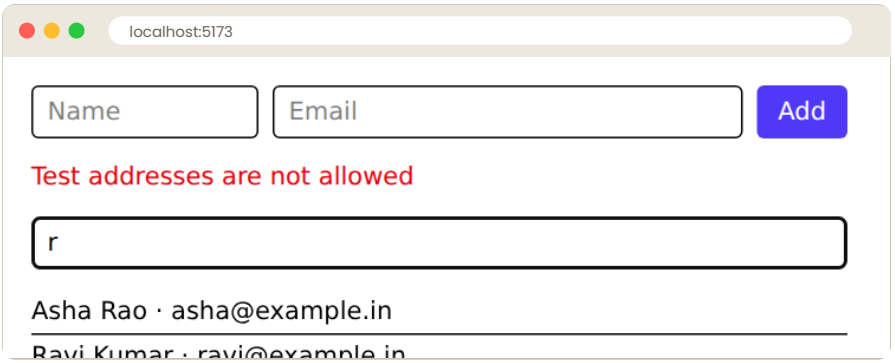
  const [formState, formAction] = useActionState(async (prev, formData) => {
    const student = { name: formData.get("name").trim(), email: formData.get("email").trim() };
    if (student.name.length < 3) return { error: "Name needs at least 3 characters" };
    addOptimistic(student);
    try {
      const saved = await saveStudent(student);
      setStudents((l) => [...l, saved]);
      return { error: null };
    } catch (e) {
      return { error: e.message };
    }
  }, { error: null });

  const [query, setQuery] = useState("");
  const [listQuery, setListQuery] = useState("");
  const [isPending, startTransition] = useTransition();

  return (
    <div className="w-[460px] space-y-3 text-sm">
      <form action={formAction} className="flex gap-2">
        <input name="name" placeholder="Name" className="w-32 rounded border px-2 py-1" />
        <input name="email" placeholder="Email" className="flex-1 rounded border px-2 py-1" />
        <SubmitButton />
      </form>
      {formState.error && <p className="text-red-600">{formState.error}</p>}
      <input value={query} placeholder="Search…" className="w-full rounded border px-2 py-1"
             onChange={(e) => { setQuery(e.target.value); startTransition(() =>
setListQuery(e.target.value)); }} />
      <div className={isPending ? "opacity-50" : ""}><SearchResults students={optimistic} query=
{listQuery} /></div>
    </div>
  );
}
```



JUST AFTER ADDING MEENA: SUBMITBUTTON SHOWS “SAVING…” AND THE OPTIMISTIC ROW IS ALREADY LISTED



AFTER SAVING, THEN A TEST ADDRESS (ROLLED BACK WITH AN ERROR) AND A SEARCH FOR “R”: THE LIST FILTERED IN A TRANSITION

Concurrent rendering lets urgent updates interrupt slow ones: mark non-urgent updates with `startTransition` (`isPending` for indicators) or lag a value with `useDeferredValue` . Suspense boundaries show fallbacks for lazy code and Suspense-enabled data; `use` reads promises (created outside render) and context. Actions are async transitions: forms take `action={fn}` with `FormData` , `useActionState` turns results into state with `isPending` , `useFormStatus` serves child buttons, and `useOptimistic` shows instant updates that roll back on failure. React 19 also brings ref as a prop with ref cleanups, `<title>` / `<meta>` hoisting, stylesheet and script support, `<Activity>` and better error reporting.

MODULE 18 · ADVANCED

Performance and React internals

Minimum duration **2 days** | Level **Advanced**

Understanding how React renders is the key to finding and fixing slow components. This module explains the virtual DOM, reconciliation and the diffing rules, the Fiber architecture and the render and commit phases; the three reasons a component re-renders; the main optimisation techniques (React.memo, useMemo and useCallback, keeping state close to where it is used, code splitting, list virtualisation, debouncing and throttling, image optimisation); the React Compiler, which memoises automatically; and how to measure with the React DevTools Profiler, the Chrome Performance panel, Lighthouse and bundle analysis.

IN THIS MODULE

18.1 How React works
18.2 Why components re-render
18.3 Optimisation techniques
18.4 React Compiler
18.5 Measuring performance

18.1 How React works

18.1.1 Virtual DOM

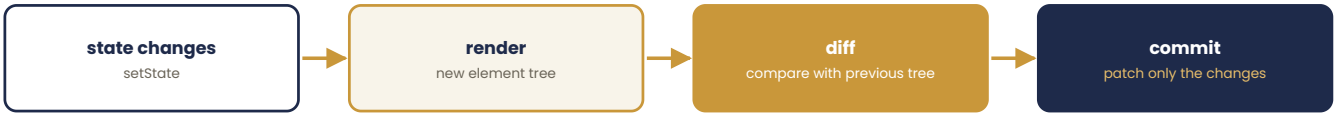
Changing the real DOM is relatively slow, and keeping it in sync by hand is error-prone. When a component renders, React does not touch the DOM: it produces a **description** of the UI, a tree of plain JavaScript objects (React elements), often called the **virtual DOM**. React then compares the new tree with the previous one and applies only the **differences** to the real DOM. You describe **what** the UI should look like for the current state; React works out **how** to update the page.

VIRTUAL-DOM.JS — main.js

```
// what JSX becomes: plain objects describing the UI
const element = {
  type: "ul",
  props: {
    className: "list",
    children: [
      { type: "li", key: "1", props: { children: "React" } },
      { type: "li", key: "2", props: { children: "Java" } },
    ],
  },
};
console.log(JSON.stringify(element, null, 1).split("\n").length, "lines of plain data, no DOM nodes");
console.log("element type:", element.type, "| first child:",
element.props.children[0].props.children);
```

OUTPUT

```
22 lines of plain data, no DOM nodes
element type: ul | first child: React
```



FROM A STATE CHANGE TO A DOM UPDATE

18.1.2 Reconciliation and the diffing algorithm

Comparing two arbitrary trees perfectly is very expensive, so React uses a fast **heuristic** based on two assumptions. (1) **Different types produce different trees**: if an element changes type (`<div>` to `<section>`, or `<Login>` to `<Dashboard>`), React destroys the old subtree, including all state inside it, and builds a new one. (2) **Keys identify list items**: among siblings, an element with the same key is treated as the same item even if it moved. If the type and the key stay the same, React keeps the component (and its state) and only updates changed props.

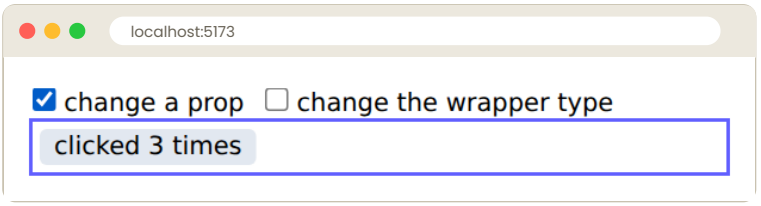
APP.JSX

JavaScript

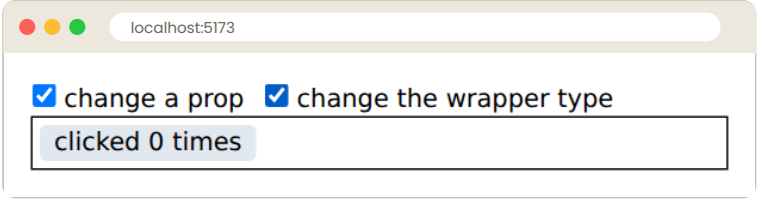
```
import { useState } from "react";

function Counter() {
  const [n, setN] = useState(0);
  return <button className="rounded bg-slate-200 px-2" onClick={() => setN(n + 1)}>clicked {n}
times</button>;
}

export default function App() {
  const [fancy, setFancy] = useState(false);
  const [boxed, setBoxed] = useState(false);
  return (
    <div className="space-y-2 text-sm">
      <label className="mr-3"><input type="checkbox" onChange={(e) => setFancy(e.target.checked)} />
change a prop</label>
      <label><input type="checkbox" onChange={(e) => setBoxed(e.target.checked)} /> change the wrapper
type</label>
      {/* section vs div: a different type, so the subtree is rebuilt */}
      {boxed
        ? <section className="p-1 ring-1"><Counter /></section>
        : <div className={fancy ? "p-1 ring-2 ring-indigo-500" : "p-1"}><Counter /></div>}
    </div>
  );
}
```



CLICKED THREE TIMES, THEN TICKED “CHANGE A PROP”: SAME TYPE, SO COUNTER KEPT ITS STATE

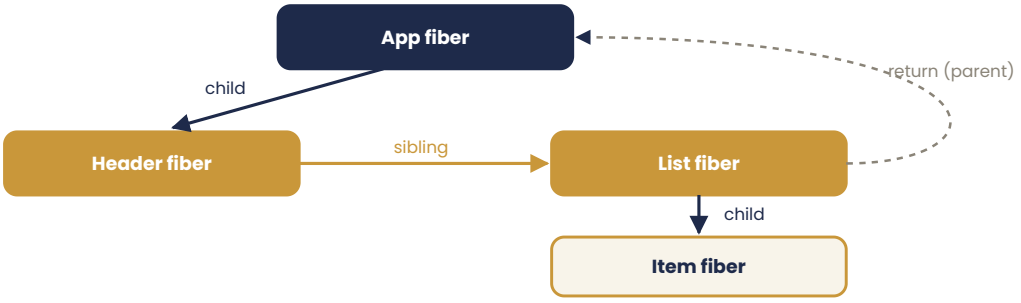


THEN TICKED “CHANGE THE WRAPPER TYPE”: DIV BECAME SECTION, SO REACT REBUILT THE SUBTREE AND THE COUNT RESET TO 0

The same rule explains why you must never **define a component inside another component**: every render creates a new function, so React sees a new type each time and throws the inner component's state away. It is also why list items need stable **keys** (Module 7): with the index as key, inserting at the top makes React match the wrong items.

18.1.3 React Fiber architecture (overview)

Fiber is the engine inside React since version 16. Every component instance and host element is represented by a **fiber**: an object that stores its type, props, state (the hooks list), and pointers to its parent, first child and next sibling. Because the work is split into many small units (one per fiber) linked in this way, React can **pause** rendering after any unit, handle something more urgent, and continue later. This is what makes concurrent features such as transitions (Module 17) possible. React keeps two trees: the **current** tree on screen and a **work-in-progress** tree being prepared; at commit they swap.



FIBERS FORM A LINKED TREE (CHILD, SIBLING, RETURN) THAT REACT CAN WALK ONE UNIT AT A TIME

18.1.4 Render phase vs commit phase

Every update has two phases. In the **render phase** React calls your components and computes what changed. This phase must be **pure**: no DOM changes, no subscriptions, no requests, because React may call components more than once (Strict Mode does so on purpose) or discard the result. In the **commit phase** React applies the changes to the DOM, then runs `useLayoutEffect` callbacks (before the browser paints) and finally `useEffect` callbacks (after painting). The console below shows the order.

APP.JSX

JavaScript

```
import { useEffect, useLayoutEffect, useState } from "react";

function Child({ n }) {
  console.log(`render: Child (n=${n})`);
  useLayoutEffect(() => { console.log("commit: Child layout effect"); });
  useEffect(() => { console.log("commit: Child effect (after paint)"); });
  return <p>{n}</p>;
}

export default function App() {
  const [n, setN] = useState(0);
  console.log("render: App");
  useEffect(() => { console.log("commit: App effect (after paint)"); });
  return <div><Child n={n} /><button onClick={() => setN(n + 1)}>+1</button></div>;
}
```

BROWSER CONSOLE: FIRST RENDER, THEN A CLICK

```
render: App
render: Child (n=0)
commit: Child layout effect
commit: Child effect (after paint)
commit: App effect (after paint)
click +1
render: App
render: Child (n=1)
commit: Child layout effect
commit: Child effect (after paint)
commit: App effect (after paint)
```

Notice that the whole tree renders first (parent, then children), and effects run afterwards, children before parents.

18.2 Why components re-render

18.2.1 State change, parent re-render, context change

A component re-renders for exactly three reasons: (1) its own **state** changed (`useState`, `useReducer`, or a store it subscribes to); (2) its **parent re-rendered**: by default every child re-renders with its parent, **whether or not its props changed**; (3) a **context** it reads changed. A change of props on its own does not trigger anything: props change only because the parent re-rendered. Re-rendering is normally cheap and fine; it becomes a problem only when a component is slow or renders very often.

APP.JSX

JavaScript

```
import { createContext, useContext, useState } from "react";

const ThemeContext = createContext("light");

function Static() { console.log("render: Static (no props!)"); return <p>I never change</p>; }
function ThemeLabel() { const t = useContext(ThemeContext); console.log("render: ThemeLabel"); return
<p>{t}</p>; }

export default function App() {
  const [count, setCount] = useState(0);
  const [theme, setTheme] = useState("light");
  console.log("render: App");
  return (
    <ThemeContext value={theme}>
      <button onClick={() => setCount(count + 1)}>count {count}</button>
      <button onClick={() => setTheme(theme === "light" ? "dark" : "light")}>theme</button>
      <Static />
      <ThemeLabel />
    </ThemeContext>
  );
}
```

BROWSER CONSOLE

```
render: App
render: Static (no props!)
render: ThemeLabel
click "count"
render: App
render: Static (no props!)
render: ThemeLabel
click "theme"
render: App
render: Static (no props!)
render: ThemeLabel
```

`Static` has no props at all, yet it re-rendered after both clicks, simply because its parent did. The next section shows how to skip such renders when they are expensive.

18.3 Optimisation techniques

Measure first (Section 18.5), then optimise the slow part. The techniques below are listed roughly from the most to the least frequently useful.

18.3.1 React.memo

`memo(Component)` returns a version of the component that **skips re-rendering** when its props are the same as last time (each prop compared with `Object.is`). Use it for components that are slow to render and often receive the same props while the parent re-renders. It has no effect if a prop is a new object, array or function on every render, which leads to the next section.

APP.JSX — JavaScript

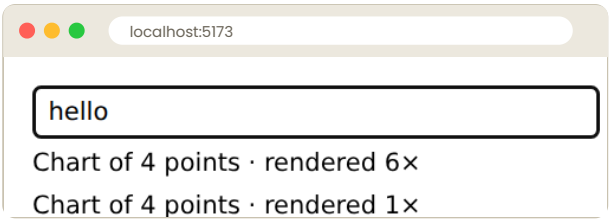
```
import { memo, useRef, useState } from "react";

function useRenders() { const r = useRef(0); r.current += 1; return r.current; }

function SlowChart({ data }) {
  const renders = useRenders();
  const start = performance.now();
  while (performance.now() - start < 30) {}              // pretend: 30 ms to draw
  return <p>Chart of {data.length} points · rendered {renders}×</p>;
}
const MemoChart = memo(SlowChart);                       // skips if props are unchanged

const points = [3, 7, 4, 9];                             // created once, outside the component

export default function App() {
  const [text, setText] = useState("");
  return (
    <div className="w-80 space-y-1 text-sm">
      <input value={text} onChange={(e) => setText(e.target.value)} placeholder="Type a note"
className="w-full rounded border px-2 py-1" />
      <SlowChart data={points} />
      <MemoChart data={points} />
    </div>
  );
}
```



AFTER TYPING FIVE CHARACTERS: THE PLAIN CHART RENDERED 6 TIMES (150 MS OF EXTRA WORK), THE MEMOISED ONE ONLY ONCE

18.3.2 useMemo and useCallback in practice

`useMemo(() => compute(a, b), [a, b])` caches a **calculated value** between renders and recomputes only when a dependency changes. Use it for (1) expensive calculations such as filtering and sorting thousands of items, and (2) keeping an object or array **referentially stable** so that a memoised child or an effect does not see a "new" value every time. `useCallback(fn, deps)` is the same for **functions**: it returns the same function object until a dependency changes. Without a memoised child or an effect that depends on them, they bring no benefit.

APP.JSX

JavaScript

```
import { memo, useCallback, useMemo, useRef, useState } from "react";

function useRenders() { const r = useRef(0); r.current += 1; return r.current; }

const StudentList = memo(function StudentList({ students, onSelect, label }) {
  return <p>{label}: {students.length} students · rendered <b>{useRenders()}×</b> <button onClick={()
=> onSelect(1)} className="text-indigo-600">select</button></p>;
});

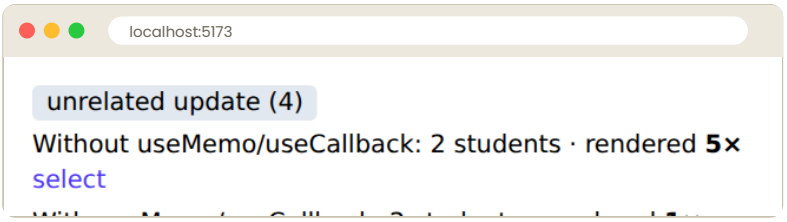
const all = [{ id: 1, name: "Asha", course: "React" }, { id: 2, name: "Ravi", course: "Java" }, { id:
3, name: "Meena", course: "React" }];

export default function App() {
  const [course, setCourse] = useState("React");
  const [clock, setClock] = useState(0);                          // unrelated state

  // new array and new function on every render: memo is useless
  const filteredBad = all.filter((s) => s.course === course);
  const selectBad = (id) => console.log(id);

  // stable between renders until `course` changes
  const filtered = useMemo(() => all.filter((s) => s.course === course), [course]);
  const select = useCallback((id) => console.log(id), []);

  return (
    <div className="space-y-1 text-sm">
      <button className="rounded bg-slate-200 px-2" onClick={() => setClock((c) => c + 1)}>unrelated
update ({clock})</button>
      <StudentList label="Without useMemo/useCallback" students={filteredBad} onSelect={selectBad} />
      <StudentList label="With useMemo/useCallback" students={filtered} onSelect={select} />
    </div>
  );
}
```



AFTER FOUR UNRELATED UPDATES: MEMO WAS DEFEATED BY NEW PROPS IN THE FIRST LIST, AND WORKED IN THE SECOND

Use	When it helps	When it is pointless
`memo`	slow child, parent re-renders often with the same props	cheap components; props that change every render anyway
`useMemo`	expensive calculation; an object/array passed to a memo child or used in effect dependencies	cheap calculations (the cache itself costs a little)
`useCallback`	a function passed to a memo child or used in effect dependencies	a handler passed only to native elements such as `<button>`

18.3.3 Keeping state close to where it is used

Often the best optimisation needs no memoisation at all. If state lives high in the tree, every change re-renders everything below it. **Move the state down** into the smallest component that needs it (state colocation), or **lift the slow content up** and pass it as `children`: a component that changes state does not re-render elements it received as children, because they were created by its parent.

APP.JSX

JavaScript

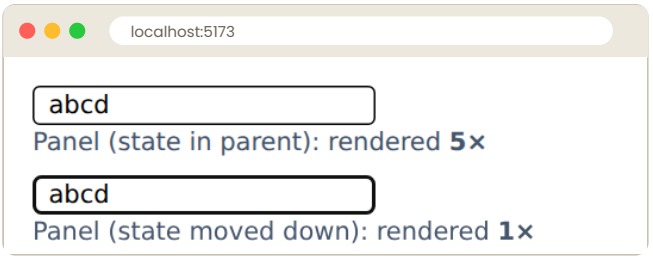
```
import { useRef, useState } from "react";

function useRenders() { const r = useRef(0); r.current += 1; return r.current; }
function SlowPanel({ label }) { return <p className="text-slate-600">{label}: rendered <b>
{useRenders()}×</b></p>; }

// before: the input state lives in the page, so the panel re-renders on every key
function PageBefore() {
  const [q, setQ] = useState("");
  return <div><input value={q} onChange={(e) => setQ(e.target.value)} placeholder="before"
className="rounded border px-2" /><SlowPanel label="Panel (state in parent)" /></div>;
}

// after: the state moved down into its own component
function SearchBox() {
  const [q, setQ] = useState("");
  return <input value={q} onChange={(e) => setQ(e.target.value)} placeholder="after"
className="rounded border px-2" />;
}
function PageAfter() {
  return <div><SearchBox /><SlowPanel label="Panel (state moved down)" /></div>;
}

export default function App() {
  return <div className="space-y-2 text-sm"><PageBefore /><PageAfter /></div>;
}
```



TYPED FOUR CHARACTERS INTO EACH BOX: ONLY THE VERSION WITH THE STATE IN THE PARENT RE-RENDERED THE PANEL

18.3.4 Code splitting with lazy() and dynamic imports

The fastest code is code that is not downloaded. Split by route (Module 13) and lazily load heavy components that are not needed immediately: charts, rich-text editors, maps, PDF viewers. Libraries used only in an event handler can be loaded with a plain **dynamic import** at the moment they are needed. Wrap lazily loaded components in Suspense (Module 17).

LAZY COMPONENTS AND DYNAMIC IMPORTS JavaScript

```
import { lazy, Suspense, useState } from "react";
const RichEditor = lazy(() => import("./RichEditor.jsx"));      // big dependency, own chunk

function Comment() {
  const [editing, setEditing] = useState(false);
  return editing
    ? <Suspense fallback={<p>Loading editor…</p>}><RichEditor /></Suspense>
    : <button onClick={() => setEditing(true)}>Write a comment</button>;
}

async function exportPdf(data) {
  const { jsPDF } = await import("jspdf");                      // loaded only when clicked
  new jsPDF().text(JSON.stringify(data), 10, 10).save("report.pdf");
}
```

18.3.5 List virtualisation (TanStack Virtual, react-window)

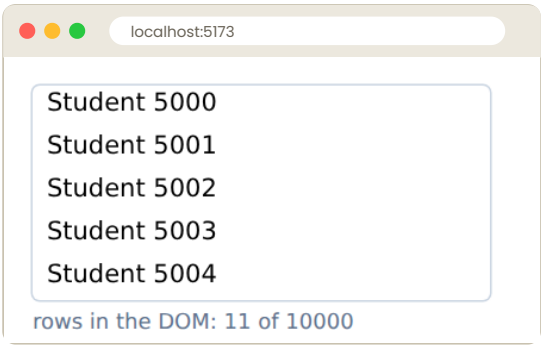
Rendering 10,000 rows creates 10,000 DOM nodes, which is slow to render, to update and to scroll, even if React itself is fast. **Virtualisation** (windowing) renders only the rows that are **visible** in the scroll area plus a few extra, and positions them inside a tall spacer so the scrollbar behaves as if all rows were there. Libraries: **TanStack Virtual** (`useVirtualizer`, headless) and **react-window** (`FixedSizeList` / `List` components). The idea is small enough to write by hand, which is what runs below:

APP.JSX JavaScript

```
import { useState } from "react";

const ROW = 24, HEIGHT = 120, OVERSCAN = 3;
const rows = Array.from({ length: 10000 }, (_, i) => `Student ${i + 1}`);

export default function VirtualList() {
  const [scrollTop, setScrollTop] = useState(0);
  const first = Math.max(0, Math.floor(scrollTop / ROW) - OVERSCAN);
  const last = Math.min(rows.length, Math.ceil((scrollTop + HEIGHT) / ROW) + OVERSCAN);
  return (
    <div className="w-64 text-sm">
      <div style={{ height: HEIGHT }} className="overflow-auto rounded ring-1 ring-slate-300"
onScroll={(e) => setScrollTop(e.currentTarget.scrollTop)}>
        <div style={{ height: rows.length * ROW, position: "relative" }}>            {/* full-height
spacer */}
          {rows.slice(first, last).map((r, i) => (
            <div key={first + i} style={{ position: "absolute", top: (first + i) * ROW, height: ROW }}
className="px-2">{r}</div>
          ))}
        </div>
      </div>
      <p className="mt-1 text-xs text-slate-500">rows in the DOM: {last - first} of {rows.length}</p>
    </div>
  );
}
```



SCROLLED TO ROW 5,000: ONLY 11 OF THE 10,000 ROWS EXIST IN THE DOM

THE SAME WITH TANSTACK VIRTUAL (REPRESENTATIVE) JavaScript

```
import { useVirtualizer } from "@tanstack/react-virtual";

function StudentList({ students }) {
  const parentRef = useRef(null);
  const virtualizer = useVirtualizer({ count: students.length, getScrollElement: () =>
parentRef.current, estimateSize: () => 24 });
  return (
    <div ref={parentRef} style={{ height: 400, overflow: "auto" }}>
      <div style={{ height: virtualizer.getTotalSize(), position: "relative" }}>
        {virtualizer.getVirtualItems().map((item) => (
          <div key={item.key} style={{ position: "absolute", top: 0, transform:
`translateY(${item.start}px)`, height: item.size }}>
            {students[item.index].name}
          </div>
        ))}
      </div>
    </div>
  );
}
```

18.3.6 Debouncing and throttling input

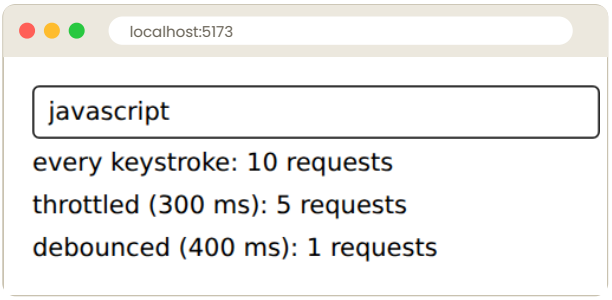
Some events fire very often: typing, scrolling, resizing, mouse moves. **Debouncing** waits until the events **stop** for a given time and then runs once (search-as-you-type: query the server only after the user pauses). **Throttling** runs at most once per interval while events keep coming (update a scroll position indicator every 100 ms). Module 11 built a `useDebounce` hook; the demo counts how many times each strategy would call the server for the same typing.

APP.JSX JavaScript

```
import { useEffect, useRef, useState } from "react";

function useDebounce(value, delay) {
  const [v, setV] = useState(value);
  useEffect(() => { const t = setTimeout(() => setV(value), delay); return () => clearTimeout(t); },
[value, delay]);
  return v;
}
function useThrottle(value, interval) {
  const [v, setV] = useState(value);
  const last = useRef(0);
  useEffect(() => {
    const wait = interval - (Date.now() - last.current);
    const t = setTimeout(() => { last.current = Date.now(); setV(value); }, Math.max(0, wait));
    return () => clearTimeout(t);
  }, [value, interval]);
  return v;
}
function useCount(value) { const c = useRef(-1); const prev = useRef(); if (prev.current !== value) {
c.current++; prev.current = value; } return c.current; }

export default function App() {
  const [text, setText] = useState("");
  const debounced = useDebounce(text, 400);
  const throttled = useThrottle(text, 300);
  return (
    <div className="w-80 space-y-1 text-sm">
      <input value={text} onChange={(e) => setText(e.target.value)} className="w-full rounded border
px-2 py-1" placeholder="Type quickly" />
      <p>every keystroke: {useCount(text)} requests</p>
      <p>throttled (300 ms): {useCount(throttled)} requests</p>
      <p>debounced (400 ms): {useCount(debounced)} requests</p>
    </div>
  );
}
```



TYPED “JAVASCRIPT” AT ABOUT 10 CHARACTERS PER SECOND, THEN PAUSED

18.3.7 Image optimisation

Images are usually the heaviest part of a page. Serve modern formats (**WebP**, **AVIF**) at the right size with `srcset` and `sizes` so phones do not download desktop images; always set `width` and `height` (or an

`aspect-ratio`) so the layout does not jump while the image loads; load below-the-fold images lazily with `loading="lazy"`; and mark the main hero image as important with `fetchPriority="high"`. Frameworks automate much of this (the Next.js `<Image>` component, Module 21).

RESPONSIVE, LAZY, LAYOUT-STABLE IMAGES JavaScript

```
<img
  src="/img/campus-800.webp"
  srcSet="/img/campus-400.webp 400w, /img/campus-800.webp 800w, /img/campus-1600.webp 1600w"
  sizes="(max-width: 640px) 100vw, 50vw"          // how wide the image is displayed
  width={800} height={450}                          // reserves space: no layout shift
  loading="lazy"                                    // only when near the viewport
  decoding="async"
  alt="Students in the LBF lab"
/>

<img src="/img/hero.avif" width={1600} height={600} fetchPriority="high" alt="..." />   {/* above the
fold */}
```

18.4 React Compiler

18.4.1 Automatic memoisation

Writing `memo`, `useMemo` and `useCallback` by hand is tedious and easy to get wrong (a missing dependency, a forgotten wrapper). The **React Compiler** (stable since 2025) is a build-time tool that analyses your components and inserts the memoisation automatically, at a finer level than you would by hand: individual JSX elements and values are cached and recomputed only when their inputs change. Code that follows the Rules of React (pure render, no mutation of props or state) needs no changes. With the compiler enabled, most manual `useMemo` / `useCallback` / `memo` calls become unnecessary.

BEFORE AND AFTER THE REACT COMPILER (REPRESENTATIVE) JavaScript

```
// what you write
function StudentList({ students, course }) {
  const filtered = students.filter((s) => s.course === course);
  return <List items={filtered} onSelect={(id) => select(id)} />;
}

// roughly what the compiler produces (simplified)
function StudentList({ students, course }) {
  const $ = useMemoCache(5);                        // a per-component cache
  let filtered;
  if ($[0] !== students || $[1] !== course) {     // recompute only when inputs change
    filtered = students.filter((s) => s.course === course);
    $[0] = students; $[1] = course; $[2] = filtered;
  } else filtered = $[2];
  let el;
  if ($[3] !== filtered) { el = <List items={filtered} onSelect={(id) => select(id)} />; $[3] =
filtered; $[4] = el; }
  else el = $[4];
  return el;
}
```

18.4.2 Enabling the compiler and the ESLint plugin

Terminal

```
$ npm install -D babel-plugin-react-compiler eslint-plugin-react-hooks@latest
```

VITE.CONFIG.JS AND ESLINT.CONFIG.JS

JavaScript

```
// vite.config.js
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [
    react({ babel: { plugins: ["babel-plugin-react-compiler"] } }),   // must run first
  ],
});

// eslint.config.js: the recommended preset includes the compiler's rules
import reactHooks from "eslint-plugin-react-hooks";
export default [reactHooks.configs.flat.recommended];
```

The ESLint rules (part of `eslint-plugin-react-hooks`) report code that breaks the Rules of React, which the compiler would skip. In React DevTools, components optimised by the compiler show a **"Memo ✨"** badge. Projects can adopt it gradually (per directory, or with the `"use memo"` directive) and opt a component out with `"use no memo"`.

18.5 Measuring performance

18.5.1 React DevTools Profiler

The **Profiler** tab of the React DevTools records a session: click record, use the app, stop. It shows every **commit** as a bar, and for each commit a **flame graph** of the components that rendered, how long each took, and (with "Record why each component rendered" enabled in the settings) **why** it rendered: props changed, state changed, parent rendered or hooks changed. Grey components did not render. The same measurements are available in code through the `<Profiler>` component, used here to measure the difference memo makes:

APP.JSX

JavaScript

```
import { Profiler, memo, useState } from "react";

function Row({ n }) {
  const start = performance.now();
  while (performance.now() - start < 0.05) {}                    // a slightly heavy row
  return <li>Row {n}</li>;
}
function Table() { return <ul className="h-10 overflow-hidden">{Array.from({ length: 2000 }, (_, i) =>
<Row key={i} n={i} />)}</ul>; }
const MemoTable = memo(Table);

function onRender(id, phase, actualDuration) {
  console.log(`${id.padEnd(9)} ${phase.padEnd(6)} ${actualDuration.toFixed(1)} ms`);
}

export default function App() {
  const [n, setN] = useState(0);
  return (
    <>
      <button onClick={() => setN(n + 1)}>update parent</button>
      <Profiler id="plain" onRender={onRender}><Table /></Profiler>
      <Profiler id="memoised" onRender={onRender}><MemoTable /></Profiler>
    </>
  );
}
```

BROWSER CONSOLE: <PROFILER> MEASUREMENTS

```
plain     mount  294.3 ms
memoised  mount  296.2 ms
click "update parent" twice
plain     update 271.5 ms
memoised  update 0.0 ms
plain     update 292.8 ms
memoised  update 0.1 ms
```

Timings are measured on the build machine and vary from run to run; the pattern is what matters: the memoised table costs almost nothing on updates.

18.5.2 Chrome Performance panel and Lighthouse

The **Performance** panel of Chrome DevTools records everything the browser does: JavaScript, style, layout, paint. Long tasks (over 50 ms) are marked in red; React 19.2 adds **React tracks** that show scheduler priorities and component renders on the same timeline. Throttle the CPU (4× or 6× slowdown) to see what users on cheaper phones experience. **Lighthouse** (in DevTools, or `npx lighthouse`) audits a page and scores performance using the **Core Web Vitals**:

Metric	Measures	Good
LCP (Largest Contentful Paint)	how quickly the main content appears	≤ 2.5 s
INP (Interaction to Next Paint)	how quickly the page responds to clicks and typing	≤ 200 ms
CLS (Cumulative Layout Shift)	how much the layout jumps while loading	≤ 0.1

18.5.3 Bundle analysis

Find out what is inside your JavaScript bundle before optimising it. For Vite, `rollup-plugin-visualizer` draws a treemap of every module and its size; `source-map-explorer` works with any bundler. Look for

large libraries used for one small feature, duplicated packages, and code that should be lazily loaded. Below is a real analysis of a minimal React app bundled for production with esbuild (the bundler inside Vite) and its `--analyze` option:

BUNDLE ANALYSIS OF A MINIMAL REACT APP main.js

```
$ esbuild main.jsx --bundle --minify --analyze
```

TERMINAL

```
dist/app.js
189.5kb  100.0%
   ├ node_modules/react-dom/cjs/react-dom-client.production.js                  172.1kb   90.8%
   ├ node_modules/react/cjs/react.production.js                                   7.4kb    3.9%
   ├ node_modules/react-dom/node_modules/scheduler/cjs/scheduler.production.js    3.6kb    1.9%
   ├ node_modules/react-dom/cjs/react-dom.production.js                           3.3kb    1.7%
   ├ node_modules/react/cjs/react-jsx-runtime.production.js                       364b     0.2%
   ├ node_modules/react-dom/client.js                                             253b     0.1%
   ├ node_modules/react-dom/index.js                                              253b     0.1%
   ├ main.jsx
218b     0.1%
   ├ node_modules/react-dom/node_modules/scheduler/index.js                        51b     0.0%
   ├ node_modules/react/index.js                                                   51b     0.0%
   └ node_modules/react/jsx-runtime.js                                             51b     0.0%
  dist/app.js  189.5kb
```

Even an empty React app ships React DOM (by far the largest part). That is fine, but it shows why every extra dependency should earn its place; check its size on bundlephobia.com before installing it.

PRACTICE CHECKPOINT

Profile a slow list of 10,000 rows, fix it with virtualisation and memoisation, and show the before-and-after Profiler results.

ROWS.JS JavaScript

```
// rows.js: 10,000 students
export const students = Array.from({ length: 10000 }, (_, i) => ({ id: i + 1, name: `Student ${i +
1}`, score: (i * 37) % 100 }));
```

BEFORE.JSX (APP.JSX BEFORE THE FIX)

JavaScript

```
import { Profiler, useState } from "react";
import { students } from "./rows.js";

function Row({ s }) { return <li className="flex justify-between px-2"><span>{s.name}</span><span>
{s.score}</span></li>; }

export default function App() {
  const [query, setQuery] = useState("");
  const [highlight, setHighlight] = useState(false);
  const list = students.filter((s) => s.name.includes(query));        // recomputed on every render
  return (
    <Profiler id="before" onRender={(id, phase, ms) => console.log(`BEFORE ${phase.padEnd(6)}
${list.length} rows rendered, ${ms.toFixed(1)} ms`)}>
      <input value={query} onChange={(e) => setQuery(e.target.value)} placeholder="Filter" />
      <button onClick={() => setHighlight(!highlight)}>toggle highlight</button>
      <ul style={{ height: 200, overflow: "auto" }} className={highlight ? "bg-yellow-50" : ""}>
        {list.map((s) => <Row key={s.id} s={s} />)}                    {/* 10,000 DOM rows */}
      </ul>
    </Profiler>
  );
}
```

BROWSER CONSOLE: BEFORE

```
BEFORE mount  10000 rows rendered, 404.2 ms
toggle highlight
BEFORE update 10000 rows rendered, 260.3 ms
type "99"
BEFORE update 280 rows rendered, 25.6 ms
```

APP.JSX: SAMPLE SOLUTION (AFTER)

JavaScript

```
import { Profiler, memo, useMemo, useState } from "react";
import { students } from "./rows.js";

const ROW = 24, HEIGHT = 200;
const Row = memo(function Row({ s, top }) {
  return <li style={{ position: "absolute", top, height: ROW, left: 0, right: 0 }} className="flex
justify-between px-2"><span>{s.name}</span><span>{s.score}</span></li>;
});

const VirtualList = memo(function VirtualList({ items }) {
  const [scrollTop, setScrollTop] = useState(0);
  const first = Math.max(0, Math.floor(scrollTop / ROW) - 3);
  const last = Math.min(items.length, Math.ceil((scrollTop + HEIGHT) / ROW) + 3);
  return (
    <ul style={{ height: HEIGHT, overflow: "auto", position: "relative" }} onScroll={(e) =>
setScrollTop(e.currentTarget.scrollTop)}>
      <div style={{ height: items.length * ROW }} />
      {items.slice(first, last).map((s, i) => <Row key={s.id} s={s} top={(first + i) * ROW} />)}
    </ul>
  );
});

export default function App() {
  const [query, setQuery] = useState("");
  const [highlight, setHighlight] = useState(false);
  const list = useMemo(() => students.filter((s) => s.name.includes(query)), [query]);   // only when
query changes
  return (
    <Profiler id="after" onRender={(id, phase, ms) => console.log(`AFTER  ${phase.padEnd(6)}
${list.length} matching, ${ms.toFixed(1)} ms`)}>
      <input value={query} onChange={(e) => setQuery(e.target.value)} placeholder="Filter" />
      <button onClick={() => setHighlight(!highlight)}>toggle highlight</button>
      <div className={highlight ? "bg-yellow-50" : ""}><VirtualList items={list} /></div>
    </Profiler>
  );
}
```

BROWSER CONSOLE: AFTER

```
AFTER  mount  10000 matching, 13.1 ms
toggle highlight
AFTER  update 10000 matching, 0.5 ms
type "99"
AFTER  update 280 matching, 3.7 ms
```

Durations come from React's Profiler on the build machine. Virtualisation cuts the work from 10,000 rows to about 15; memo makes the highlight toggle almost free; useMemo avoids filtering again when only the highlight changes.

React renders a tree of plain objects, diffs it with the previous tree (different type = rebuild, same type and key = update) and commits only the changes; Fiber splits the work into interruptible units; the render phase must be pure, effects run after commit. Components re-render because of their own state, a parent render or a context change. Measure first, then: memo slow children, stabilise props with useMemo/useCallback, move state down or pass slow content as children, split code with lazy, virtualise long lists, debounce or throttle frequent events, and optimise images. The React Compiler adds memoisation automatically. Measure with the React Profiler, the Chrome Performance panel, Lighthouse (LCP, INP, CLS) and bundle analysis.

MODULE 19 · ADVANCED

Advanced patterns and application architecture

Minimum duration **2 days** Level **Advanced**

As applications grow, the way components are designed and files are organised decides how easy the code is to change. This module covers the component patterns used by professional teams and UI libraries (composition, container and presentational components, compound components, render props, higher-order components, controlled and uncontrolled APIs, headless components), error boundaries and fallback UI, portals for modals, tooltips and toasts, project architecture (folder structures, path aliases, environment variables, linting and formatting), accessibility in React, and an overview of internationalisation.

IN THIS MODULE

19.1 Component patterns
19.2 Error handling
19.3 Portals
19.4 Project architecture
19.5 Accessibility in React
19.6 Internationalisation (overview)

19.1 Component patterns

19.1.1 Composition over inheritance

React components are reused by **composing** them, never by inheriting from each other. A component that does not know its content in advance accepts it as `children`; when it has several “holes”, it accepts elements through named props, often called **slots**. Specialised components are built by wrapping general ones with fixed props, not by subclassing.

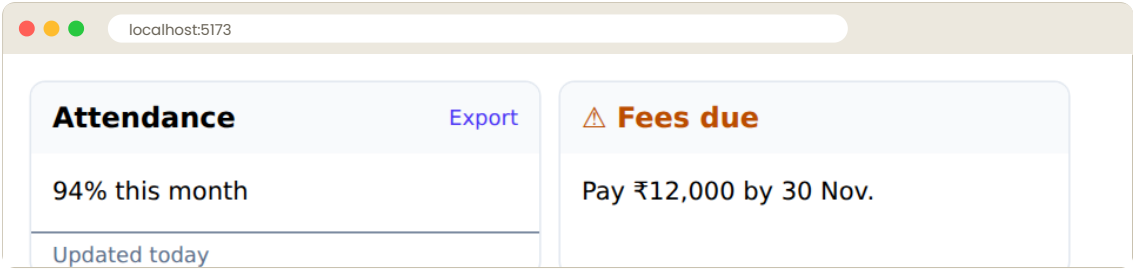
APP.JSX

JavaScript

```
function Card({ title, actions, footer, children }) {          // slots: title, actions, footer,
children
  return (
    <section className="w-72 overflow-hidden rounded-lg ring-1 ring-slate-200">
      <header className="flex items-center justify-between bg-slate-50 px-3 py-2">
        <h3 className="font-semibold">{title}</h3>{actions}
      </header>
      <div className="p-3 text-sm">{children}</div>
      {footer && <footer className="border-t px-3 py-1 text-xs text-slate-500">{footer}</footer>}
    </section>
  );
}

// specialisation by composition: a Card with some props fixed
function WarningCard(props) {
  return <Card {...props} title={<span className="text-amber-700">⚠ {props.title}</span>} />;
}

export default function App() {
  return (
    <div className="flex gap-3">
      <Card title="Attendance" actions={<button className="text-xs text-indigo-600">Export</button>}
footer="Updated today">
        <p>94% this month</p>
      </Card>
      <WarningCard title="Fees due">Pay ₹12,000 by 30 Nov.</WarningCard>
    </div>
  );
}
```



ONE CARD COMPONENT, FILLED THROUGH SLOTS AND SPECIALISED BY WRAPPING

19.1.2 Container and presentational components

A classic way to separate concerns: **presentational** components only describe how things look (they receive data and callbacks through props, have no fetching and little state, and are easy to reuse and test), while **container** components know how things work (they fetch data, hold state, and pass it down). With hooks the container is often replaced by a **custom hook** that holds the logic, but the idea of keeping UI components “dumb” remains valuable.

APP.JSX

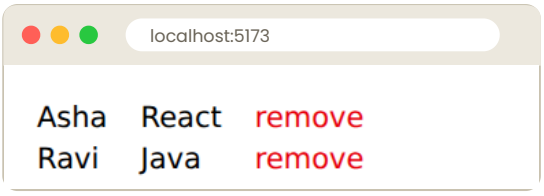
JavaScript

```
import { useEffect, useState } from "react";

// presentational: just props in, UI out
function StudentTable({ students, onRemove }) {
  return (
    <table className="text-sm">
      <tbody>{students.map((s) => (
        <tr key={s.id}><td className="pr-4">{s.name}</td><td className="pr-4">{s.course}</td>
          <td><button className="text-red-600" onClick={() => onRemove(s.id)}>remove</button></td>
</tr>
      ))}</tbody>
    </table>
  );
}

// logic in a custom hook (the modern “container”)
function useStudents() {
  const [students, setStudents] = useState([]);
  useEffect(() => { setStudents([{ id: 1, name: "Asha", course: "React" }, { id: 2, name: "Ravi",
course: "Java" }]); }, []);
  const remove = (id) => setStudents((list) => list.filter((s) => s.id !== id));
  return { students, remove };
}

// container: connects the logic to the view
export default function StudentTableContainer() {
  const { students, remove } = useStudents();
  return <StudentTable students={students} onRemove={remove} />;
}
```



THE TABLE KNOWS NOTHING ABOUT WHERE THE DATA COMES FROM

19.1.3 Compound components (Tabs, Accordion)

Compound components are a set of components that work together and share state implicitly, like `<select>` and `<option>` in HTML. The parent holds the state in a context, and the children read it. The user of the component arranges the parts freely, with no long list of configuration props. Attaching the parts as properties (`Tabs.List` , `Tabs.Tab` , `Tabs.Panel`) makes the relationship obvious.

TABS.JSX JavaScript

```
import { createContext, useContext, useState, useId } from "react";

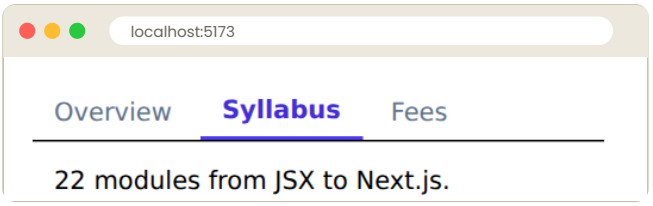
const TabsContext = createContext(null);
const useTabs = () => {
  const ctx = useContext(TabsContext);
  if (!ctx) throw new Error("Tabs.* must be used inside <Tabs>");
  return ctx;
};

export function Tabs({ defaultValue, children }) {
  const [value, setValue] = useState(defaultValue);
  const id = useId();
  return <TabsContext value={{ value, setValue, id }}><div className="w-80">{children}</div>
</TabsContext>;
}
Tabs.List = function TabList({ children }) {
  return <div role="tablist" className="flex gap-1 border-b">{children}</div>;
};
Tabs.Tab = function Tab({ value, children }) {
  const { value: active, setValue, id } = useTabs();
  const selected = active === value;
  return (
    <button role="tab" id={`${id}-tab-${value}`} aria-selected={selected} aria-controls={`${id}-
panel-${value}`} onClick={() => setValue(value)}
            className={`px-3 py-1 text-sm ${selected ? "border-b-2 border-indigo-600 font-semibold
text-indigo-700" : "text-slate-500"}`}>{children}</button>
  );
};
Tabs.Panel = function TabPanel({ value, children }) {
  const { value: active, id } = useTabs();
  if (active !== value) return null;
  return <div role="tabpanel" id={`${id}-panel-${value}`} aria-labelledby={`${id}-tab-${value}`}
className="p-3 text-sm">{children}</div>;
};
```

APP.JSX JavaScript

```
import { Tabs } from "./Tabs.jsx";

export default function App() {
  return (
    <Tabs defaultValue="overview">
      <Tabs.List>
        <Tabs.Tab value="overview">Overview</Tabs.Tab>
        <Tabs.Tab value="syllabus">Syllabus</Tabs.Tab>
        <Tabs.Tab value="fees">Fees</Tabs.Tab>
      </Tabs.List>
      <Tabs.Panel value="overview">48 days, beginner to advanced.</Tabs.Panel>
      <Tabs.Panel value="syllabus">22 modules from JSX to Next.js.</Tabs.Panel>
      <Tabs.Panel value="fees">₹24,000, payable in two instalments.</Tabs.Panel>
    </Tabs>
  );
}
```



AFTER CLICKING "SYLLABUS": THE PARTS SHARE STATE THROUGH CONTEXT

19.1.4 Render props

A **render prop** is a prop whose value is a function that returns UI. The component does the work (tracking, fetching, measuring) and calls the function with the result, so the **caller decides what to render**. `children` can be the function too. Custom hooks have replaced most render props, but the pattern is still common in libraries (for example the virtualised list and form libraries) and whenever the logic must stay inside a component.

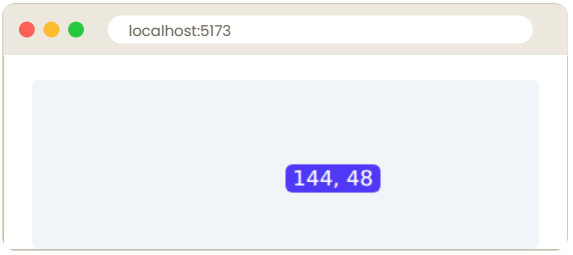
APP.JSX

JavaScript

```
import { useState } from "react";

function Hoverable({ children }) {                      // does the work...
  const [pos, setPos] = useState(null);
  return (
    <div className="relative h-24 w-72 rounded bg-slate-100"
         onMouseMove={(e) => { const r = e.currentTarget.getBoundingClientRect(); setPos({ x:
Math.round(e.clientX - r.left), y: Math.round(e.clientY - r.top) }); }}
         onMouseLeave={() => setPos(null)}>
      {children(pos)}                                    {/* ...the caller decides the UI */}
    </div>
  );
}

export default function App() {
  return (
    <Hoverable>
      {(pos) => pos
        ? <span className="absolute rounded bg-indigo-600 px-1 text-xs text-white" style={{ left:
pos.x, top: pos.y }}>{pos.x}, {pos.y}</span>
        : <p className="p-2 text-sm text-slate-500">Move the mouse here</p>}
    </Hoverable>
  );
}
```



THE MOUSE WAS MOVED INTO THE BOX: THE RENDER PROP RECEIVED THE POSITION

19.1.5 Higher-order components (HOC)

A **higher-order component** is a function that takes a component and returns a new component with extra behaviour: `const Enhanced = withSomething(Base)`. HOCs were the main reuse pattern before hooks (Redux's old `connect()`, React Router's old `withRouter`). Today prefer custom hooks; you will still meet HOCs in older code and for cross-cutting wrappers such as permission checks or error boundaries. Name them `withX`, pass all other props through, and set a `displayName` for the DevTools.

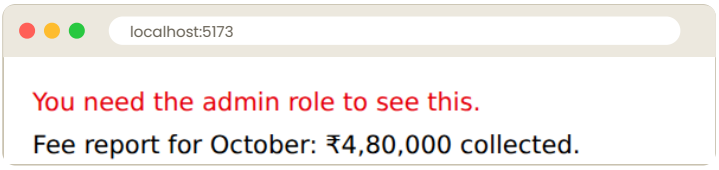
APP.JSX

JavaScript

```
function withPermission(Component, role) {
  function Guarded({ user, ...props }) {
    if (user?.role !== role) return <p className="text-sm text-red-600">You need the {role} role to
see this.</p>;
    return <Component {...props} user={user} />;            // pass everything through
  }
  Guarded.displayName = `withPermission(${Component.name})`;
  return Guarded;
}

function FeeReport({ month }) { return <p className="text-sm">Fee report for {month}: ₹4,80,000
collected.</p>; }
const AdminFeeReport = withPermission(FeeReport, "admin");

export default function App() {
  return (
    <div className="space-y-1">
      <AdminFeeReport user={{ name: "Asha", role: "student" }} month="October" />
      <AdminFeeReport user={{ name: "Priya", role: "admin" }} month="October" />
    </div>
  );
}
```



THE SAME WRAPPED COMPONENT FOR A STUDENT AND FOR AN ADMIN

19.1.6 Controlled vs uncontrolled component APIs

Reusable inputs should work both ways, like native inputs do. **Uncontrolled**: the component keeps its own state, and the parent may give a starting value with `defaultValue`. **Controlled**: the parent owns the state and passes `value` and `onChange`. A common implementation checks whether `value` was provided and picks the right source.

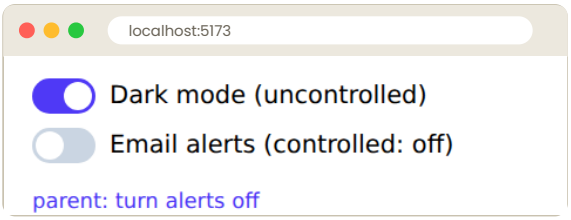
APP.JSX

JavaScript

```
import { useState } from "react";

function Toggle({ value, defaultValue = false, onChange, label }) {
  const [internal, setInternal] = useState(defaultValue);
  const isControlled = value !== undefined;
  const on = isControlled ? value : internal;
  function flip() {
    if (!isControlled) setInternal(!on);                  // uncontrolled: own state
    onChange?.(!on);                                      // tell the parent either way
  }
  return (
    <button role="switch" aria-checked={on} onClick={flip} className="flex items-center gap-2 text-
sm">
      <span className={`h-5 w-9 rounded-full p-0.5 ${on ? "bg-indigo-600" : "bg-slate-300"}`}>
        <span className={`block h-4 w-4 rounded-full bg-white transition ${on ? "translate-x-4" :
""}`} />
      </span>{label}
    </button>
  );
}

export default function App() {
  const [emails, setEmails] = useState(true);
  return (
    <div className="space-y-2">
      <Toggle label="Dark mode (uncontrolled)" defaultValue={false} />
      <Toggle label={`Email alerts (controlled: ${emails ? "on" : "off"})`} value={emails} onChange=
{setEmails} />
      <button className="text-xs text-indigo-600" onClick={() => setEmails(false)}>parent: turn alerts
off</button>
    </div>
  );
}
```



CLICKED THE UNCONTROLLED TOGGLE, AND THE PARENT SWITCHED THE CONTROLLED ONE OFF

19.1.7 Headless components

A **headless** component or hook provides **behaviour and accessibility** (state, keyboard handling, ARIA attributes) but **no styling or markup** of its own. You render whatever you like and spread the props it gives you. This is how modern libraries such as TanStack Table, Downshift, React Aria, Radix Primitives and Headless UI work: one tested implementation of tricky behaviour, any design.

USEDISCLOSURE.JS JavaScript

```
import { useId, useState } from "react";

// headless: state + accessible props, no markup
export function useDisclosure(initial = false) {
  const [open, setOpen] = useState(initial);
  const id = useId();
  return {
    open,
    getButtonProps: () => ({ "aria-expanded": open, "aria-controls": id, onClick: () => setOpen((o) =>
!o) }),
    getPanelProps: () => ({ id, hidden: !open }),
  };
}
```

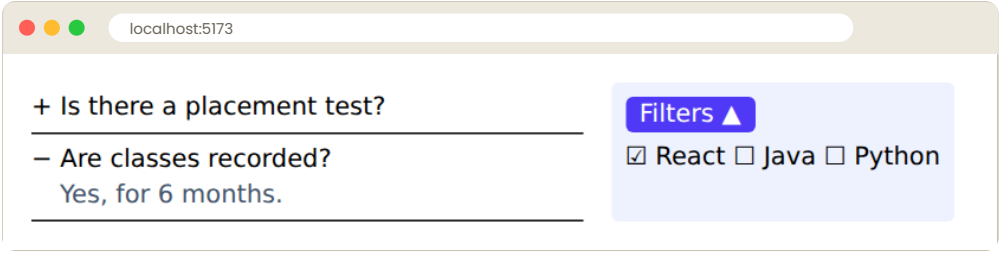
APP.JSX JavaScript

```
import { useDisclosure } from "./useDisclosure.js";

function FaqItem({ q, a }) {                                    // design 1: an FAQ
  const d = useDisclosure();
  return (
    <div className="border-b py-1">
      <button {...d.getButtonProps()} className="w-full text-left font-medium">{d.open ? "−" : "+"}
{q}</button>
      <p {...d.getPanelProps()} className="pl-4 text-slate-600">{a}</p>
    </div>
  );
}

function FilterPanel() {                                        // design 2: a filter drawer
  const d = useDisclosure(true);
  return (
    <div className="rounded bg-indigo-50 p-2">
      <button {...d.getButtonProps()} className="rounded bg-indigo-600 px-2 text-white">Filters
{d.open ? "▲" : "▼"}</button>
      <div {...d.getPanelProps()} className="pt-1">☑ React ☐ Java ☐ Python</div>
    </div>
  );
}

export default function App() {
  return (
    <div className="flex w-[520px] gap-4 text-sm">
      <div className="flex-1"><FaqItem q="Is there a placement test?" a="No, only an interview." />
<FaqItem q="Are classes recorded?" a="Yes, for 6 months." /></div>
      <FilterPanel />
    </div>
  );
}
```



ONE HEADLESS HOOK, TWO COMPLETELY DIFFERENT DESIGNS (THE SECOND FAQ ITEM WAS OPENED)

19.2 Error handling

19.2.1 Error boundaries

If a component throws during rendering, React unmounts the **whole** tree: the user sees a blank page. An **error boundary** catches errors thrown while rendering its children (including in lifecycle methods and constructors) and shows a **fallback UI** instead, so the rest of the app keeps working. Error boundaries must still be written as **class components** using `static getDerivedStateFromError` (to show the fallback) and `componentDidCatch` (to log). They do **not** catch errors in event handlers, in asynchronous code (timers, promises) or on the server; handle those with `try/catch`.

ERRORBOUNDARY.JSX — JavaScript

```
import { Component } from "react";

export class ErrorBoundary extends Component {
  state = { error: null };

  static getDerivedStateFromError(error) {            // render phase: switch to the fallback
    return { error };
  }
  componentDidCatch(error, info) {                    // commit phase: log it
    console.log("logged to monitoring:", error.message);
  }
  reset = () => this.setState({ error: null });

  render() {
    if (this.state.error) {
      return this.props.fallback
        ? this.props.fallback({ error: this.state.error, reset: this.reset })
        : <p>Something went wrong.</p>;
    }
    return this.props.children;
  }
}
```

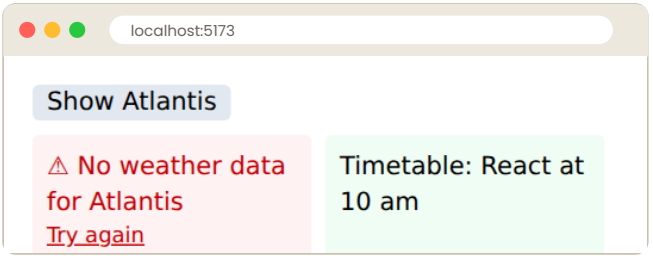
APP.JSX

JavaScript

```
import { useState } from "react";
import { ErrorBoundary } from "./ErrorBoundary.jsx";

function WeatherWidget({ city }) {
  if (city === "Atlantis") throw new Error(`No weather data for ${city}`);   // a bug or bad data
  return <p>{city}: 29°C, sunny</p>;
}

export default function App() {
  const [city, setCity] = useState("Pune");
  return (
    <div className="w-80 space-y-2 text-sm">
      <button className="rounded bg-slate-200 px-2" onClick={() => setCity("Atlantis")}>Show
Atlantis</button>
      <div className="grid grid-cols-2 gap-2">
        <ErrorBoundary fallback={({ error, reset }) => (
          <div className="rounded bg-red-50 p-2 text-red-700">⚠ {error.message}
            <button className="block text-xs underline" onClick={() => { setCity("Pune"); reset();
}}>Try again</button></div>
        )}>
          <div className="rounded bg-sky-50 p-2"><WeatherWidget city={city} /></div>
        </ErrorBoundary>
        <div className="rounded bg-green-50 p-2">Timetable: React at 10 am</div>     {/* unaffected
*/}
      </div>
    </div>
  );
}
```



AFTER CLICKING “SHOW ATLANTIS”: ONLY THE CRASHING WIDGET WAS REPLACED BY ITS FALLBACK

19.2.2 react-error-boundary

Instead of writing the class yourself, most projects use the small **react-error-boundary** package. Its `<ErrorBoundary>` accepts `fallback`, `FallbackComponent` or `fallbackRender`, an `onError` callback for logging, and `resetKeys`: when one of those values changes, the boundary resets automatically. The `useErrorBoundary` hook lets you send errors from event handlers and async code to the nearest boundary.

REACT-ERROR-BOUNDARY (REPRESENTATIVE) JavaScript

```
import { ErrorBoundary, useErrorBoundary } from "react-error-boundary";

function Fallback({ error, resetErrorBoundary }) {
  return (
    <div role="alert">
      <p>Could not load the widget: {error.message}</p>
      <button onClick={resetErrorBoundary}>Try again</button>
    </div>
  );
}

<ErrorBoundary FallbackComponent={Fallback} onError={(error, info) => logToSentry(error, info)}
resetKeys={[city]}>
  <WeatherWidget city={city} />
</ErrorBoundary>

function SaveButton() {
  const { showBoundary } = useErrorBoundary();
  return <button onClick={() => save().catch(showBoundary)}>Save</button>;   // async error ->
boundary
}
```

19.2.3 Fallback UI and error logging

Place boundaries at several levels: one around the whole app (a friendly "something went wrong, reload" page), one per route (so the navigation stays usable), and around risky widgets (third-party embeds, charts, user-generated content). A good fallback explains what happened in plain words, offers a way out (try again, go home) and does not show stack traces to users. Send the error and the component stack to a monitoring service (Sentry, Datadog, LogRocket) from `componentDidCatch` / `onError` and from the React 19 root options `onCaughtError` and `onUncaughtError`.

BROWSER CONSOLE: WHAT WAS LOGGED

```
click "Show Atlantis"
logged to monitoring: No weather data for Atlantis
Error: No weather data for Atlantis

The above error occurred in the <WeatherWidget> component.

React will try to recreate this component tree from scratch using the error boundary you provided,
ErrorBoundary.
```

React itself also reports the caught error once (in development, with the component stack) so that it is never silently swallowed.

19.3 Portals

19.3.1 createPortal for modals, tooltips and toasts

`createPortal(children, domNode)` renders children into a **different DOM node**, usually one directly under `<body>`, while keeping them in the same place in the **React tree**. That solves a common CSS problem: a modal or tooltip rendered inside a container with `overflow: hidden`, a `transform` or a low `z-index` gets clipped or hidden. Through the portal it escapes the container visually, yet props, context and even events still flow as if it were nested: a click inside the portal bubbles to React ancestors.

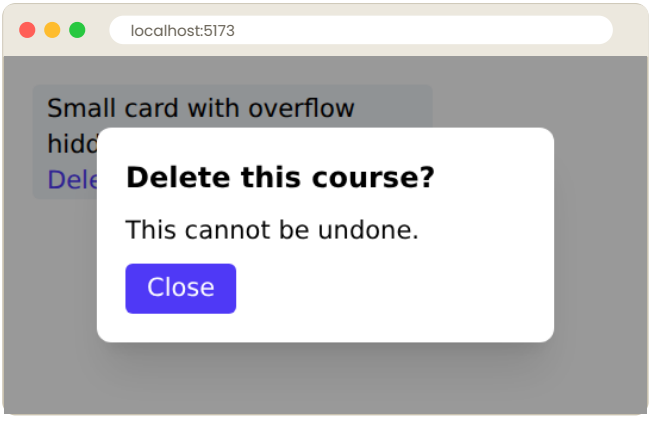
APP.JSX

JavaScript

```
import { useState } from "react";
import { createPortal } from "react-dom";

function Modal({ title, onClose, children }) {
  return createPortal(
    <div className="fixed inset-0 grid place-items-center bg-black/40" onClick={onClose}>
      <div role="dialog" aria-modal="true" aria-label={title} className="w-64 rounded-lg bg-white p-4
shadow-xl" onClick={(e) => e.stopPropagation()}>
        <h2 className="font-semibold">{title}</h2>
        <div className="py-2 text-sm">{children}</div>
        <button className="rounded bg-indigo-600 px-3 py-1 text-sm text-white" onClick=
{onClose}>Close</button>
      </div>
    </div>,
    document.body                                    // rendered at the end of <body>
  );
}

export default function App() {
  const [open, setOpen] = useState(false);
  return (
    <div className="h-48">
      <div className="h-16 w-56 overflow-hidden rounded bg-slate-100 p-2 text-sm">   {/* clips its
content */}
        Small card with overflow hidden
        <button className="block text-indigo-600" onClick={() => setOpen(true)}>Delete course…
</button>
        {open && <Modal title="Delete this course?" onClose={() => setOpen(false)}>This cannot be
undone.</Modal>}
      </div>
    </div>
  );
}
```



THE MODAL IS DECLARED INSIDE THE CLIPPED CARD, BUT THE PORTAL RENDERED IT OVER THE WHOLE PAGE

Toasts follow the same idea: a `ToastProvider` holds the list of messages in context and renders them through a portal into a fixed corner of the screen, while any component can call `useToast().show("Saved")`.

19.4 Project architecture

19.4.1 Folder structures: by type vs by feature

Small projects are often organised **by type**: all components in one folder, all hooks in another. As the app grows, files that change together end up far apart. Organising **by feature** keeps everything for one feature (components, hooks, API calls, tests) together; shared building blocks live in a separate folder. Each feature exposes a small public API through an `index.js`, and features do not import each other's internals.

Two common folder structures

```
# by type (fine for small apps)
src/
├── components/
│   ├── StudentTable.jsx
│   ├── StudentForm.jsx
│   └── CourseCard.jsx
├── hooks/
│   ├── useStudents.js
│   └── useCourses.js
├── services/
│   └── api.js
├── pages/
│   └── StudentsPage.jsx
└── App.jsx
```

```
# by feature (scales to big apps)
src/
├── app/            router, providers, store
├── features/
│   ├── students/
│   │   ├── components/   StudentTable.jsx, StudentForm.jsx
│   │   ├── api.js        REST calls
│   │   ├── hooks.js      useStudents, useAddStudent
│   │   ├── students.test.jsx
│   │   └── index.js      public exports
│   └── courses/ ...
├── shared/
│   ├── ui/           Button, Modal, Tabs
│   ├── hooks/        useDebounce, useLocalStorage
│   └── lib/          apiClient, formatters
└── main.jsx
```

19.4.2 Absolute imports and path aliases

Deep relative paths such as `../../../shared/ui/Button` are hard to read and break when files move. A **path alias** maps a short prefix, conventionally `@/`, to `src/`. Configure it for the bundler (Vite) and, for editor autocompletion and TypeScript, in `tsconfig.json` / `jsconfig.json`.

CONFIGURING THE @ ALIAS

JavaScript

```
// vite.config.js
import { fileURLToPath, URL } from "node:url";
export default defineConfig({
  plugins: [react()],
  resolve: { alias: { "@": fileURLToPath(new URL("./src", import.meta.url)) } },
});

// tsconfig.json (or jsconfig.json)
{ "compilerOptions": { "baseUrl": ".", "paths": { "@/*": ["src/*"] } } }

// before
import { Button } from "../../../shared/ui/Button";
// after
import { Button } from "@/shared/ui/Button";
```

19.4.3 Environment variables

Values that differ between environments (API URLs, feature flags, public keys) go into `.env` files. Vite loads `.env`, `.env.local`, `.env.[mode]` and `.env.[mode].local`, and exposes only variables that start with `VITE_` to the browser code as `import.meta.env.VITE_...`. Built-in values: `import.meta.env.MODE`, `DEV`,

`PROD`. Everything in a `VITE_` variable ends up in the public bundle, so **never put secrets there** (Module 22). Commit a `.env.example` listing the variables, and keep `.env.local` out of Git.

File	Loaded when	Commit to Git?
`.env`	always	yes (defaults only)
`.env.development`	`npm run dev`	yes
`.env.production`	`npm run build`	yes
`.env.local`, `.env.*.local`	always / that mode, overrides the others	no (personal values)

19.4.4 Linting and formatting setup

ESLint finds bugs and enforces conventions (unused variables, missing hook dependencies, rules of hooks, accessibility problems); **Prettier** formats code automatically so that nobody argues about spacing. The Vite template already includes ESLint with the React Hooks and React Refresh plugins. Add Prettier, turn off ESLint's formatting rules with `eslint-config-prettier`, format on save in the editor, and run both in CI. **Husky** with **lint-staged** can run them on the changed files before each commit.

ESLINT AND PRETTIER

JavaScript

```
// eslint.config.js (flat config)
import js from "@eslint/js";
import reactHooks from "eslint-plugin-react-hooks";
import reactRefresh from "eslint-plugin-react-refresh";
import jsxA11y from "eslint-plugin-jsx-a11y";
import prettier from "eslint-config-prettier";

export default [
  js.configs.recommended,
  reactHooks.configs.flat.recommended,          // rules of hooks, exhaustive deps, compiler rules
  reactRefresh.configs.vite,
  jsxA11y.flatConfigs.recommended,              // accessibility checks in JSX
  prettier,                                     // last: disable rules that conflict with Prettier
];

// package.json scripts
"lint": "eslint .",
"format": "prettier --write ."
```

19.5 Accessibility in React

19.5.1 Semantic HTML in JSX

Accessibility starts with the right HTML elements, which JSX supports fully. Use `<button>` for actions (it is focusable and works with Enter and Space), `<a href>` / `Link` for navigation, real headings in order, `<nav>`, `<main>`, `<ul>` for lists, and `<label htmlFor>` for every input. A `<div onClick>` is invisible to keyboard and screen-reader users. Remember the JSX spellings: `htmlFor`, `className`, and `aria-*` / `data-*` attributes stay hyphenated. Use `useId()` to generate ids that connect labels, descriptions and errors.

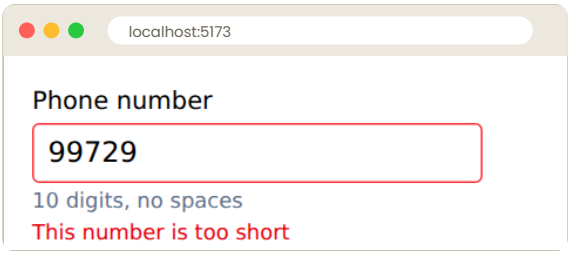
APP.JSX

JavaScript

```
import { useId } from "react";

function Field({ label, hint, error, ...props }) {
  const id = useId();
  return (
    <div className="space-y-0.5">
      <label htmlFor={id} className="block text-sm font-medium">{label}</label>
      <input id={id} aria-describedby={`${id}-hint ${id}-error`} aria-invalid={!!error} className={`w-
64 rounded border px-2 py-1 ${error ? "border-red-500" : ""}`} {...props} />
      <p id={`${id}-hint`} className="text-xs text-slate-500">{hint}</p>
      {error && <p id={`${id}-error`} role="alert" className="text-xs text-red-600">{error}</p>}
    </div>
  );
}

export default function App() {
  return <Field label="Phone number" hint="10 digits, no spaces" error="This number is too short"
defaultValue="99729" />;
}
```



LABEL, HINT AND ERROR ARE CONNECTED TO THE INPUT WITH IDS FROM USEID, SO SCREEN READERS ANNOUNCE ALL THREE

19.5.2 Focus management in modals and route changes

When a **modal** opens, move focus into it (the first field or the dialog itself), keep Tab and Shift+Tab **trapped** inside while it is open, close it with **Escape**, and when it closes return focus to the button that opened it. The native `<dialog>` element with `showModal()` does most of this for you. After a client-side **route change**, nothing tells a screen-reader user that the page changed: move focus to the new page's main heading (with `tabIndex={-1}`) or announce the new title in a live region.

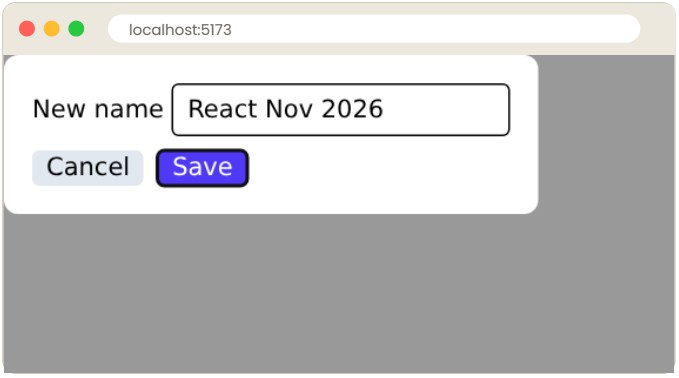
APP.JSX

JavaScript

```
import { useEffect, useRef, useState } from "react";

function Dialog({ onClose, children }) {
  const ref = useRef(null);
  useEffect(() => {
    const dialog = ref.current;
    dialog.showModal();                                  // native modal: focus trap, Esc, backdrop
    dialog.querySelector("input")?.focus();
    return () => dialog.close();
  }, []);
  return <dialog ref={ref} onClose={onClose} className="rounded-lg p-4 backdrop:bg-black/40">
{children}</dialog>;
}

export default function App() {
  const [open, setOpen] = useState(false);
  const opener = useRef(null);
  function close() { setOpen(false); opener.current.focus(); }   // return focus to the opener
  return (
    <div className="h-40 text-sm">
      <button ref={opener} onClick={() => setOpen(true)} className="rounded bg-indigo-600 px-3 py-1
text-white">Rename batch</button>
      {open && (
        <Dialog onClose={close}>
          <label className="block">New name <input className="rounded border px-2 py-1"
defaultValue="React Nov 2026" /></label>
          <div className="mt-2 flex gap-2"><button onClick={close} className="rounded bg-slate-200 px-
2">Cancel</button><button onClick={close} className="rounded bg-indigo-600 px-2 text-
white">Save</button></div>
        </Dialog>
      )}
      <p className="mt-2 text-xs text-slate-500">focused element: <span id="f" /></p>
    </div>
  );
}
```



OPENED THE DIALOG (FOCUS WENT TO ITS INPUT) AND PRESSED TAB TWICE: FOCUS MOVED BETWEEN THE DIALOG CONTROLS WHILE THE PAGE BEHIND IS INERT

19.5.3 ARIA attributes and accessible component libraries

ARIA attributes add meaning that HTML alone cannot express: `aria-expanded` on a disclosure button, `aria-selected` on tabs, `aria-live="polite"` on a region whose updates should be announced, `aria-label` for icon-only buttons, `aria-current="page"` on the active link. The first rule of ARIA: do not use it when a native element already does the job. Complex widgets (combobox, menu, date picker, dialog) have detailed keyboard and ARIA requirements, so use a tested, accessible library rather than building them from scratch.

Library	Style	Notes
Radix Primitives	headless, unstyled	dialogs, menus, tabs, popovers; the base of shadcn/ui
React Aria (Adobe)	hooks and unstyled components	very thorough accessibility and internationalisation
Headless UI	unstyled, made for Tailwind	menus, listboxes, dialogs, comboboxes
MUI, **Chakra UI**, **Mantine**	fully styled	large component sets with accessibility built in

TIP

Test with the keyboard alone (Tab, Shift+Tab, Enter, Space, Escape, arrows), run the Lighthouse or axe DevTools accessibility audit, keep `eslint-plugin-jsx-a11y` on, and try a screen reader (NVDA on Windows, VoiceOver on macOS) on your main flows.

19.6 Internationalisation (overview)

19.6.1 react-i18next

Internationalisation (i18n) means preparing the app for several languages and regions: no hard-coded text, translations in resource files, plurals and interpolation handled per language, and dates, numbers and currencies formatted for the user's locale. **react-i18next** is the most widely used library: you call `t("key")` from the `useTranslation` hook, and switch languages with `i18n.changeLanguage("hi")`. Number and date formatting is built into JavaScript through `Intl`, shown running below.

REACT-I18NEXT (REPRESENTATIVE) JavaScript

```
// i18n.js
import i18n from "i18next";
import { initReactI18next } from "react-i18next";

i18n.use(initReactI18next).init({
  lng: "en",
  fallbackLng: "en",
  resources: {
    en: { translation: { welcome: "Welcome, {{name}}!", classes_one: "{{count}} class today",
classes_other: "{{count}} classes today" } },
    hi: { translation: { welcome: "स्वागत है, {{name}}!", classes_one: "आज {{count}} कक्षा", classes_other:
"आज {{count}} कक्षाएँ" } },
  },
});

// a component
import { useTranslation } from "react-i18next";
function Greeting() {
  const { t, i18n } = useTranslation();
  return (
    <>
      <h1>{t("welcome", { name: "Asha" })}</h1>
      <p>{t("classes", { count: 3 })}</p>                {/* picks classes_other */}
      <button onClick={() => i18n.changeLanguage("hi")}>हिन्दी</button>
    </>
  );
}
```

INTL-DEMO.JS (NODE.JS) main.js

```
const amount = 2450000;
const date = new Date("2026-11-30T10:00:00Z");
for (const locale of ["en-IN", "en-US", "de-DE", "hi-IN"]) {
  const money = new Intl.NumberFormat(locale, { style: "currency", currency: "INR" }).format(amount);
  const when = new Intl.DateTimeFormat(locale, { dateStyle: "medium", timeZone: "Asia/Kolkata"
}).format(date);
  console.log(locale.padEnd(6), money.padEnd(18), when);
}
const plural = new Intl.PluralRules("en-IN");
console.log("1 ->", plural.select(1), "| 3 ->", plural.select(3));
```

OUTPUT

```
en-IN  ₹24,50,000.00      30 Nov 2026
en-US  ₹2,450,000.00      Nov 30, 2026
de-DE  2.450.000,00 ₹     30.11.2026
hi-IN  ₹24,50,000.00      30 नव॰ 2026
1 -> one | 3 -> other
```

PRACTICE CHECKPOINT

Build a reusable compound Tabs component, a portal-based Modal with focus trapping, and add an error boundary around a crashing widget.

The sample solution reuses the compound `Tabs` from Section 19.1.3 and the `ErrorBoundary` class from Section 19.2.1, and adds a `Modal` rendered through a portal with a hand-written focus trap (Tab and Shift+Tab wrap around, Escape closes, focus returns to the opener).

MODAL.JSX JavaScript

```
import { useEffect, useRef } from "react";
import { createPortal } from "react-dom";

const FOCUSABLE = 'button, [href], input, select, textarea, [tabindex]:not([tabindex="-1"])';

export function Modal({ title, onClose, children }) {
  const panel = useRef(null);
  useEffect(() => {
    const opener = document.activeElement;                    // remember who opened us
    const nodes = () => [...panel.current.querySelectorAll(FOCUSABLE)];
    nodes()[0]?.focus();
    function onKey(e) {
      if (e.key === "Escape") onClose();
      if (e.key !== "Tab") return;
      const list = nodes(), first = list[0], last = list[list.length - 1];
      if (e.shiftKey && document.activeElement === first) { e.preventDefault(); last.focus(); }
      else if (!e.shiftKey && document.activeElement === last) { e.preventDefault(); first.focus(); }
    }
    document.addEventListener("keydown", onKey);
    return () => { document.removeEventListener("keydown", onKey); opener?.focus(); };
  }, [onClose]);

  return createPortal(
    <div className="fixed inset-0 grid place-items-center bg-black/40">
      <div ref={panel} role="dialog" aria-modal="true" aria-labelledby="modal-title" className="w-72
rounded-lg bg-white p-4 shadow-xl">
        <h2 id="modal-title" className="font-semibold">{title}</h2>
        {children}
      </div>
    </div>,
    document.body
  );
}
```

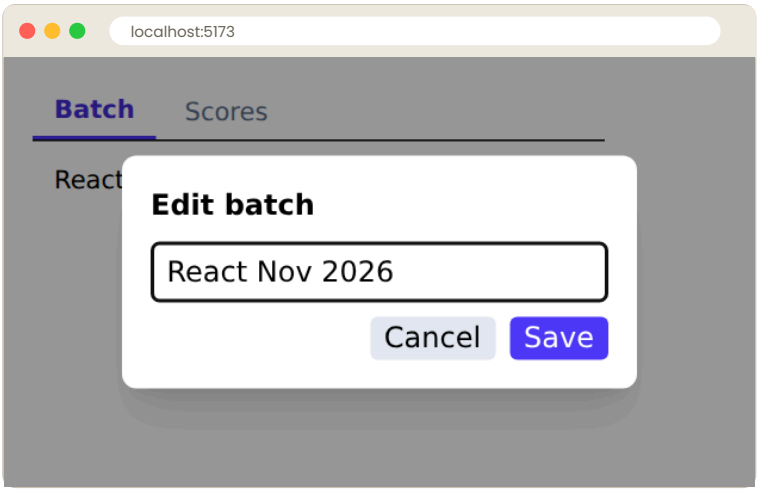
APP.JSX: SAMPLE SOLUTION

JavaScript

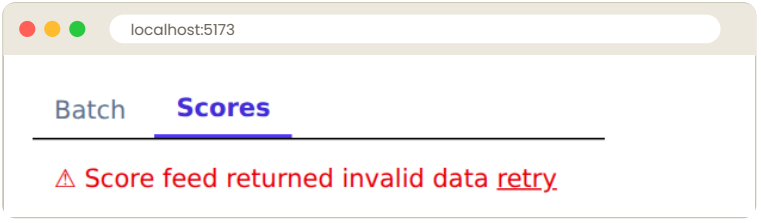
```
import { useCallback, useState } from "react";
import { Tabs } from "./Tabs.jsx";
import { Modal } from "./Modal.jsx";
import { ErrorBoundary } from "./ErrorBoundary.jsx";

function LiveScores() {
  const [n, setN] = useState(0);
  if (n > 1) throw new Error("Score feed returned invalid data");
  return <p>Live quiz scores: 82, 91, 77 <button className="text-xs underline" onClick={() => setN(n +
1)}>refresh ({n})</button></p>;
}

export default function App() {
  const [open, setOpen] = useState(false);
  const close = useCallback(() => setOpen(false), []);
  return (
    <div className="h-56 space-y-2 text-sm">
      <Tabs defaultValue="batch">
        <Tabs.List>
          <Tabs.Tab value="batch">Batch</Tabs.Tab>
          <Tabs.Tab value="scores">Scores</Tabs.Tab>
        </Tabs.List>
        <Tabs.Panel value="batch">
          React Nov 2026 · 24 students <button className="ml-2 text-indigo-600" onClick={() =>
setOpen(true)}>Edit</button>
        </Tabs.Panel>
        <Tabs.Panel value="scores">
          <ErrorBoundary fallback={({ error, reset }) => <p className="text-red-600">⚠ {error.message}
<button className="underline" onClick={reset}>retry</button></p>}>
            <LiveScores />
          </ErrorBoundary>
        </Tabs.Panel>
      </Tabs>
      {open && (
        <Modal title="Edit batch" onClose={close}>
          <input className="my-2 w-full rounded border px-2 py-1" defaultValue="React Nov 2026" />
          <div className="flex justify-end gap-2"><button onClick={close} className="rounded bg-slate-
200 px-2">Cancel</button><button onClick={close} className="rounded bg-indigo-600 px-2 text-
white">Save</button></div>
        </Modal>
      )}
    </div>
  );
}
```



EDIT OPENED THE PORTAL MODAL; THREE TAB PRESSES WRAPPED AROUND FROM SAVE BACK TO THE INPUT



ON THE SCORES TAB, REFRESHING TWICE CRASHED THE WIDGET: THE ERROR BOUNDARY CONTAINED IT INSIDE THE TAB

Compose components with children and slots instead of inheritance; keep presentational components separate from logic (custom hooks). Compound components share state through context; render props and HOCs pass behaviour around (prefer hooks today); good reusable inputs support controlled and uncontrolled use; headless hooks give behaviour without markup. Error boundaries (class components or react-error-boundary) contain rendering errors and log them. Portals render modals and toasts outside clipping containers. Organise big apps by feature, use the @ alias, VITE_ environment variables without secrets, and ESLint with Prettier. Build accessibility in with semantic HTML, useId, focus management and ARIA, and use Intl and react-i18next for internationalisation.

MODULE 20 · ADVANCED

Testing React applications

Minimum duration **2 days** | Level **Advanced**

Tests let you change code with confidence: they catch regressions before users do and document how components are meant to behave. This module explains unit, integration and end-to-end tests and what is worth testing; sets up Vitest and writes tests with describe, it, expect and mocks; tests components the way users use them with React Testing Library (queries by role, label and text, user-event, async UI with findBy and waitFor, forms, context and routing); mocks APIs with Mock Service Worker; tests custom hooks with renderHook; runs end-to-end tests with Playwright; and covers coverage reports and running tests in GitHub Actions.

IN THIS MODULE

20.1 Testing fundamentals
20.2 Vitest
20.3 React Testing Library
20.4 Mocking APIs
20.5 Testing hooks
20.6 End-to-end testing
20.7 Coverage and CI

HOW THE TESTS WERE RUN

Vitest, React Testing Library, user-event and MSW are not installed in the offline build. The test files are written exactly as for the real packages; they were run in Chromium by a small stand-in runner with the same API, which prints a report in Vitest's format (timings come from that run). The Playwright end-to-end tests in Section 20.6 were run with the **real** Playwright test runner (version 1.56).

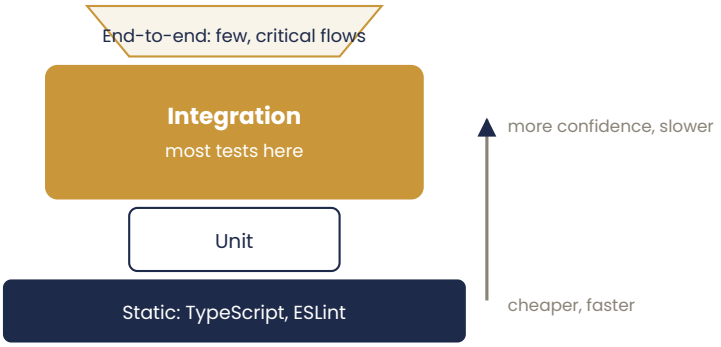
20.1 Testing fundamentals

20.1.1 Unit, integration and end-to-end tests

Kind	What it tests	Example	Speed	Tools
Static	types and code problems, without running anything	a prop of the wrong type, an unused variable	instant	TypeScript, ESLint
Unit	one function or one small component in isolation	`formatFee(24000)` returns "₹24,000"	milliseconds	Vitest
Integration	several units working together, as a user sees them	filling the student form adds a row to the list	tens of ms	Vitest + Testing Library + MSW
End-to-end (E2E)	the real app in a real browser, against a real or test back end	a student logs in, enrols and sees the course	seconds	Playwright, Cypress

20.1.2 The testing trophy and what to test

Kent C. Dodds' **testing trophy** suggests where to spend effort in front-end projects: a base of static analysis, some unit tests, **most effort on integration tests**, and a few end-to-end tests for the critical journeys. Integration tests give the most confidence per minute because they exercise real components together, the way users use them. The guiding principle of Testing Library: *"The more your tests resemble the way your software is used, the more confidence they can give you."*



THE TESTING TROPHY: SPEND MOST EFFORT ON INTEGRATION TESTS

Test this	Avoid testing this
what the user sees: text, roles, enabled/disabled, error messages	internal state variables and private functions
what happens when the user clicks, types, submits	which hooks a component uses, how many times it renders
the data sent to callbacks and APIs	CSS class names (unless they are the behaviour)
edge cases: empty list, loading, server error, invalid input	third-party libraries themselves

20.2 Vitest

20.2.1 Setup with Vite

Vitest is a test runner built on Vite: it reuses your Vite config (aliases, plugins, JSX), runs tests in parallel, re-runs only affected tests in watch mode, and offers a Jest-compatible API. Component tests need a browser-like DOM, provided by **jsdom** (or Vitest's real-browser mode).

Terminal

```
$ npm install -D vitest jsdom @testing-library/react @testing-library/dom @testing-library/user-event
@testing-library/jest-dom
```

VITE.CONFIG.JS, SETUP.JS AND SCRIPTS

JavaScript

```
// vite.config.js
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  test: {
    environment: "jsdom",              // a simulated browser DOM
    globals: true,                     // describe/it/expect without imports (optional)
    setupFiles: "./src/test/setup.js",
  },
});

// src/test/setup.js
import "@testing-library/jest-dom/vitest";   // adds toBeInTheDocument(), toHaveTextContent()...
import { afterEach } from "vitest";
import { cleanup } from "@testing-library/react";
afterEach(() => cleanup());                  // unmount after each test

// package.json
"scripts": { "test": "vitest", "test:run": "vitest run", "coverage": "vitest run --coverage" }
```

Test files sit next to the code they test and end in `.test.js` / `.test.jsx` (or `.spec`). `npm test` starts watch mode; `vitest run` runs once (for CI).

20.2.2 describe, it, expect

`describe` groups related tests; `it` (or `test`) defines one test with a sentence that states the expected behaviour; `expect(value)` followed by a **matcher** makes an assertion. A good test follows **Arrange, Act, Assert**: set up the input, run the code, check the result. Start with plain functions:

SRC/FORMAT.JS

JavaScript

```
export function formatFee(amount) {
  return "₹" + amount.toLocaleString("en-IN");
}

export function grade(score) {
  if (score < 0 || score > 100) throw new Error("score must be between 0 and 100");
  if (score >= 90) return "A";
  if (score >= 75) return "B";
  if (score >= 50) return "C";
  return "F";
}
```

SRC/FORMAT.TEST.JS

JavaScript

```
import { describe, it, expect } from "vitest";
import { formatFee, grade } from "./format.js";

describe("formatFee", () => {
  it("adds the rupee sign and Indian digit grouping", () => {
    expect(formatFee(240000)).toBe("₹2,40,000");
  });
});

describe("grade", () => {
  it("returns A for 90 and above", () => {
    expect(grade(90)).toBe("A");
    expect(grade(100)).toBe("A");
  });
  it("uses the boundaries 75 and 50", () => {
    expect([grade(75), grade(74), grade(50), grade(49)]).toEqual(["B", "C", "C", "F"]);
  });
  it("rejects impossible scores", () => {
    expect(() => grade(101)).toThrow("between 0 and 100");
  });
});
```

TERMINAL: npx vitest run

```
✓ src/format.test.js (4 tests) 20ms
   ✓ formatFee > adds the rupee sign and Indian digit grouping 20ms
   ✓ grade > returns A for 90 and above 0ms
   ✓ grade > uses the boundaries 75 and 50 0ms
   ✓ grade > rejects impossible scores 0ms

 Test Files  1 passed (1)
      Tests  4 passed (4)
   Duration  20ms
```

A failing test shows exactly what was expected and what was received. If someone changes the B boundary to 80 by mistake:

TERMINAL: npx vitest run (after a bug)

```
❯ src/format.test.js (4 tests | 1 failed) 14ms
   ✓ formatFee > adds the rupee sign and Indian digit grouping 14ms
   ✓ grade > returns A for 90 and above 0ms
   × grade > uses the boundaries 75 and 50 0ms
   ✓ grade > rejects impossible scores 0ms

 FAIL  src/format.test.js > grade > uses the boundaries 75 and 50
AssertionError: expected ["C","C","C","F"] to deeply equal ["B","C","C","F"]

 Test Files  1 failed (1)
      Tests  1 failed | 3 passed (4)
   Duration  14ms
```

Matcher	Checks
`toBe(x)`	same value (`Object.is`): numbers, strings, the same object
`toEqual(x)`	deep equality of objects and arrays
`toContain(x)` , `toHaveLength(n)`	arrays and strings
`toThrow(msg)`	the function throws (pass a function!)

Matcher	Checks
`toBeTruthy()`, `toBeNull()`	truthiness, null
`.not`	negates any matcher: `expect(x).not.toBe(0)`

20.2.3 Mocking functions and modules

A **mock function** (`vi.fn()`) records every call, so a test can check that a component called a callback with the right arguments; it can also return fixed values (`mockReturnValue`, `mockResolvedValue`). `vi.spyOn(object, "method")` wraps an existing method. `vi.mock("./module")` replaces a whole imported module, for example an analytics or API module, with fakes for the test file.

SRC/MOCKS.TEST.JS — JavaScript

```
import { describe, it, expect, vi } from "vitest";

function applyDiscount(price, getRate) {
  const rate = getRate("STUDENT");
  return Math.round(price * (1 - rate));
}

describe("mock functions", () => {
  it("records calls and returns a fixed value", () => {
    const getRate = vi.fn().mockReturnValue(0.2);
    expect(applyDiscount(24000, getRate)).toBe(19200);
    expect(getRate).toHaveBeenCalledTimes(1);
    expect(getRate).toHaveBeenCalledWith("STUDENT");
  });
  it("spies on an existing method", () => {
    const spy = vi.spyOn(console, "warn").mockImplementation(() => {});   // silence it
    console.warn("low attendance");
    expect(spy).toHaveBeenCalledWith("low attendance");
    spy.mockRestore();
  });
});
```

TERMINAL: npx vitest run

```
✓ src/mocks.test.js (2 tests) 0ms
   ✓ mock functions > records calls and returns a fixed value 0ms
   ✓ mock functions > spies on an existing method 0ms

 Test Files  1 passed (1)
      Tests  2 passed (2)
   Duration  1ms
```

MOCKING A WHOLE MODULE WITH VI.MOCK (REPRESENTATIVE) — JavaScript

```
import { vi, it, expect } from "vitest";
import { enrol } from "./enrol.js";
import * as analytics from "./analytics.js";

vi.mock("./analytics.js", () => ({ track: vi.fn() }));     // replaces the real module in this file

it("tracks the enrolment", async () => {
  await enrol({ course: "React" });
  expect(analytics.track).toHaveBeenCalledWith("enrolled", { course: "React" });
});
```

20.3 React Testing Library

20.3.1 Rendering components

React Testing Library (RTL) renders a component into a DOM container and gives you tools to find elements the way a user would: by what they **see and hear**, not by implementation details. `render(<Component />)` mounts it; the `screen` object queries the whole document. With the jest-dom matchers you can assert things like `toBeInTheDocument()`, `toHaveTextContent()`, `toBeDisabled()` and `toHaveValue()`.

SRC/COUNTER.JSX JavaScript

```
import { useState } from "react";

export function Counter({ initial = 0, max = 3 }) {
  const [count, setCount] = useState(initial);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)} disabled={count >= max}>Increment</button>
      {count >= max && <p role="alert">Maximum reached</p>}
    </div>
  );
}
```

SRC/COUNTER.TEST.JSX JavaScript

```
import { describe, it, expect } from "vitest";
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import { Counter } from "./Counter.jsx";

describe("Counter", () => {
  it("starts at the initial value", () => {
    render(<Counter initial={2} />);
    expect(screen.getByText("Count: 2")).toBeInTheDocument();
  });

  it("increments when the button is clicked", async () => {
    const user = userEvent.setup();
    render(<Counter />);
    await user.click(screen.getByRole("button", { name: "Increment" }));
    expect(screen.getByText("Count: 1")).toBeInTheDocument();
  });

  it("disables the button and warns at the maximum", async () => {
    const user = userEvent.setup();
    render(<Counter max={2} />);
    const button = screen.getByRole("button", { name: /increment/i });
    await user.click(button);
    await user.click(button);
    expect(button).toBeDisabled();
    expect(screen.getByRole("alert")).toHaveTextContent("Maximum reached");
  });
});
```

```
TERMINAL: npx vitest run
✓ src/Counter.test.jsx (3 tests) 32ms
   ✓ Counter > starts at the initial value 17ms
   ✓ Counter > increments when the button is clicked 11ms
   ✓ Counter > disables the button and warns at the maximum 4ms

 Test Files  1 passed (1)
      Tests  3 passed (3)
   Duration  33ms
```

20.3.2 Querying by role, label and text

Queries come in three families: `getBy...` returns the element or **throws** if there is not exactly one (use it when the element should be there); `queryBy...` returns `null` instead of throwing (use it to assert that something is **absent**); `findBy...` returns a promise and **waits** for the element to appear (for async UI). Each has an `All` version for several elements. Prefer queries in this order, because it matches how users and assistive technology find things:

Priority	Query	Finds	Example
1	`ByRole`	elements by ARIA role and accessible name	`getByRole("button", { name: "Save" })`
2	`ByLabelText`	form fields by their label	`getByLabelText("Email")`
3	`ByPlaceholderText`	fields by placeholder (only if there is no label)	`getByPlaceholderText("Search")`
4	`ByText`	non-interactive text	`getByText(/welcome/i)`
5	`ByDisplayValue`, `ByAltText`, `ByTitle`	current input values, images, titles	`getByAltText("Logo")`
last	`ByTestId`	`data-testid` attributes; when nothing else works	`getByTestId("chart")`

TIP

Queries by role double as an **accessibility check**: if `getByRole("button", { name: "Delete" })` cannot find your icon button, a screen reader cannot announce it either. Add an `aria-label` to fix both. `within(element)` limits queries to part of the page, for example one table row.

20.3.3 Simulating user events with user-event

`@testing-library/user-event` simulates complete interactions as a real user performs them: `user.type` focuses the field and fires keydown, keypress, input and keyup for each character; `user.click` fires pointer and mouse events and moves focus. Create it with `userEvent.setup()` at the start of each test and `await` every call. The lower-level `fireEvent` dispatches a single event and is only needed for unusual cases.

SRC/LOGINFORM.JSX JavaScript

```
import { useState } from "react";

export function LoginForm({ onSubmit }) {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  function handleSubmit(e) {
    e.preventDefault();
    if (!email.includes("@")) return setError("Enter a valid email");
    if (password.length < 8) return setError("Password must have at least 8 characters");
    setError("");
    onSubmit({ email, password });
  }
  return (
    <form onSubmit={handleSubmit}>
      <label htmlFor="email">Email</label>
      <input id="email" value={email} onChange={(e) => setEmail(e.target.value)} />
      <label htmlFor="password">Password</label>
      <input id="password" type="password" value={password} onChange={(e) =>
setPassword(e.target.value)} />
      {error && <p role="alert">{error}</p>}
      <button type="submit">Log in</button>
    </form>
  );
}
```

SRC/LOGINFORM.TEST.JSX

JavaScript

```
import { describe, it, expect, vi } from "vitest";
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import { LoginForm } from "./LoginForm.jsx";

describe("LoginForm", () => {
  it("submits email and password", async () => {
    const user = userEvent.setup();
    const onSubmit = vi.fn();
    render(<LoginForm onSubmit={onSubmit} />);

    await user.type(screen.getByLabelText("Email"), "asha@example.in");
    await user.type(screen.getByLabelText("Password"), "react2026");
    await user.click(screen.getByRole("button", { name: "Log in" }));

    expect(onSubmit).toHaveBeenCalledWith({ email: "asha@example.in", password: "react2026" });
  });

  it("shows an error and does not submit a short password", async () => {
    const user = userEvent.setup();
    const onSubmit = vi.fn();
    render(<LoginForm onSubmit={onSubmit} />);

    await user.type(screen.getByLabelText("Email"), "asha@example.in");
    await user.type(screen.getByLabelText("Password"), "abc{Enter}");    // Enter submits

    expect(screen.getByRole("alert")).toHaveTextContent(/at least 8 characters/);
    expect(onSubmit).not.toHaveBeenCalled();
  });

  it("starts without an error message", () => {
    render(<LoginForm onSubmit={() => {}} />);
    expect(screen.queryByRole("alert")).toBeNull();                    // queryBy for absence
  });
});
```

TERMINAL: npx vitest run

```
✓ src/LoginForm.test.jsx (3 tests) 77ms
   ✓ LoginForm > submits email and password 56ms
   ✓ LoginForm > shows an error and does not submit a short password 19ms
   ✓ LoginForm > starts without an error message 2ms

 Test Files  1 passed (1)
      Tests  3 passed (3)
   Duration  77ms
```

20.3.4 Testing async UI with findBy and waitFor

When the UI updates after a promise (a fetch, a timer, a transition), the element is not there immediately. `await screen.findByText(...)` retries the query until it succeeds or times out (1 second by default). `await waitFor(() => expect(...))` retries any assertion the same way, which is useful for things that are not elements, such as a mock having been called. Use `queryBy` with `waitFor` to wait for something to **disappear**, such as a spinner.

SRC/QUOTE.JSX

JavaScript

```
import { useEffect, useState } from "react";

export function Quote({ load }) {
  const [quote, setQuote] = useState(null);
  const [error, setError] = useState("");
  useEffect(() => { load().then(setQuote).catch((e) => setError(e.message)); }, [load]);
  if (error) return <p role="alert">Could not load: {error}</p>;
  if (!quote) return <p>Loading…</p>;
  return <blockquote>{quote}</blockquote>;
}
```

SRC/QUOTE.TEST.JSX

JavaScript

```
import { it, expect, vi } from "vitest";
import { render, screen, waitFor } from "@testing-library/react";
import { Quote } from "./Quote.jsx";

it("shows a loading message, then the quote", async () => {
  const load = vi.fn(() => new Promise((r) => setTimeout(() => r("Practice makes progress."), 50)));
  render(<Quote load={load} />);
  expect(screen.getByText("Loading…")).toBeInTheDocument();
  expect(await screen.findByText("Practice makes progress.")).toBeInTheDocument();   // waits
  expect(screen.queryByText("Loading…")).toBeNull();
});

it("shows an error when loading fails", async () => {
  render(<Quote load={vi.fn().mockRejectedValue(new Error("timeout"))} />);
  await waitFor(() => expect(screen.getByRole("alert")).toHaveTextContent("Could not load: timeout"));
});
```

TERMINAL: npx vitest run

```
✓ src/Quote.test.jsx (2 tests) 104ms
   ✓ shows a loading message, then the quote 79ms
   ✓ shows an error when loading fails 25ms

 Test Files  1 passed (1)
      Tests  2 passed (2)
   Duration  104ms
```

20.3.5 Testing forms, context and routing

Components that read **context** or use the **router** need their providers in the test too. Pass a `wrapper` to `render`, or write a custom `renderWithProviders` helper used by every test. For routing, `MemoryRouter` with `initialEntries` starts the test at any URL without a browser address bar.

SRC/THEME.JSX JavaScript

```
import { createContext, useContext, useState } from "react";

const ThemeContext = createContext(null);
export function ThemeProvider({ children, initial = "light" }) {
  const [theme, setTheme] = useState(initial);
  return <ThemeContext value={{ theme, toggle: () => setTheme((t) => (t === "light" ? "dark" :
"light")) }}>{children}</ThemeContext>;
}
export const useTheme = () => useContext(ThemeContext);

export function ThemeButton() {
  const { theme, toggle } = useTheme();
  return <button onClick={toggle}>Theme: {theme}</button>;
}
```

SRC/APPROUTES.JSX JavaScript

```
import { Routes, Route, Link, useParams } from "react-router";

function Course() { const { id } = useParams(); return <h1>Course {id}</h1>; }
export function AppRoutes() {
  return (
    <Routes>
      <Route path="/" element={<Link to="/courses/react">React course</Link>} />
      <Route path="/courses/:id" element={<Course />} />
      <Route path="*" element={<h1>Not found</h1>} />
    </Routes>
  );
}
```

SRC/PROVIDERS.TEST.JSX JavaScript

```
import { describe, it, expect } from "vitest";
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import { MemoryRouter } from "react-router";
import { ThemeProvider, ThemeButton } from "./theme.jsx";
import { AppRoutes } from "./AppRoutes.jsx";

describe("with providers", () => {
  it("toggles the theme from context", async () => {
    const user = userEvent.setup();
    render(<ThemeButton />, { wrapper: ({ children }) => <ThemeProvider initial="dark">{children}
</ThemeProvider> });
    await user.click(screen.getByRole("button", { name: "Theme: dark" }));
    expect(screen.getByRole("button")).toHaveTextContent("Theme: light");
  });

  it("navigates to a course page", async () => {
    const user = userEvent.setup();
    render(<MemoryRouter initialEntries={["/"]}><AppRoutes /></MemoryRouter>);
    await user.click(screen.getByRole("link", { name: "React course" }));
    expect(screen.getByRole("heading")).toHaveTextContent("Course react");
  });

  it("shows the 404 page for unknown URLs", () => {
    render(<MemoryRouter initialEntries={["/nope"]}><AppRoutes /></MemoryRouter>);
    expect(screen.getByRole("heading", { name: "Not found" })).toBeInTheDocument();
  });
});
```

```
TERMINAL: npx vitest run
✓ src/providers.test.jsx (3 tests) 37ms
   ✓ with providers > toggles the theme from context 30ms
   ✓ with providers > navigates to a course page 6ms
   ✓ with providers > shows the 404 page for unknown URLs 1ms

 Test Files  1 passed (1)
      Tests  3 passed (3)
   Duration  37ms
```

20.4 Mocking APIs

20.4.1 Mock Service Worker (MSW)

Mocking `fetch` by hand couples tests to how the code calls the API. **Mock Service Worker** intercepts requests at the **network level** instead: you describe handlers such as "GET /api/students returns this JSON", and your real code, with its real `fetch` or Axios calls, receives those responses. In tests MSW runs as `setupServer` (Node); in the browser the same handlers can run in a service worker (`setupWorker`) to develop the UI before the back end exists. `server.use(...)` overrides a handler for one test, for example to simulate a server error.

SRC/HANDLERS.JS

JavaScript

```javascript
import { http, HttpResponse } from "msw";

export const handlers = [
  http.get("/api/students", () =>
    HttpResponse.json([{ id: 1, name: "Asha Rao" }, { id: 2, name: "Ravi Kumar" }])),
  http.post("/api/students", async ({ request }) => {
    const body = await request.json();
    return HttpResponse.json({ id: 3, ...body }, { status: 201 });
  }),
];
```

SRC/STUDENTLIST.JSX

JavaScript

```
import { useEffect, useState } from "react";

export function StudentList() {
  const [students, setStudents] = useState([]);
  const [status, setStatus] = useState("loading");
  const [name, setName] = useState("");
  useEffect(() => {
    fetch("/api/students")
      .then((r) => { if (!r.ok) throw new Error(r.status); return r.json(); })
      .then((data) => { setStudents(data); setStatus("ready"); })
      .catch(() => setStatus("error"));
  }, []);
  async function add(e) {
    e.preventDefault();
    const res = await fetch("/api/students", { method: "POST", body: JSON.stringify({ name }) });
    const created = await res.json();
    setStudents((s) => [...s, created]);
    setName("");
  }
  if (status === "loading") return <p>Loading…</p>;
  if (status === "error") return <p role="alert">Could not load students</p>;
  return (
    <>
      <ul>{students.map((s) => <li key={s.id}>{s.name}</li>)}</ul>
      <form onSubmit={add}>
        <label htmlFor="n">Name</label><input id="n" value={name} onChange={(e) =>
setName(e.target.value)} />
        <button>Add</button>
      </form>
    </>
  );
}
```

SRC/STUDENTLIST.TEST.JSX

JavaScript

```
import { describe, it, expect, beforeAll, afterEach, afterAll } from "vitest";
import { render, screen, within } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import { setupServer } from "msw/node";
import { http, HttpResponse } from "msw";
import { handlers } from "./handlers.js";
import { StudentList } from "./StudentList.jsx";

const server = setupServer(...handlers);
beforeAll(() => server.listen());
afterEach(() => server.resetHandlers());          // remove per-test overrides
afterAll(() => server.close());

describe("StudentList", () => {
  it("loads students from the API", async () => {
    render(<StudentList />);
    const items = await screen.findAllByRole("listitem");
    expect(items).toHaveLength(2);
    expect(items[0]).toHaveTextContent("Asha Rao");
  });

  it("adds a student with POST", async () => {
    const user = userEvent.setup();
    render(<StudentList />);
    await user.type(await screen.findByLabelText("Name"), "Meena Iyer");
    await user.click(screen.getByRole("button", { name: "Add" }));
    const list = screen.getByRole("list");
    expect(await within(list).findByText("Meena Iyer")).toBeInTheDocument();
  });

  it("shows an error when the server fails", async () => {
    server.use(http.get("/api/students", () => HttpResponse.json({ message: "boom" }, { status: 500
})));
    render(<StudentList />);
    expect(await screen.findByRole("alert")).toHaveTextContent("Could not load students");
  });
});
```

TERMINAL: npx vitest run

```
✓ src/StudentList.test.jsx (3 tests) 125ms
   ✓ StudentList > loads students from the API 44ms
   ✓ StudentList > adds a student with POST 59ms
   ✓ StudentList > shows an error when the server fails 22ms

 Test Files  1 passed (1)
      Tests  3 passed (3)
   Duration  125ms
```

20.5 Testing hooks

20.5.1 renderHook

Custom hooks are usually tested through the components that use them. When a hook contains logic worth testing on its own, `renderHook(() => useMyHook(args))` calls it inside a tiny test component and gives you `result.current`, the latest return value. Wrap calls that update state in `act(...)` so React processes the update before you assert. `rerender(newProps)` re-runs the hook with new arguments.

SRC/USELOCALSTORAGE.JS JavaScript

```
import { useState } from "react";

export function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved !== null ? JSON.parse(saved) : initial;
  });
  function set(next) {
    const v = typeof next === "function" ? next(value) : next;
    setValue(v);
    localStorage.setItem(key, JSON.stringify(v));
  }
  return [value, set];
}
```

SRC/USELOCALSTORAGE.TEST.JS JavaScript

```
import { describe, it, expect, beforeEach } from "vitest";
import { renderHook } from "@testing-library/react";
import { act } from "react";
import { useLocalStorage } from "./useLocalStorage.js";

describe("useLocalStorage", () => {
  beforeEach(() => localStorage.clear());

  it("returns the initial value when nothing is saved", () => {
    const { result } = renderHook(() => useLocalStorage("theme", "light"));
    expect(result.current[0]).toBe("light");
  });

  it("saves updates to localStorage", () => {
    const { result } = renderHook(() => useLocalStorage("theme", "light"));
    act(() => result.current[1]("dark"));
    expect(result.current[0]).toBe("dark");
    expect(localStorage.getItem("theme")).toBe('"dark"');
  });

  it("reads a value saved earlier", () => {
    localStorage.setItem("cart", JSON.stringify(["React"]));
    const { result } = renderHook(() => useLocalStorage("cart", []));
    expect(result.current[0]).toEqual(["React"]);
  });
});
```

TERMINAL: npx vitest run

```
✓ src/useLocalStorage.test.js (3 tests) 16ms
   ✓ useLocalStorage > returns the initial value when nothing is saved 13ms
   ✓ useLocalStorage > saves updates to localStorage 2ms
   ✓ useLocalStorage > reads a value saved earlier 1ms

 Test Files  1 passed (1)
      Tests  3 passed (3)
   Duration  16ms
```

20.6 End-to-end testing

20.6.1 Playwright or Cypress

End-to-end tests drive the **real application in a real browser**: they catch problems that component tests cannot (routing, real network calls, the build itself, browser differences). They are slower and more fragile, so keep them for the critical journeys: sign-up, login, enrolment, checkout. The two leading tools:

	Playwright	Cypress
Browsers	Chromium, Firefox, WebKit (Safari engine)	Chromium family, Firefox (WebKit experimental)
Language	JavaScript/TypeScript, Python, Java, .NET	JavaScript/TypeScript
Model	runs outside the browser; parallel by default; multiple tabs and origins	runs inside the browser; excellent interactive runner
Waiting	auto-waits for elements and assertions	auto-retries commands and assertions
Extras	codegen recorder, trace viewer, API testing	time-travel debugging UI, component testing

Playwright's locators use the same ideas as Testing Library (`getByRole`, `getByLabel`, `getByText`), and its `expect` assertions wait automatically. The test below was run with the real Playwright runner against the enrolment app built from the React code shown:

SRC/APP.JSX (THE APP UNDER TEST) JavaScript

```
import { useState } from "react";

export default function App() {
  const [step, setStep] = useState("form");
  const [data, setData] = useState({ name: "", course: "React" });
  if (step === "done") return <h1 className="text-xl">Thanks {data.name}, you are enrolled in
{data.course}!</h1>;
  return (
    <form className="space-y-2" onSubmit={(e) => { e.preventDefault(); if (data.name.trim())
setStep("done"); }}>
      <h1 className="text-xl font-semibold">Enrol at LBF</h1>
      <label className="block">Full name <input className="rounded border px-2" value={data.name}
onChange={(e) => setData({ ...data, name: e.target.value })} /></label>
      <label className="block">Course
        <select className="rounded border" value={data.course} onChange={(e) => setData({ ...data,
course: e.target.value })}>
          <option>React</option><option>Java</option><option>Python</option>
        </select>
      </label>
      <button className="rounded bg-indigo-600 px-3 py-1 text-white">Enrol</button>
    </form>
  );
}
```

PLAYWRIGHT.CONFIG.JS JavaScript

```
import { defineConfig, devices } from "@playwright/test";

export default defineConfig({
  testDir: "./tests",
  reporter: "list",
  use: { baseURL: "http://localhost:4173", trace: "on-first-retry" },
  projects: [{ name: "chromium", use: { ...devices["Desktop Chrome"] } }],
  webServer: { command: "python3 -m http.server 4173 -d dist", url: "http://localhost:4173", stdout:
"ignore", stderr: "ignore" },
});
```

TESTS/ENROL.SPEC.JS JavaScript

```
import { test, expect } from "@playwright/test";

test.describe("enrolment", () => {
  test("a visitor enrols in the Java course", async ({ page }) => {
    await page.goto("/");
    await expect(page.getByRole("heading", { name: "Enrol at LBF" })).toBeVisible();
    await page.getByLabel("Full name").fill("Asha Rao");
    await page.getByLabel("Course").selectOption("Java");
    await page.getByRole("button", { name: "Enrol" }).click();
    await expect(page.getByRole("heading")).toHaveText("Thanks Asha Rao, you are enrolled in Java!");
  });

  test("an empty name does not submit", async ({ page }) => {
    await page.goto("/");
    await page.getByRole("button", { name: "Enrol" }).click();
    await expect(page.getByRole("heading", { name: "Enrol at LBF" })).toBeVisible();
  });
});
```

PLAYWRIGHT TEST RUN (REAL PLAYWRIGHT 1.56) main.js

```
$ npx playwright test
```

TERMINAL

```
$ npx playwright test
Running 2 tests using 1 worker
  ✓  1 [chromium] › tests/enrol.spec.js:4:3 › enrolment › a visitor enrols in the Java course (692ms)
  ✓  2 [chromium] › tests/enrol.spec.js:13:3 › enrolment › an empty name does not submit (414ms)
  2 passed (3.2s)
```

Useful Playwright commands

```
$ npm init playwright@latest           # installs @playwright/test, browsers and an example
$ npx playwright test --ui             # interactive mode with time travel
$ npx playwright codegen localhost:5173   # record a test by clicking through the app
$ npx playwright show-report           # open the HTML report
```

20.7 Coverage and CI

20.7.1 Coverage reports

Coverage measures which lines, branches and functions ran during the tests. `vitest run --coverage` (with the `@vitest/coverage-v8` package) prints a table and writes an HTML report where untested lines

are highlighted. Use it to **find gaps** (an error branch never tested), not as a goal in itself: 100% coverage does not mean the assertions are meaningful. Many teams set a sensible minimum threshold so coverage cannot silently drop.

COVERAGE CONFIGURATION JavaScript

```
// vite.config.js
test: {
  coverage: {
    provider: "v8",
    reporter: ["text", "html"],
    include: ["src/**/*.{js,jsx}"],
    exclude: ["src/**/*.test.*", "src/main.jsx"],
    thresholds: { lines: 80, branches: 75 },    // the run fails below these
  },
}
```

A TYPICAL COVERAGE TABLE (REPRESENTATIVE OUTPUT) main.js

```
$ npx vitest run --coverage
 % Coverage report from v8
-------------------|---------|----------|---------|---------|-------------------
File               | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-------------------|---------|----------|---------|---------|-------------------
All files          |   91.42 |    84.61 |   93.33 |   91.42 |
 Counter.jsx       |     100 |      100 |     100 |     100 |
 LoginForm.jsx     |     100 |      100 |     100 |     100 |
 StudentList.jsx   |   86.36 |       75 |   83.33 |   86.36 | 14-15
 format.js         |   83.33 |    77.77 |     100 |   83.33 | 8
-------------------|---------|----------|---------|---------|-------------------
```

20.7.2 Running tests in GitHub Actions

Continuous integration runs the checks on every push and pull request, so broken code cannot be merged unnoticed. A GitHub Actions workflow is a YAML file in `.github/workflows/`. The one below installs dependencies, then runs linting, type checking, unit and integration tests with coverage, builds the app, and runs the Playwright tests, uploading the report if they fail. Protect the main branch so that a pull request can only be merged when this workflow passes.

CI.YML

JavaScript

```yaml
# .github/workflows/ci.yml
name: CI
on:
  push: { branches: [main] }
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npx tsc --noEmit            # if the project uses TypeScript
      - run: npx vitest run --coverage
      - run: npm run build
      - run: npx playwright install --with-deps chromium
      - run: npx playwright test
      - uses: actions/upload-artifact@v4
        if: failure()
        with: { name: playwright-report, path: playwright-report/ }
```

PRACTICE CHECKPOINT

Write component tests for the student form and list (with MSW-mocked APIs), a hook test for useLocalStorage, and one Playwright end-to-end test.

The hook test (Section 20.5) and the Playwright test (Section 20.6) already cover two parts. The sample solution below adds a student form with validation and a page that combines it with the API-backed list, plus integration tests for the whole flow with MSW.

SRC/STUDENTFORM.JSX JavaScript

```
import { useState } from "react";

export function StudentForm({ onAdd }) {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [error, setError] = useState("");
  const [saving, setSaving] = useState(false);
  async function submit(e) {
    e.preventDefault();
    if (name.trim().length < 3) return setError("Name needs at least 3 characters");
    if (!/^\S+@\S+\.\S+$/.test(email)) return setError("Enter a valid email");
    setError("");
    setSaving(true);
    await onAdd({ name: name.trim(), email });
    setSaving(false);
    setName(""); setEmail("");
  }
  return (
    <form onSubmit={submit} aria-label="Add student">
      <label htmlFor="name">Name</label><input id="name" value={name} onChange={(e) =>
setName(e.target.value)} />
      <label htmlFor="email">Email</label><input id="email" value={email} onChange={(e) =>
setEmail(e.target.value)} />
      {error && <p role="alert">{error}</p>}
      <button disabled={saving}>{saving ? "Saving…" : "Add student"}</button>
    </form>
  );
}
```

SRC/STUDENTSPAGE.JSX JavaScript

```
import { useEffect, useState } from "react";
import { StudentForm } from "./StudentForm.jsx";

export function StudentsPage() {
  const [students, setStudents] = useState(null);
  const [error, setError] = useState("");
  useEffect(() => {
    fetch("/api/students").then((r) => (r.ok ? r.json() : Promise.reject(new Error(`HTTP
${r.status}`))))
      .then(setStudents).catch((e) => setError(e.message));
  }, []);
  async function add(student) {
    const res = await fetch("/api/students", { method: "POST", body: JSON.stringify(student) });
    if (!res.ok) return setError("Could not save");
    const created = await res.json();
    setStudents((s) => [...s, created]);
  }
  if (error) return <p role="alert">{error}</p>;
  if (!students) return <p>Loading…</p>;
  return (
    <main>
      <h1>Students ({students.length})</h1>
      <ul>{students.map((s) => <li key={s.id}>{s.name} · {s.email}</li>)}</ul>
      <StudentForm onAdd={add} />
    </main>
  );
}
```

SRC/HANDLERS.JS JavaScript

```
import { http, HttpResponse } from "msw";

let db;
export const resetDb = () => { db = [{ id: 1, name: "Asha Rao", email: "asha@example.in" }]; };
export const handlers = [
  http.get("/api/students", () => HttpResponse.json(db)),
  http.post("/api/students", async ({ request }) => {
    const s = { id: db.length + 1, ...(await request.json()) };
    db.push(s);
    return HttpResponse.json(s, { status: 201 });
  }),
];
```

SRC/STUDENTS.TEST.JSX: SAMPLE SOLUTION JavaScript

```
import { describe, it, expect, vi, beforeAll, beforeEach, afterEach, afterAll } from "vitest";
import { render, screen, within } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import { setupServer } from "msw/node";
import { http, HttpResponse } from "msw";
import { handlers, resetDb } from "./handlers.js";
import { StudentForm } from "./StudentForm.jsx";
import { StudentsPage } from "./StudentsPage.jsx";

const server = setupServer(...handlers);
beforeAll(() => server.listen());
beforeEach(() => resetDb());
afterEach(() => server.resetHandlers());
afterAll(() => server.close());

describe("StudentForm", () => {
  it("validates the name and the email", async () => {
    const user = userEvent.setup();
    const onAdd = vi.fn();
    render(<StudentForm onAdd={onAdd} />);
    await user.type(screen.getByLabelText("Name"), "Jo");
    await user.click(screen.getByRole("button", { name: "Add student" }));
    expect(screen.getByRole("alert")).toHaveTextContent("at least 3 characters");
    await user.type(screen.getByLabelText("Name"), "hn");
    await user.type(screen.getByLabelText("Email"), "john@");
    await user.click(screen.getByRole("button", { name: "Add student" }));
    expect(screen.getByRole("alert")).toHaveTextContent("valid email");
    expect(onAdd).not.toHaveBeenCalled();
  });

  it("sends trimmed data and clears the fields", async () => {
    const user = userEvent.setup();
    const onAdd = vi.fn().mockResolvedValue(undefined);
    render(<StudentForm onAdd={onAdd} />);
    await user.type(screen.getByLabelText("Name"), "  Meena Iyer ");
    await user.type(screen.getByLabelText("Email"), "meena@example.in");
    await user.click(screen.getByRole("button", { name: "Add student" }));
    expect(onAdd).toHaveBeenCalledWith({ name: "Meena Iyer", email: "meena@example.in" });
    expect(await screen.findByLabelText("Name")).toHaveValue("");
  });
});

describe("StudentsPage (integration with MSW)", () => {
  it("lists students and adds a new one through the API", async () => {
    const user = userEvent.setup();
    render(<StudentsPage />);
```

```
    expect(await screen.findByRole("heading", { name: "Students (1)" })).toBeInTheDocument();
    await user.type(screen.getByLabelText("Name"), "Ravi Kumar");
    await user.type(screen.getByLabelText("Email"), "ravi@example.in");
    await user.click(screen.getByRole("button", { name: "Add student" }));
    expect(await screen.findByRole("heading", { name: "Students (2)" })).toBeInTheDocument();
    expect(within(screen.getByRole("list")).getByText("Ravi Kumar ·
ravi@example.in")).toBeInTheDocument();
  });

  it("shows the server error", async () => {
    server.use(http.get("/api/students", () => HttpResponse.json({}, { status: 503 })));
    render(<StudentsPage />);
    expect(await screen.findByRole("alert")).toHaveTextContent("HTTP 503");
  });
});
```

TERMINAL: npx vitest run (component, integration and hook tests)

```
✓ src/students.test.jsx (4 tests) 171ms
   ✓ StudentForm > validates the name and the email 49ms
   ✓ StudentForm > sends trimmed data and clears the fields 26ms
   ✓ StudentsPage (integration with MSW) > lists students and adds a new one through the API 73ms
   ✓ StudentsPage (integration with MSW) > shows the server error 23ms
✓ src/useLocalStorage.test.js (3 tests) 2ms
   ✓ useLocalStorage > returns the initial value when nothing is saved 1ms
   ✓ useLocalStorage > saves updates to localStorage 1ms
   ✓ useLocalStorage > reads a value saved earlier 0ms

 Test Files  2 passed (2)
      Tests  7 passed (7)
   Duration  173ms
```

Together with the Playwright run in Section 20.6, the project now has the full trophy: static checks, unit tests, integration tests with a mocked API, and end-to-end tests of a real build.

Spend most effort on integration tests that use components the way users do, add unit tests for pure logic and a few end-to-end tests for critical journeys. Vitest runs tests with `describe` / `it` / `expect` and mocks with `vi.fn`, `vi.spyOn` and `vi.mock`. React Testing Library renders components and queries them by role, label and text (`getBy`, `queryBy` for absence, `findBy` and `waitFor` for async UI); user-event simulates real typing and clicking; wrappers provide context and `MemoryRouter`. MSW mocks APIs at the network level; `renderHook` tests custom hooks. Playwright tests the real app in real browsers. Track coverage to find gaps and run everything in CI on every pull request.

MODULE 21 · ADVANCED

Full-stack React with Next.js

Minimum duration **4 days** Level **Advanced**

Next.js is the most widely used React framework. It adds file-based routing, server rendering and static generation, React Server Components, Server Actions, API routes, image and font optimisation and a deployment story, turning React into a full-stack tool. This module explains rendering strategies (CSR, SSR, SSG, ISR) and their SEO and performance benefits; the App Router with layouts, dynamic routes, loading and error files and the Metadata API; Server and Client Components, data fetching, streaming and caching; Server Actions for mutations; route handlers, proxy (middleware), databases and authentication; deployment; and other React frameworks.

IN THIS MODULE

21.1 Why a framework
21.2 Next.js App Router
21.3 React Server Components
21.4 Server Functions
21.5 Back-end features
21.6 Deploying Next.js
21.7 Other React frameworks (overview)

HOW THE EXAMPLES WERE RUN

Next.js is not installed in the offline build, so the Next.js listings are representative (written for Next.js 16 with the App Router). The rendering and streaming demonstrations in Sections 21.1 and 21.3 are real: they use React 19.2's own server renderer (`react-dom/server`) in Node.js 22, which is the same machinery Next.js uses underneath.

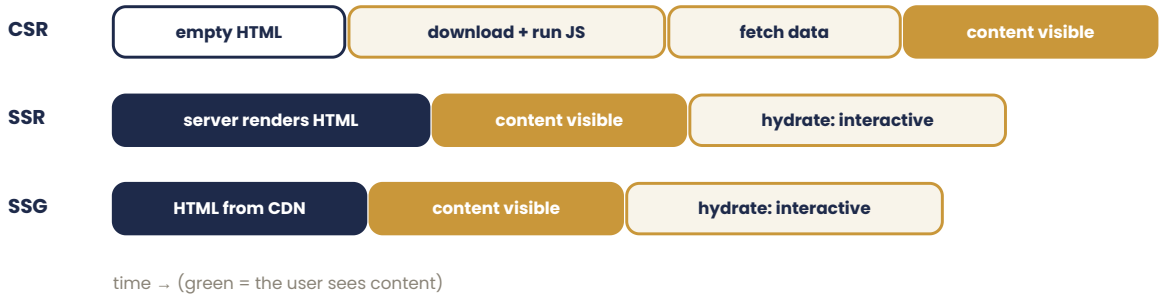
21.1 Why a framework

21.1.1 Rendering strategies: CSR, SSR, SSG, ISR

A Vite single-page app uses **client-side rendering (CSR)**: the server sends an almost empty HTML page and a JavaScript bundle, and the browser builds the whole UI. That is simple and great for dashboards behind a login, but the first paint waits for the JavaScript, and crawlers and link previews see an empty page. Frameworks can render React **on the server** instead, in three ways:

Strategy	HTML produced	Data	Best for
CSR (client-side rendering)	in the browser, after the JS loads	fetched by the browser	apps behind a login, highly interactive tools
SSR (server-side rendering)	on the server, for every request	fresh on every request	personalised or frequently changing pages (a dashboard, search results)
SSG (static site generation)	once, at build time	as of the last build	pages that rarely change (marketing, docs, blog)

Strategy	HTML produced	Data	Best for
ISR (incremental static regeneration)	at build time, then re-generated in the background	refreshed every N seconds or on demand	large catalogues: product and course pages



SERVER RENDERING SHOWS CONTENT BEFORE THE JAVASCRIPT HAS FINISHED

Hydration is the step that makes server HTML interactive: the browser downloads the same components, React walks the existing HTML and attaches event handlers instead of re-creating it. Here is the difference, produced by React's real server renderer:

SERVER.JSX (NODE.JS, REACT-DOM/SERVER) JavaScript

```
import { renderToString } from "react-dom/server";

function CoursePage({ course }) {
  return (
    <main>
      <h1>{course.title}</h1>
      <p>{course.weeks} weeks · ₹{course.fee.toLocaleString("en-IN")}</p>
      <button>Enrol</button>
    </main>
  );
}

const course = { title: "React", weeks: 10, fee: 24000 };

console.log("CSR: what the server sends");
console.log('<body><div id="root"></div><script src="/assets/app.js"></script></body>');
console.log();
console.log("SSR/SSG: what the server sends");
console.log(`<body><div id="root">${renderToString(<CoursePage course={course} />)}</div><script
src="/assets/app.js"></script></body>`);
```

EXAMPLE main.js

```
$ node server.jsx
```

OUTPUT

```
CSR: what the server sends
<body><div id="root"></div><script src="/assets/app.js"></script></body>

SSR/SSG: what the server sends
<body><div id="root"><main><h1>React</h1><p>10<!-- --> weeks · ₹<!-- -->24,000</p>
<button>Enrol</button></main></div><script src="/assets/app.js"></script></body>
```

21.1.2 SEO and performance benefits

- **SEO and sharing:** search engines and social previews (WhatsApp, LinkedIn, X) read the HTML. With server rendering the title, description and content are in the first response; Google can run JavaScript, but later and less reliably, and most preview bots do not run it at all.
- **Faster first content (LCP):** the user sees the page before the JavaScript has downloaded, which matters most on slow phones and networks.
- **Less JavaScript:** with Server Components, code that only renders (markdown parsers, date libraries, database access) stays on the server and is never sent to the browser.
- **Data close to the source:** server code can query the database directly, without an extra public API and without request waterfalls from the browser.
- **One project for front end and back end:** pages, API routes, server logic and deployment in one codebase.

21.2 Next.js App Router

21.2.1 Creating a Next.js app

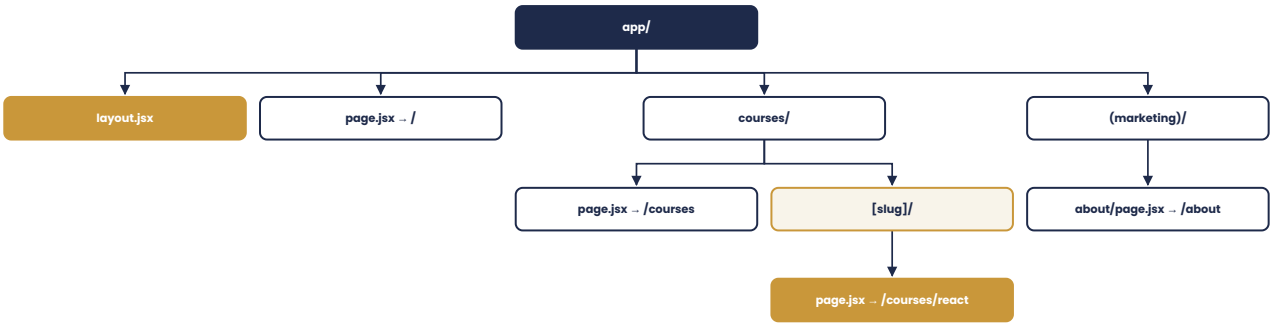
Terminal (typical output)

```
$ npx create-next-app@latest lbf-portal
✔ Would you like to use the recommended Next.js defaults? › Yes (TypeScript, ESLint, Tailwind CSS, App
Router, Turbopack)
$ cd lbf-portal && npm run dev
   ▲ Next.js 16
   - Local:   http://localhost:3000

lbf-portal/
├── app/
│   ├── layout.tsx         root layout: <html> and <body>
│   ├── page.tsx           the page at "/"
│   └── globals.css
├── public/                static files served as-is
├── next.config.ts
└── package.json           "dev": "next dev", "build": "next build", "start": "next start"
```

21.2.2 File-based routing: pages, layouts, nested and dynamic routes

In the App Router, **folders** inside `app/` define URL segments and a `page.jsx` file makes a segment publicly reachable. A `layout.jsx` wraps all pages below it and **stays mounted** while the user navigates between them (state and scroll are kept). A folder name in brackets creates a **dynamic segment**: `app/courses/[slug]/page.jsx` matches `/courses/react`, `/courses/java` and so on. Folders in parentheses, `(marketing)`, group routes without affecting the URL.



FOLDERS DEFINE THE URL; PAGE.JSX MAKES IT REACHABLE; LAYOUT.JSX WRAPS EVERYTHING BELOW

LAYOUTS AND A DYNAMIC ROUTE (REPRESENTATIVE) JavaScript

```
// app/layout.jsx: the root layout (required)
export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        <header>Learn Being Forward</header>
        {children}                    {/* the current page or nested layout */}
      </body>
    </html>
  );
}

// app/courses/[slug]/page.jsx: a dynamic route (params is a promise in Next.js 15+)
export default async function CoursePage({ params }) {
  const { slug } = await params;
  const course = await getCourse(slug);            // runs on the server
  return <h1>{course.title}</h1>;
}

// pre-render known slugs at build time (SSG)
export async function generateStaticParams() {
  return [{ slug: "react" }, { slug: "java" }, { slug: "python" }];
}
```

21.2.3 Loading and error UI files

Special files in a route folder give each segment its own states, implemented with React's Suspense and error boundaries under the hood: `loading.jsx` is shown instantly while the page's data loads (it wraps the page in `<Suspense>`); `error.jsx` is shown if rendering throws (it must be a Client Component and receives `error` and `reset`); `not-found.jsx` is rendered when you call `notFound()`. Layouts stay visible in all three cases.

LOADING.JSX, ERROR.JSX AND NOTFOUND() (REPRESENTATIVE) JavaScript

```
// app/courses/loading.jsx
export default function Loading() {
  return <p className="animate-pulse">Loading courses…</p>;
}

// app/courses/error.jsx
"use client";
export default function Error({ error, reset }) {
  return (
    <div>
      <p>Could not load courses.</p>
      <button onClick={() => reset()}>Try again</button>
    </div>
  );
}

// app/courses/[slug]/page.jsx
import { notFound } from "next/navigation";
export default async function CoursePage({ params }) {
  const course = await getCourse((await params).slug);
  if (!course) notFound();                  // renders not-found.jsx
  return <h1>{course.title}</h1>;
}
```

21.2.4 Link, Image and Font optimisation

`<Link href>` from `next/link` navigates on the client and **prefetches** the target route when the link appears on screen. `<Image>` from `next/image` serves resized, modern-format images (WebP/AVIF) per device, lazy-loads them and reserves their space to prevent layout shift. `next/font` downloads Google or local fonts **at build time** and self-hosts them, so there is no request to an external font server and no layout shift when the font loads.

NEXT/LINK, NEXT/IMAGE AND NEXT/FONT (REPRESENTATIVE) JavaScript

```
import Link from "next/link";
import Image from "next/image";
import { Poppins } from "next/font/google";

const poppins = Poppins({ subsets: ["latin"], weight: ["400", "600"] });   // self-hosted at build
time

export default function Home() {
  return (
    <main className={poppins.className}>
      <Image src="/campus.jpg" alt="LBF campus" width={1200} height={600} priority />   {/* hero: load
early */}
      <Link href="/courses/react">React course</Link>                                    {/*
prefetched */}
    </main>
  );
}
```

21.2.5 Metadata API for SEO

Export a `metadata` object (static) or a `generateMetadata` function (dynamic, it can fetch data) from a layout or page. Next.js turns it into `<title>`, `<meta name="description">`, Open Graph and Twitter tags, canonical links and more, merged from the root layout down. Files named `sitemap.js`, `robots.js` and `opengraph-image.jsx` generate those resources too.

STATIC AND DYNAMIC METADATA (REPRESENTATIVE) JavaScript

```
// app/layout.jsx
export const metadata = {
  title: { default: "Learn Being Forward", template: "%s | Learn Being Forward" },
  description: "Job-ready courses in Java, Python and React.",
};

// app/courses/[slug]/page.jsx
export async function generateMetadata({ params }) {
  const course = await getCourse((await params).slug);
  return {
    title: course.title,                                  // -> "React | Learn Being Forward"
    description: course.summary,
    openGraph: { images: [course.coverImage] },
  };
}
```

21.3 React Server Components

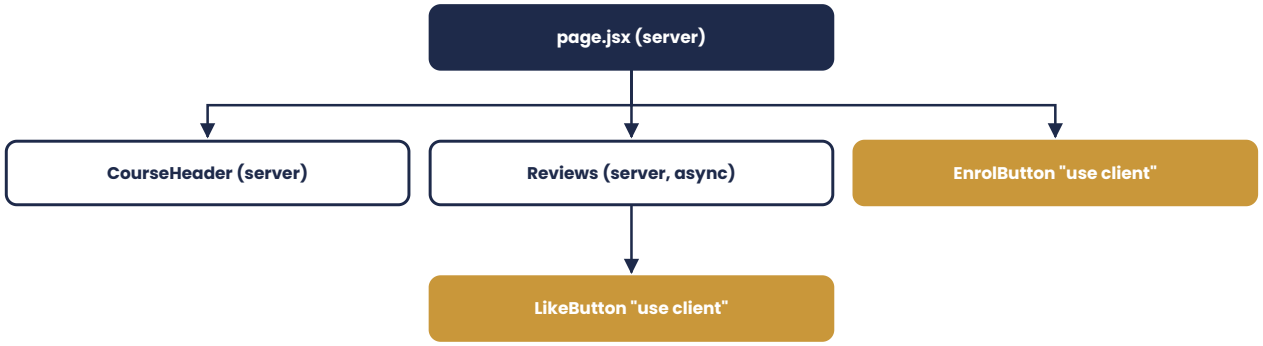
21.3.1 Server vs Client Components

In the App Router every component is a **Server Component** by default. Server Components run **only on the server** (at build time or per request): they can be `async`, read databases and files directly, use secrets, and their code is **never sent to the browser**; only their rendered result is. But they cannot use state, effects, event handlers or browser APIs. Components that need interactivity are **Client Components**: they are rendered on the server for the first HTML too, then hydrated and run in the browser like ordinary React components.

Need	Server Component	Client Component
fetch data, query a database, read files	✓ (async/await directly)	via an API or Server Action
use secrets (API keys, DB passwords)	✓	✗ never
keep big dependencies off the client	✓	✗ they ship to the browser
useState, useEffect, useReducer, custom hooks with state	✗	✓
onClick, onChange and other event handlers	✗	✓
browser APIs: window, localStorage, geolocation	✗	✓

21.3.2 "use client" boundary

Put `"use client"` at the top of a file to mark it, **and everything it imports**, as client code. That line is a **boundary** in the module graph, so place it as low (as close to the leaves) as possible: keep pages and layouts on the server and make only the interactive pieces (a like button, a search box, a cart) client components. A Server Component can render a Client Component and pass it serialisable props (strings, numbers, plain objects, even promises) or server-rendered elements as `children`.



ONLY THE GOLD LEAVES (AND WHAT THEY IMPORT) ARE SENT TO THE BROWSER AS JAVASCRIPT

A SERVER COMPONENT RENDERING A CLIENT COMPONENT (REPRESENTATIVE)

JavaScript

```
// app/courses/[slug]/EnrolButton.jsx
"use client";                                  // this file runs in the browser too
import { useState } from "react";

export function EnrolButton({ courseId }) {
  const [done, setDone] = useState(false);
  return <button onClick={() => setDone(true)}>{done ? "Enrolled ✓" : "Enrol"}</button>;
}

// app/courses/[slug]/page.jsx: a Server Component (no directive)
import { EnrolButton } from "./EnrolButton";
import { db } from "@/lib/db";

export default async function CoursePage({ params }) {
  const course = await db.course.findUnique({ where: { slug: (await params).slug } });   // direct DB
access
  return (
    <main>
      <h1>{course.title}</h1>
      <EnrolButton courseId={course.id} />       {/* serialisable props only */}
    </main>
  );
}
```

21.3.3 Fetching data on the server

Server Components fetch data with plain `async` / `await`, with no `useEffect`, loading flags or API layer. Start independent requests together with `Promise.all` to avoid waterfalls; when the same data is needed in several components, wrap the function in React's `cache()` so it runs once per request. Treat everything a Server Component renders as public: only pass the fields the client needs.

PARALLEL DATA FETCHING IN A SERVER COMPONENT (REPRESENTATIVE) JavaScript

```
import { cache } from "react";

export const getStudent = cache(async (id) => {             // de-duplicated per request
  return db.student.findUnique({ where: { id } });
});

export default async function Dashboard({ params }) {
  const { id } = await params;
  const [student, courses, notices] = await Promise.all([   // in parallel, not one after another
    getStudent(id),
    db.enrolment.findMany({ where: { studentId: id } }),
    fetch("https://api.example.com/notices").then((r) => r.json()),
  ]);
  return <h1>Welcome {student.name}: {courses.length} courses, {notices.length} notices</h1>;
}
```

21.3.4 Streaming with Suspense

With SSR alone, the server must finish **all** data fetching before it sends any HTML, so one slow query delays the whole page. **Streaming** sends the HTML in pieces: the page shell and fast parts first, with Suspense fallbacks in place of slow parts, then each slow part (plus a tiny inline script that swaps it in) as soon as its data is ready, all in the same response. In Next.js a `loading.jsx` file or a `<Suspense>` boundary around a slow async component is all it takes. The demonstration below streams a page with React's real server renderer and prints each chunk with the time it left the server:

STREAM.JSX (NODE.JS, REACT-DOM/SERVER) JavaScript

```
import { Suspense } from "react";
import { renderToReadableStream } from "react-dom/server";

const wait = (ms) => new Promise((r) => setTimeout(r, ms));

async function Reviews() {                              // a slow async Server Component
  await wait(1200);
  return <ul><li>"Clear explanations" (5★)</li><li>"Great projects" (4★)</li></ul>;
}

function Page() {
  return (
    <html><body>
      <h1>React course</h1>
      <p>10 weeks · ₹24,000</p>
      <Suspense fallback={<p>Loading reviews…</p>}>
        <Reviews />
      </Suspense>
    </body></html>
  );
}

const start = Date.now();
const stream = await renderToReadableStream(<Page />);
const decoder = new TextDecoder();
for await (const chunk of stream) {
  const text = decoder.decode(chunk).replace(/<script>.*?<\/script>/gs, "<script>…swap code…
</script>");
  console.log(`--- chunk at ${Math.round((Date.now() - start) / 100) * 100} ms ---`);
  console.log(text.length > 420 ? text.slice(0, 420) + "…" : text);
}
```

EXAMPLE

main.js

```
$ node stream.jsx
```

OUTPUT

```
--- chunk at 0 ms ---
<!DOCTYPE html><html><head></head><body><h1>React course</h1><p>10 weeks · ₹24,000</p><!--$?-->
<template id="B:0"></template><p>Loading reviews…</p><!--/$--><script
id="_R_">requestAnimationFrame(function(){$RT=performance.now()});</script>
--- chunk at 1200 ms ---
<div hidden id="S:0"><ul><li>&quot;Clear explanations&quot; (5★)</li><li>&quot;Great projects&quot;
(4★)</li></ul></div><script>…swap code…</script>
--- chunk at 1200 ms ---
</body></html>
```

Times are rounded to 100 ms. The browser shows the first chunk immediately; the second chunk replaces the fallback when the reviews are ready.

21.3.5 Caching and revalidation

Rendering and fetching on the server for every request is expensive, so Next.js caches. In Next.js 16, with **Cache Components** enabled (`cacheComponents: true` in `next.config`), caching is explicit: pages are dynamic by default, and you mark a component or function with the `"use cache"` directive to cache its result, set its lifetime with `cacheLife`, and label it with `cacheTag`. After a change, a Server Action calls `updateTag(tag)` to expire the tagged data immediately (the user sees their own change), while `revalidateTag(tag, "max")` refreshes it in the background (stale-while-revalidate) and `revalidatePath(path)` refreshes one route (ISR on demand). Older App Router code controls caching through `fetch` options instead, such as `{ next: { revalidate: 3600 } }` or `{ cache: "no-store" }`.

CACHING WITH "USE CACHE" AND REVALIDATION (REPRESENTATIVE)

JavaScript

```
// next.config.ts
export default { cacheComponents: true };

// lib/courses.js
import { cacheLife, cacheTag } from "next/cache";

export async function getCourses() {
  "use cache";                      // cache the result of this function
  cacheLife("hours");               // keep it for about an hour
  cacheTag("courses");              // a label for targeted revalidation
  return db.course.findMany();
}

// after an admin edits a course (in a Server Action)
import { updateTag, revalidateTag } from "next/cache";
updateTag("courses");               // expire now: the next request gets fresh data
revalidateTag("courses", "max");    // or: serve stale data while refreshing in the background

// older style, still common
await fetch("https://api.example.com/courses", { next: { revalidate: 3600 } });  // ISR: refresh
hourly
await fetch("https://api.example.com/cart", { cache: "no-store" });              // always fresh
```

21.4 Server Functions

21.4.1 "use server" and Server Actions for mutations

A **Server Function** is an async function marked with `"use server"` (at the top of the function or of a whole file). It always runs on the server, yet a Client Component can import and call it like a normal function: React and Next.js turn the call into a POST request behind the scenes. Used for mutations, they are called **Server Actions**. Because they are public HTTP endpoints, **always validate the input and check the user's permissions inside them**.

A SERVER ACTION (REPRESENTATIVE) — JavaScript

```
// app/actions.js
"use server";                                          // every export runs on the server
import { z } from "zod";
import { revalidatePath } from "next/cache";
import { auth } from "@/auth";
import { db } from "@/lib/db";

const ReviewSchema = z.object({ courseId: z.string(), rating: z.coerce.number().min(1).max(5), text:
z.string().min(10) });

export async function addReview(prevState, formData) {
  const session = await auth();                        // who is calling?
  if (!session) return { error: "Please log in" };
  const parsed = ReviewSchema.safeParse(Object.fromEntries(formData));
  if (!parsed.success) return { error: parsed.error.issues[0].message };
  await db.review.create({ data: { ...parsed.data, userId: session.user.id } });
  revalidatePath(`/courses/${parsed.data.courseId}`);  // show the new review
  return { ok: true };
}
```

21.4.2 Forms with Server Actions

Pass a Server Action to a form's `action` prop. The form then works **even before JavaScript has loaded** (progressive enhancement): it submits as a normal HTML form to the server. Once hydrated, React submits it without a reload and you get all the React 19 form features from Module 17: `useActionState` for the returned state and pending flag, `useFormStatus` in the submit button, and `useOptimistic`.

A FORM CALLING A SERVER ACTION (REPRESENTATIVE) JavaScript

```
// app/courses/[slug]/ReviewForm.jsx
"use client";
import { useActionState } from "react";
import { addReview } from "@/app/actions";

export function ReviewForm({ courseId }) {
  const [state, formAction, isPending] = useActionState(addReview, {});
  return (
    <form action={formAction}>
      <input type="hidden" name="courseId" value={courseId} />
      <select name="rating" defaultValue="5">{[5, 4, 3, 2, 1].map((n) => <option key={n}>{n}
</option>)}</select>
      <textarea name="text" placeholder="What did you think?" />
      <button disabled={isPending}>{isPending ? "Posting…" : "Post review"}</button>
      {state.error && <p role="alert">{state.error}</p>}
      {state.ok && <p>Thanks for your review!</p>}
    </form>
  );
}
```

21.5 Back-end features

21.5.1 Route handlers (API routes)

A `route.js` file in `app/` creates an **HTTP endpoint** instead of a page. Export functions named after HTTP methods (`GET`, `POST`, `PUT`, `DELETE` …); they receive a standard Web `Request` and return a `Response` (or `NextResponse`). Use route handlers for webhooks, a public API used by a mobile app, file downloads and OAuth callbacks; for your own forms and buttons, Server Actions are usually simpler.

ROUTE HANDLERS (REPRESENTATIVE) JavaScript

```
// app/api/courses/route.js   ->  GET/POST /api/courses
import { NextResponse } from "next/server";
import { db } from "@/lib/db";

export async function GET(request) {
  const level = new URL(request.url).searchParams.get("level");
  const courses = await db.course.findMany({ where: level ? { level } : {} });
  return NextResponse.json(courses);
}

export async function POST(request) {
  const body = await request.json();
  const course = await db.course.create({ data: body });
  return NextResponse.json(course, { status: 201 });
}

// app/api/courses/[id]/route.js   ->  DELETE /api/courses/7
export async function DELETE(request, { params }) {
  await db.course.delete({ where: { id: (await params).id } });
  return new Response(null, { status: 204 });
}
```

21.5.2 Middleware

Code that should run **before** a request is completed, such as redirecting users who are not logged in, rewriting URLs for A/B tests or setting headers, goes into one file at the project root. It was called

`middleware.js`; Next.js 16 renamed it to `proxy.js` (exporting a `proxy` function) to make clear that it sits in front of the app like a network proxy. A `matcher` limits which paths it runs on. Keep it fast and light, and do not rely on it alone for security: check permissions again where the data is read.

REDIRECTING ANONYMOUS USERS (REPRESENTATIVE) JavaScript

```
// proxy.js (middleware.js before Next.js 16)
import { NextResponse } from "next/server";

export function proxy(request) {
  const token = request.cookies.get("session")?.value;
  if (!token) {
    const login = new URL("/login", request.url);
    login.searchParams.set("next", request.nextUrl.pathname);   // come back after login
    return NextResponse.redirect(login);
  }
  return NextResponse.next();
}

export const config = { matcher: ["/dashboard/:path*", "/admin/:path*"] };
```

21.5.3 Connecting a database with Prisma or Drizzle

Because Server Components, Server Actions and route handlers run on the server, they can talk to a database through an **ORM**. **Prisma** uses a schema file and generates a fully typed client; **Drizzle** defines tables in TypeScript and writes SQL-like queries. Both work with PostgreSQL, MySQL and SQLite and with hosted databases (Neon, Supabase, PlanetScale). Create the client once in `lib/db.js` and import it only from server code.

PRISMA SCHEMA, CLIENT AND A QUERY (REPRESENTATIVE) JavaScript

```
// prisma/schema.prisma
model Course {
  id      String   @id @default(cuid())
  slug    String   @unique
  title   String
  weeks   Int
  reviews Review[]
}
model Review {
  id       String @id @default(cuid())
  rating   Int
  text     String
  course   Course @relation(fields: [courseId], references: [id])
  courseId String
}

// $ npx prisma migrate dev --name init      creates the tables and the typed client

// lib/db.js
import { PrismaClient } from "@prisma/client";
export const db = globalThis.prisma ?? new PrismaClient();
if (process.env.NODE_ENV !== "production") globalThis.prisma = db;   // one client in dev (hot reload)

// in a Server Component
const course = await db.course.findUnique({ where: { slug: "react" }, include: { reviews: true } });
```

21.5.4 Authentication with Auth.js or Clerk (overview)

Auth.js (formerly NextAuth.js) is an open-source library: you configure providers (Google, GitHub, email magic links, username and password), it manages sessions in cookies, and `auth()` returns the session in Server Components, Server Actions, route handlers and the proxy. **Clerk** is a hosted service with ready-made sign-in pages and user management, quicker to start, paid beyond a free tier. Whichever you choose, check the session on the **server** for every protected page and action.

AUTH.JS (REPRESENTATIVE) JavaScript

```
// auth.js
import NextAuth from "next-auth";
import Google from "next-auth/providers/google";
export const { handlers, auth, signIn, signOut } = NextAuth({ providers: [Google] });

// app/api/auth/[...nextauth]/route.js
export { GET, POST } from "@/auth";   // re-export the handlers

// app/dashboard/page.jsx
import { auth } from "@/auth";
import { redirect } from "next/navigation";
export default async function Dashboard() {
  const session = await auth();
  if (!session) redirect("/login");
  return <h1>Welcome, {session.user.name}</h1>;
}
```

21.6 Deploying Next.js

21.6.1 Vercel and other hosts

Vercel, the company behind Next.js, offers the simplest path: connect the GitHub repository, and every push to `main` deploys to production while every pull request gets its own **preview URL**. Environment variables are set in the project settings. Next.js also runs anywhere Node.js runs: `next build` then `next start` on a VPS, in a Docker container (with `output: "standalone"` for a small image), or on platforms such as Netlify, Railway, Render, AWS Amplify and Cloudflare (through adapters). A site with no server features can be exported as plain static files with `output: "export"`.

A typical production build summary (representative)

```
$ npm run build
   ▲ Next.js 16
   Creating an optimized production build ...
 ✓ Compiled successfully
 ✓ Generating static pages (12/12)

Route (app)
┌ ○ /                    static
├ ○ /about               static
├ ● /courses/[slug]      SSG (generateStaticParams)
├ ƒ /dashboard           dynamic (server-rendered on demand)
└ ƒ /api/courses         route handler

○ Static   ● SSG   ƒ Dynamic
$ npm start              # production server on port 3000
```

21.7 Other React frameworks (overview)

21.7.1 React Router framework mode, Remix and Expo for React Native

Framework	What it is	Choose it when
Next.js	full-stack React framework with RSC, Server Actions, many rendering modes	most new full-stack or content-heavy React projects
React Router (framework mode)	the successor of Remix 2: loaders, actions, SSR and pre-rendering on Vite (Module 13.7)	you like web-standard loaders/actions and want to stay close to plain React and Vite
Remix 3	the Remix team's next framework, a new direction under active development	following its evolution; not yet a default choice
TanStack Start	full-stack framework built on TanStack Router, type-safe end to end	TypeScript-heavy apps already using TanStack tools
Astro	content-first site builder; React components as interactive "islands"	blogs, docs and marketing sites with little interactivity
Expo (React Native)	framework for native iOS and Android apps written in React, with file-based routing	mobile apps; reuse React knowledge and some code

PRACTICE CHECKPOINT

A full-stack blog or job portal in Next.js with Server Components, a database, Server Actions for create/edit, authentication and SEO metadata, deployed live.

A suggested structure for the **job portal** option, combining every part of the module. Build it in this order: data model, read pages, then write actions, then authentication, then polish (metadata, loading and error states), then deploy.

Job portal: project outline

```
app/
├── layout.jsx                  metadata template, header with session
├── page.jsx                    latest jobs (Server Component, "use cache" + cacheTag("jobs"))
├── jobs/
│   ├── page.jsx                search with searchParams (?q=react&city=pune)
│   ├── loading.jsx
│   ├── [id]/
│   │   ├── page.jsx            job detail + generateMetadata (title, description, Open Graph)
│   │   └── ApplyButton.jsx     "use client"
│   └── new/page.jsx            protected: post a job (form -> Server Action)
├── dashboard/page.jsx          protected: my postings, edit and delete
├── api/auth/[...nextauth]/route.js
├── actions.js                  "use server": createJob, updateJob, deleteJob (zod + auth +
updateTag)
├── sitemap.js, robots.js
lib/db.js                       Prisma client (PostgreSQL on Neon)
prisma/schema.prisma            User, Job, Application
auth.js                         Auth.js with Google sign-in
proxy.js                        redirect anonymous users away from /dashboard and /jobs/new
```

APP/ACTIONS.JS: SAMPLE SOLUTION CORE (REPRESENTATIVE)

JavaScript

```
// app/actions.js (core of the write path)
"use server";
import { z } from "zod";
import { auth } from "@/auth";
import { db } from "@/lib/db";
import { updateTag } from "next/cache";
import { redirect } from "next/navigation";

const Job = z.object({ title: z.string().min(5), company: z.string().min(2), city: z.string(), salary:
z.coerce.number().positive() });

export async function createJob(prev, formData) {
  const session = await auth();
  if (!session) return { error: "Log in to post a job" };
  const parsed = Job.safeParse(Object.fromEntries(formData));
  if (!parsed.success) return { error: parsed.error.issues[0].message };
  const job = await db.job.create({ data: { ...parsed.data, ownerId: session.user.id } });
  updateTag("jobs");
  redirect(`/jobs/${job.id}`);
}

export async function deleteJob(id) {
  const session = await auth();
  const job = await db.job.findUnique({ where: { id } });
  if (!session || job.ownerId !== session.user.id) throw new Error("Not allowed");   // authorise!
  await db.job.delete({ where: { id } });
  updateTag("jobs");
}
```

- **Server Components:** list, search and detail pages read from Prisma directly; only `ApplyButton` and the forms are client components.
- **SEO:** `generateMetadata` per job, a sitemap listing every job, and server-rendered HTML for every page.
- **Deploy:** push to GitHub, import the repository in Vercel, add `DATABASE_URL`, `AUTH_SECRET` and the Google keys as environment variables, and run `prisma migrate deploy` during the build.

Frameworks render React on the server: SSR per request, SSG at build time, ISR in between, with hydration making the HTML interactive, which helps SEO and first paint. The Next.js App Router maps folders to URLs (`page`, `layout`, `[slug]`, `loading`, `error`, `not-found`), optimises links, images and fonts, and generates SEO tags from `metadata`. Components are Server Components by default (async data access, no client JavaScript); `"use client"` marks interactive leaves. Streaming with Suspense sends fast parts first; `"use cache"` and revalidation control caching. Server Actions (`"use server"`) handle mutations from forms (validate and authorise inside). Route handlers build APIs, `proxy.js` runs before requests, Prisma or Drizzle connect databases, Auth.js or Clerk handle login, and Vercel or any Node host deploys the app.

MODULE 22 · ADVANCED

Deployment, projects and interview preparation

Minimum duration **4 days** | Level **Advanced**

The final module turns knowledge into shipped work and job offers. It covers production builds and environment variables, deploying single-page apps to Netlify, Vercel or GitHub Pages with the right routing configuration, CI/CD and error monitoring; front-end security (XSS and dangerouslySetInnerHTML, storing tokens, keeping secrets out of the bundle); five capstone projects; and interview preparation: a core-topics checklist, output-prediction questions verified in the browser, machine-coding rounds with solutions, the JavaScript fundamentals asked alongside React, and presenting your portfolio. It ends with next steps: React Native and Node.js back ends.

IN THIS MODULE

22.1 Deployment
22.2 Security
22.3 Capstone projects
22.4 Interview preparation
22.5 Next steps

22.1 Deployment

22.1.1 Production builds and environment variables

`npm run build` creates an optimised production version in `dist/`: JavaScript and CSS are bundled, minified and split into chunks, file names get a **content hash** (`index-a1b2c3.js`) so browsers can cache them forever, and React runs in production mode (faster, no development warnings). `npm run preview` serves the build locally: always check it before deploying, because some bugs appear only in production builds. Environment variables are **baked into the bundle at build time**: set `VITE_API_URL` in the hosting provider's settings (or a `.env.production` file) **before** building.

A REAL PRODUCTION BUILD (ESBUILD, THE BUNDLER USED BY VITE) — main.js

```
$ esbuild src/main.jsx --bundle --splitting --minify --entry-names=[name]-[hash] ...
```

TERMINAL

```
dist/assets/Reports-FRP3FSQK.js              0.2 kB
dist/assets/chunk-JWQPBWFW.js                1.3 kB
dist/assets/main-RV7K52IB.js               188.4 kB

$ grep -l "api.learnbeingforward.in" dist/assets/*
dist/assets/Reports-FRP3FSQK.js
```

The build produced hashed files, a separate chunk for the lazily loaded `Reports` page, and the value of `VITE_API_URL` written literally into that chunk, which is exactly why such variables must never hold secrets (Section 22.2.3).

22.1.2 Deploying SPAs to Netlify, Vercel or GitHub Pages

Host	How	Notes
Netlify	connect the Git repository; build command `npm run build`, publish directory `dist`	preview deploys for pull requests, forms, serverless functions
Vercel	import the repository; Vite is detected automatically	first-class for Next.js; preview URLs per branch
GitHub Pages	build in GitHub Actions and publish `dist`; set `base: "/repo-name/"` in `vite.config.js`	free for public repos; static only; needs a 404 trick for routing
Cloudflare Pages, **Render**, **Firebase Hosting**	similar Git-based flow	all serve static files from a global CDN

22.1.3 SPA routing configuration (redirects)

A client-side router works only once `index.html` is loaded. If a user opens `/courses/react` directly or refreshes the page, the browser asks the **server** for that path, and a static host answers **404 Not Found** because there is no such file. Configure the host to serve `index.html` for every unknown path (a "rewrite"); React Router then reads the URL and shows the right page.

REWRITES FOR CLIENT-SIDE ROUTING

JavaScript

```
# Netlify: public/_redirects  (copied into dist/ by Vite)
/*    /index.html   200

// Vercel: vercel.json
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }

# Nginx (own server)
location / {
  try_files $uri $uri/ /index.html;
}

# GitHub Pages has no rewrites: copy index.html to 404.html after the build
"build": "vite build && cp dist/index.html dist/404.html"
```

22.1.4 CI/CD basics

Continuous integration (CI) checks every change automatically: install, lint, type-check, test, build (the workflow from Module 20). **Continuous deployment** (CD) ships every change that passes to production automatically, usually when it is merged into `main`, while pull requests get **preview deployments** that reviewers can click through. Hosts like Netlify and Vercel do CD for you once the repository is connected; GitHub Actions can deploy anywhere.



A TYPICAL CI/CD PIPELINE

22.1.5 Monitoring errors with Sentry (overview)

Users rarely report errors; they just leave. An error-monitoring service such as **Sentry** captures uncaught exceptions and rejected promises in your users' browsers, with the stack trace (mapped to your original

code through uploaded **source maps**), the browser, the URL, the user's last actions (breadcrumbs) and even session replays, and alerts you. It also measures performance (Web Vitals, slow requests).

SENTRY SETUP (REPRESENTATIVE) JavaScript

```
// main.jsx
import * as Sentry from "@sentry/react";

Sentry.init({
  dsn: import.meta.env.VITE_SENTRY_DSN,       // public identifier, safe in the bundle
  environment: import.meta.env.MODE,
  tracesSampleRate: 0.2,                      // performance data for 20% of sessions
});

createRoot(document.getElementById("root"), {
  onUncaughtError: Sentry.reactErrorHandler(),
  onCaughtError: Sentry.reactErrorHandler(),
}).render(<App />);
```

22.2 Security

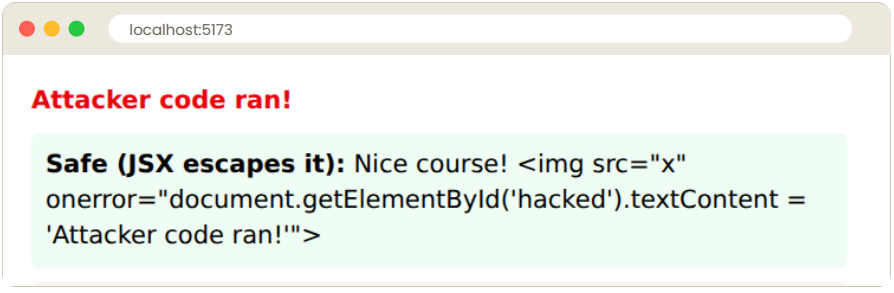
22.2.1 XSS and dangerouslySetInnerHTML

Cross-site scripting (XSS) happens when text from users (a comment, a profile name, a URL parameter) is inserted into the page as HTML, so an attacker's script runs in other users' browsers, where it can steal tokens or act on their behalf. React protects you by default: everything inside `{ }` in JSX is **escaped** and shown as text. The escape hatch `dangerouslySetInnerHTML` inserts raw HTML and switches that protection off. Use it only with HTML you trust, or **sanitise** it first with a library such as DOMPurify. Also never put user input into `href` without checking that it starts with `http`, because `javascript:` URLs run code.

APP.JSX JavaScript

```
const comment = `Nice course! <img src="x" onerror="document.getElementById('hacked').textContent =
'Attacker code ran!'">`;

export default function App() {
  return (
    <div className="w-[460px] space-y-2 text-sm">
      <p id="hacked" className="font-bold text-red-600"></p>
      <div className="rounded bg-green-50 p-2"><b>Safe (JSX escapes it):</b> {comment}</div>
      <div className="rounded bg-red-50 p-2"><b>Unsafe (dangerouslySetInnerHTML):</b> <span
dangerouslySetInnerHTML={{ __html: comment }} /></div>
    </div>
  );
}
```



THE SAME COMMENT TWICE: SHOWN AS HARMLESS TEXT IN THE FIRST BOX; IN THE SECOND IT BECAME A REAL <IMG> WHOSE ONERROR CODE RAN

SANITISING HTML BEFORE INSERTING IT (REPRESENTATIVE) JavaScript

```
import DOMPurify from "dompurify";

function Comment({ html }) {
  const clean = DOMPurify.sanitize(html);          // removes scripts, on* handlers, javascript: URLs
  return <div dangerouslySetInnerHTML={{ __html: clean }} />;
}
```

22.2.2 Storing tokens safely

Where you keep the authentication token decides what an XSS bug can steal. Anything in `localStorage` or `sessionStorage` can be read by **any** script running on the page, including injected ones. The safest common setup is an **httpOnly, Secure, SameSite** cookie set by the server: JavaScript cannot read it at all, the browser sends it automatically, and `SameSite` (plus a CSRF token for older browsers) protects against cross-site requests.

Storage	Readable by JavaScript (and XSS)?	Sent automatically?	Verdict
`localStorage`	yes	no (you add the header)	simple, but exposed to XSS; keep tokens short-lived
memory (a variable / React state)	yes, but gone on reload	no	good for short-lived access tokens, with a refresh cookie
httpOnly Secure SameSite cookie	**no**	yes	recommended for session or refresh tokens

22.2.3 Never exposing secrets in front-end code

Everything shipped to the browser is **public**: anyone can open DevTools, read the JavaScript and see every string, including environment variables that were baked in at build time (the build in Section 22.1.1 showed the API URL sitting in plain text). Private API keys (payment providers, OpenAI, database passwords, admin tokens) must stay on a **server**: a small back end, a serverless function, a Next.js route handler or Server Action, which the front end calls and which adds the secret itself. Keys that are designed to be public (a Firebase config, a Stripe *publishable* key, a Sentry DSN, a Maps key restricted to your domain) are fine in the bundle.

KEEPING A PRIVATE KEY ON THE SERVER JavaScript

```
// ✗ the secret ends up in dist/assets/*.js for everyone to read
const res = await fetch("https://api.openai.com/v1/responses", {
  headers: { Authorization: `Bearer ${import.meta.env.VITE_OPENAI_KEY}` },
});

// ✓ the browser calls your server; the server adds the secret
const res = await fetch("/api/summarise", { method: "POST", body: JSON.stringify({ text }) });

// server (Express, serverless function or Next.js route handler)
app.post("/api/summarise", async (req, res) => {
  const r = await fetch("https://api.openai.com/v1/responses", {
    headers: { Authorization: `Bearer ${process.env.OPENAI_KEY}` },   // never sent to the client
    method: "POST", body: JSON.stringify({ model: "gpt-x", input: req.body.text }),
  });
  res.json(await r.json());
});
```

22.3 Capstone projects

Recruiters look for **finished, deployed** projects with clean code, a README and tests, more than for many half-done tutorials. Choose two (one single-page app and one Next.js full-stack app, as the checkpoint requires) and build them well. Each project below lists the features that show the skills from this course.

22.3.1 E-commerce store with cart, filters, auth and checkout flow

- Product list with search, category and price filters and sorting kept in the URL (Module 13), pagination or infinite scroll.
- Product detail pages, cart with quantities in a Zustand or Redux Toolkit store persisted to `localStorage` (Module 16).
- Login and protected checkout and order-history routes; a multi-step checkout form with React Hook Form and Zod (Module 14).
- Server data through TanStack Query with optimistic "add to wishlist"; tests for the cart logic and the checkout form (Module 20).

22.3.2 Admin dashboard with charts, tables and CRUD

- A data table with sorting, filtering, pagination and row selection (TanStack Table), and virtualisation for large data (Module 18).
- Charts (Recharts or Chart.js) for enrolments and revenue, with date-range filters.
- Create, edit and delete with forms in modals (portals, focus management, Module 19), optimistic updates and toasts.
- Role-based access (admin and viewer), dark mode, and TypeScript throughout (Module 15).

22.3.3 Social media or chat app with real-time updates

- Authentication, profiles with image upload and previews (Module 14).
- A feed with infinite scroll and likes with `useOptimistic` (Module 17).
- Real-time chat or notifications through WebSockets (Socket.IO) or a service such as Firebase or Supabase realtime, with presence ("online") and typing indicators.
- Careful effect cleanup for subscriptions (Module 9) and accessibility for live updates (`aria-live`).

22.3.4 Movie or recipe search app using a public API

- Debounced search-as-you-type (Module 11) with race-condition handling, against a public API such as TMDB or TheMealDB.
- Detail pages with dynamic routes, favourites saved locally, loading skeletons and error states.
- A responsive, polished UI with Tailwind CSS and good Lighthouse scores (Module 18).
- Good first project to finish in a week; deploy it to Netlify or Vercel.

22.3.5 Full-stack MERN or Next.js application

- Either MERN (MongoDB, Express, React, Node.js) or Next.js with Prisma and PostgreSQL (Module 21).
- Real authentication, a relational or document data model, validation on the server, and role-based authorisation.
- Server Components and Server Actions (Next.js) or a REST API with an Express back end and a React front end.

- CI with tests, deployment of both parts, environment variables and a seeded demo account for recruiters.

22.4 Interview preparation

22.4.1 Core topics checklist

Front-end interviews for React roles usually test these topics. For each, be ready to explain it in two or three sentences, give an example, and mention a common mistake.

Topic	What to be able to explain	Module
Virtual DOM and reconciliation	element trees, diffing by type and key, render vs commit phase	18
Keys	why stable unique keys matter; what goes wrong with the index	7, 18
Props vs state	read-only inputs vs component memory; lifting state up; one-way data flow	5, 6
Rules of hooks	top level only, React functions only; why (the hooks list is ordered)	11
useEffect dependencies	when effects run, cleanup, stale closures, why not to fetch without cleanup	9
Memoisation	memo, useMemo, useCallback, when they help, the React Compiler	18
Context vs Redux/Zustand	context re-renders consumers; stores with selectors; server state separately	12, 16
Controlled vs uncontrolled	value + onChange vs refs/defaultValue; when to use each	8, 19
Server Components	server vs client components, "use client", what never reaches the browser	21
React 19 features	Actions, useActionState, useOptimistic, use, ref as a prop	17

22.4.2 Output-prediction questions on renders and state

Interviewers show a short component and ask what it displays or logs. Predict the answer, then compare it with the real result below each question, which was produced by running the code in Chromium.

Q1. What is logged after one click?

Q1 JavaScript

```
import { useState } from "react";

export default function App() {
  const [count, setCount] = useState(0);
  function handleClick() {
    setCount(count + 1);
    setCount(count + 1);
    setCount(count + 1);
    console.log("inside handler:", count);
  }
  console.log("render with count =", count);
  return <button onClick={handleClick}>+3?</button>;
}
```

```
REAL RESULT
render with count = 0
click
inside handler: 0
render with count = 1
```

Answer: the count becomes **1**, not 3. All three calls compute `0 + 1` from the same render's `count`, the handler logs the old value 0 (state is a snapshot), and React batches the updates into a single re-render. Fix: `setCount(c => c + 1)`.

Q2. In what order are these logged on mount and after one click?

Q2 JavaScript

```
import { useEffect, useState } from "react";

export default function App() {
  const [n, setN] = useState(0);
  useEffect(() => {
    console.log("effect, n =", n);
    return () => console.log("cleanup, n =", n);
  }, [n]);
  console.log("render, n =", n);
  return <button onClick={() => setN(n + 1)}>next</button>;
}
```

```
REAL RESULT
render, n = 0
effect, n = 0
click
render, n = 1
cleanup, n = 0
effect, n = 1
```

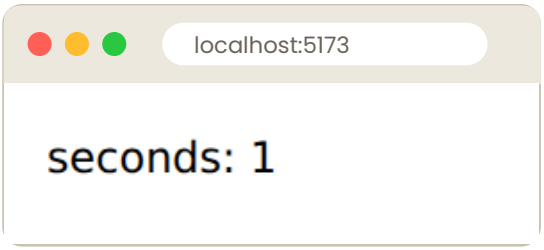
Answer: render first, then the effect. On the update React renders with the new value, then runs the **previous** effect's cleanup (which still sees `n = 0` through its closure), then the new effect. (In development with Strict Mode you would also see an extra mount, cleanup, mount at the start.)

Q3. This timer runs for about 2 seconds. What does it show?

Q3 JavaScript

```
import { useEffect, useState } from "react";

export default function App() {
  const [seconds, setSeconds] = useState(0);
  useEffect(() => {
    const id = setInterval(() => setSeconds(seconds + 1), 300);
    return () => clearInterval(id);
  }, []);                                    // empty dependency array
  return <p>seconds: {seconds}</p>;
}
```



REAL RESULT AFTER ABOUT 2 SECONDS

Answer: it is stuck at **1**. The interval callback was created in the first render, so it always sees `seconds = 0` (a **stale closure**) and keeps setting 1. Fix: `setSeconds(s => s + 1)`, or list `seconds` as a dependency.

Q4. Type "S123" into the field, then click *switch*. Is the text kept?

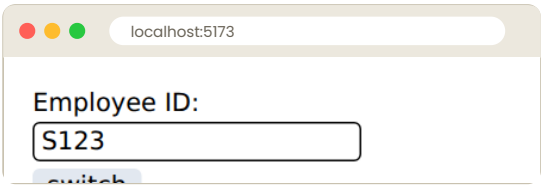
Q4

JavaScript

```
import { useState } from "react";

function Field({ label }) {
  const [text, setText] = useState("");
  return <label className="block">{label}: <input value={text} onChange={(e) =>
setText(e.target.value)} className="rounded border px-1" /></label>;
}

export default function App() {
  const [isStudent, setIsStudent] = useState(true);
  return (
    <div className="space-y-1 text-sm">
      {isStudent ? <Field label="Student ID" /> : <Field label="Employee ID" />}
      <button className="rounded bg-slate-200 px-2" onClick={() =>
setIsStudent(!isStudent)}>switch</button>
    </div>
  );
}
```



REAL RESULT: TYPED S123, THEN CLICKED SWITCH

Answer: yes, "S123" stays, under the new label. Both branches render a `Field` at the **same position**, so React keeps the same component instance and its state; only the `label` prop changes. To reset it, give each branch a different `key` (`key="student"` / `key="employee"`).

22.4.3 Machine-coding rounds: to-do, star rating, autocomplete, pagination, infinite scroll, modal

In a machine-coding round you build a small working feature in 45 to 90 minutes, usually without libraries. Interviewers watch how you break the problem down, name things, handle edge cases (empty, loading, error, keyboard) and talk through trade-offs. Practise each of these until you can build it calmly in the time. Three model solutions follow; the to-do list (Module 6/7) and the modal with focus trap (Module 19) were built earlier in the course.

Star rating (20 minutes): hover preview, click to select, keyboard accessible, configurable count.

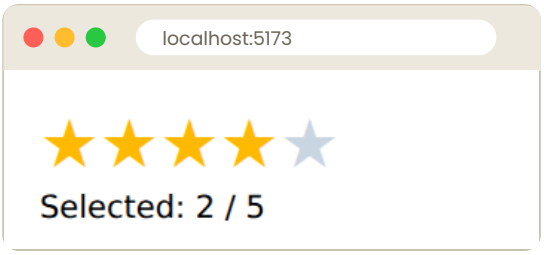
APP.JSX

JavaScript

```
import { useState } from "react";

function StarRating({ max = 5, value, onChange }) {
  const [hover, setHover] = useState(0);
  const shown = hover || value;
  return (
    <div role="radiogroup" aria-label="Rating" className="flex text-3xl" onMouseLeave={() =>
setHover(0)}>
      {Array.from({ length: max }, (_, i) => i + 1).map((n) => (
        <button key={n} role="radio" aria-checked={value === n} aria-label={`${n} star${n > 1 ? "s" :
""}`}
                onMouseEnter={() => setHover(n)} onClick={() => onChange(n)}
                className={n <= shown ? "text-amber-400" : "text-slate-300"}>★</button>
      ))}
    </div>
  );
}

export default function App() {
  const [rating, setRating] = useState(2);
  return <div className="text-sm"><StarRating value={rating} onChange={setRating} /><p>Selected:
{rating} / 5</p></div>;
}
```



SELECTED 2, NOW HOVERING OVER THE FOURTH STAR: THE PREVIEW SHOWS 4 WHILE THE VALUE STAYS 2

Autocomplete (45 minutes): debounced search, keyboard navigation with arrow keys and Enter, highlight of the matching text, and stale-response protection.

APP.JSX

JavaScript

```
import { useEffect, useState } from "react";

const COURSES = ["React", "React Native", "Redux Toolkit", "Java", "JavaScript", "Python",
"PostgreSQL", "Next.js", "Node.js", "TypeScript"];
const search = (q) => new Promise((r) => setTimeout(() => r(COURSES.filter((c) =>
c.toLowerCase().includes(q.toLowerCase()))), 150));

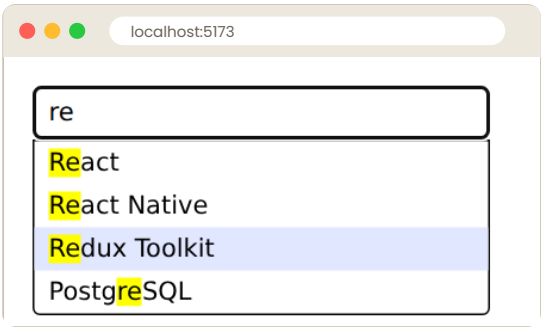
function Highlight({ text, q }) {
  const i = text.toLowerCase().indexOf(q.toLowerCase());
  return <>{text.slice(0, i)}<mark>{text.slice(i, i + q.length)}</mark>{text.slice(i + q.length)}</>;
}

export default function Autocomplete() {
  const [q, setQ] = useState("");
  const [items, setItems] = useState([]);
  const [active, setActive] = useState(0);
  const [chosen, setChosen] = useState("");

  useEffect(() => {
    if (!q) { setItems([]); return; }
    let ignore = false;                                    // stale-response guard
    const t = setTimeout(() => search(q).then((r) => { if (!ignore) { setItems(r); setActive(0); } }),
```

```
200);   // debounce
    return () => { ignore = true; clearTimeout(t); };
  }, [q]);

  function onKeyDown(e) {
    if (e.key === "ArrowDown") { e.preventDefault(); setActive((a) => Math.min(a + 1, items.length -
1)); }
    if (e.key === "ArrowUp") { e.preventDefault(); setActive((a) => Math.max(a - 1, 0)); }
    if (e.key === "Enter" && items[active]) { setChosen(items[active]); setQ(""); }
  }
  return (
    <div className="w-64 text-sm">
      <input value={q} onChange={(e) => setQ(e.target.value)} onKeyDown={onKeyDown} role="combobox"
aria-expanded={items.length > 0}
             placeholder="Search courses" className="w-full rounded border px-2 py-1" />
      {items.length > 0 && (
        <ul role="listbox" className="rounded-b border">
          {items.map((it, i) => (
            <li key={it} role="option" aria-selected={i === active} onMouseDown={() => {
setChosen(it); setQ(""); }}
                 className={`px-2 py-0.5 ${i === active ? "bg-indigo-100" : ""}`}><Highlight text={it}
q={q} /></li>
          ))}
        </ul>
      )}
      {chosen && <p className="mt-1">Chosen: <b>{chosen}</b></p>}
    </div>
  );
}
```



TYPED “RE” AND PRESSED THE DOWN ARROW TWICE: DEBOUNCED RESULTS WITH THE MATCH HIGHLIGHTED AND THE THIRD OPTION ACTIVE

Pagination (30 minutes): page buttons with ellipses, previous/next, and a page-size choice; the helper that computes the page numbers is the interesting part, so keep it pure and testable.

APP.JSX

JavaScript

```
import { useState } from "react";

// pure helper: [1, "…", 4, 5, 6, "…", 20]
export function pageList(current, total, around = 1) {
  const pages = [];
  for (let p = 1; p <= total; p++) {
    if (p === 1 || p === total || Math.abs(p - current) <= around) pages.push(p);
    else if (pages[pages.length - 1] !== "…") pages.push("…");
  }
  return pages;
}

const rows = Array.from({ length: 97 }, (_, i) => `Student ${i + 1}`);

export default function App() {
  const [page, setPage] = useState(1);
  const size = 5;
  const total = Math.ceil(rows.length / size);
  return (
    <div className="w-96 space-y-2 text-sm">
      <ul>{rows.slice((page - 1) * size, page * size).map((r) => <li key={r}>{r}</li>)}</ul>
      <nav aria-label="Pagination" className="flex gap-1">
        <button disabled={page === 1} onClick={() => setPage(page - 1)} className="px-2
disabled:opacity-30">‹</button>
        {pageList(page, total).map((p, i) => p === "…"
          ? <span key={`gap${i}`} className="px-1">…</span>
          : <button key={p} aria-current={p === page ? "page" : undefined} onClick={() => setPage(p)}
                    className={`rounded px-2 ${p === page ? "bg-indigo-600 text-white" : "bg-slate-
100"}`}>{p}</button>)}
        <button disabled={page === total} onClick={() => setPage(page + 1)} className="px-2
disabled:opacity-30">›</button>
      </nav>
    </div>
  );
}
```



WENT TO PAGE 9 OF 20: FIRST AND LAST PAGES, NEIGHBOURS AND ELLIPSES

Infinite scroll hint: render a sentinel `<div ref={sentinelRef} />` after the list and observe it with an `IntersectionObserver` in an effect; when it becomes visible and more data exists, load the next page; disconnect in the cleanup and guard against loading twice. **Modal**: portal, focus trap, Escape to close, return focus to the opener, prevent background scroll (Module 19).

22.4.4 JavaScript fundamentals asked alongside React

React interviews almost always include JavaScript questions, because React code is JavaScript. The most common ones, run in Node.js:

JS-FUNDAMENTALS.JS

main.js

```
// 1. Closures and var vs let in loops
var fs = [];
for (var i = 0; i < 3; i++) fs.push(() => i);
let gs = [];
for (let j = 0; j < 3; j++) gs.push(() => j);
console.log("var:", fs.map((f) => f()), "let:", gs.map((g) => g()));

// 2. Event loop: sync, microtasks (promises), macrotasks (timers)
console.log("A");
setTimeout(() => console.log("D (timeout)"), 0);
Promise.resolve().then(() => console.log("C (promise)"));
console.log("B");

// 3. this and arrow functions
const course = {
  title: "React",
  normal() { return this.title; },
  arrow: () => typeof this,
};
console.log(course.normal(), "|", course.arrow());

// 4. Equality and reference
console.log([1, 2] === [1, 2], { a: 1 } == { a: 1 }, "5" == 5, "5" === 5);

// 5. Spread copies one level only
const a = { user: { name: "Asha" }, tags: ["x"] };
const b = { ...a };
b.user.name = "Ravi";
console.log(a.user.name, "(shallow copy shares nested objects)");
```

OUTPUT

```
var: [ 3, 3, 3 ] let: [ 0, 1, 2 ]
A
B
React | object
false false true false
Ravi (shallow copy shares nested objects)
C (promise)
D (timeout)
```

Topic	Typical question
Closures	Write a `debounce` / `once` / counter factory.
Event loop	Order of logs with promises, timers and async/await.
this, call/apply/bind	What does `this` refer to here? Implement `bind`.
Promises	Implement `Promise.all`; difference between all, allSettled, race, any.
Array methods	Implement `map` / `filter` / `reduce`; flatten a nested array; group by.
Copying	Shallow vs deep copy; `structuredClone`; why immutability matters in React.
Modules and tooling	ES modules, tree shaking, what a bundler does.

22.4.5 Presenting your portfolio and GitHub

- **Portfolio site:** one page with a short introduction, your 3 to 4 best projects (live link, GitHub link, a screenshot or 30-second video, the stack, and what you personally solved), skills and contact details. Make it fast and accessible; it is itself a React project.
- **Each repository:** a README with what the app does, a screenshot, live demo link, demo login, tech stack, how to run it locally, and the interesting technical decisions; meaningful commit history; tests and a CI badge; no secrets committed.
- **GitHub profile:** pin your best repositories, add a profile README, and contribute regularly; small fixes to open-source projects count.
- **Talking about a project:** prepare a 2-minute story: the problem, your role, the architecture, one hard bug and how you solved it, what you would improve. Be ready to share your screen and walk through the code.
- **Resume:** one page, impact-focused bullet points ("Built a dashboard with React, TypeScript and TanStack Query; virtualised a 10,000-row table, cutting render time from 400 ms to 15 ms").

22.5 Next steps

22.5.1 React Native for mobile apps

React Native uses the same React (components, props, state, hooks, context) to build **native** iOS and Android apps. Instead of `div` and `span` you use native components such as `View`, `Text`, `Image`, `ScrollView` and `FlatList`, styled with a JavaScript subset of CSS (flexbox by default). **Expo** is the recommended way to start: `npx create-expo-app`, file-based routing with Expo Router, and running the app on your phone by scanning a QR code.

REACT NATIVE WITH EXPO (REPRESENTATIVE) — JavaScript

```
import { useState } from "react";
import { View, Text, Pressable, FlatList, StyleSheet } from "react-native";

export default function Courses() {
  const [saved, setSaved] = useState([]);
  const courses = ["React", "Java", "Python"];
  return (
    <View style={styles.screen}>
      <Text style={styles.title}>LBF Courses</Text>
      <FlatList data={courses} keyExtractor={(c) => c} renderItem={({ item }) => (
        <Pressable onPress={() => setSaved([...saved, item])} style={styles.row}>
          <Text>{item} {saved.includes(item) ? "★" : "☆"}</Text>
        </Pressable>
      )} />
    </View>
  );
}
const styles = StyleSheet.create({ screen: { flex: 1, padding: 24 }, title: { fontSize: 22,
fontWeight: "600" }, row: { paddingVertical: 12 } });
```

22.5.2 Back-end with Node.js and Express

Knowing a back end makes you a **full-stack** developer and helps you build complete capstone projects. With Node.js you keep using JavaScript: **Express** (or Fastify, Hono) handles HTTP routes, a database (PostgreSQL with Prisma, or MongoDB with Mongoose) stores the data, and JWT or session cookies

handle authentication. The small API below is a real Express-style server written with Node's built-in `http` module so that it runs without installing anything:

SERVER.MJS (NODE.JS) main.js

```
import http from "node:http";

const students = [{ id: 1, name: "Asha Rao" }];

const server = http.createServer(async (req, res) => {
  res.setHeader("Content-Type", "application/json");
  if (req.method === "GET" && req.url === "/api/students") return res.end(JSON.stringify(students));
  if (req.method === "POST" && req.url === "/api/students") {
    let body = "";
    for await (const chunk of req) body += chunk;
    const s = { id: students.length + 1, ...JSON.parse(body) };
    students.push(s);
    res.statusCode = 201;
    return res.end(JSON.stringify(s));
  }
  res.statusCode = 404;
  res.end(JSON.stringify({ error: "Not found" }));
});

server.listen(0, async () => {
  const base = `http://localhost:${server.address().port}`;
  const created = await fetch(`${base}/api/students`, { method: "POST", body: JSON.stringify({ name:
"Ravi Kumar" }) });
  console.log("POST", created.status, await created.text());
  const list = await fetch(`${base}/api/students`);
  console.log("GET ", list.status, await list.text());
  const missing = await fetch(`${base}/api/nothing`);
  console.log("GET ", missing.status, await missing.text());
  server.close();
});
```

OUTPUT

```
POST 201 {"id":2,"name":"Ravi Kumar"}
GET  200 [{"id":1,"name":"Asha Rao"},{"id":2,"name":"Ravi Kumar"}]
GET  404 {"error":"Not found"}
```

EXPRESS (REPRESENTATIVE) JavaScript

```
// the same with Express
import express from "express";
const app = express();
app.use(express.json());
app.get("/api/students", (req, res) => res.json(students));
app.post("/api/students", (req, res) => {
  const s = { id: students.length + 1, ...req.body };
  students.push(s);
  res.status(201).json(s);
});
app.listen(3000);
```

PRACTICE CHECKPOINT

Build and deploy two capstone projects (one SPA, one Next.js full-stack app) with tests, and complete five timed machine-coding challenges.

Deliverable	Done when
Capstone 1: SPA (Vite + React)	deployed on Netlify/Vercel with SPA rewrites; README with live link; TanStack Query, a state store, forms with validation; Vitest + Testing Library tests and one Playwright test in CI
Capstone 2: Next.js full-stack	deployed on Vercel; Server Components, Server Actions, database, authentication, metadata and sitemap; seeded demo login
Security review	no secrets in the bundle (search `dist/`), no unsanitised `dangerouslySetInnerHTML`, tokens in httpOnly cookies or memory
Monitoring	Sentry (or similar) receiving errors from production with source maps

#	Timed machine-coding challenge	Time	Must include
1	To-do list with filters (all, active, done) and local persistence	45 min	add, toggle, delete, edit inline, counts, localStorage
2	Star rating component	20 min	hover preview, click, keyboard and ARIA, configurable size
3	Autocomplete / typeahead	45 min	debounce, keyboard navigation, highlight, no stale results
4	Paginated table with sorting	40 min	page list with ellipses, page size, sort by column
5	Infinite scroll feed or accessible modal	45 min	IntersectionObserver with loading guard / portal, focus trap, Escape

Practise each challenge twice: once with this book open, once against the clock with nothing but react.dev. Record yourself explaining your decisions aloud: that commentary is half of what interviewers assess.

Ship with `npm run build` (hashed, minified, split), set `VITE_` variables at build time, add SPA rewrites on static hosts, automate CI/CD with preview deploys, and monitor errors with Sentry. React escapes JSX by default; treat `dangerouslySetInnerHTML` as dangerous and sanitise; keep tokens out of reach of scripts (httpOnly cookies) and every secret on a server, since the bundle is public. Build and deploy finished capstones. For interviews, master the core-topics checklist, predict renders and state updates (snapshots, batching, stale closures, identity by position and key), practise the classic machine-coding rounds, revise JavaScript fundamentals, and present a polished portfolio. Next: React Native with Expo for mobile, and Node.js with Express for the back end.